

SensoVibe

AI Powered Industrial Vibration Intelligence Platform

Software Design & Technical Documentation

Document version 1.0

Project	SensoVibe — AI Powered Industrial Vibration Intelligence Platform
Repository	VibrationMonitoring
Product version	Backend 1.1.0 · Frontend 1.0.0 · Schema rev 011
Document version	1.0
Document type	Software Design Document (SDD) / Complete Technical Documentation
Document scope	Frontend + Backend + Database + Configuration + Assets + APIs + Project Structure
Source of truth	Extracted exclusively from the source code in this repository
Source branch	laxman-dev
Date	26 July 2026
Prepared by	Engineering — SensoVibe Platform Team
Reviewed by	<i>pending</i>
Approved by	<i>pending</i>
Company	SensoVibe
Classification	Internal — Engineering / Stakeholder distribution
Status	Baselined

Document Control

Purpose and scope

This document is the complete technical specification of the SensoVibe platform. It covers the frontend application, the backend service, the database schema, every REST endpoint, all configuration files, the deployment topology, the test position, performance characteristics, and the security posture.

Reading note. Every statement in this document is derived from code that exists in the repository. Where a feature is declared but not implemented (for example the Change Password screen), this document says so explicitly rather than describing intended behaviour.

Intended audience

Audience	Recommended reading
New developers	\$1, \$2, \$3, \$4, \$10, \$11, \$12
Backend engineers	\$2, \$4, \$5, \$6, \$7, \$8, \$14
Frontend engineers	\$2, \$3, \$8, \$9, \$10
Database administrators	\$6, \$11, \$12, \$14
Security reviewers	\$7, \$15
QA engineers	\$5, \$9, \$13, \$15
DevOps / SRE	\$11, \$12, \$14, \$15
Project managers and stakeholders	\$1, \$8, \$9, \$12, \$13

Conventions used

Convention	Meaning
<code>monospace</code>	File path, identifier, command, or literal value
\$n.n	Cross-reference to a numbered section of this document
✓ / ✗	Implemented / not implemented
<i>Figure n</i>	Numbered diagram; indexed in the List of Figures
<i>Table n</i>	Numbered table; indexed in the List of Tables
"stated plainly" notes	Deliberate disclosure of a gap, limitation, or unused artefact

Page numbers are applied by the PDF renderer at export time; section, figure, and table numbering are fixed within this document.

Revision History

Version	Date	Author	Description of change	Status
0.1	26 July 2026	Engineering	Initial draft — Project Overview and System Architecture (§1–§2)	Superseded
0.2	26 July 2026	Engineering	Added Frontend Documentation (§3)	Superseded
0.3	26 July 2026	Engineering	Added Backend Documentation (§4)	Superseded
0.4	26 July 2026	Engineering	Added REST API Documentation (§5)	Superseded
0.5	26 July 2026	Engineering	Added Database Documentation and Authentication & Security (§6–§7)	Superseded
0.6	26 July 2026	Engineering	Added Business Logic, User Flows and Module Documentation (§8–§10)	Superseded
0.7	26 July 2026	Engineering	Added Configuration, Deployment, Testing, Performance, Troubleshooting, Appendix and References (§11–§17)	Superseded
1.0	26 July 2026	Engineering	Consolidated master document: merged all parts, added cover page, document control, revision history, clickable table of contents, list of figures, list of tables, sequential figure and table numbering, and page-break formatting for PDF export.	Current

Document baseline

Item	Value
Source revision	Working tree of branch <code>laxman-dev</code>
Backend version	1.1.0 (<code>backend/app/main.py</code>)
Frontend version	1.0.0 (<code>frontend/package.json</code>)
Database schema	Alembic revision <code>011</code>
Endpoints documented	46
Database tables documented	17
Database columns documented	215
Figures	40
Tables	214

Table of Contents

- Cover Page
- Document Control
- Revision History
- Table of Contents
- List of Figures
- List of Tables
- 1.0 Project Overview
 - 1.1 Project Name
 - 1.2 Purpose
 - 1.3 Business Problem
 - 1.4 Objectives
 - 1.5 Feature Inventory
 - 1.5.1 Declared-but-not-implemented features
 - 1.6 Technologies Used
 - 1.6.1 Backend runtime (backend/requirements.txt)
 - 1.6.2 Frontend runtime (frontend/package.json)
 - 1.6.3 Infrastructure
 - 1.7 Software Architecture
 - 1.7.1 Architectural decisions and their rationale
 - 1.8 High-Level Workflow
 - 1.8.1 The upload pipeline in detail
 - 1.9 Complete Folder Structure
 - 1.9.1 File-count summary
- 2.0 System Architecture
 - 2.1 Overall Architecture
 - 2.2 Client–Server Communication
 - 2.2.1 Transport contract
 - 2.2.2 CORS
 - 2.2.3 Two development paths to the API
 - 2.3 Frontend Architecture
 - 2.3.1 Layered module responsibilities
 - 2.4 Backend Architecture
 - 2.4.1 Application startup sequence
 - 2.5 Database Architecture
 - 2.6 Authentication Flow
 - 2.6.1 Silent refresh with request queueing
 - 2.6.2 Refresh-token rotation
 - 2.7 API Communication Flow
 - 2.7.1 React Query cache keys in use
 - 2.8 Data Flow
 - 2.8.1 Parsed-data document shape
 - 2.9 Request Lifecycle
 - 2.10 Response Lifecycle
- 3.0 Frontend Documentation
 - 3.1 Project Structure — folder by folder
 - 3.2 Entry Points
 - 3.2.1 index.html
 - 3.2.2 main.tsx
 - 3.2.3 App.tsx
 - 3.3 Routing
 - 3.3.1 ProtectedRoute decision order
 - 3.4 State Management
 - 3.4.1 AuthContext (contexts/AuthContext.tsx)
 - 3.4.2 LayoutContext

- 3.4.3 ThemeContext
 - 3.4.4 Toast context (components/ui/Toast.tsx)
 - 3.5 Custom Hooks
 - 3.5.1 useEchartsResize(getInstance, deps)
 - 3.5.2 useHealthStatusData({uploadId, channel, samplingRateHz, enabled})
 - 3.5.3 useFeatureHealthDashboard({sensorId, uploadId, channel, primaryBaseline, baselineList, enabled})
 - 3.5.4 useUploadFactorTrends({uploadId, channel, baselineId, enabled})
 - 3.5.5 useHistoricalTrendData(...) — implemented, not mounted
 - 3.5.6 useVibrationSettings()
 - 3.6 API Layer
 - 3.6.1 api/client.ts — axios configuration
 - 3.6.2 api/auth.ts
 - 3.6.3 api/equipment.ts
 - 3.6.4 api/measurements.ts
 - 3.6.5 api/baselines.ts
 - 3.6.6 Response normalisation — lib/feature-api-normalize.ts
 - 3.7 Styling Strategy
 - 3.7.1 Approach
 - 3.7.2 Design tokens
 - 3.7.3 Component classes defined in index.css
 - 3.7.4 Accessibility and motion
 - 3.7.5 The card contracts
 - 3.8 Responsive Design
 - 3.9 Charts
 - 3.9.1 Chart architecture
 - 3.9.2 GraphWorkspace — the reusable shell
 - 3.9.3 GraphToolbar — 10 possible actions
 - 3.9.4 EchartsGraphViewport
 - 3.9.5 Per-plot-type option builders
 - 3.9.6 Threshold overlay system (lib/threshold-overlay.ts)
 - 3.9.7 Chart statistics
 - 3.10 Tables
 - 3.11 Forms and Validation
 - 3.11.1 The equipment wizard
 - 3.11.2 Login validation
 - 3.11.3 Settings validation
 - 3.11.4 Server-side validation surfaced to the user
 - 3.12 Error Handling (frontend)
 - 3.13 Loading States
 - 3.14 Pagination, Search, Filtering, Sorting
 - 3.15 Exports and Imports
 - 3.16 Environment Variables and Build
 - 3.17 Screen-by-Screen Documentation
 - 3.17.1 Login (/login)
 - 3.17.2 Dashboard (/)
 - 3.17.3 Equipment Master List (/equipment)
 - 3.17.4 Equipment Create / Edit (/equipment/new, /equipment/:id/edit)
 - 3.17.5 Vibration Analysis (/analysis)
 - 3.17.6 Settings (/settings)
 - 3.17.7 Unauthorized (/unauthorized)
 - 3.17.8 Change Password (/change-password)
 - 3.17.9 Application shell (all authenticated routes)
 - 3.18 Complete Frontend Component Reference
 - 3.18.1 components/ui
 - 3.18.2 components/charts
 - 3.18.3 components/analysis
 - 3.18.4 components/analysis/health
 - 3.18.5 components/equipment
 - 3.18.6 components/settings
 - 3.18.7 components/brand
 - 3.19 Complete lib/ Reference
 - 3.20 Complete types/ Reference
- 4.0 Backend Documentation
 - 4.1 Folder Structure and Layering

- 4.2 Application Startup — app/main.py
 - 4.2.1 custom_openapi() in detail
 - 4.3 Configuration — app/config.py
 - 4.4 Database Access — app/database.py
 - 4.5 Dependencies — app/dependencies/auth.py
 - 4.6 Models — app/models/
 - 4.6.1 Equipment (equipment_masters)
 - 4.6.2 SensorConfiguration (sensor_configurations)
 - 4.6.3 Measurement models (models/measurement.py)
 - 4.6.4 User models (models/user.py)
 - 4.7 Schemas — app/schemas/
 - 4.7.1 schemas/equipment.py
 - 4.7.2 schemas/measurement.py
 - 4.7.3 schemas/feature.py
 - 4.7.4 schemas/auth.py, schemas/baseline.py, schemas/acquisition.py
 - 4.8 CRUD Layer — app/crud/
 - 4.8.1 crud/equipment.py
 - 4.8.2 crud/measurement.py
 - 4.8.3 crud/baseline.py
 - 4.8.4 crud/feature.py
 - 4.8.5 crud/user.py
 - 4.9 Services — app/services/
 - 4.9.1 auth_service.py
 - 4.9.2 seed.py
 - 4.9.3 pdf_parser.py
 - 4.9.4 signal_processing.py
 - 4.9.5 plot_generator.py
 - 4.9.6 plot_storage.py
 - 4.9.7 baseline_storage.py
 - 4.9.8 feature_extraction.py
 - 4.9.9 threshold_evaluator.py
 - 4.9.10 feature_storage.py
 - 4.9.11 acquisition_config.py
 - 4.10 Routers — endpoint inventory
 - 4.11 Middleware, Filters, and Interceptors
 - 4.12 Logging
 - 4.13 Caching
 - 4.14 Transactions
 - 4.15 Dependency Injection
 - 4.16 Scheduler, Async Processing, Background Jobs
 - 4.17 Exception Handling (backend)
 - 4.18 Alembic Migrations
 - 4.18.1 alembic/env.py
 - 4.19 Backend Utility Scripts
 - 4.20 Edge Acquisition Script — scripts/vibration.py
- 5.0 REST API Documentation
 - 5.1 Conventions
 - 5.2 Complete Endpoint Index (46 endpoints)
 - 5.3 Health
 - 5.3.1 GET /health
 - 5.4 Authentication API
 - 5.4.1 POST /api/v1/auth/login
 - 5.4.2 POST /api/v1/auth/token
 - 5.4.3 POST /api/v1/auth/refresh
 - 5.4.4 POST /api/v1/auth/logout
 - 5.4.5 GET /api/v1/auth/me
 - 5.5 Equipment API
 - 5.5.1 POST /api/v1/equipment/ — create
 - 5.5.2 GET /api/v1/equipment/ — list
 - 5.5.3 GET /api/v1/equipment/{equipment_id}
 - 5.5.4 PUT and PATCH /api/v1/equipment/{equipment_id}
 - 5.5.5 DELETE /api/v1/equipment/{equipment_id}
 - 5.5.6 POST /api/v1/equipment/{equipment_id}/image
 - 5.5.7 GET /api/v1/equipment/{equipment_id}/image

- 5.5.8 DELETE /api/v1/equipment/{equipment_id}/image
 - 5.5.9 Sensor endpoints
 - 5.5.10 GET /api/v1/equipment/{equipment_id}/ai-readiness
- 5.6 Lookups API
 - 5.6.1 GET /api/v1/lookups/
 - 5.6.2 GET /api/v1/lookups/{lookup_name}
- 5.7 Measurements API
 - 5.7.1 POST /api/v1/measurements/configure
 - 5.7.2 GET /api/v1/measurements/configure/{sensor_id}
 - 5.7.3 PUT /api/v1/measurements/configure/{sensor_id}
 - 5.7.4 Edge acquisition endpoints (26–28)
 - 5.7.5 POST /api/v1/measurements/upload
 - 5.7.6 GET /api/v1/measurements/uploads
 - 5.7.7 GET /api/v1/measurements/uploads/{upload_id}
 - 5.7.8 GET /api/v1/measurements/uploads/{upload_id}/plots
 - 5.7.9 GET /api/v1/measurements/uploads/{upload_id}/plots/{plot_type}
 - 5.7.10 GET /api/v1/measurements/plot-types
 - 5.7.11 GET /api/v1/measurements/uploads/{upload_id}/features
 - 5.7.12 GET /api/v1/measurements/uploads/{upload_id}/factor-trends
 - 5.7.13 GET /api/v1/measurements/uploads/{upload_id}/features/compare
- 5.8 Baselines API
 - 5.8.1 GET /api/v1/baselines
 - 5.8.2 GET /api/v1/baselines/primary
 - 5.8.3 GET /api/v1/baselines/{baseline_id}
 - 5.8.4 PATCH /api/v1/baselines/{baseline_id}/primary
 - 5.8.5 POST /api/v1/baselines/upload
 - 5.8.6 POST /api/v1/baselines/from-upload/{upload_id}
 - 5.8.7 GET /api/v1/baselines/{baseline_id}/plots
 - 5.8.8 GET /api/v1/baselines/{baseline_id}/plots/{plot_type}
 - 5.8.9 GET /api/v1/baselines/{baseline_id}/features
- 5.9 Cross-Cutting API Behaviour
 - 5.9.1 Which endpoints require write access
 - 5.9.2 Validation-rule summary
 - 5.9.3 Endpoint → table matrix
- 6.0 Database Documentation
 - 6.1 Database Identity
 - 6.2 Entity Relationship Diagram
 - 6.3 Relationship Catalogue
 - 6.4 Table Reference
 - 6.4.1 equipment_masters
 - 6.4.2 sensor_configurations
 - 6.4.3 plot_configurations
 - 6.4.4 sensor_data_uploads
 - 6.4.5 measurement_upload_data
 - 6.4.6 plot_results
 - 6.4.7 sensor_baselines
 - 6.4.8 baseline_plot_results
 - 6.4.9 feature_definitions
 - 6.4.10 feature_threshold_rules
 - 6.4.11 measurement_channel_features
 - 6.4.12 measurement_channel_feature_trends
 - 6.4.13 baseline_channel_features
 - 6.4.14 roles
 - 6.4.15 users
 - 6.4.16 user_roles
 - 6.4.17 refresh_tokens
 - 6.5 Constraint Summary
 - 6.5.1 Primary keys
 - 6.5.2 Unique constraints and unique indexes
 - 6.5.3 Check constraints
 - 6.5.4 Default values
 - 6.5.5 Complete index inventory (24 indexes)
 - 6.6 Triggers, Views, Stored Procedures, Functions
 - 6.7 Normalisation Analysis

- 6.8 Data-Volume Model
- 6.9 SQL Examples
- 6.10 Performance Considerations (database)
- 6.11 Backup and Recovery
- 6.12 Migration Runbook
- 6.13 Data Dictionary Quick Reference
- 6.14 End-to-End Database Flow
- 7.0 Authentication & Security
 - 7.1 Authentication Model
 - 7.1.1 Access-token payload
 - 7.1.2 Refresh-token design
 - 7.2 Authorisation Model
 - 7.2.1 Role hierarchy
 - 7.2.2 Enforcement points
 - 7.2.3 Client-side gating inventory
 - 7.3 Sessions and Cookies
 - 7.4 Password Storage
 - 7.5 Encryption
 - 7.6 Role and Permission Management
 - 7.7 Security Filters and Middleware
 - 7.8 CORS
 - 7.9 CSRF
 - 7.10 Rate Limiting and Brute-Force Resistance
 - 7.11 Input Validation and Injection Resistance
 - 7.11.1 SQL injection
 - 7.11.2 Cross-site scripting
 - 7.11.3 File-upload validation
 - 7.11.4 Other validation surfaces
 - 7.12 Security Posture Summary
- 8.0 Business Logic Documentation
 - 8.1 Module BL-1 — Identity and Session Management
 - 8.2 Module BL-2 — Equipment Master Data
 - 8.3 Module BL-3 — Sensor and Acquisition Configuration
 - 8.4 Module BL-4 — Measurement Ingestion
 - 8.5 Module BL-5 — Signal Processing and Plot Generation
 - 8.6 Module BL-6 — Feature Extraction and Health Evaluation
 - 8.7 Module BL-7 — Baseline Management
 - 8.8 Module BL-8 — Vibration Settings (client-side)
 - 8.9 Module BL-9 — Data Visualisation and Interaction
- 9.0 Complete User Flows
 - 9.1 Flow map
 - 9.2 UF-1 — Login
 - 9.3 UF-2 — Create equipment
 - 9.4 UF-3 — Upload and analyse a capture
 - 9.5 UF-4 — Browse capture history
 - 9.6 UF-5 — Create and use a baseline
 - 9.7 UF-6 — Configure vibration settings
 - 9.8 UF-7 — Search, filter, and paginate the register
 - 9.9 UF-8 — Export a chart
 - 9.10 UF-9 — Logout
 - 9.11 UF-10 — Session expiry during work
 - 9.12 UF-11 — Read-only user journey
- 10.0 Module Documentation
 - 10.1 Module inventory
 - 10.2 M-1 Authentication & Session
 - 10.3 M-2 Equipment Master
 - 10.4 M-3 Measurement & Plot Configuration
 - 10.5 M-4 Measurement Ingestion
 - 10.6 M-5 Signal Processing & Charts
 - 10.7 M-6 Feature Analytics & Health
 - 10.8 M-7 Baseline Management

- 10.9 M-8 Vibration Settings
 - 10.10 M-9 Application Shell & Design System
 - 10.11 Code Walkthrough — Application Entry to Shutdown
 - 10.11.1 Backend
 - 10.11.2 Frontend
 - 10.12 UI Element Catalogue
- 11.0 Configuration Documentation
 - 11.1 Configuration file inventory
 - 11.2 .env — complete reference
 - 11.3 docker-compose.yml
 - 11.4 backend/Dockerfile
 - 11.5 frontend/Dockerfile
 - 11.6 frontend/nginx.conf
 - 11.7 frontend/vite.config.ts
 - 11.8 frontend/tsconfig.json
 - 11.9 frontend/tailwind.config.js
 - 11.10 backend/alembic.ini
 - 11.11 .gitignore
 - 11.12 Configuration precedence
- 12.0 Deployment Guide
 - 12.1 Deployment topology
 - 12.2 Local development (from START.md)
 - 12.3 Full container deployment
 - 12.4 Frontend build process
 - 12.5 Production readiness checklist
 - 12.6 Reverse-proxy example
 - 12.7 Cloud deployment notes
 - 12.8 CI/CD
- 13.0 Testing Documentation
 - 13.1 Current state — stated plainly
 - 13.2 The existing smoke test
 - 13.3 Implicit quality gates
 - 13.4 Manual test plan
 - 13.4.1 Authentication
 - 13.4.2 Authorisation
 - 13.4.3 Equipment
 - 13.4.4 Measurement and analysis
 - 13.4.5 Features and health
 - 13.4.6 Baselines
 - 13.4.7 Settings
 - 13.4.8 Responsive and accessibility
 - 13.5 Recommended automated test suite
- 14.0 Performance & Optimisation
 - 14.1 Implemented optimisations
 - 14.2 Not implemented
 - 14.3 The dominant performance characteristic
 - 14.4 Payload sizes
 - 14.5 Frontend rendering
 - 14.6 Known query weaknesses
 - 14.7 Recommended maintenance jobs
 - 14.8 Scalability profile
 - 14.9 Memory profile of an upload
- 15.0 Troubleshooting & Error Handling
 - 15.1 Error-handling architecture
 - 15.2 Backend error catalogue
 - 15.3 Frontend error surfaces
 - 15.4 Diagnostic runbook
 - 15.5 Log locations
- 16.0 Appendix

- 16.1 Glossary
- 16.2 Abbreviations
- 16.3 API summary table
- 16.4 Database summary table
- 16.5 Backend class / module summary
- 16.6 Frontend component summary
- 16.7 Folder-by-folder file index
- 16.8 Dependency reference
 - 16.8.1 Backend
 - 16.8.2 Frontend — runtime
 - 16.8.3 Frontend — development
 - 16.8.4 Infrastructure images
- 16.9 Constants quick reference
- 16.10 Diagram index
- 17.0 References
 - 17.1 Domain and standards references cited in the code
 - 17.2 Technology documentation
 - 17.3 Internal source references

List of Figures

Figure	Subject
1	1.7 Software Architecture
2	1.8 High-Level Workflow
3	1.8.1 The upload pipeline in detail
4	2.1 Overall Architecture
5	2.2.3 Two development paths to the API
6	2.3 Frontend Architecture
7	2.4 Backend Architecture
8	2.4.1 Application startup sequence
9	2.5 Database Architecture
10	2.6 Authentication Flow
11	2.6.1 Silent refresh with request queueing
12	2.7 API Communication Flow
13	2.8 Data Flow
14	2.10 Response Lifecycle
15	3.3.1 ProtectedRoute decision order
16	3.9.1 Chart architecture
17	4.2.1 custom_openapi() in detail
18	4.9.4 signal_processing.py
19	4.9.6 plot_storage.py
20	4.15 Dependency Injection
21	5.4.3 POST /api/v1/auth/refresh
22	5.5.1 POST /api/v1/equipment/ — create
23	5.7.11 GET /api/v1/measurements/uploads/{upload_id}/features
24	5.8.6 POST /api/v1/baselines/from-upload/{upload_id}
25	6.2 Entity Relationship Diagram
26	6.14 End-to-End Database Flow
27	6.14 End-to-End Database Flow
28	7.2.2 Enforcement points
29	7.6 Role and Permission Management
30	8.4 Module BL-4 — Measurement Ingestion
31	8.6 Module BL-6 — Feature Extraction and Health Evaluation
32	9.1 Flow map
33	9.4 UF-3 — Upload and analyse a capture
34	10.11.1 Backend
35	10.11.2 Frontend
36	11.12 Configuration precedence
37	12.1 Deployment topology
38	12.4 Frontend build process
39	12.8 CI/CD
40	15.1 Error-handling architecture

List of Tables

Table	Subject
1	1.1 Project Name
2	1.3 Business Problem
3	1.5 Feature Inventory
4	1.5.1 Declared-but-not-implemented features
5	1.6.1 Backend runtime (backend/requirements.txt)
6	1.6.2 Frontend runtime (frontend/package.json)
7	1.6.3 Infrastructure
8	1.7.1 Architectural decisions and their rationale
9	1.9.1 File-count summary
10	2.1 Overall Architecture
11	2.2.1 Transport contract
12	2.3.1 Layered module responsibilities
13	2.7.1 React Query cache keys in use
14	2.9 Request Lifecycle
15	2.10 Response Lifecycle
16	3.1 Project Structure — folder by folder
17	3.3 Routing
18	3.4 State Management
19	3.4.1 AuthContext (contexts/AuthContext.tsx)
20	3.5.6 useVibrationSettings()
21	3.6.1 api/client.ts — axios configuration
22	3.6.2 api/auth.ts
23	3.6.3 api/equipment.ts
24	3.6.6 Response normalisation — lib/feature-api-normalize.ts
25	3.7.2 Design tokens
26	3.7.3 Component classes defined in index.css
27	3.8 Responsive Design
28	3.9.4 EchartsGraphViewport
29	3.9.5 Per-plot-type option builders
30	3.9.6 Threshold overlay system (lib/threshold-overlay.ts)
31	3.9.6 Threshold overlay system (lib/threshold-overlay.ts)
32	3.10 Tables
33	3.12 Error Handling (frontend)
34	3.13 Loading States
35	3.14 Pagination, Search, Filtering, Sorting
36	3.16 Environment Variables and Build
37	3.16 Environment Variables and Build
38	3.17.4 Equipment Create / Edit (/equipment/new, /equipment/:id/edit)
39	3.17.5 Vibration Analysis (/analysis)
40	3.18.1 components/ui
41	3.18.2 components/charts
42	3.18.3 components/analysis
43	3.18.4 components/analysis/health
44	3.18.5 components/equipment

Table	Subject
45	3.18.6 components/settings
46	3.18.7 components/brand
47	3.19 Complete lib/ Reference
48	3.20 Complete types/ Reference
49	4.2 Application Startup — app/main.py
50	4.3 Configuration — app/config.py
51	4.4 Database Access — app/database.py
52	4.5 Dependencies — app/dependencies/auth.py
53	4.6.1 Equipment (equipment_masters)
54	4.6.3 Measurement models (models/measurement.py)
55	4.6.4 User models (models/user.py)
56	4.8.1 crud/equipment.py
57	4.9.1 auth_service.py
58	4.9.3 pdf_parser.py
59	4.9.4 signal_processing.py
60	4.9.5 plot_generator.py
61	4.9.8 feature_extraction.py
62	4.9.9 threshold_evaluator.py
63	4.9.10 feature_storage.py
64	4.9.11 acquisition_config.py
65	4.10 Routers — endpoint inventory
66	4.11 Middleware, Filters, and Interceptors
67	4.12 Logging
68	4.13 Caching
69	4.14 Transactions
70	4.17 Exception Handling (backend)
71	4.18 Alembic Migrations
72	4.19 Backend Utility Scripts
73	5.1 Conventions
74	5.2 Complete Endpoint Index (46 endpoints)
75	5.4.1 POST /api/v1/auth/login
76	5.4.1 POST /api/v1/auth/login
77	5.4.1 POST /api/v1/auth/login
78	5.4.3 POST /api/v1/auth/refresh
79	5.4.5 GET /api/v1/auth/me
80	5.5.1 POST /api/v1/equipment/ — create
81	5.5.1 POST /api/v1/equipment/ — create
82	5.5.2 GET /api/v1/equipment/ — list
83	5.5.6 POST /api/v1/equipment/{equipment_id}/image
84	5.5.9 Sensor endpoints
85	5.5.10 GET /api/v1/equipment/{equipment_id}/ai-readiness
86	5.6.1 GET /api/v1/lookups/
87	5.7.1 POST /api/v1/measurements/configure
88	5.7.1 POST /api/v1/measurements/configure
89	5.7.4 Edge acquisition endpoints (26–28)
90	5.7.5 POST /api/v1/measurements/upload

Table	Subject
91	5.7.5 POST /api/v1/measurements/upload
92	5.7.5 POST /api/v1/measurements/upload
93	5.7.6 GET /api/v1/measurements/uploads
94	5.7.8 GET /api/v1/measurements/uploads/{upload_id}/plots
95	5.7.11 GET /api/v1/measurements/uploads/{upload_id}/features
96	5.7.12 GET /api/v1/measurements/uploads/{upload_id}/factor-trends
97	5.7.13 GET /api/v1/measurements/uploads/{upload_id}/features/compare
98	5.8.5 POST /api/v1/baselines/upload
99	5.8.6 POST /api/v1/baselines/from-upload/{upload_id}
100	5.9.1 Which endpoints require write access
101	5.9.2 Validation-rule summary
102	5.9.3 Endpoint → table matrix
103	6.1 Database Identity
104	6.3 Relationship Catalogue
105	6.4.1 equipment_masters
106	6.4.2 sensor_configurations
107	6.4.3 plot_configurations
108	6.4.4 sensor_data_uploads
109	6.4.5 measurement_upload_data
110	6.4.6 plot_results
111	6.4.7 sensor_baselines
112	6.4.8 baseline_plot_results
113	6.4.9 feature_definitions
114	6.4.9 feature_definitions
115	6.4.10 feature_threshold_rules
116	6.4.10 feature_threshold_rules
117	6.4.11 measurement_channel_features
118	6.4.12 measurement_channel_feature_trends
119	6.4.13 baseline_channel_features
120	6.4.14 roles
121	6.4.15 users
122	6.4.16 user_roles
123	6.4.17 refresh_tokens
124	6.5.2 Unique constraints and unique indexes
125	6.5.3 Check constraints
126	6.5.4 Default values
127	6.5.5 Complete index inventory (24 indexes)
128	6.6 Triggers, Views, Stored Procedures, Functions
129	6.7 Normalisation Analysis
130	6.7 Normalisation Analysis
131	6.8 Data-Volume Model
132	6.10 Performance Considerations (database)
133	6.11 Backup and Recovery
134	6.13 Data Dictionary Quick Reference
135	7.1 Authentication Model
136	7.1.2 Refresh-token design

Table	Subject
137	7.2.1 Role hierarchy
138	7.2.3 Client-side gating inventory
139	7.3 Sessions and Cookies
140	7.4 Password Storage
141	7.5 Encryption
142	7.6 Role and Permission Management
143	7.7 Security Filters and Middleware
144	7.10 Rate Limiting and Brute-Force Resistance
145	7.11.2 Cross-site scripting
146	7.11.3 File-upload validation
147	7.11.4 Other validation surfaces
148	7.12 Security Posture Summary
149	8.1 Module BL-1 — Identity and Session Management
150	8.2 Module BL-2 — Equipment Master Data
151	8.3 Module BL-3 — Sensor and Acquisition Configuration
152	8.4 Module BL-4 — Measurement Ingestion
153	8.5 Module BL-5 — Signal Processing and Plot Generation
154	8.5 Module BL-5 — Signal Processing and Plot Generation
155	8.6 Module BL-6 — Feature Extraction and Health Evaluation
156	8.6 Module BL-6 — Feature Extraction and Health Evaluation
157	8.7 Module BL-7 — Baseline Management
158	8.8 Module BL-8 — Vibration Settings (client-side)
159	8.9 Module BL-9 — Data Visualisation and Interaction
160	9.2 UF-1 — Login
161	9.3 UF-2 — Create equipment
162	9.6 UF-5 — Create and use a baseline
163	9.8 UF-7 — Search, filter, and paginate the register
164	9.12 UF-11 — Read-only user journey
165	10.1 Module inventory
166	10.12 UI Element Catalogue
167	11.1 Configuration file inventory
168	11.2 .env — complete reference
169	11.3 docker-compose.yml
170	11.5 frontend/Dockerfile
171	11.7 frontend/vite.config.ts
172	11.8 frontend/tsconfig.json
173	11.9 frontend/tailwind.config.js
174	12.3 Full container deployment
175	12.5 Production readiness checklist
176	12.7 Cloud deployment notes
177	13.1 Current state — stated plainly
178	13.3 Implicit quality gates
179	13.4.1 Authentication
180	13.4.2 Authorisation
181	13.4.3 Equipment
182	13.4.4 Measurement and analysis

Table	Subject
183	13.4.5 Features and health
184	13.4.6 Baselines
185	13.4.7 Settings
186	13.4.8 Responsive and accessibility
187	13.5 Recommended automated test suite
188	13.5 Recommended automated test suite
189	14.1 Implemented optimisations
190	14.2 Not implemented
191	14.4 Payload sizes
192	14.5 Frontend rendering
193	14.6 Known query weaknesses
194	14.8 Scalability profile
195	15.2 Backend error catalogue
196	15.3 Frontend error surfaces
197	15.4 Diagnostic runbook
198	15.5 Log locations
199	16.1 Glossary
200	16.2 Abbreviations
201	16.3 API summary table
202	16.4 Database summary table
203	16.5 Backend class / module summary
204	16.6 Frontend component summary
205	16.7 Folder-by-folder file index
206	16.8.1 Backend
207	16.8.2 Frontend — runtime
208	16.8.3 Frontend — development
209	16.8.4 Infrastructure images
210	16.9 Constants quick reference
211	16.10 Diagram index
212	17.1 Domain and standards references cited in the code
213	17.2 Technology documentation
214	17.3 Internal source references

1.0 Project Overview

1.1 Project Name

SensoVibe — AI Powered Industrial Vibration Intelligence Platform.

The name appears in three places in the code:

Table 1 — 1.1 Project Name

Location	Value
frontend/index.html → <title>	SensoVibe – AI Powered Industrial Vibration Intelligence Platform
backend/app/main.py → FastAPI(title=...)	AI Vibration Intelligence Platform
frontend/src/components/layout/Sidebar.tsx → TAGLINE	AI Powered Vibration Intelligence

The npm package is `vibration-platform-frontend`; the Docker Compose project deploys containers named `vibration_platform_db`, `vibration_platform_pgadmin`, `vibration_platform_backend`, `vibration_platform_frontend`.

1.2 Purpose

The platform is a **condition-monitoring system for rotating industrial machinery**. It performs four things:

- 1. **Asset master data management** — captures a complete "digital twin" record of every machine (plant hierarchy, mechanical specification, rotating components, operating envelope, lubrication history, sensor mounting layout).
- 2. **Vibration measurement ingestion** — accepts raw multi-channel time-series captures as CSV or PDF, parses them into per-channel sample arrays, and stores both the original bytes and parsed arrays in PostgreSQL.
- 3. **Signal processing and diagnostics** — computes five diagnostic plot types (time waveform, circular time waveform, FFT spectrum, envelope spectrum, trend plot) and ten scalar vibration features per channel, with per-segment trend series for each feature.
- 4. **Health evaluation against baselines and thresholds** — compares each capture against a stored reference "baseline" capture and against configurable threshold rules to classify every feature as `normal`, `warning`, `critical`, or `no_baseline`.

1.3 Business Problem

Rotating equipment (pumps, motors, fans, compressors, gearboxes, turbines) fails progressively. The mechanical degradation signature appears in vibration long before functional failure. The business problems the code solves are:

Table 2 — 1.3 Business Problem

Problem	How the code addresses it
Asset context is scattered across CMMS, drawings, and tribal knowledge, so vibration data cannot be interpreted	<code>equipment_masters</code> stores 40 attributes including <code>rated_rpm</code> , <code>bearing_number_de/nde</code> , <code>gear_teeth</code> , <code>motor_pole_count</code> , <code>fan_blades</code> , <code>pump_vanes</code> — the exact inputs required to compute shaft and bearing defect frequencies
Raw sensor exports (CSV/PDF) are unusable without processing	<code>app/services/pdf_parser.py</code> normalises heterogeneous exports into <code>{timestamps, channels: {ch0..chN}}</code>
Recomputing FFTs on every page view is expensive	<code>plot_results</code> / <code>baseline_plot_results</code> cache computed x/y arrays in JSONB, keyed by a <code>config_fingerprint</code> SHA-256 hash of the processing parameters
"Is this vibration level bad?" cannot be answered without a reference	<code>sensor_baselines</code> stores append-only reference captures; <code>feature_threshold_rules</code> stores absolute, range, percent-of-RMS, and percent-of-baseline rules
Machine data must be traceable and auditable	Baselines are never deleted or overwritten — <code>create_baseline</code> only inserts; <code>is_primary</code> is a display flag, not a retention flag
Different plant roles need different capabilities	Three roles (<code>super_admin</code> , <code>admin</code> , <code>user</code>) with read/write separation enforced by a FastAPI dependency and mirrored in the React router

1.4 Objectives

Derived from the code structure and docstrings:

- 1. **O-1 — Single source of asset truth.** One `Equipment` row per machine, with cascade-owned `SensorConfiguration` children.
- 2. **O-2 — Deterministic, reproducible signal processing.** Every stored plot carries the fingerprint of the configuration that produced it (`compute_config_fingerprint`, `ALGORITHM_VERSION = "v1"`).

- 3. **O-3 — Append-only historical record for future machine learning.** `sensor_baselines` is documented in code as "*Historical baseline records — all rows kept (append-only) for RAG / learning.*"
- 4. **O-4 — Edge-device interoperability.** `/api/v1/measurements/acquisition` returns a Sensovibe-compatible acquisition JSON for UDP acquisition scripts, keyed by MAC-style `device_id`.
- 5. **O-5 — Industrial-standard visualisation.** `lib/industrial-viz-standards.ts` cites *Condition Monitoring with Vibration Signals* (Randall/Antoni), *The Scientist and Engineer's Guide to DSP* (Smith), and ISO 10816 as the basis for axis conventions, Nyquist markers, harmonic markers, and alarm-zone colour coding.
- 6. **O-6 — Role-based access control.** Read for all authenticated users; write restricted to `super_admin` and `admin`.

1.5 Feature Inventory

Table 3 — 1.5 Feature Inventory

#	Feature	Frontend entry point	Backend endpoints
F-01	Email/password login with JWT access + opaque refresh tokens	<code>pages/Login.tsx</code>	POST <code>/api/v1/auth/login</code> , <code>/token</code>
F-02	Silent token refresh with request queueing	<code>api/client.ts</code> interceptor	POST <code>/api/v1/auth/refresh</code>
F-03	Logout with server-side refresh-token revocation	<code>contexts/AuthContext.tsx</code>	POST <code>/api/v1/auth/logout</code>
F-04	Session bootstrap from <code>sessionStorage</code>	<code>AuthContext</code> <code>bootstrap()</code>	GET <code>/api/v1/auth/me</code>
F-05	Role-guarded routes and nav items	<code>components/auth/ProtectedRoute.tsx</code> , <code>nav-config.ts</code>	<code>require_write_access</code> dependency
F-06	Equipment register with search, type/criticality filters, pagination	<code>pages/EquipmentMasterList.tsx</code>	GET <code>/api/v1/equipment/</code>
F-07	6-step equipment creation/edit wizard with live completeness scoring	<code>components/equipment/EquipmentForm.tsx</code>	POST / PATCH <code>/api/v1/equipment/</code>
F-08	Equipment image upload / fetch / delete	<code>tabs/BasicDetailsTab.tsx</code>	POST
F-09	Sensor configuration CRUD (nested under equipment)	<code>tabs/SensorsOrientationTab.tsx</code>	<code>/api/v1/equipment/{id}/sensors...</code>
F-10	AI readiness scoring (5 checks → percentage)	<code>AssetHealthPanel.tsx</code> (client-side), backend endpoint	GET <code>/api/v1/equipment/{id}/ai-readiness</code>
F-11	Dropdown lookup catalogue (18 lists)	<code>api/equipment.ts</code> <code>getLookup</code>	GET <code>/api/v1/lookups/</code>
F-12	Plot configuration upsert per sensor	<code>DetailedAnalysisTab.tsx</code>	POST
F-13	Edge acquisition JSON generation	— (machine-facing)	GET <code>/api/v1/measurements/acquisition</code>
F-14	CSV/PDF measurement upload with synchronous parse → plots → features pipeline	<code>pages/VibrationAnalysis.tsx</code>	POST <code>/api/v1/measurements/upload</code>
F-15	Capture timeline browser with date-range filter and day chips	<code>analysis/CaptureTimeline.tsx</code>	GET <code>/api/v1/measurements/uploads</code>
F-16	Five diagnostic plot types with zoom/pan/crosshair/threshold overlays/PNG export/fullscreen	<code>charts/GraphWorkspace.tsx</code> + <code>EchartsDiagnosticChart.tsx</code>	GET <code>/uploads/{id}/plots</code>
F-17	Ten-feature health dashboard with summary cards and category-grouped tables	<code>health/StatusHealthTab.tsx</code>	GET <code>/uploads/{id}/features</code>
F-18	Feature comparison versus baseline with % difference	<code>health/FeatureComparisonSection.tsx</code>	GET <code>/uploads/{id}/features/compare</code>
F-19	Per-feature 32-segment trend cards	<code>health/FeatureTrendCardsSection.tsx</code>	GET <code>/uploads/{id}/factor-trends</code>
F-20	Baseline management: list, search, filter, set-primary, load-for-analysis	<code>baseline/BaselineManagementPanel.tsx</code>	<code>/api/v1/baselines...</code>
F-21	Create baseline from an existing upload	<code>analysis/SaveBaselineModal.tsx</code>	POST <code>/api/v1/baselines/from-upload/{id}</code>
F-22	Direct baseline file upload	— (API only)	POST <code>/api/v1/baselines/upload</code>
F-23	Statistics tab (12 statistical parameters)	<code>workspace/StatisticsTab.tsx</code>	reuses <code>/plots</code>
F-24	Vibration Settings: 8-channel mapping + threshold matrix (localStorage-backed)	<code>settings/vibration/VibrationSettingsModule.tsx</code>	none (client-only)
F-25	Light/dark theme toggle scaffolding	<code>contexts/ThemeContext.tsx</code>	none
F-26	Toast notification system	<code>components/ui/Toast.tsx</code>	none
F-27	Collapsible sidebar + plant selector + user menu	<code>layout/Sidebar.tsx</code> , <code>layout/TopNav.tsx</code>	none

1.5.1 Declared-but-not-implemented features

Documented here because the code shows them explicitly:

Table 4 — 1.5.1 Declared-but-not-implemented features

Item	Evidence
Change Password	<code>pages/ChangePassword.tsx</code> renders the text "Password change API is not yet available. Contact your administrator." No backend endpoint exists.
Operations Dashboard	<code>pages/Dashboard.tsx</code> renders <code><ComingSoon></code> and four KPI tiles whose values are literal <code>"—"</code> .
Platform Settings module	<code>settings/SettingsTabNav.tsx</code> marks the <code>platform</code> tab <code>available: false</code> .
Trend Analysis tab	<code>workspace/TrendAnalysisTab.tsx</code> renders a notice that factor trends moved to Status (Health).
Global search box	<code>layout/TopNav.tsx</code> renders an input with no submit handler.
Notification bell	<code>layout/TopNav.tsx</code> shows a hard-coded count of <code>3</code> .
Plant selector	<code>layout/nav-config.ts</code> <code>PLANTS</code> is a hard-coded array; selection updates <code>LayoutContext</code> only and filters nothing.
<code>useHistoricalTrendData</code> hook	Fully implemented (<code>hooks/useHistoricalTrendData.ts</code>) but not imported by any component.
<code>AssetHealthPanel</code> , <code>CompletenessEngine</code> , <code>AssetIntelligencePanel</code> , <code>StatusHealthSection</code> , <code>BaselineSelectionPanel</code> , <code>IndustrialEmptyState</code> , <code>CriticalityIndicator</code> , <code>SectionCard</code> -based review helpers	Present and functional but not currently wired into the active render tree (verified by import graph).

1.6 Technologies Used

1.6.1 Backend runtime (`backend/requirements.txt`)

Table 5 — 1.6.1 Backend runtime (backend/requirements.txt)

Package	Version	Role in this project
<code>fastapi</code>	0.115.0	HTTP framework, dependency injection, OpenAPI generation
<code>uvicorn[standard]</code>	0.30.6	ASGI server (<code>uvicorn app.main:app</code>)
<code>sqlalchemy</code>	2.0.35	ORM; declarative models, session management
<code>alembic</code>	1.13.3	Schema migrations (11 revisions)
<code>psycopg2-binary</code>	2.9.9	PostgreSQL driver
<code>python-multipart</code>	0.0.12	<code>multipart/form-data</code> parsing for file uploads
<code>python-dotenv</code>	1.0.1	<code>.env</code> loading inside <code>alembic/env.py</code>
<code>pillow</code>	10.4.0	Installed for image handling (equipment images are written with plain <code>openC</code>) ; Pillow is not imported anywhere in <code>app/</code>)
<code>pydantic</code>	2.9.2	Request/response schemas, validators
<code>pydantic-settings</code>	2.5.2	<code>Settings(BaseSettings)</code> env binding
<code>aiofiles</code>	24.1.0	Declared; not imported in <code>app/</code>
<code>python-jose[cryptography]</code>	3.3.0	JWT encode/decode (HS256)
<code>passlib[bcrypt]</code>	1.7.4	Password hashing context
<code>bcrypt</code>	4.0.1	bcrypt backend for passlib
<code>email-validator</code>	2.2.0	Backs Pydantic <code>EmailStr</code>
<code>numpy</code>	1.26.4	Array maths for all DSP
<code>scipy</code>	1.13.1	<code>scipy.fft.fft</code> , <code>fftfreq</code> , <code>scipy.signal.hilbert</code>
<code>pdfplumber</code>	0.11.4	PDF table/text extraction

1.6.2 Frontend runtime (`frontend/package.json`)

Table 6 — 1.6.2 Frontend runtime (frontend/package.json)

Package	Version	Role in this project
react / react-dom	^18.3.1	UI runtime
react-router-dom	^6.27.0	Routing, nested layout routes, <code>Navigate</code> guards
@tanstack/react-query	^5.59.20	Server state: <code>useQuery</code> , <code>useQueries</code> , <code>useMutation</code> , cache invalidation
axios	^1.7.7	HTTP client + request/response interceptors
echarts	^6.1.0	Charting engine
echarts-for-react	^3.0.6	React wrapper exposing the ECharts instance
react-hook-form	^7.53.2	Equipment wizard form state, <code>useFieldArray</code> for sensors
@hookform/resolvers	^3.9.0	Bridges Zod to react-hook-form
zod	^3.23.8	<code>equipmentSchema</code> , <code>sensorSchema</code> validation
framer-motion	^12.40.0	Card entrance animations, sidebar width transition, toast transitions
lucide-react	^0.454.0	Icon set (all icons in the app)
tailwindcss	^3.4.14	Utility-first styling
tailwindcss-animate	^1.0.7	Tailwind animation plugin
clsx + tailwind-merge	^2.1.1 / ^2.5.4	<code>cn()</code> class merge helper
class-variance-authority	^0.7.0	Declared; not imported by any source file
date-fns	^4.1.0	<code>format</code> , <code>parseISO</code> , <code>subDays</code> for capture timeline and date ranges
@radix-ui/* (dialog, dropdown-menu, label, popover, select, separator, slot, toast)	^1.x/^2.x	Declared; no Radix import exists in <code>src/</code> — modals/menus are hand-rolled
vite	^5.4.10	Dev server + bundler
@vitejs/plugin-react	^4.3.3	React fast refresh + JSX transform
typescript	^5.6.3	Type checking (<code>tsc</code> && <code>vite build</code>)
postcss , autoprefixer	^8.4.47 / ^10.4.20	CSS pipeline
@types/node , @types/react , @types/react-dom	—	Type definitions

1.6.3 Infrastructure

Table 7 — 1.6.3 Infrastructure

Component	Version / image	Notes
PostgreSQL	postgres:16	Host port 5433 → container 5432
pgAdmin 4	dpage/pgadmin4:latest	Host port 5050
Backend image	python:3.11-slim	Installs <code>gcc</code> , <code>libjpeg-dev</code> , <code>zlib1g-dev</code>
Frontend image	node:20-alpine (build) → nginx:alpine (preview)	Multi-stage: <code>base</code> → <code>dev</code> / <code>build</code> → <code>preview</code>
Web server (prod)	nginx	<code>try_files \$uri \$uri/ /index.html</code> SPA fallback on port 4173

1.7 Software Architecture

The system is a **three-tier layered architecture** with an optional fourth edge tier.

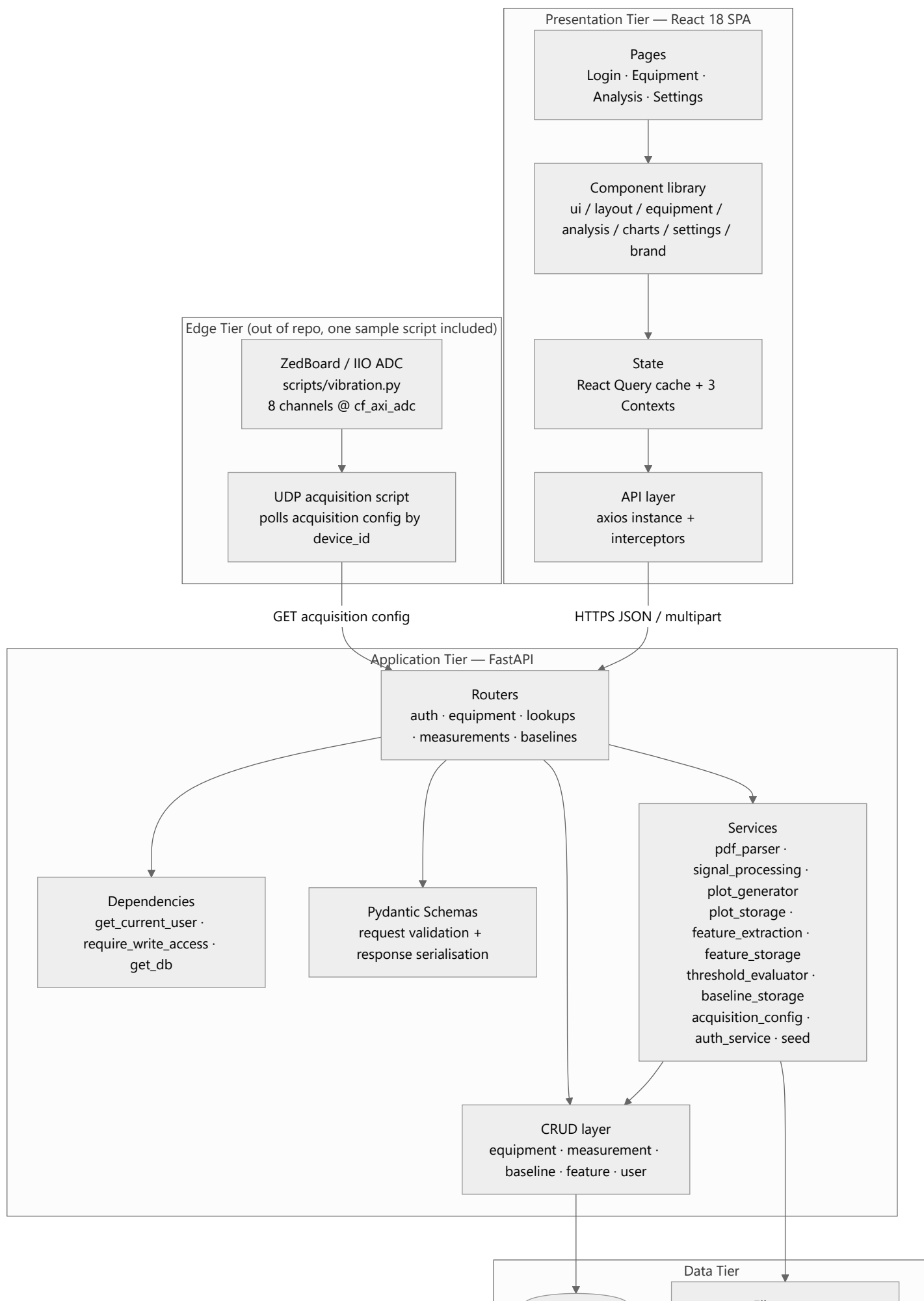




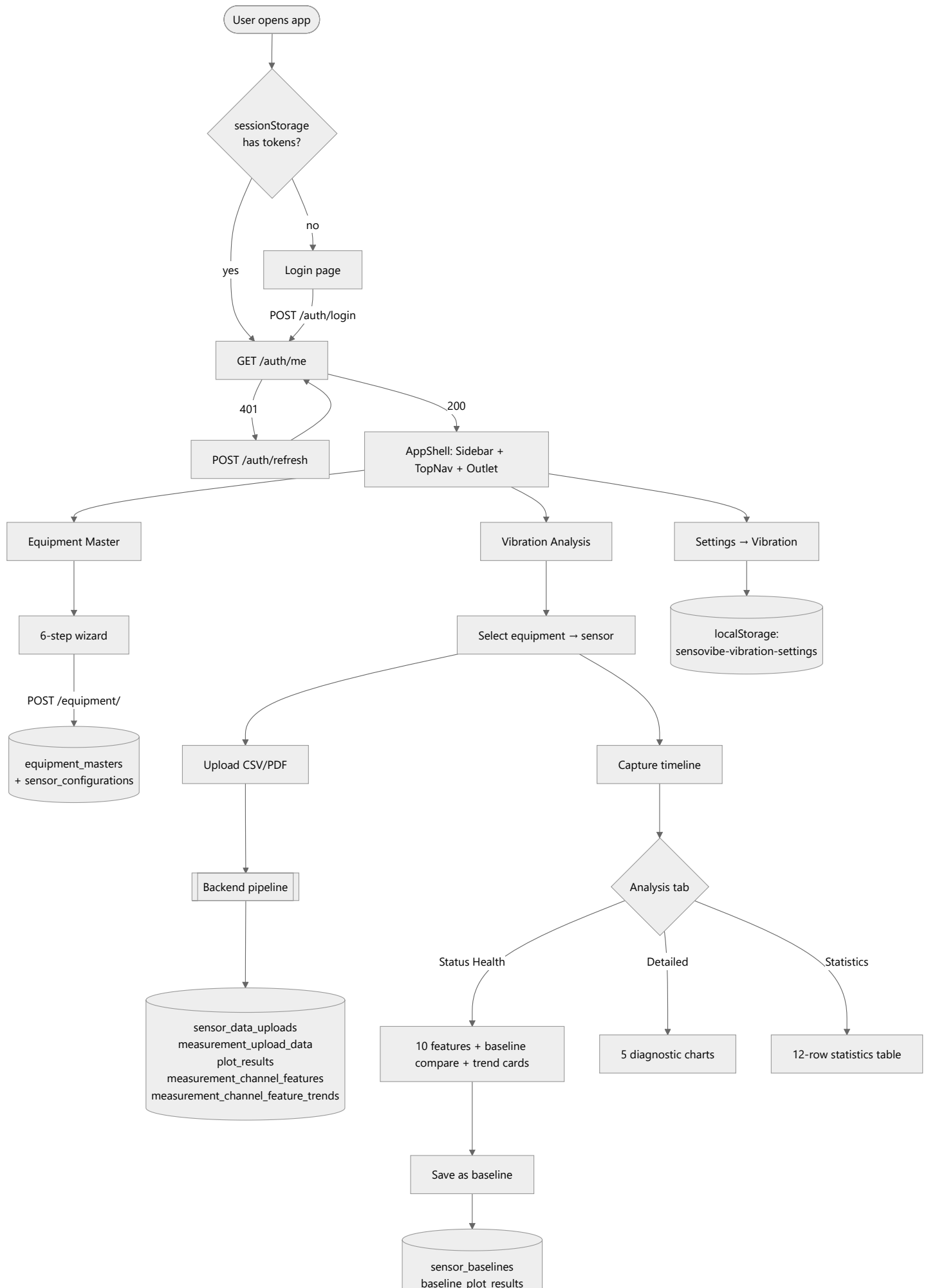
Figure 1 — 1.7 Software Architecture

1.7.1 Architectural decisions and their rationale

Table 8 — 1.7.1 Architectural decisions and their rationale

Decision	Where	Why the code does this
Router → Service → CRUD → ORM rather than fat controllers	app/routers/*	Routers only handle HTTP concerns (status codes, <code>HTTPException</code>); business rules live in <code>services/</code> ; SQL lives in <code>crud/</code>
Pydantic schemas separate from ORM models	app/schemas/ vs app/models/	Enables <code>EquipmentUpdate</code> partial semantics (<code>exclude_unset=True</code>) and prevents accidental exposure of internal columns such as <code>file_content</code>
Computed plots persisted as JSONB, not recomputed	plot_results, baseline_plot_results	FFT/Hilbert on 100k+ samples is expensive; caching is keyed by <code>config_fingerprint</code> so a config change automatically invalidates
Fingerprint instead of TTL cache invalidation	services/plot_storage.py::compute_config_fingerprint	Deterministic: the hash covers <code>algorithm_version</code> , <code>sampling_rate_hz</code> , <code>fft_lines</code> , <code>frequency_max_hz</code> , <code>data_type</code> , sorted <code>enabled_plots</code>
Original file bytes stored in the DB (<code>LargeBinary</code>) as well as on disk	measurement_upload_data.file_content, sensor_baselines.file_content	Guarantees a baseline can be reproduced even if the filesystem volume is lost; <code>create_baseline_from_upload</code> reads bytes from the DB, not disk
Append-only baselines	crud/baseline.py::create_baseline	Only <code>INSERT</code> ; <code>set_baseline_primary</code> merely flips a boolean. No delete path exists in the API
Opaque refresh tokens hashed with SHA-256, JWT only for access	services/auth_service.py	Refresh tokens are revocable server-side (<code>revoked_at</code>); access tokens stay stateless and short-lived (30 min default)
Two axios instances	api/client.ts	<code>authClient</code> has no interceptors, which structurally prevents an infinite refresh loop when <code>/auth/refresh</code> itself returns 401
Frontend-generated time axis for waveforms	lib/waveform-time-axis.ts	Sensor exports frequently place a Unix-epoch batch ID in the timestamp column; the frontend regenerates <code>t[i] = i/fs × 1000</code> ms for display while leaving amplitudes untouched
display:none tab panels instead of unmounting	EquipmentForm.tsx, VibrationAnalysis.tsx	Keeps uncontrolled inputs and chart instances alive so values and zoom state survive tab switches
Client-only Vibration Settings	hooks/useVibrationSettings.ts	Persisted to <code>localStorage</code> under <code>sensovibe-vibration-settings</code> ; no server endpoint exists for it yet

1.8 High-Level Workflow



baseline_channel_features

Figure 2 — 1.8 High-Level Workflow

1.8.1 The upload pipeline in detail

POST /api/v1/measurements/upload executes eight sequential stages inside one request (routers/measurements.py:184–260):

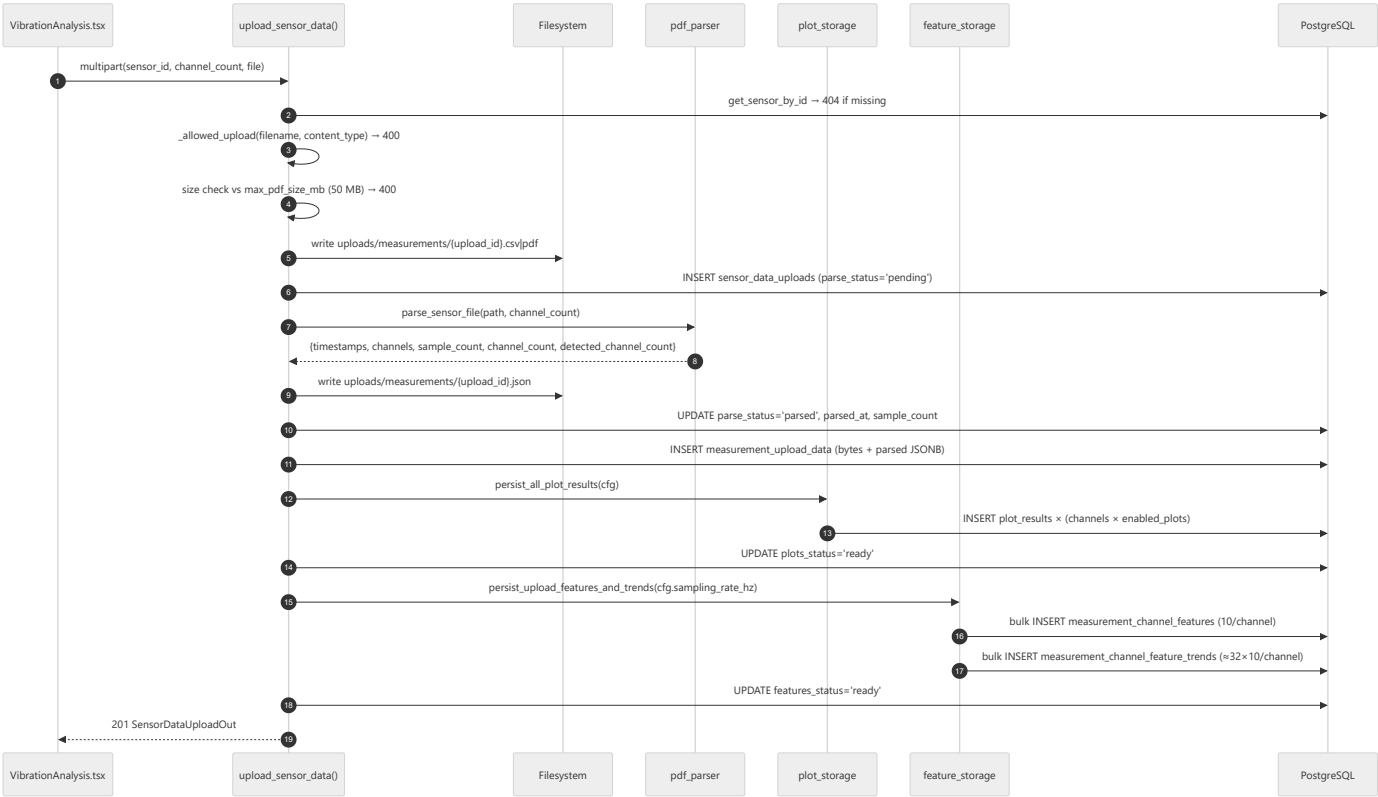


Figure 3 — 1.8.1 The upload pipeline in detail

Failure semantics. Plot failure and feature failure are caught independently (mark_upload_plots_failed , mark_upload_features_failed) and do not abort the request — the upload still returns 201 with the failure recorded on the row. Parse failure is fatal: mark_upload_failed is called and the endpoint raises 422 .

1.9 Complete Folder Structure

```

VibrationMonitoring/
├── .env                                # Single env file consumed by compose, backend, and Vite
├── .gitignore                          # Excludes .env, __pycache__, node_modules, dist, uploads
├── START.md                           # 4-step local bring-up guide
├── docker-compose.yml                 # postgres + pgadmin + backend + frontend
├── package-lock.json                 # Empty root lockfile (no root package.json)
├──
├── backend/
│   ├── .dockerignore
│   ├── Dockerfile                    # python:3.11-slim, runs alembic upgrade head then uvicorn
│   ├── alembic.ini                   # script_location=alembic; url overridden in env.py
│   ├── requirements.txt               # 18 pinned dependencies
│   └── setup_and_run.bat              # Windows venv bootstrap + migrate + serve
├── alembic/
│   ├── env.py                       # Loads ../../.env, injects DATABASE_URL, imports app.models
│   ├── script.py.mako                # Migration template
│   └── versions/
│       ├── 001_initial_schema.py     # equipment_masters, sensor_configurations
│       ├── 002_machine_id_nullable.py # machine_id → NULL allowed
│       ├── 003_measurement_tables.py  # plot_configurations, sensor_data_uploads
│       ├── 004_auth_tables.py         # roles, users, user_roles, refresh_tokens (+5 seed roles)
│       ├── 005_sensor_device_id.py    # sensor_configurations.device_id (unique)
│       ├── 006_plot_results.py        # plot_results + 3 plots_* columns on uploads
│       ├── 007_upload_data_and_baselines.py # measurement_upload_data, sensor_baselines, baseline_plot_results
│       ├── 008_add_user_role_column.py # users.role + CHECK constraint + role backfill
│       ├── 009_upload_history_fields.py # original_filename, source, (sensor_id, created_at) index
│       ├── 010_channel_features_and_trends.py # feature_definitions, feature_threshold_rules,
│       │                                       # measurement_channel_features, baseline_channel_features
│       │                                       # + 3 features_* columns + 10 definition & 10 rule seeds
│       └── 011_feature_trends_table.py # measurement_channel_feature_trends
├── app/
│   ├── __init__.py                  # empty package marker
│   ├── config.py                    # Settings(BaseSettings) - 16 settings + effective_jwt_secret
│   ├── database.py                  # engine, SessionLocal, Base, get_db() generator
│   └── main.py                      # FastAPI app, lifespan seeding, CORS, custom OpenAPI, /health
├── dependencies/
│   ├── __init__.py
│   └── auth.py                     # HTTPBearer, get_current_user, require_write_access, WRITE_ROLES
├── models/
│   ├── __init__.py                  # SQLAlchemy declarative models
│   ├── equipment.py                 # Re-exports all models so Alembic autogenerate sees them
│   ├── sensor.py                    # Equipment (equipment_masters)
│   ├── measurement.py               # SensorConfiguration
│   │                               # 9 models: PlotConfiguration, SensorDataUpload, PlotResult,
│   │                               # MeasurementUploadData, SensorBaseline, FeatureDefinition,
│   │                               # FeatureThresholdRule, MeasurementChannelFeature,
│   │                               # MeasurementChannelFeatureTrend, BaselineChannelFeature,
│   │                               # BaselinePlotResult
│   └── user.py                      # Role, User, UserRole, RefreshToken
├── schemas/
│   ├── __init__.py                  # Pydantic v2 models
│   ├── acquisition.py               # Re-exports equipment schemas
│   ├── auth.py                     # EdgeAcquisitionConfigOut + 2 nested models
│   ├── baseline.py                  # LoginRequest, TokenResponse, RefreshRequest, LogoutRequest, UserMeResponse
│   ├── equipment.py                 # BaselineOut, BaselineListOut, BaselineCreateFromUpload, BaselineSetPrimary
│   ├── feature.py                   # Equipment* + SensorConfig* + AIReadinessOut + PaginatedEquipment
│   └── measurement.py               # 8 feature/compare/trend response models
│                                       # PLOT_TYPES, PlotConfig*, SensorDataUploadOut, PlotSeriesOut, AllPlotsOut
├── crud/
│   ├── __init__.py                  # Data-access functions (no HTTP awareness)
│   ├── baseline.py                  # Re-exports equipment CRUD as app.crud.*
│   ├── equipment.py                 # upload-data + baseline + baseline-plot queries
│   ├── feature.py                   # equipment + sensor CRUD + compute_ai_readiness
│   ├── measurement.py               # threshold rules, definitions, feature/trend reads & deletes
│   └── user.py                      # plot config, upload lifecycle, plot result queries, config dicts
│                                       # user/role/refresh-token queries, primary_role resolution
├── routers/
│   └── __init__.py                  # HTTP layer

```

auth.py	# 5 endpoints
baselines.py	# 8 endpoints
equipment.py	# 13 endpoints
lookups.py	# 2 endpoints + LOOKUPS dictionary (18 lists)
measurements.py	# 14 endpoints
services/	# Business logic
__init__.py	
acquisition_config.py	# Edge JSON builder + acquisition formula maths
auth_service.py	# bcrypt hashing, JWT issue/decode, refresh lifecycle
baseline_storage.py	# Baseline plot persistence + row→schema mapping
feature_extraction.py	# 10 scalar features + 32-segment trends (numpy/scipy)
feature_storage.py	# Threshold evaluation, persistence, baseline copy, summaries
pdf_parser.py	# CSV/PDF → {timestamps, channels}
plot_generator.py	# PLOT_COMPUTERS registry, channel resolution, JSON I/O
plot_storage.py	# Fingerprinting, persistence, cache-or-compute reads
seed.py	# Idempotent super_admin/admin/user seeding at startup
signal_processing.py	# 5 DSP routines + timestamp heuristics
threshold_evaluator.py	# 5 rule types → normal/warning/critical/no_baseline
scripts/	
create_sample_sensor_pdf.py	# Generates a 512-sample 2-channel test PDF (needs reportlab)
test_auth_phase1.py	# TestClient smoke test of the full auth cycle
frontend/	
.dockerignore	
Dockerfile	# base → dev build → preview(nginx)
index.html	# SPA shell, <html class="light">, #root
nginx.conf	# listen 4173, SPA fallback
package.json	# 3 scripts, 25 deps, 9 devDeps
package-lock.json	
postcss.config.js	# tailwindcss + autoprefixer
setup_and_run.bat	# npm install && npm run dev
tailwind.config.js	# darkMode:class, full design-token theme extension
tsconfig.json	# strict, bundler resolution, @/* → ./src/*
tsconfig.node.json	# composite config for vite.config.ts
vite.config.ts	# react plugin, @ alias, port 5173, /api proxy → :8000
public/	
favicon.svg	
src/	
main.tsx	# ReactDOM root, QueryClient, BrowserRouter
App.tsx	# Provider stack + 8 routes
index.css	# 1015 lines: tokens, base, 100+ component classes, keyframes
vite-env.d.ts	# Vite client types + image module declarations
api/	# 5 files – every network call in the app
components/	
analysis/	# 12 root + charts/(4) + baseline/(3) + health/(13) + workspace/(7)
auth/	# ProtectedRoute
brand/	# 5 decorative SVG/animation backdrops
charts/	# 6 reusable chart-shell components + barrel index
equipment/	# 9 root + industrial/(3) + tabs/(6)
layout/	# AppShell, Sidebar, TopNav, PageHero, ComingSoon, nav-config
settings/	# 3 root + vibration/(7)
ui/	# Button, FormField, GlassCard, MultiSelect, SectionCard, Toast # + CARD_HOVER.md, CARD_SIZING.md design contracts
contexts/	# AuthContext, LayoutContext, ThemeContext
hooks/	# 6 custom hooks
images/	# 6 assets + typed barrel index.ts
lib/	# 30 pure-function modules (no JSX except baseline-utils.tsx)
pages/	# 9 page components
types/	# 9 type/constant modules
scripts/	
vibration.py	# Standalone libiiio 8-channel ADC reader for a ZedBoard at 192.168.1.34

1.9.1 File-count summary

Table 9 — 1.9.1 File-count summary

Area	Files	Notes
Backend Python (<code>app/</code>)	41	5 routers, 12 services, 6 CRUD, 5 model modules, 8 schema modules
Alembic	13	<code>env.py</code> , template, 11 revisions
Backend scripts	2	Both standalone utilities
Frontend source	~185	<code>.tsx / .ts</code> under <code>src/</code>
Frontend assets	7	5 SVG, 1 JPEG, 1 public favicon
Configuration	14	Root + backend + frontend config files

2.0 System Architecture

2.1 Overall Architecture

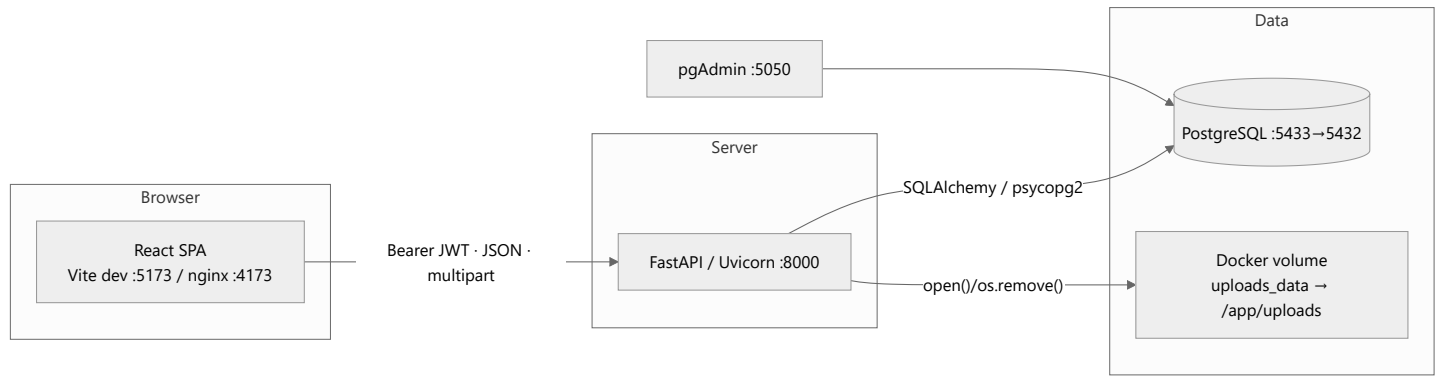


Figure 4 — 2.1 Overall Architecture

Deployment topology (`docker-compose.yml`):

Table 10 — 2.1 Overall Architecture

Service	Container	Host port	Depends on	Volume
postgres	vibration_platform_db	5433	—	postgres_data:/var/lib/postgresql/data
pgadmin	vibration_platform_pgadmin	5050	postgres	pgadmin_data:/var/lib/pgadmin
backend	vibration_platform_backend	8000	postgres (healthcheck service_healthy)	uploads_data:/app/uploads
frontend	vibration_platform_frontend	4173	backend	—

The Postgres healthcheck runs `pg_isready -U $POSTGRES_USER -d $POSTGRES_DB` every 10 s (5 s timeout, 5 retries), and the backend will not start until it passes. This matters because the backend's `lifespan` hook writes seed users to the database on the very first request cycle.

2.2 Client–Server Communication

2.2.1 Transport contract

Table 11 — 2.2.1 Transport contract

Aspect	Value	Defined in
Base URL	<code>import.meta.env.VITE_API_BASE_URL</code> else <code>http://localhost:8000</code>	<code>api/client.ts:6</code>
Default content type	<code>application/json</code>	<code>axios.create</code> headers
Upload content type	<code>multipart/form-data</code> (set per-request)	<code>uploadSensorPdf</code> , <code>uploadEquipmentImage</code>
Auth header	<code>Authorization: Bearer <access_token></code>	request interceptor
Credentials	<code>allow_credentials=True</code> on the server; the client does not set <code>withCredentials</code> — tokens travel in the header, not cookies	<code>main.py</code> , <code>client.ts</code>
Long-running reads	<code>timeout: 120_000</code> ms on <code>/features</code> and <code>/factor-trends</code>	<code>api/measurements.ts</code>

2.2.2 CORS

`main.py` registers `CORSMiddleware` with an explicit six-entry origin allow-list:

```
http://localhost:5173    http://127.0.0.1:5173    (Vite dev)
http://localhost:4173    http://127.0.0.1:4173    (Vite/nginx preview)
http://localhost:3000    http://127.0.0.1:3000    (reserved)
```

with `allow_methods=["*"]`, `allow_headers=["*"]`, `allow_credentials=True`.

2.2.3 Two development paths to the API

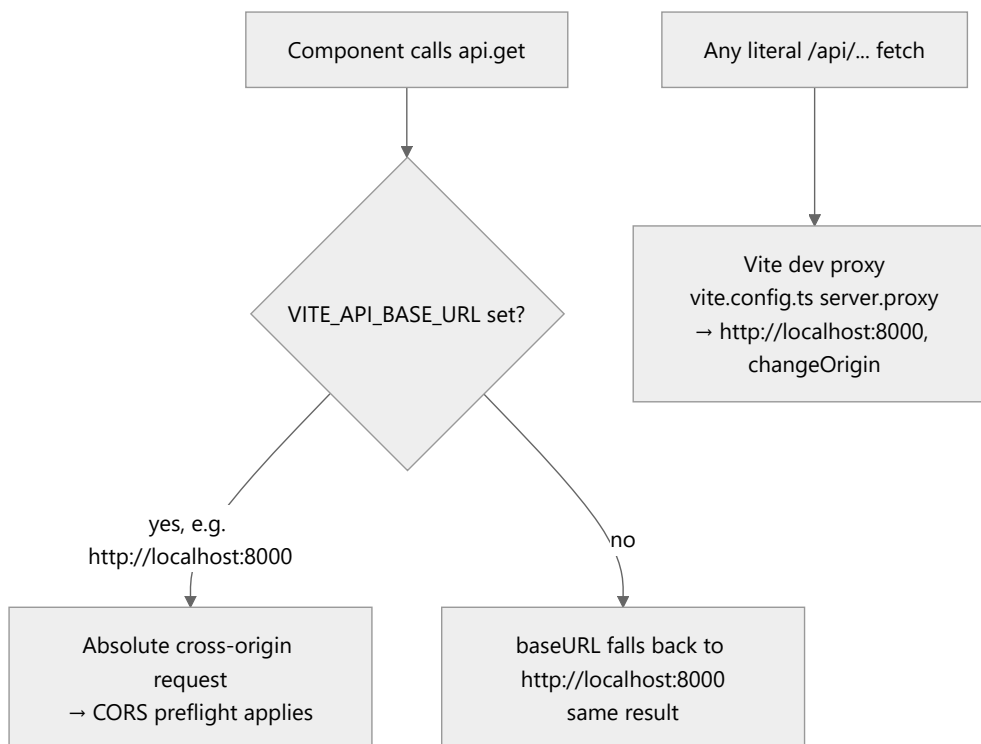


Figure 5 — 2.2.3 Two development paths to the API In practice every call in `src/api/*` uses the axios instance with an absolute `baseUrl`, so the Vite `/api` proxy is a convenience path that the current code does not exercise.

2.3 Frontend Architecture



Figure 6 — 2.3 Frontend Architecture

Provider ordering is deliberate: `AuthProvider` sits *inside* `ToastProvider` because `AuthContext` calls `useToast()` to raise the "Session expired" toast, and inside `BrowserRouter` because it calls `useNavigate()` on logout.

2.3.1 Layered module responsibilities

Table 12 — 2.3.1 Layered module responsibilities

Layer	Directory	Rule observed by the code
Pages	<code>src/pages</code>	Own routing-level state and orchestrate queries/mutations; contain no DSP or formatting maths
Feature components	<code>src/components/<feature></code>	Presentational + local UI state; receive data via props
Shared chart shell	<code>src/components/charts</code>	Chart chrome (toolbar, fullscreen, statistics slot) with a render-prop child; chart-library-agnostic at the shell level
Hooks	<code>src/hooks</code>	Compose React Query calls and derive view models
Lib	<code>src/lib</code>	Pure functions only — maths, option builders, formatters, tokens
Types	<code>src/types</code>	Interfaces plus the constant catalogues that drive UI (<code>PLOT_TYPES</code> , <code>ANALYSIS_TABS</code> , <code>THRESHOLD_PARAMETERS</code>)
API	<code>src/api</code>	The only place <code>axios</code> is referenced (except <code>Login.tsx</code> , which imports <code>axios</code> solely for <code>axios.isAxiosError</code>)

2.4 Backend Architecture

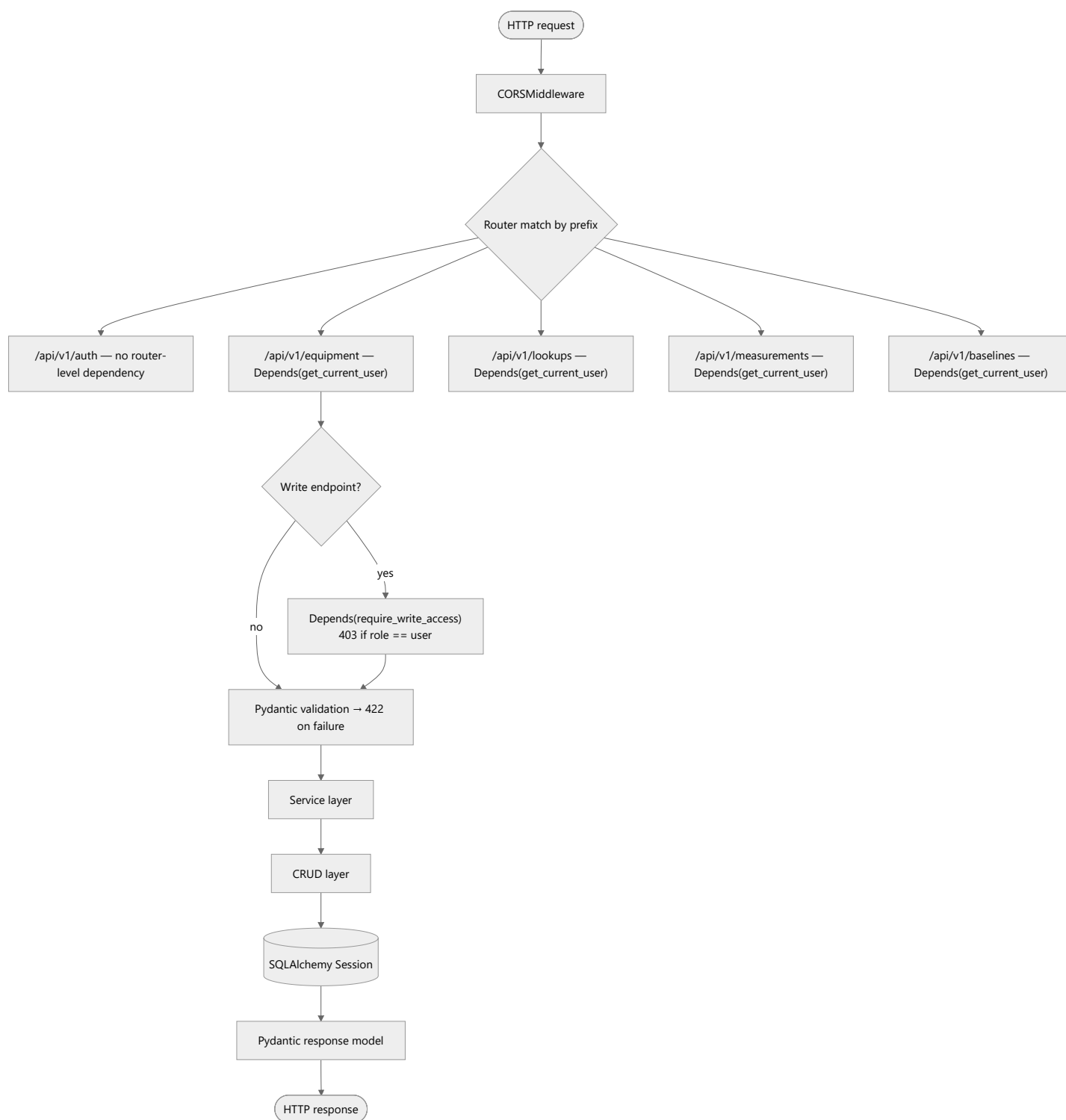


Figure 7 — 2.4 Backend Architecture

2.4.1 Application startup sequence

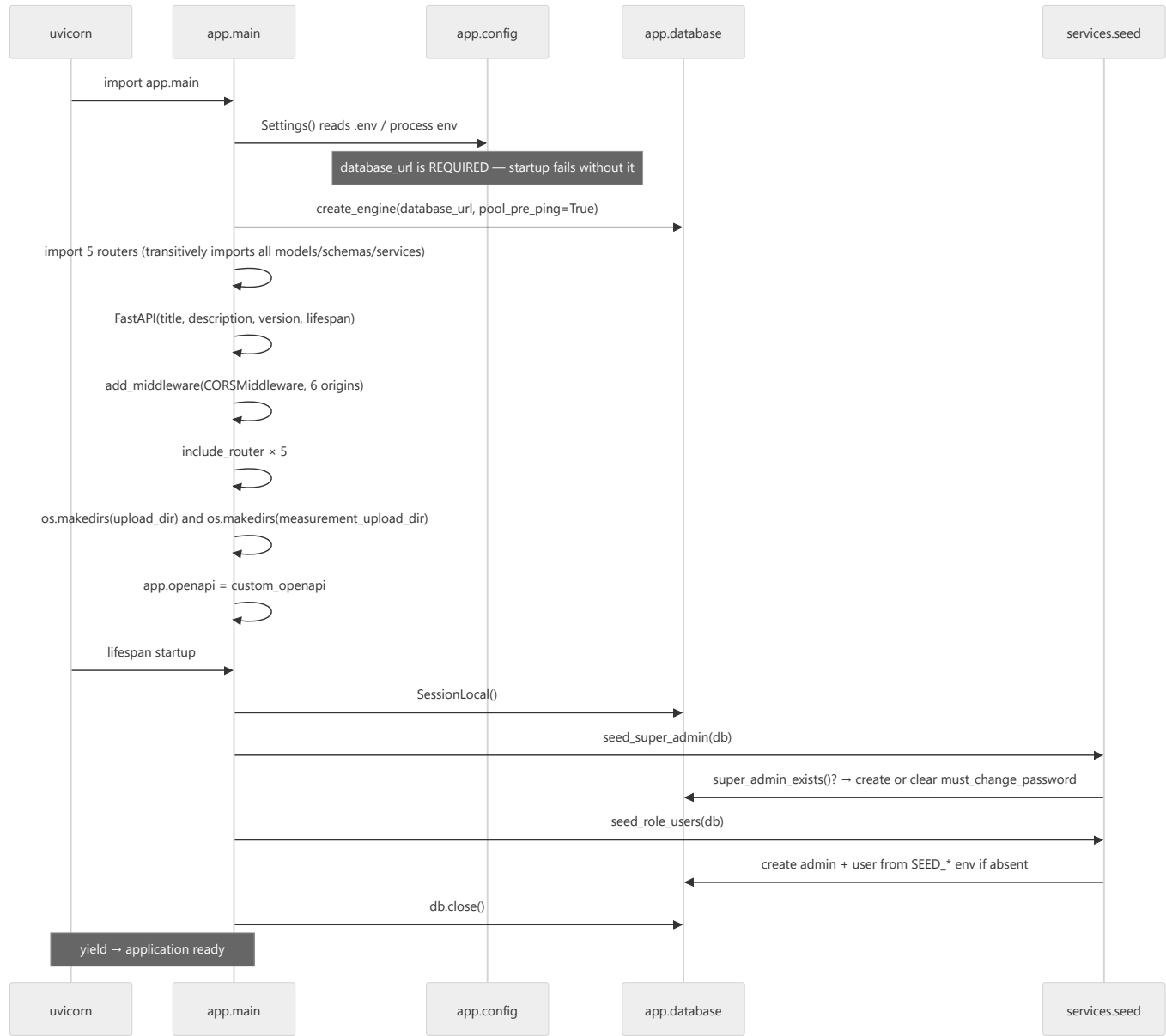


Figure 8 — 2.4.1 Application startup sequence

`os.makedirs(..., exist_ok=True)` runs at **import time**, not inside `lifespan`, so the upload directories exist before the first request even if `lifespan` were skipped.

2.5 Database Architecture

Seventeen tables in four functional clusters:

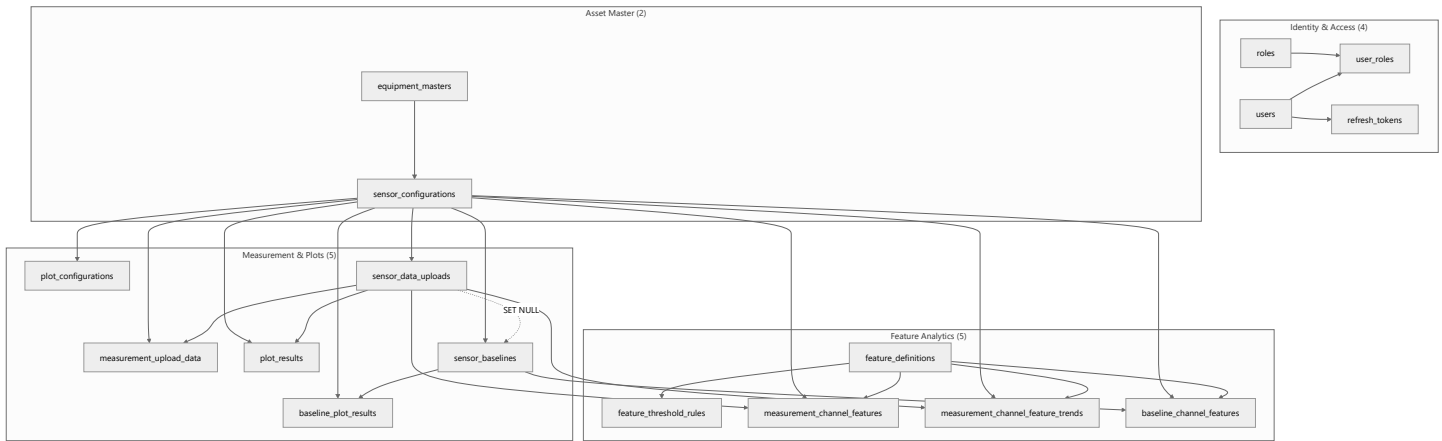


Figure 9 — 2.5 Database Architecture

Cascade policy. Every foreign key that points at `sensor_configurations`, `sensor_data_uploads`, `sensor_baselines`, `users`, or `roles` uses `ON DELETE CASCADE`, with a single exception: `sensor_baselines.source_upload_id` uses `ON DELETE SET NULL` so that deleting an upload never destroys the baseline derived from it.

Full column-level documentation is in **Section 6.0**.

2.6 Authentication Flow

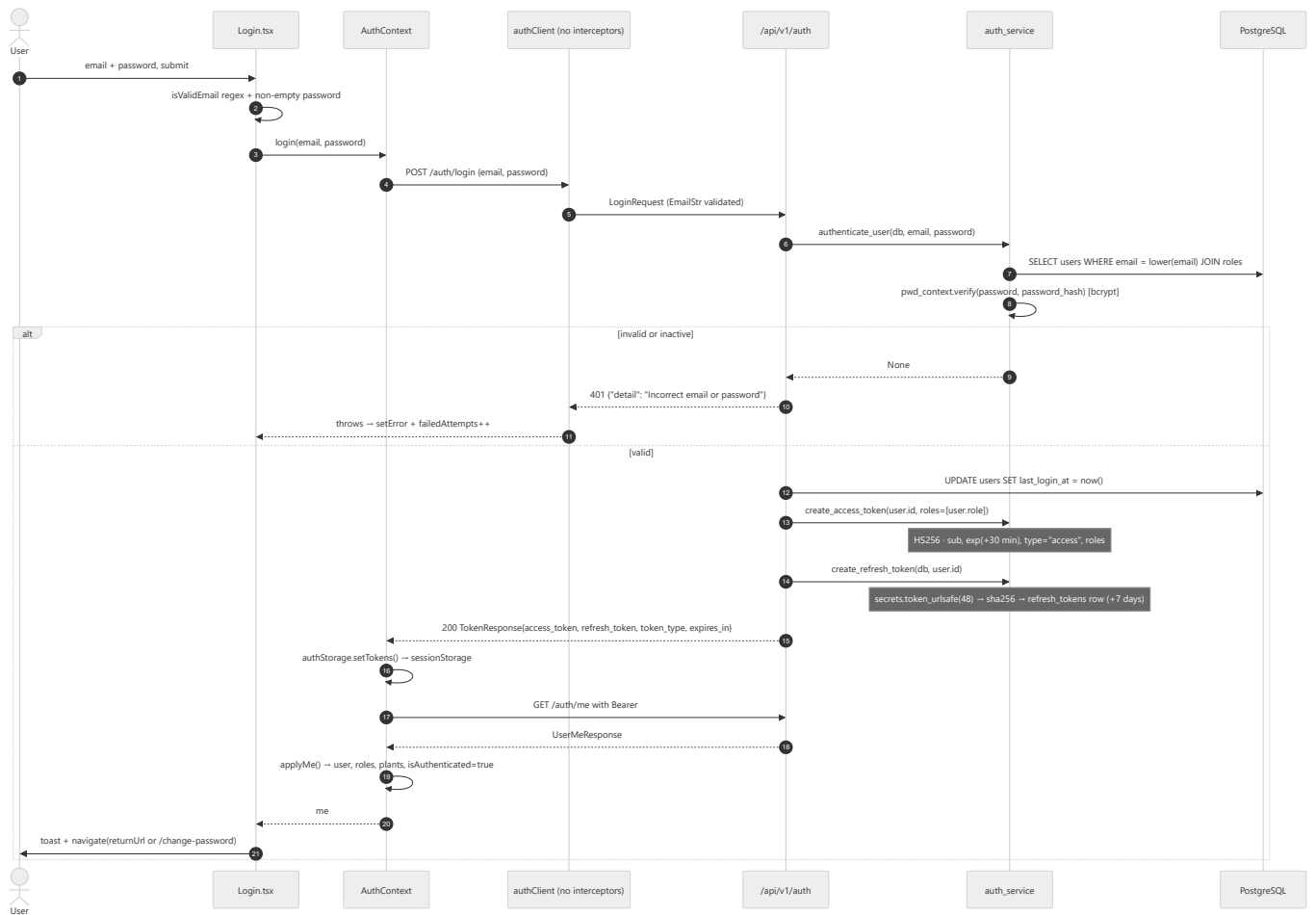


Figure 10 — 2.6 Authentication Flow

2.6.1 Silent refresh with request queueing

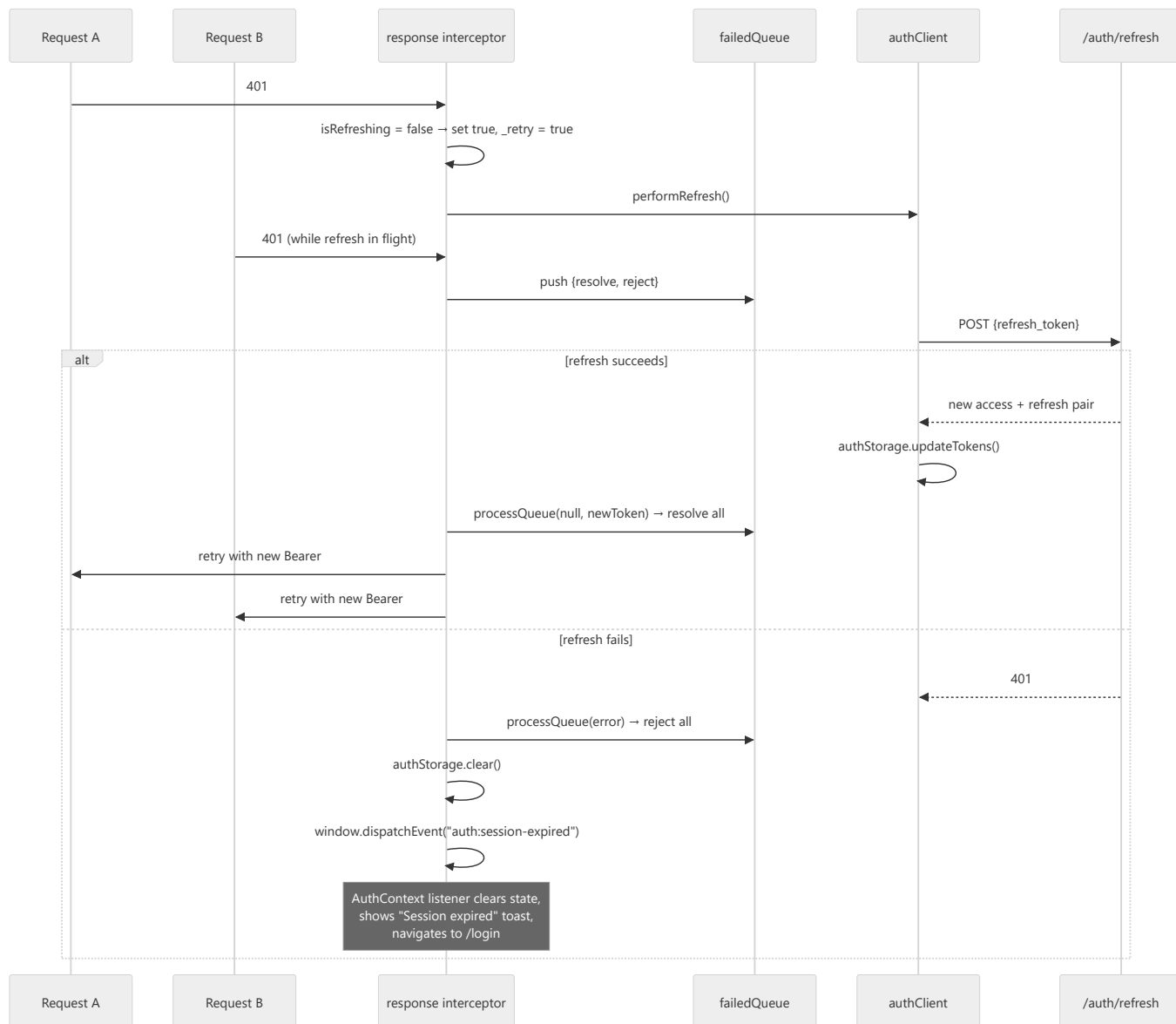


Figure 11 — 2.6.1 Silent refresh with request queueing

Guards that make this safe:

- `shouldSkipRefresh(url)` skips `/auth/login`, `/auth/logout`, `/auth/refresh`.
- `originalRequest._retry` ensures each request is retried at most once.
- `authClient` has **no** interceptors, so the refresh call itself can never recurse.

2.6.2 Refresh-token rotation

`POST /auth/refresh` (`routers/auth.py:66-75`) does the following in order:

1. `validate_refresh_token` — hash lookup, `revoked_at` is `None`, not expired, `user.is_active`.
2. `revoke_refresh_token(record)` — sets `revoked_at = now()` on the **presented** token.
3. `_issue_tokens(db, record.user)` — issues a brand-new access token *and* a brand-new refresh token row.

This is full rotation: a refresh token is single-use.

2.7 API Communication Flow

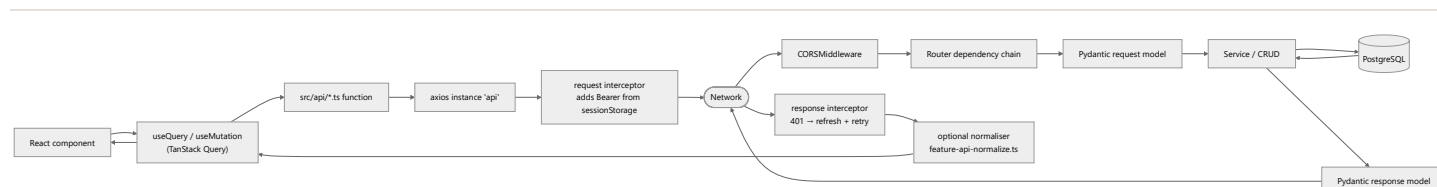


Figure 12 — 2.7 API Communication Flow

2.7.1 React Query cache keys in use

Table 13 — 2.7.1 React Query cache keys in use

Key pattern	Producer	Invalidated by
<code>["equipment", page, filterType, filterCriticality]</code>	EquipmentMasterList	<code>deleteMutation → invalidateQueries(["equipment"])</code>
<code>["equipment", id]</code>	EditEquipmentPage	—
<code>["equipment", "count"]</code>	AssetIntelligencePanel	—
<code>["equipment-list-analysis"]</code>	VibrationAnalysis	—
<code>["equipment-detail", equipmentId]</code>	VibrationAnalysis	—
<code>["plot-config", sensorId]</code>	VibrationAnalysis	<code>saveConfigMutation</code>
<code>["sensor-uploads", sensorId]</code>	VibrationAnalysis	<code>uploadMutation</code>
<code>["sensor-uploads-timeline", sensorId, fromDate, toDate, refreshKey]</code>	CaptureTimelinePanel	<code>uploadMutation + refreshKey bump</code>
<code>["plots", plotSource, uploadId, baselineId, channel]</code>	VibrationAnalysis	upload/select/load handlers
<code>["primary-baseline", sensorId], ["baseline-list", sensorId]</code>	VibrationAnalysis	<code>saveBaselineMutation, setPrimaryMutation</code>
<code>["upload-features", uploadId, channel]</code>	useFeatureHealthDashboard	upload/select handlers
<code>["upload-features-compare", uploadId, channel, baselineId]</code>	useFeatureHealthDashboard	—
<code>["upload-factor-trends", uploadId, channel]</code>	useUploadFactorTrends	upload/select handlers; self-polls every 3 s while <code>features_status ∉ {ready, failed}</code>
<code>["upload-features-for-trends", ...], ["upload-features-compare-for-trends", ...]</code>	useUploadFactorTrends	—
<code>["health-status-plots", uploadId, channel]</code>	useHealthStatusData	—
<code>["trend-uploads", ...], ["trend-upload-plots", ...], ["trend-baseline-plots", ...]</code>	useHistoricalTrendData (unused)	—

2.8 Data Flow

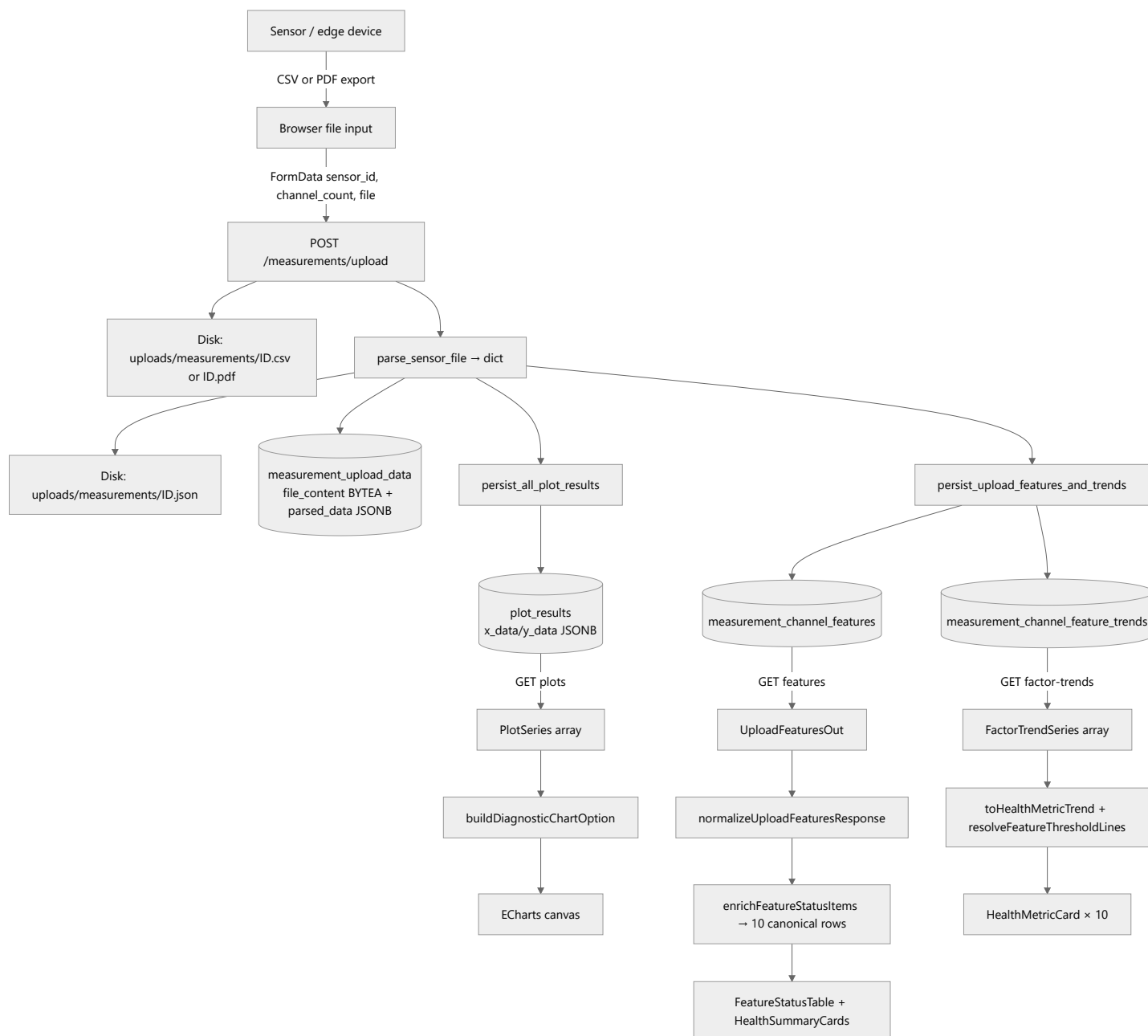


Figure 13 — 2.8 Data Flow

2.8.1 Parsed-data document shape

`parse_measurement_text` returns and `measurement_upload_data.parsed_data` / `sensor_baselines.parsed_data` store:

```

{
  "timestamps": [1777747212.0, 1777747212.0, "..."],
  "channels": { "ch0": [0.0123, -0.0087, "..."], "ch1": ["..."] },
  "sample_count": 4096,
  "channel_count": 7,
  "detected_channel_count": 7
}

```

`channel_count` is the **effective** count actually used; `detected_channel_count` is what the header scan found (`null` when no header was present). When a header declares `ch0...ch6`, the detected value (7) overrides a user-supplied `channel_count` of 8.

2.9 Request Lifecycle

Worked example: `GET /api/v1/measurements/uploads/{upload_id}/features?channel=0`

Table 14 — 2.9 Request Lifecycle

#	Stage	Code
1	Browser attaches <code>Authorization: Bearer ...</code>	<code>client.ts</code> request interceptor
2	CORS preflight (cross-origin) then actual request	<code>CORSMiddleware</code>
3	Route match on prefix <code>/api/v1/measurements</code>	<code>router = APIRouter(prefix=..., dependencies=[Depends(get_current_user)])</code>
4	<code>get_db()</code> opens a <code>SessionLocal</code>	<code>database.py</code>
5	<code>HTTPBearer(auto_error=False)</code> extracts credentials; <code>None</code> /non-bearer → 401	<code>dependencies/auth.py</code>
6	<code>decode_access_token</code> verifies HS256 signature, <code>exp</code> , and <code>type == "access"</code>	<code>auth_service.py</code>
7	<code>UUID(payload["sub"]); JWTError / ValueError / KeyError</code> → 401	<code>dependencies/auth.py</code>
8	<code>get_user_by_id</code> with <code>joinedload(User.roles)</code> ; missing or <code>is_active == False</code> → 401	<code>crud/user.py</code>
9	Path/query params coerced: <code>upload_id: UUID, 'channel: int</code> None with <code>ge=0, le=31</code> → 422 on violation	
10	Handler loads the upload; 404 if absent; 422 if <code>parse_status != "parsed"</code>	<code>routers/measurements.py</code>
11	<code>_resolve_config</code> reads <code>plot_configurations</code> or falls back to defaults	<code>crud/measurement.py</code>
12	<code>ensure_upload_features_ready</code> returns early if features exist, else computes and persists them	<code>services/feature_storage.py</code>
13	Rows read, mapped to <code>ChannelFeatureOut</code> , summarised, overview derived	handler helpers
14	<code>UploadFeaturesOut</code> serialised to JSON	Pydantic
15	<code>finally: db.close()</code> in the <code>get_db</code> generator	<code>database.py</code>

2.10 Response Lifecycle

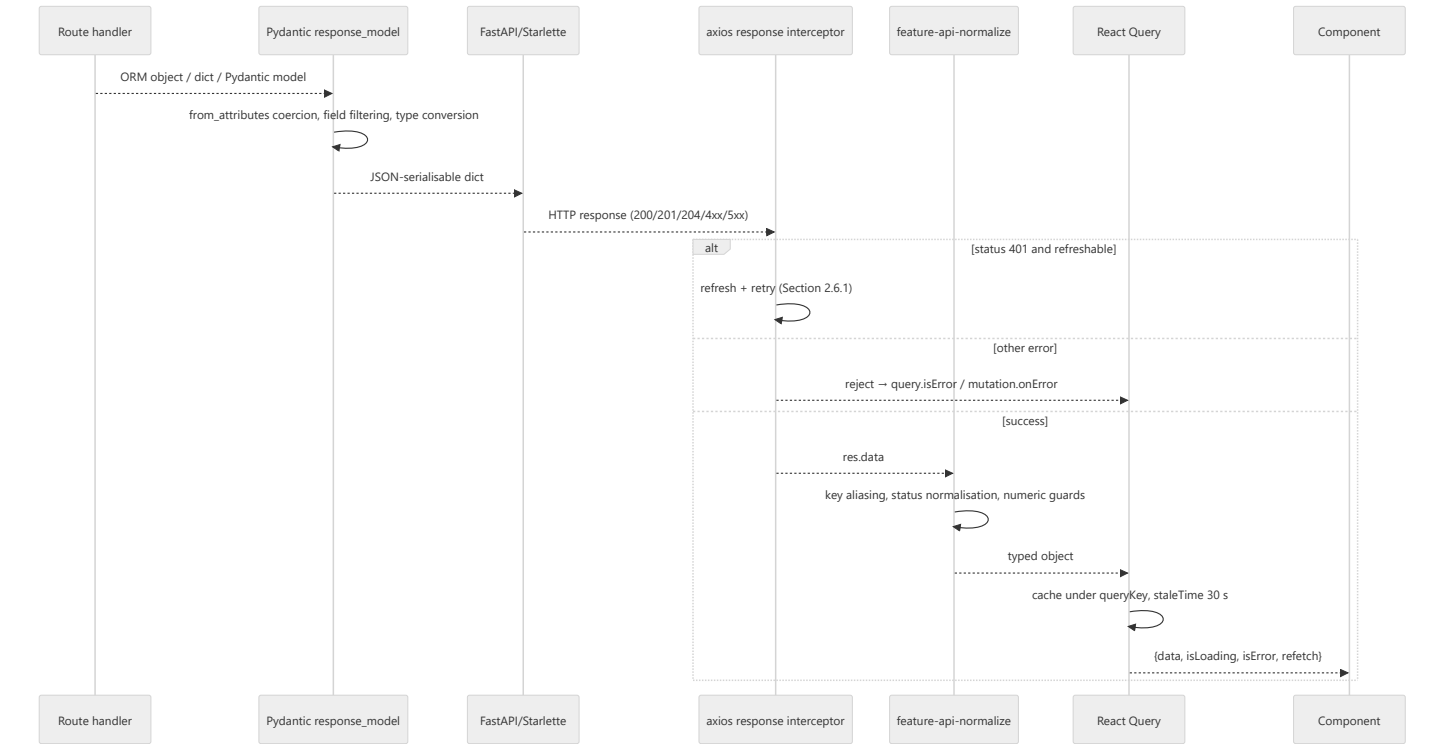


Figure 14 — 2.10 Response Lifecycle

Status codes emitted by the backend

Table 15 — 2.10 Response Lifecycle

Code	Where
200	All successful <code>GET</code> , <code>PUT</code> , <code>PATCH</code> , and <code>POST /auth/*</code>
201	<code>POST /equipment/</code> , <code>POST /equipment/{id}/sensors</code> , <code>POST /measurements/upload</code> , <code>POST /baselines/upload</code> , <code>POST /baselines/from-upload/{id}</code>
204	<code>DELETE /equipment/{id}</code> , <code>DELETE /equipment/{id}/image</code> , <code>DELETE .../sensors/{id}</code> , <code>POST /auth/logout</code>
400	Bad file type, oversized image, invalid plot type, <code>from_date > to_date</code>
401	Missing/invalid/expired token, inactive user, bad credentials, invalid refresh token
403	<code>require_write_access</code> rejects role <code>user</code>
404	Entity not found, image missing on disk, lookup name unknown, no primary baseline
409	Duplicate <code>machine_id</code> on create
422	Pydantic validation failure; parse failure; unparsed upload; feature-compute failure; baseline plot-compute failure

3.0 Frontend Documentation

3.1 Project Structure — folder by folder

Table 16 — 3.1 Project Structure — folder by folder

Folder	Purpose	Contents
src/api	The only network boundary in the app	client.ts (axios + interceptors), auth.ts, equipment.ts, measurements.ts, baselines.ts
src/components/analysis	Vibration Analysis feature components	timeline, chart wrappers, plot selector, baseline modal, layout tokens
src/components/analysis/charts	ECharts diagnostic chart + 3 deprecated re-exports	EchartsDiagnosticChart.tsx, ChartContainer.tsx, ChartToolbar.tsx, FftSpectrumChart.tsx
src/components/analysis/baseline	Baseline browsing UI	BaselineManagementPanel, BaselineDetailCard, baseline-utils.tsx
src/components/analysis/health	Status (Health) tab — 13 components	summary cards, overview card, feature tables, comparison, trend cards, badges, banners
src/components/analysis/workspace	Analysis tab shell and tab bodies	AnalysisWorkspace, AnalysisTabNav, DetailedAnalysisTab, StatisticsTab, TrendAnalysisTab, SelectedCapturePanel, BaselineSelectionPanel
src/components/auth	Route protection	ProtectedRoute.tsx
src/components/brand	Decorative SVG backdrops and animations	HeroIntelligenceBg, LoginIntelligenceBg, SensorPulseRings, VibrationWave, VibrationIntelligenceBg (deprecated shim)
src/components/charts	Reusable, chart-agnostic shells	GraphWorkspace, GraphToolbar, GraphStatisticsPanel, GraphChannelSelector, ThresholdZoneLegend, EchartsGraphViewport, index.ts barrel
src/components/equipment	Equipment Master feature	form shell, stepper, digital-twin header, visualisation, intelligence panels
src/components/equipment/industrial	Small industrial UI atoms	ProgressRing, CriticalityIndicator, IndustrialEmptyState
src/components/equipment/tabs	The 6 wizard steps	Basic, Mechanical, Rotating, Operating, Sensors, Review
src/components/layout	App chrome	AppShell, Sidebar, TopNav, PageHero, ComingSoon, nav-config.ts
src/components/settings	Settings shell + vibration module	SettingsSectionCard, SettingsTabNav, ToggleSwitch, vibration/*
src/components/ui	Design-system primitives + written contracts	Button, FormField, GlassCard, MultiSelect, SectionCard, Toast, CARD_HOVER.md, CARD_SIZING.md
src/contexts	Cross-cutting React context	AuthContext, LayoutContext, ThemeContext
src/hooks	Data-composition hooks	6 hooks
src/images	Static assets + typed barrel	logo, mark, favicon, empty state, 2 accent SVGs
src/lib	Pure functions: maths, chart options, formatters, tokens	30 modules
src/pages	Route-level components	9 files
src/types	Interfaces + constant catalogues	9 modules

3.2 Entry Points

3.2.1 index.html

```
<html lang="en" class="light">
```

The `light` class is applied statically, and `ThemeContext` swaps it for `dark` at runtime. `<div id="root">` is the mount node; `/src/main.tsx` is loaded as an ES module.

3.2.2 main.tsx

```
const queryClient = new QueryClient({
  defaultOptions: { queries: { retry: 1, staleTime: 30_000 } },
});
```

Every query therefore retries once on failure and treats data as fresh for 30 seconds unless a hook overrides `staleTime` (`useUploadFactorTrends` uses 30 s + `gcTime` 300 s + `refetchOnMount: "always"`; `useHistoricalTrendData` 's per-upload plot queries use 60 s).

Render tree: `React.StrictMode` → `QueryClientProvider` → `BrowserRouter` → `App` .

3.2.3 App.tsx

Declares the provider stack and all eight routes (see §2.3). Route protection is applied at three levels:

- 1. **Shell level** — the layout route is wrapped in a bare `<ProtectedRoute>` (authentication only).
- 2. **Page level** — each child route is *additionally* wrapped in `<ProtectedRoute roles={[[...]]}>` .
- 3. **Navigation level** — `Sidebar` filters `NAV_ITEMS` by `hasRole(item.roles)` so unauthorised destinations are not even rendered.

3.3 Routing

Table 17 — 3.3 Routing

Path	Element	Guard	Notes
<code>/login</code>	<code>LoginPage</code>	none	Redirects to <code>returnUrl</code> if already authenticated
<code>/unauthorized</code>	<code>UnauthorizedPage</code>	none	Offers "Return to Dashboard" and "Sign Out"
<code>/change-password</code>	<code>ChangePasswordPage</code>	authenticated	Reached automatically when <code>must_change_password</code> is true
<code>/</code> (layout)	<code>AppShell</code>	authenticated	Renders <code>Sidebar</code> + <code>TopNav</code> + <code><Outlet/></code>
<code>/</code>	<code>Dashboard</code>	<code>ALL_ROLES</code>	Coming-soon placeholder
<code>/equipment</code>	<code>EquipmentMasterList</code>	<code>ALL_ROLES</code>	
<code>/equipment/new</code>	<code>NewEquipmentPage</code>	<code>WRITE_ROLES</code>	
<code>/equipment/:id/edit</code>	<code>EditEquipmentPage</code>	<code>WRITE_ROLES</code>	Loads via <code>useParams</code> + <code>useQuery</code>
<code>/analysis</code>	<code>VibrationAnalysisPage</code>	<code>ALL_ROLES</code>	Write actions disabled for role <code>user</code>
<code>/settings</code>	<code>SettingsPage</code>	<code>ADMIN_ROLES</code>	
<code>*</code>	<code>Navigate to="/" replace</code>	inherits shell	Catch-all inside the protected shell

3.3.1 ProtectedRoute decision order

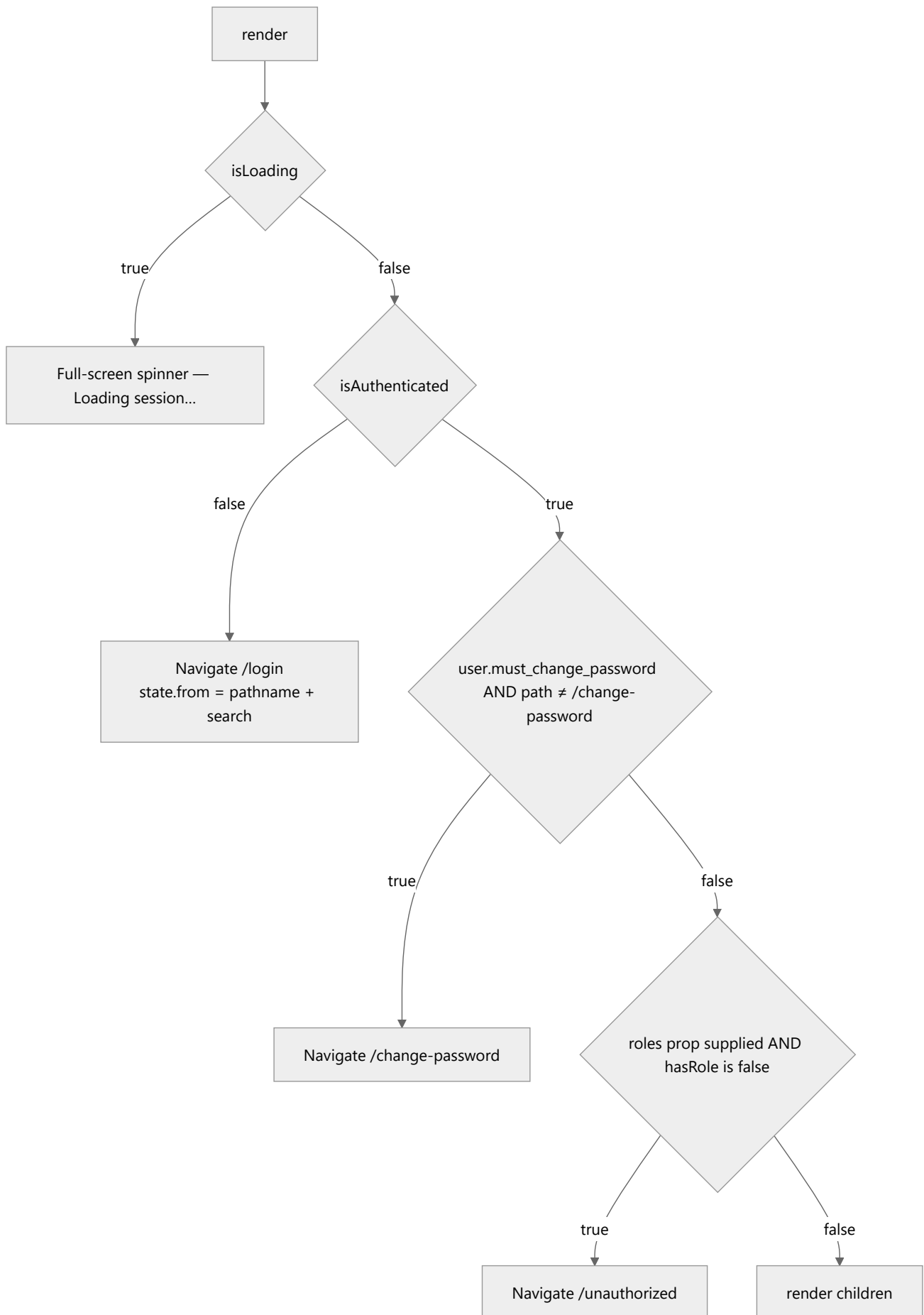


Figure 15 — 3.3.1 ProtectedRoute decision order

The `state.from` value is consumed by `Login.tsx` (`returnUrl`), producing deep-link-preserving login.

3.4 State Management

The application deliberately uses **no global store library**. State is split three ways:

Table 18 — 3.4 State Management

Kind	Mechanism	Examples
Server state	TanStack Query cache	equipment lists, uploads, plots, features, baselines
Cross-cutting UI state	React Context	auth session, sidebar collapse + plant, theme, toasts
Local state	<code>useState</code> / <code>useRef</code> inside components	selected channel, active tab, modal open, zoom range, file input ref

3.4.1 AuthContext (contexts/AuthContext.tsx)

Exposed value:

Table 19 — 3.4.1 AuthContext (contexts/AuthContext.tsx)

Member	Type	Source
<code>user</code>	<code>UserMeResponse</code> <code>null</code>	<code>GET /auth/me</code>
<code>roles</code>	<code>string[]</code>	<code>me.roles</code>
<code>plants</code>	<code>string[]</code>	<code>me.plants</code> (backend always returns <code>[]</code>)
<code>accessToken</code> , <code>refreshToken</code>	<code>string</code> <code>null</code>	mirrored from <code>sessionStorage</code>
<code>isAuthenticated</code>	<code>boolean</code>	set by <code>applyMe</code> , cleared by <code>clearSession</code>
<code>isLoading</code>	<code>boolean</code>	true until <code>bootstrap()</code> settles
<code>login(email, password)</code>	<code>Promise<UserMeResponse></code>	<code>loginApi</code> → <code>setTokens</code> → <code>getMeApi</code> → <code>applyMe</code>
<code>logout()</code>	<code>Promise<void></code>	best-effort <code>logoutApi</code> , then <code>clearSession</code> , then <code>navigate('/login', {replace:true})</code>
<code>refreshSession()</code>	<code>Promise<void></code>	<code>refreshApi</code> → <code>updateTokens</code> → <code>getMeApi</code> → <code>applyMe</code>
<code>hasRole(role roles)</code>	<code>boolean</code>	<code>hasAnyRole(roles, required)</code> ; empty array ⇒ <code>true</code>

Two `useEffect` hooks:

- **Bootstrap** — if `authStorage.hasTokens()` , call `getMeApi()` . On failure, try `refreshSession()` . On failure again, `clearSession()` . `isLoading` is cleared in `finally` .
- **Session-expired listener** — subscribes to the `auth:session-expired` window event dispatched by the axios interceptor; clears session, shows an error toast, navigates to `/login` .

`applyMe` and `clearSession` are module-level functions taking a memoised `setters` object, which keeps the `useCallback` dependency arrays stable.

3.4.2 LayoutContext

`{ sidebarCollapsed, toggleSidebar, selectedPlant, setSelectedPlant }` . Default plant is "All Plants" . Not persisted; resets on reload.

3.4.3 ThemeContext

Lazily initialises from `localStorage["vibration-theme"]` (default `"light"`) . On change it removes both `light` / `dark` classes from `document.documentElement` , adds the current one, and writes back to `localStorage` . `toggleTheme` exists but **no component currently calls it** — the toggle UI has not been built.

3.4.4 Toast context (components/ui/Toast.tsx)

`showToast(message, type)` where `type` ∈ `{"success","error"}` . Toasts auto-dismiss after 4000 ms and render bottom-right in a fixed stack with `AnimatePresence` . Success uses `machine-healthy` colouring with `CheckCircle` ; error uses `destructive` with `AlertCircle` .

Implementation note: the id counter is declared as `let counter = 0` inside the component body, so it resets on every render. IDs can therefore repeat, which React uses as `key` . In practice the 4-second lifetime plus low toast volume makes collisions unlikely, but this is a latent duplicate-key risk.

3.5 Custom Hooks

3.5.1 useEchartsResize(getInstance, deps)

Keeps an ECharts instance sized to its container. On every dependency change it resizes immediately, again inside `requestAnimationFrame`, and again after a 150 ms timeout; then subscribes to `window.resize` and a `ResizeObserver` on the instance's parent element. All three are needed because fullscreen transitions and CSS layout settle asynchronously.

3.5.2 useHealthStatusData({uploadId, channel, samplingRateHz, enabled})

Queries `getAllPlots(uploadId, channel)` and derives a `HealthStatusSnapshot` via `computeHealthMetrics`. Returns the full query object plus `snapshot`. Used by `StatusHealthSection` (which is not currently mounted).

3.5.3 useFeatureHealthDashboard({sensorId, uploadId, channel, primaryBaseline, baselineList, enabled})

The engine behind the Status (Health) tab.

- **Baseline auto-selection** — an effect tracks `{uploadId, channel}` in a ref; when the upload changes (or nothing is selected), it prefers `primaryBaseline.id`, else the first baseline, else `""`.
- **Two queries** — `getUploadFeatures(uploadId, channel)` and `compareUploadFeatures(uploadId, compareBaselineId, channel)` (only when a baseline is selected).
- **Enrichment** — `enrichFeatureStatusItems` / `enrichFeatureCompareItems` map API rows onto the canonical 10-feature catalogue, filling absent features with `no_baseline` placeholders. The table therefore always shows exactly 10 rows.
- **Summary fallback** — if the API summary has `total === 0`, it recomputes counts client-side via `summarizeFeatureItems`.
- **Overview fallback** — prefers the API `channel_overview`, then the compare response's, then a locally derived one from `deriveHealthState(summary)`.

Derived flags returned: `isLoading`, `hasFeatureData` (at least one non-null value), `hasFeatureTable`, `usesApiFeatures`, `usesApiCompare`, `compareIsLoading`, `compareHasApiError`.

3.5.4 useUploadFactorTrends({uploadId, channel, baselineId, enabled})

Three queries:

1. `getUploadFactorTrends` — **self-polling**: `refetchInterval` returns 3000 while `features_status` is neither `ready` nor `failed`, otherwise `false`. This is how the UI waits out a first-time feature computation.
2. `getUploadFeatures` — supplies channel RMS for `percent_rms` threshold rules.
3. `compareUploadFeatures` — supplies per-feature baseline values for `percent_baseline` rules.

`toHealthMetricTrend` converts each `FactorTrendSeries` into a `HealthMetricTrend`, resolving the display threshold lines through `resolveFeatureThresholdLines(featureCode, {channelRms, baselineValue})` and mapping `critical→danger`, `warning→warning`, `normal→healthy`, `other→neutral`. Units are normalised: `scaled_eng` → "scaled", `scaled_eng_sq` → "scaled²", `dimensionless / -` → "".

3.5.5 useHistoricalTrendData(...) — implemented, not mounted

Builds cross-capture trends: lists uploads in a date range (default last 30 days), fires one `getAllPlots` query **per upload** via `useQueries`, extracts the time waveform, computes six metrics per capture with `computeScalarFromSamples`, sorts by `created_at`, and optionally appends the primary baseline as a final point labelled "`<Metric> Trend (incl. baseline)`".

3.5.6 useVibrationSettings()

A local-storage-backed settings store with draft/saved separation.

Table 20 — 3.5.6 useVibrationSettings()

Concern	Implementation
Storage key	<code>sensovibe-vibration-settings</code>
Load	180 ms <code>setTimeout</code> (simulated latency), then <code>migrateSettings(stored ?? defaults)</code>
Migration	<code>migrateThresholds</code> rebuilds the full <code>channels × THRESHOLD_PARAMETERS</code> grid, preserving values whose parameter still exists; channels are clamped to <code>device.maxChannelCount</code>
Dirty tracking	<code>settingsStatesEqual = JSON.stringify(a) === JSON.stringify(b)</code>
Row-level editing	<code>editingChannels: Set<number></code> , <code>editingThresholds: Set<string></code> where the key is <code>`\${channelNo}-\${parameter}`</code>
Mutators	<code>updateChannel</code> , <code>updateThreshold</code> , <code>resetChannelRow</code> , <code>resetThresholdRow</code> , <code>addChannelRow</code> (respects max), <code>removeChannelRow</code> (renumbers), <code>toggleChannelEdit</code> , <code>toggleThresholdEdit</code>
Commit	<code>save()</code> copies draft→saved, persists, clears both editing sets
Revert	<code>cancel()</code> / <code>resetAll()</code> copy saved→draft

3.6 API Layer

3.6.1 api/client.ts — axios configuration

Two instances:

Table 21 — 3.6.1 api/client.ts — axios configuration

Instance	Interceptors	Used by
api (default export)	request: inject Bearer; response: 401 → refresh + retry	every non-auth call
authClient (named export)	none	loginApi, refreshApi, logoutApi, and the interceptor's own performRefresh

Module-level state: `isRefreshing: boolean` and `failedQueue: Array<{resolve, reject}>`. `processQueue(error, token)` drains the queue and resets it. `emitSessionExpired()` dispatches `new CustomEvent("auth:session-expired")` on `window` — the decoupling mechanism that lets a non-React module notify React state.

3.6.2 api/auth.ts

Table 22 — 3.6.2 api/auth.ts

Function	Call
loginApi(data)	authClient.post("/api/v1/auth/login", data)
getMeApi()	api.get("/api/v1/auth/me")
refreshApi(refresh_token)	authClient.post("/api/v1/auth/refresh", {refresh_token})
logoutApi(refresh_token)	authClient.post("/api/v1/auth/logout", {refresh_token})

3.6.3 api/equipment.ts

Table 23 — 3.6.3 api/equipment.ts

Function	Method + path
createEquipment(data)	POST /api/v1/equipment/
updateEquipment(id, data)	PATCH /api/v1/equipment/{id}
getEquipment(id)	GET /api/v1/equipment/{id}
listEquipment(params)	GET /api/v1/equipment/ with page, page_size, plant_name, machine_type, machine_criticality
deleteEquipment(id)	DELETE /api/v1/equipment/{id}
uploadEquipmentImage(id, file)	POST /api/v1/equipment/{id}/image (FormData)
addSensor / updateSensor / deleteSensor	POST/PUT/DELETE /api/v1/equipment/{id}/sensors[/sensorId]
getAIReadiness(id)	GET /api/v1/equipment/{id}/ai-readiness
getLookup(name)	GET /api/v1/lookups/{name} → returns res.data.values
getAllLookups()	GET /api/v1/lookups/

3.6.4 api/measurements.ts

Beyond the plain wrappers, three functions carry logic:

- `savePlotConfig(sensorId, data, hasExisting)` — if `hasExisting` it goes straight to `PUT`; otherwise it tries `POST /configure` and falls back to `PUT` on HTTP 409. (The backend's `POST /configure` actually upserts, so the 409 branch is defensive.)
- `listUploads(sensorId, filters)` — tolerates both a bare array and the paginated envelope, defaulting `page_size` to 200.
- `getUploadFeatures / compareUploadFeatures` — pass `timeout: 120_000` and run the response through the normaliser.

3.6.5 api/baselines.ts

`listBaselines`, `getPrimaryBaseline` (returns `null` on 404 rather than throwing), `createBaselineFromUpload`, `setBaselinePrimary`, `getBaselinePlots`.

3.6.6 Response normalisation — lib/feature-api-normalize.ts

A defensive adapter between backend payloads and UI types:

Table 24 — 3.6.6 Response normalisation — lib/feature-api-normalize.ts

Concern	Behaviour
Key aliasing	Accepts <code>items</code> / <code>features</code> / <code>parameters</code> / <code>comparisons</code> ; <code>feature_key</code> / <code>featureKey</code> / <code>feature_code</code> / <code>feature</code> / <code>name</code> / <code>parameter</code> ; <code>computed_at</code> / <code>computedAt</code> ; <code>no_baseline</code> / <code>noBaseline</code>
Status mapping	<code>ok</code> / <code>healthy</code> → <code>normal</code> ; <code>caution</code> → <code>warning</code> ; <code>danger</code> / <code>alarm</code> → <code>critical</code> ; anything unknown → <code>no_baseline</code>
Numeric guard	<code>readNumber</code> returns <code>null</code> unless <code>typeof value === "number" && Number.isFinite(value)</code>
Feature identity	<code>resolveVibrationFeatureKey</code> maps <code>fft_band_energy_0_500</code> → <code>fft_band_energy</code> , <code>1x</code> / <code>amplitude1x</code> → <code>amplitude_1x</code> , etc.
<code>formatDifferencePercent(v)</code>	<code>" +37%"</code> / <code>" -12%"</code> / <code>" -"</code>

3.7 Styling Strategy

3.7.1 Approach

Tailwind CSS utility classes plus a hand-written component layer in `index.css` (1015 lines). No CSS-in-JS. No CSS modules. The `cn()` helper (`clsx` + `tailwind-merge`) resolves conflicting utilities so later classes win predictably.

3.7.2 Design tokens

All colours are HSL triples in `:root` and are surfaced to Tailwind through `hsl(var(--token))` in `tailwind.config.js`.

Table 25 — 3.7.2 Design tokens

Token	Value	Meaning
<code>--background</code>	<code>40 100% 99% (#FFDF8)</code>	Warm white page background
<code>--foreground</code>	<code>214 67% 25% (#15366D)</code>	Brand navy — all primary text
<code>--signal-light</code>	<code>38 90% 55% (#F5A623)</code>	Focus rings, current step, highlights
<code>--signal-dark</code>	<code>38 100% 43% (#D98C00)</code>	Metrics, completed states
<code>--signal-deep</code>	<code>33 100% 36% (#B86E00)</code>	Hover / critical actions
<code>--cta</code>	<code>24 100% 50% (#FF6B00)</code>	Primary call-to-action
<code>--machine-healthy</code>	<code>142 71% 45%</code>	Green
<code>--machine-warning</code>	<code>38 90% 55%</code>	Amber
<code>--machine-critical</code>	<code>0 84% 60%</code>	Red
<code>--sensor-offline</code>	<code>215 16% 65%</code>	Grey
<code>--radius</code>	<code>0.5rem</code>	Base radius; <code>md</code> = <code>-2px</code> , <code>sm</code> = <code>-4px</code>
<code>--orange-border-gradient</code>	<code>linear-gradient(90deg, #ff6b00, #ffb26b, #ffe3c5)</code>	The signature gradient hairline

Typography scale is **deliberately enlarged** for control-room readability (`tailwind.config.js`): `xs` = `0.875rem` , `sm` = `1rem` , `base` = `1.125rem` , `lg` = `1.25rem` , `xl` = `1.375rem` , `2xl` = `1.75rem` , ... `5xl` = `3rem` , plus a custom `overline` size with `0.05em` tracking and weight 600. Font family is Inter, imported from Google Fonts at the top of `index.css`.

3.7.3 Component classes defined in `index.css`

Table 26 — 3.7.3 Component classes defined in index.css

Group	Classes
Signal accents	<code>.signal-gradient</code> , <code>.signal-gradient-v</code> , <code>.signal-indicator</code> , <code>.signal-nav-rail</code> , <code>.brand-divider</code> , <code>.brand-divider-wide</code>
Sidebar	<code>.sidebar-shell</code> (+ <code>::before</code> / <code>::after</code> gradient edges), <code>.sidebar-nav-item</code> , <code>.sidebar-nav-item-active</code> , <code>.sidebar-logo-divider</code> , <code>.logo-zone</code>
Cards	<code>.content-card</code> (+ <code>::before</code> / <code>::after</code> gradient ring), <code>.card-auto</code> , <code>.card-equal</code> , <code>.card-grid-equal</code> , <code>.card-kpi-body</code> , <code>.card-scroll-region</code> , <code>.card-scroll-region-sm</code> , <code>.card-scroll-fill</code> , <code>.card-state-center</code>
Hover system	<code>.card-hover</code> , <code>.card-hover-soft</code> , <code>.card-hover-kpi</code> , <code>.card-hover-interactive</code> , <code>.panel-hover</code> , <code>.card-hover-upload</code> + gradient-ring intensification rules
Gradient borders	<code>.orange-gradient-border</code> , <code>.orange-gradient-border-subtle</code> , <code>.orange-gradient-border-top</code>
Typography	<code>.text-overline</code> , <code>.text-helper</code> , <code>.text-kpi-value</code> , <code>.text-table-header</code> , <code>.text-card-title</code> , <code>.text-section-title</code> , <code>.text-page-title</code> , <code>.metric-value</code>
Buttons	<code>.btn-cta</code> , <code>.btn-cta-outline</code> (+ hover/focus-visible/disabled)
Forms	<code>.field-label</code> , <code>.required-star</code> , <code>.input-unit</code> , <code>.input-focus</code> , <code>.form-actions-bar</code>
Login page	20+ classes: <code>.login-form-panel</code> , <code>.login-field-box</code> , <code>.login-field-input</code> , <code>.login-submit-btn</code> , <code>.login-features-panel</code> , <code>.login-form-alert-error/--warning</code> , autofill overrides, ...
Animations	<code>@keyframes</code> <code>sensor-pulse</code> , <code>sensor-pulse-slow</code> , <code>login-pulse-ring</code> , <code>login-fft-peak</code> , <code>login-sensor-node</code> , <code>login-sensor-ring</code> , <code>login-wave-drift</code> , <code>login-link-flow</code> , <code>login-gauge-needle</code>
Misc	<code>.page-bg</code> , <code>.scrollbar-thin</code> , <code>.diagnostic-chart-shell:fullscreen</code> , <code>.tab-active/.tab-completed/.tab-inactive</code>

3.7.4 Accessibility and motion

`index.css` contains two `@media` (`prefers-reduced-motion: reduce`) blocks: one disables card lift on hover (background/shadow/border still transition), the other stops all nine login-background animations. `SensorPulseRings` uses an 18-second cycle specifically to stay unobtrusive.

3.7.5 The card contracts

Two Markdown files inside `components/ui` are treated as normative by the code:

- `CARD_SIZING.md` — three height modes: *auto* (default), *equal height* (opt-in, requires `cardSizing.gridEqual` on the parent grid), *scrollable* (opt-in for tables and long lists). The tokens live in `lib/card-sizing.ts`.
- `CARD_HOVER.md` — four hover modes with a table mapping surface type → token. Tokens live in `lib/card-hover.ts`. `GlassCard` picks `interactive` → `kpi` (when `equalHeight`) → `passive`; `SectionCard` picks `interactive` → `soft`.

3.8 Responsive Design

Table 27 — 3.8 Responsive Design

Breakpoint usage	Where
<code>sm</code> :	Toolbar dividers, KPI grids (<code>grid-cols-2 sm:grid-cols-4</code>), settings action bar
<code>md</code> :	TopNav search (<code>hidden md:block</code>), metric card grids, form column splits
<code>lg</code> :	Login split (<code>flex-col lg:flex-row</code> , left panel <code>lg:w-[54%]</code>), equipment 3-column tab layouts, hero action alignment
<code>xl</code> :	Analysis tab nav (<code>xl:grid-cols-4</code>), equipment form sidebar (<code>xl:w-[28%]</code> <code>xl:max-w-[300px]</code> <code>xl:sticky</code>), health summary (<code>xl:grid-cols-5</code>)
Horizontal scroll	Every wide table wraps in <code>overflow-x-auto</code> with an explicit <code>min-w-[...]</code> (<code>min-w-[1080px]</code> channel table, <code>min-w-[920px]</code> threshold table, <code>min-w-[880px]</code> coverage matrix, <code>min-w-[720px]</code> compare table)
Sidebar	Animated width 320 px ↔ 80 px via <code>framer-motion</code> ; labels and the "Coming Soon" hints are hidden when collapsed

3.9 Charts

3.9.1 Chart architecture

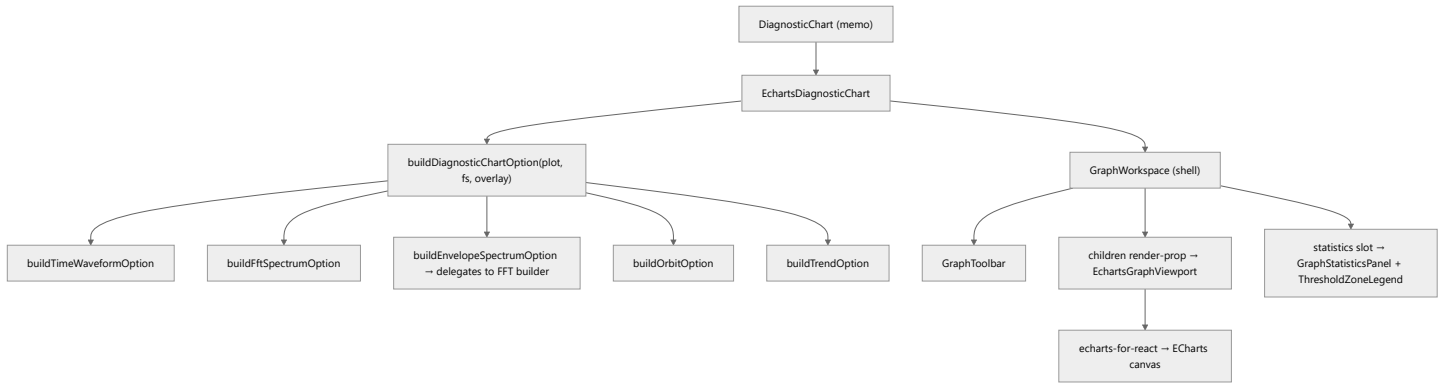


Figure 16 — 3.9.1 Chart architecture

3.9.2 GraphWorkspace — the reusable shell

Props of note: `variant` (`primary` 580 px / `compact` 220 px), `toolbarActions`, `statistics`, `headerExtra`, `channelSlot`, `hint`, and the render-prop `children({height, isFullscreen})`.

Fullscreen is handled with a two-tier strategy:

- 1. Try the native Fullscreen API on the shell element.
- 2. If `requestFullscreen()` throws, fall back to a `createPortal(shell, document.body)` overlay with `position:fixed inset=0 z-[200]`, `document.body.style.overflow = "hidden"`, and an `Escape` key handler.

In both modes `scheduleChartResize` fires the resize callback four times (immediately, next animation frame, +100 ms, +300 ms) and a `ResizeObserver` watches the chart area, because ECharts cannot infer a size change from a CSS-only layout transition.

3.9.3 GraphToolbar — 10 possible actions

`zoomIn`, `zoomOut`, `pan`, `reset`, `crosshair`, `thresholds`, `autoscale`, `refresh`, `export`, `fullscreen`. Buttons render only when listed in `actions` and disable themselves when the corresponding handler is undefined. Toggle buttons (`pan`, `crosshair`, `thresholds`, `fullscreen`) show an active ring.

3.9.4 EchartsGraphViewport

Wraps `<ReactECharts notMerge lazyUpdate opts={{renderer:"canvas"}}/>` and wires three ECharts events:

Table 28 — 3.9.4 EchartsGraphViewport

Event	Behaviour
<code>datazoom</code>	Reads the zoom range, applies adaptive line width, calls <code>onDataZoom</code> (which drives the live statistics recomputation)
<code>dblclick</code>	<code>resetChartZoom</code> then resets the range to 0–100 %
<code>finished</code>	Re-applies adaptive line width after render completes

Adaptive line width (`lib/graph-interactions.ts`): `width = clamp(0.35, baseWidth / sqrt(1/visibleFraction), baseWidth)`. Zooming in therefore thins the trace so dense waveforms stay legible.

3.9.5 Per-plot-type option builders

Table 29 — 3.9.5 Per-plot-type option builders

Builder	Key behaviours
<code>buildTimeWaveformOption</code>	Replaces the X axis with a frontend-generated millisecond axis (<code>withGeneratedTimeAxis</code>); min/max bucket downsampling to 8192 points (<code>downsampleWaveformSeries</code>) so peaks survive; symmetric zero-centred Y axis (<code>computeSymmetricYAxisBounds</code> , ± 10 % padding); tooltip shows time, amplitude+unit, sample rate, sample count, and record length
<code>buildFftSpectrumOption</code>	Uniform decimation to 2000 points; Y axis floored at 0; X axis capped at Nyquist when the sample rate is known; mark-lines for Nyquist, $1\times/2\times/3\times$ harmonics (when <code>rpm</code> metadata exists), and the dominant peak; a highlighted peak mark-point; tooltip adds Δf and Nyquist
<code>buildEnvelopeSpectrumOption</code>	Delegates entirely to the FFT builder (identical frequency/magnitude structure); the trace colour differs because <code>INDUSTRIAL_TRACE_COLORS.envelope_spectrum</code> is <code>#C2410C</code>
<code>buildOrbitOption</code>	Square symmetric bounds derived from both X and Y (<code>computeOrbitAxisBounds</code>); waveform-style downsampling
<code>buildTrendOption</code>	Shows symbols (<code>symbolSize: 5</code>) because trend points are sparse; standard padded Y bounds
<code>buildHealthTrendOption</code>	Compact grid (44/12/16/48), hidden X labels, smoothed line with a navy gradient area fill, Y padding of 12 % expanded to include threshold values

3.9.6 Threshold overlay system (lib/threshold-overlay.ts)

Three levels with fixed visual language:

Table 30 — 3.9.6 Threshold overlay system (lib/threshold-overlay.ts)

Level	Colour	Line	Shade
normal	#2E7D32 green	solid	rgba(46,125,50,0.06)
warning	#D98C00 amber	dashed	rgba(217,140,0,0.10)
critical	#DC2626 red	dotted	rgba(220,38,38,0.12)

Four artefacts are generated per chart:

- 1. markLine — horizontal lines labelled "<Level> Threshold: <value>" .
- 2. markArea — shaded bands from each threshold up to computeShadeUpperBound (max of data max and threshold max, $\times 1.12 + 0.001$).
- 3. markPoint — threshold crossings found by linear interpolation between consecutive samples, capped at 24 per level with an index step so long series stay cheap.
- 4. ThresholdZoneLegend — the green/amber/red key rendered under the statistics panel.

Threshold values are resolved from plot metadata by resolveGraphThresholds , which accepts normal_threshold / healthy_threshold , warning_threshold / caution_threshold / caution_limit , and danger_threshold / critical_threshold / alarm_threshold / warning_limit .

For feature trend cards the values instead come from lib/feature-threshold-lines.ts , which mirrors the backend's seeded feature_threshold_rules and converts rule types into display values:

Table 31 — 3.9.6 Threshold overlay system (lib/threshold-overlay.ts)

Rule type	Display conversion
absolute_max , absolute_db , range	Use normalMax / warningMax directly; critical = warning $\times$ 1.25 (or `warning + max(
percent_rms	normal = channelRms $\times$ normalMax/100 , warning = channelRms $\times$ warningMax/100
percent_baseline	normal = baseline $\times$ normalMax/100 , warning = baseline $\times$ warningMax/100 , critical = baseline $\times$ criticalPercent/100

If the required context (channel RMS or baseline value) is missing, the function returns {} and no overlay is drawn.

3.9.7 Chart statistics

computeChartStatistics(values , context) returns RMS, peak, peak-to-peak, mean, min, max, crest factor, sampling rate, RPM, and sensor status. sliceValuesByZoomPercent slices the series to the visible zoom window first, so the statistics panel updates live as the user zooms. GraphStatisticsPanel renders 10 tiles, emphasising RMS / Peak / Peak-to-Peak / Crest Factor with a ring and larger type.

3.10 Tables

Table 32 — 3.10 Tables

Table	Component	Features
Equipment register	EquipmentMasterList	7 columns, per-row motion.tr stagger (delay: i*0.03), hover row highlight, role-gated edit/delete, AnimatePresence on removal
Feature status	FeatureStatusTable	Grouped into 4 collapsible categories, row tinting by status, badge column
Feature comparison	FeatureComparisonSection	5 columns including signed % difference, same category grouping
Health thresholds	HealthThresholdsTable	7 columns: parameter, latest, delta vs prior, range, caution, warning, status
Statistics	StatisticsTab	12 parameter/value rows computed client-side
Channel configuration	ChannelConfigurationSection	Sticky header, per-row edit mode, inline selects/toggles, add/remove rows
Threshold configuration	ThresholdConfigurationSection	Sticky header, numeric inputs, inline validation ("Danger must exceed warning")
Threshold coverage matrix	ThresholdCoverageMatrix	Sticky first column, 8 channels $\times$ 10 parameters colour-coded cells
Sensor mounting	SensorsOrientationTab	6 fixed rows with mounting/orientation selects feeding the SVG diagram

3.11 Forms and Validation

3.11.1 The equipment wizard

`react-hook-form` with `zodResolver(equipmentSchema)` and `mode: "onChange"`. All six tab panels stay mounted (`style={{display: activeTab === n ? undefined : "none"}})` — the code comments this explicitly: *"Keep all tabs mounted so uncontrolled inputs never lose their values."*

Zod schema highlights (`types/equipment.ts`):

- Almost every field is `.optional().nullable()` — the wizard is intentionally permissive so partial drafts can be saved.
- `z.coerce.number()` converts string inputs from `<input type="number">`.
- Constrained numerics: `rated_power_kw`, `rated_rpm`, `gearbox_ratio`, `gear_teeth`, `fan_blades`, `pump_vanes` are `.positive()`; integers use `.int()`.
- `sensors` is an array of `sensorSchema` where `sensor_type`, `mounting_location`, and `orientation` are `.min(1)` required, and `is_active` defaults to `true`.

Conditional fields (`RotatingComponentsTab`): pole count appears only for Motor/Generator/DG Set; fan blades for Fan/Blower; pump vanes for Pump; gearbox ratio + gear teeth when `machine_type === "Gearbox"` **or** `drive_type === "Gear Drive"`.

Dynamic sensor rows: `useFieldArray({name: "sensors"})` with `append` seeding an empty sensor and `remove(idx)` deleting one. Custom sampling rate and frequency range inputs appear only when the corresponding select is `"Custom"`.

3.11.2 Login validation

Client-side only, in `Login.tsx`: `isValidEmail` regex `^[^\s@]+@[^\s@]+\.[^\s@]+$` plus a non-empty password. Field errors clear on the next keystroke. Caps Lock detection uses `e.getModifierState("CapsLock")` on both keydown and keyup. After five failures a persistent warning banner appears (no lockout is enforced).

3.11.3 Settings validation

`isThresholdValid(row)` returns `true` when the row is disabled; otherwise both thresholds must be present and `danger > warning`. `VibrationSettingsModule.handleSave` blocks the save and raises an error toast when any enabled row is invalid; the offending row is tinted and shows an inline `AlertTriangle` message.

3.11.4 Server-side validation surfaced to the user

`EquipmentForm.onSubmit` catches the axios error and shows `err?.response?.data?.detail` (for example the 409 "Machine ID 'X' already exists"), falling back to a generic message.

3.12 Error Handling (frontend)

Table 33 — 3.12 Error Handling (frontend)

Layer	Mechanism
Network 401	axios interceptor → refresh → retry; terminal failure → <code>auth:session-expired</code> → toast + redirect
Query failure	<code>isError</code> renders a message; several panels add a <code>Retry</code> button calling <code>refetch()</code> (<code>BaselineManagementPanel</code> , <code>FeatureComparisonSection</code>)
Mutation failure	<code>onError</code> → <code>showToast(..., "error")</code> (equipment delete/save)
API detail extraction	<code>(err as {response?:{data?:{detail?:string}}})?.response?.data?.detail</code> pattern used in <code>DetailedAnalysisTab</code> , <code>FeatureTrendCardsSection</code> , <code>EquipmentForm</code>
Empty vs error distinction	Separate branches: loading skeleton → error → empty state → data (see <code>StatusHealthTab</code> , <code>BaselineManagementPanel</code>)
Storage failures	<code>useVibrationSettings</code> wraps <code>JSON.parse</code> in try/catch and falls back to defaults with a visible banner
Missing-data guards	<code>getPrimaryBaseline</code> converts 404 to <code>null</code> ; <code>computeChartStatistics</code> returns all- <code>null</code> for an empty series; <code>formatFeatureValue</code> returns <code>"–"</code>

There is **no React error boundary** in the tree — a render-time exception would blank the page.

3.13 Loading States

Table 34 — 3.13 Loading States

Pattern	Example
Full-screen spinner	<code>ProtectedRoute</code> while <code>isLoading</code>
Rotating square	<code>EquipmentMaster</code> <code>LoadingState</code> , <code>EquipmentMasterList</code> table loader
Skeleton blocks	<code>HealthSummaryCardsSkeleton</code> , <code>ChannelHealthOverviewSkeleton</code> , <code>FeatureStatusTableSkeleton</code> , <code>BaselineListSkeleton</code>
Inline text	"Loading analysis data...", "Loading statistics...", "Loading health metrics..."
Spinner in button	<code>Login</code> submit shows <code>Loader2</code> + "Signing in..."
Progressive computation notice	<code>FeatureTrendCardsSection</code> shows "Computing factor trends for this capture (first load may take a few seconds)..." while the 3-second poll runs
Disabled + spinner	<code>BaselineManagementPanel</code> "Set as Primary" swaps its star icon for a spinning <code>Loader2</code> for the specific row being mutated

3.14 Pagination, Search, Filtering, Sorting

Table 35 — 3.14 Pagination, Search, Filtering, Sorting

Concern	Implementation
Pagination	Server-side on equipment (<code>page</code> , <code>page_size=20</code>); Previous/Next buttons appear only when <code>total > 20</code> ; label reads <code>Showing X-Y of N</code>
Search	Client-side on the current equipment page (<code>machine_name</code> , <code>machine_id</code> , <code>plant_name</code> , case-insensitive <code>includes</code>); client-side on baselines (name + formatted date)
Filtering	Server-side <code>machine_type</code> and <code>machine_criticality</code> (both included in the query key so a change refetches); client-side baseline status filter (<code>all</code> / <code>primary</code> / <code>ready</code> / <code>pending</code> / <code>failed</code>); date-range filter on uploads (server-side <code>from_date</code> / <code>to_date</code>); day-chip filter on the timeline (client-side)
Sorting	No user-facing sort control. Server sorts equipment and uploads by <code>created_at</code> DESC , baselines by <code>created_at</code> DESC . The timeline re-sorts ascending for chronological display; historical trend points sort ascending by <code>created_at</code> ; factor trends sort by the backend <code>sort_order</code> of each feature definition

3.15 Exports and Imports

Export. PNG only, per chart, via `instance.getDataURL({type:"png", pixelRatio:2, backgroundColor:"#FFFDF8"})` and a synthetic `<a download>` click. Filenames: `sensovibe-{plot_type}-ch{channel}.png` for diagnostic charts and `sensovibe-health-{metric.key}-{channelLabel}.png` for health cards. There is no CSV/PDF/Excel export.

Import. Two paths: equipment images (`image/jpeg|png|webp|gif`, ≤10 MB) and measurement files (`.csv` / `.pdf`, ≤50 MB). The measurement input declares `accept=".pdf,.csv,application/pdf,text/csv"` ; the selected file is previewed with name, size (`formatFileSize`), and MIME type, and can be removed before upload.

3.16 Environment Variables and Build

Table 36 — 3.16 Environment Variables and Build

Variable	Consumer	Default
<code>VITE_API_BASE_URL</code>	<code>api/client.ts</code>	<code>http://localhost:8000</code>
<code>import.meta.env.DEV</code>	<code>lib/auth-debug.ts</code>	Vite built-in — gates <code>[Auth]</code> console logging

Build configuration:

Table 37 — 3.16 Environment Variables and Build

File	Key settings
<code>vite.config.ts</code>	React plugin; <code>@</code> → <code>./src</code> ; dev port 5173; <code>/api</code> proxy to <code>http://localhost:8000</code> with <code>changeOrigin</code> ; <code>optimizeDeps.include</code> still lists <code>plotly.js-dist-min</code> and <code>react-plotly.js</code> although neither package is installed or imported (a leftover from a previous charting library)
<code>tsconfig.json</code>	<code>target: ES2020</code> , <code>strict: true</code> , <code>noEmit</code> , <code>jsx: react-jsx</code> , <code>moduleResolution: bundler</code> , <code>noFallthroughCasesInSwitch: true</code> , <code>noUnusedLocals/Parameters: false</code> , path alias <code>@/*</code>
<code>package.json</code> scripts	<code>dev = vite</code> ; <code>build = tsc && vite build</code> (type errors fail the build); <code>preview = vite preview</code>
<code>Dockerfile</code>	<code>base</code> (npm ci + copy) → <code>dev</code> (<code>vite --host 0.0.0.0</code>) or <code>build</code> (<code>ARG VITE_API_BASE_URL</code> baked at build time) → <code>preview</code> (nginx serving <code>/app/dist</code>)

Because `VITE_API_BASE_URL` is inlined at build time, the compose file passes it as a build arg (`http://localhost:8000`), not as a runtime environment variable.

3.17 Screen-by-Screen Documentation

3.17.1 Login (/login)

Purpose. Authenticate the user and establish the session.

UI components. Two-panel layout. Left: `LoginIntelligenceBg` `variant="left"` (animated dark canvas with drifting waveforms, an animated sensor mesh with travelling link pulses, FFT bars, and a gauge needle), the "Sensovibe" wordmark, tagline, headline, brand divider, and a six-item capability grid (`Real-Time Monitoring`, `FFT Spectrum Analysis`, `Time Waveform Analysis`, `Predictive Maintenance`, `Anomaly Detection`, `Equipment Health Scoring`) with `framer-motion` stagger of 0.04 s. Right: `LoginIntelligenceBg` `variant="right"`, heading "Welcome Back", email field with `Mail` icon, password field with `Lock` icon and an `Eye / EyeOff` toggle, alerts, and the submit button.

Data source. None on load. `POST /api/v1/auth/login` then `GET /api/v1/auth/me` on submit.

APIs used. `/auth/login`, `/auth/me` (via `AuthContext.login`).

Workflow. `validate` → `login()` → `setTokens` → `getMeApi` → `applyMe` → `clear password` → `success toast` → `navigate to /change-password` if `must_change_password`, else to `returnUrl`.

Navigation. Entry point for unauthenticated users; redirects away immediately if already authenticated.

Business logic. Failed-attempt counter; Caps Lock warning; deep-link preservation through `location.state.from`.

Validation. Email format, password presence; errors clear on input.

Error handling. Extracts `detail` from an axios error; otherwise "Sign in failed. Please check your credentials."

Screenshot placeholder. [SCREENSHOT: Login - dual-panel with animated intelligence backdrop]

3.17.2 Dashboard (/)

Purpose. Landing page; currently a placeholder.

UI components. `ComingSoon` (icon tile, "Coming Soon" heading, four feature chips, CTA to Equipment Master); a "Fleet Overview" heading with a `Preview` chip; four `GlassCard` KPI tiles (Fleet Health, AI Predictions, Anomaly Score, Uptime) all displaying `"-"`; a bottom CTA card linking to Equipment Master.

Data source / APIs. None.

Screenshot placeholder. [SCREENSHOT: Dashboard - coming-soon state with four placeholder KPI tiles]

3.17.3 Equipment Master List (/equipment)

Purpose. Browse, search, filter, and manage the machine register.

UI components.

- `PageHero` with breadcrumbs, `vibrationBg`, live equipment count, and an "Add Equipment" button (write roles only).
- Four KPI `GlassCard` s: Total Equipment (from `data.total`), Critical Assets, High Priority, Active Status (the last three counted from the **current page** only).
- Filter card: search input with `Search` icon, machine-type select, criticality select.
- Table card: 7 columns — Machine (icon + name + manufacturer), ID (monospace chip), Type, Plant/Area (two lines), Criticality (dot + coloured pill), Status (pill), Actions.
- Pagination row and an "AI Tip" hint strip.

Data source. `useQuery(["equipment", page, filterType, filterCriticality])` → `listEquipment`.

APIs used. `GET /api/v1/equipment/`, `DELETE /api/v1/equipment/{id}`.

Workflow. `Filter/paginate` → `server refetch`; `search` filters the fetched page in memory; `Edit` navigates to `/equipment/{id}/edit`; `Delete` opens `window.confirm` then mutates and invalidates `["equipment"]`.

Business logic. `canWrite = hasRole(WRITE_ROLES)` hides Add/Edit/Delete for role `user`. Criticality and status colours come from `CRITICALITY_COLORS`, `CRITICALITY_DOT`, `ASSET_STATUS_COLORS` in `types/equipment.ts`.

Error handling. Distinct loading, error ("Make sure the backend is running on port 8000"), and empty (`empty-equipment.svg` + CTA) states.

Screenshot placeholder. [SCREENSHOT: Equipment Master - KPI row, filters, register table]

3.17.4 Equipment Create / Edit (/equipment/new , /equipment/:id/edit)

Purpose. Build or maintain the machine digital twin.

UI components. `Breadcrumb` → `DigitalTwinHeader` (hero backdrop, "Create/Edit Digital Twin", machine name · ID) → `FormStepper` (progress bar + six clickable step nodes with completed/active/upcoming/locked states) → the active tab panel → an action bar (Back / Continue or Save & Finish) → a sticky `MachineVisualizationPanel` aside.

The six steps:

Table 38 — 3.17.4 Equipment Create / Edit (/equipment/new, /equipment/{id}/edit)

Step	Component	Fields
1 Basic	BasicDetailsTab	Location Hierarchy (plant, area, line) · Machine Identification (name, ID, type, criticality with colour dot) · Manufacturer & Model (manufacturer, model, serial) · Equipment Image (drop zone + preview + clear)
2 Mechanical	MechanicalDetailsTab	rated_power_kw (kW), rated_rpm (RPM), drive_type, load_type, foundation_type, coupling_details
3 Rotating	RotatingComponentsTab	bearing_details, bearing_number_de/nde + conditional motor_pole_count / fan_blades / pump_vanes / gearbox_ratio + gear_teeth, direction_of_rotation
4 Operating	OperatingProcessTab	speed range, load range, normal load, process details, operating environment (MultiSelect), lubrication type, installation & last-maintenance dates, maintenance notes
5 Sensors	SensorsOrientationTab	6 fixed mounting rows feeding SensorMountingDiagram ; dynamic "Additional Sensors" list with 7 fields each
6 Review	ReviewSaveTab	Read-only summary in 5 sections + Asset Configuration (status select + 3 readiness checkboxes)

Data source. EditEquipmentPage loads via useQuery(["equipment", id]) and normalises dates by splitting on "T".

APIs used. GET /equipment/{id} , POST /equipment/ , PATCH /equipment/{id} , POST /equipment/{id}/image .

Workflow. Wizard navigation marks the previous step complete; step nodes beyond maxReachableStep are disabled; submit persists the equipment, then uploads the pending image if one was chosen, then marks all six steps complete and navigates back to the list after 1200 ms.

Business logic. MachineVisualizationPanel selects one of nine hand-drawn SVG machine illustrations by machine_type (case-insensitive), falling back to a dashed placeholder, behind SensorPulseRings .

Validation. Zod on submit; server 409 for duplicate machine_id surfaced as a toast.

Screenshot placeholders. [SCREENSHOT: Wizard step 1 – Basic Details] [SCREENSHOT: Wizard step 5 – Sensors & mounting diagram] [SCREENSHOT: Wizard step 6 – Review & Save]

3.17.5 Vibration Analysis (/analysis)

Purpose. The analytical workspace: select an asset and sensor, upload or select a capture, and analyse it.

Page composition (top to bottom):

1. PageHero — "Vibration Analysis".
2. SaveBaselineModal — rendered always, visible when baselineModalOpen .
3. Equipment & Sensor card — two selects; changing equipment clears the sensor; changing the sensor clears the selected upload.
4. BaselineManagementPanel — search, status filter, baseline cards with Primary/Loaded badges, "Set as Primary" and "Load for Analysis" actions, and a collapsible detail card.
5. Upload Sensor Data card — file input or selected-file chip with size/type and a Remove button; upload button; a read-only notice for role user .
6. CaptureTimelineSection — date-range bar + horizontal timeline.
7. SelectedCapturePanel — capture label plus five metric tiles (Samples, Channels, Parse Status, Plots Status, Features Status).
8. AnalysisWorkspace — four-tab nav and four panels rendered with hidden rather than unmounting.

The capture timeline (CaptureTimeline.tsx) deserves specific description: two summary tiles (Total Files in Range, Selected Date Range), day chips derived from groupUploadsByDay , a gradient rail on which each upload is positioned by ((created_at - rangeStart) / rangeSpan) × 100 % , selected dots enlarged with an orange halo, failed parses outlined in red, hover tooltips, and a Previous/Next navigator showing n of N .

The four tabs:

Table 39 — 3.17.5 Vibration Analysis (/analysis)

Tab	Component	Contents
Status (Health)	StatusHealthTab	Channel selector · 5 summary cards · Channel Health Overview · Feature Status Table (4 categories × 10 features) · Feature Comparison vs Baseline · 10 Feature Trend cards
Trend	TrendAnalysisTab	Notice that factor trends moved to Status (Health)
Detailed Analysis	DetailedAnalysisTab	Sampling rate / FFT lines / channel count inputs + Save Plot Configuration · summary + channel buttons · GraphChannelSelector · PlotSelector (5 tabs) · the full DiagnosticChart
Statistics	StatisticsTab	12-row statistics table computed from the time waveform

APIs used. GET /equipment/ , GET /equipment/{id} , GET /measurements/configure/{sensorId} , GET /measurements/uploads , POST /measurements/upload , GET /measurements/uploads/{id}/plots , .../features , .../features/compare , .../factor-trends , GET /baselines , GET /baselines/primary , GET /baselines/{id}/plots , POST /baselines/from-upload/{id} , PATCH /baselines/{id}/primary , POST|PUT /measurements/configure .

Business logic.

- `plotSource` switches the plot query between `getAllPlots(uploadId)` and `getBaselinePlots(baselineId)`.
- `plotChannelCount` prefers the selected upload's or baseline's `channel_count` over the form value.
- An effect clamps `activeChannel` whenever `plotChannelCount` shrinks.
- After a successful upload the page selects the new upload, resets to channel 0, bumps `timelineRefreshKey`, invalidates five query keys, switches to the Trend tab, clears the file input, and refetches plots after 100 ms.

Error handling. Per-section: plot errors show the API `detail`; feature errors show a channel-specific message; trend errors distinguish "computing", "failed", and "no data".

Screenshot placeholders. [SCREENSHOT: Analysis – equipment/sensor + baseline management] [SCREENSHOT: Capture timeline with day chips] [SCREENSHOT: Status (Health) tab – summary cards + feature table] [SCREENSHOT: Detailed Analysis – FFT spectrum with threshold overlay] [SCREENSHOT: Chart fullscreen mode]

3.17.6 Settings (/settings)

Purpose. Configure the vibration acquisition device: channel mapping and alarm thresholds.

UI components. `PageHero` → `SettingsTabNav` (Vibration Settings / Platform — the latter disabled) → `VibrationSettingsModule`, which renders:

- Header block with title, brand divider, description.
- Error banner when settings could not be loaded.
- `DeviceInfoCard` — device label, MAC-style ID in monospace, and mode (`MEMS`).
- `ChannelConfigurationSection` — "N of M channels configured", Add Channel Row, and a 7-column table (Channel No, Axis, Data Type, Engineering Unit, Measurement Point Name, Active toggle, Actions) where fields are disabled until the row's Edit is pressed.
- `ChannelMappingOverview` — a legend plus auto-fill grid of channel tiles coloured by readiness (Configured / Partial / Not Configured).
- `ThresholdConfigurationSection` — 6-column table over `channels × 10 parameters` with warning/danger numeric inputs, an Enabled toggle, and inline validation.
- `ThresholdCoverageMatrix` — 8×10 dot matrix with a four-state legend (Saved / Incomplete / Disabled / Not Configured).
- `SettingsPageActions` — a sticky bottom bar with Save Changes / Reset / Cancel, all disabled unless `isDirty`, and a status line.

Data source. `localStorage["sensovibe-vibration-settings"]` via `useVibrationSettings`. **No API calls.**

Business logic. Defaults seed five configured channels (MDE, DE, NDE, Motor Drive End, Pump Housing) and threshold values for channels 1–4. The threshold parameter list is derived from `VIBRATION_FEATURE_CATALOG`, so the Settings page automatically stays in sync with the ten features analysed on the Status tab.

Validation. Save is blocked while any enabled row violates `danger > warning`.

Screenshot placeholders. [SCREENSHOT: Settings – channel configuration table] [SCREENSHOT: Settings – threshold coverage matrix]

3.17.7 Unauthorized (/unauthorized)

Centred `GlassCard` over `HeroIntelligenceBg` at 30 % opacity: `ShieldAlert` icon, "Access Restricted", divider, explanation, and two actions — "Return to Dashboard" (link to `/`) and "Sign Out" (calls `logout()`).

[SCREENSHOT: Unauthorized]

3.17.8 Change Password (/change-password)

Centred `GlassCard`: `KeyRound` icon, "Password Change Required", the signed-in email, the explicit notice *"Password change API is not yet available. Contact your administrator."*, and a Sign Out button. **There is no password form** — the screen is a terminal state until the backend endpoint exists.

[SCREENSHOT: Change Password – placeholder state]

3.17.9 Application shell (all authenticated routes)

AppShell — `flex h-screen overflow-hidden`; `Sidebar` (fixed) + a column containing `TopNav` (sticky) and a scrollable `<main class="page-bg">` wrapping `<Outlet/>` in a fade-in `motion.div` with `px-6 lg:px-8 py-6`.

Sidebar — animated width $320 \leftrightarrow 80$ px; logo zone showing the full JPEG logo + a superscript "TM" + tagline when expanded and the SVG mark when collapsed; a "Modules" overline; role-filtered nav items with an active orange rail, an icon tile, and a "Coming Soon" sub-label for items whose `active` flag is false (currently only Dashboard); a Collapse toggle at the bottom.

TopNav — search input (`hidden md:block`, widens on focus, non-functional), plant dropdown over `PLANTS`, notification bell with a hard-coded badge of 3, and a user menu showing full name, role badge (`roleLabel(primaryRole(roles))`), email, and Sign Out. Both dropdowns use a full-screen transparent click-catcher plus `AnimatePresence`.

[SCREENSHOT: Sidebar expanded] [SCREENSHOT: Sidebar collapsed] [SCREENSHOT: TopNav user menu]

3.18 Complete Frontend Component Reference

3.18.1 components/ui

Table 40 — 3.18.1 components/ui

Component	Props	Behaviour
Button	variant (primary/secondary/ghost/danger/warning), size (sm/md/lg), icon, all button attrs	motion.button with whileTap: scale 0.98; primary / warning → .btn-cta, secondary → .btn-cta-outline
FormField	label, required, error, hint, compact, className	Compact mode renders a small uppercase label; hint renders an info panel; error renders red helper text
TextInput	unit, error + input attrs	Optional right-aligned unit suffix with pr-16 padding
SelectInput	options, placeholder, error	Renders <option> list with optional empty placeholder
TextareaInput	error	resize-none
RangeInput	unit, valueMin/Max, onChangeMin/Max, type	Two inputs separated by an em dash
GlassCard	hover, interactive, delay, equalHeight, scrollable	Entrance animation (opacity 0→1, y 8→0, 0.35 s); picks the hover token per §3.7.5
SectionCard	title, description, icon, equalHeight, scrollBody, hover, interactive	Header with an amber gradient-ring icon tile + p-8 body
MultiSelect	options, value, onChange, placeholder, error	Chip-based multi-select with outside-click close and a custom checkbox
Toast / ToastProvider / useToast	—	4 s auto-dismiss, bottom-right stack

3.18.2 components/charts

Table 41 — 3.18.2 components/charts

Component	Role
GraphWorkspace	Chart shell: header, controls slot, toolbar, measured chart area, statistics slot, hint; owns fullscreen (native + portal fallback)
GraphToolbar	10 configurable actions with active-state rings
GraphStatisticsPanel	10 statistic tiles; primary four emphasised
GraphChannelSelector	Segmented CH-n control
ThresholdZoneLegend	Green/amber/red legend, hidden when thresholds are off
EchartsGraphViewport	ReactECharts + resize hook + zoom/dblclick/finished handlers + adaptive line width

3.18.3 components/analysis

Table 42 — 3.18.3 components/analysis

Component	Role
AnalysisSectionHeader	Icon tile + title + optional subtitle + bottom rule
AnalysisSummaryPanel	Capture metrics, channel buttons, "Save as baseline" (write-only)
CaptureTimeline	The timeline visualisation described in §3.17.5
CaptureTimelinePanel	Owns date range + day filter; queries uploads; renders the bar and timeline
CaptureTimelineSection	GlassCard + header wrapper around the panel
ChartHeader	Title + ch{n} badge (used by legacy chart layouts)
CompactDateRangeBar	From/To date inputs with reciprocal min/max clamping
DiagnosticChart	memo wrapper delegating to EchartsDiagnosticChart
PlotChart	Deprecated alias of DiagnosticChart
PlotSelector	Five plot-type tabs; unavailable types render disabled at 40 % opacity
SaveBaselineModal	Name, description, "set as primary" checkbox; resets on open; disables Save while the name is blank
analysis-layout.ts	Shared spacing/typography tokens for the analysis dashboard

3.18.4 components/analysis/health

Table 43 — 3.18.4 components/analysis/health

Component	Role
StatusHealthTab	Orchestrates the whole Status tab through useFeatureHealthDashboard
HealthChannelSelector	CH-n select (default 8 channels) + current-channel badge
HealthSummaryCards (+ skeleton)	Total / Normal / Warning / Critical / No Baseline counts
ChannelHealthOverviewCard (+ skeleton)	Health state, feature count, last computation, baseline used
FeatureStatusTable (+ skeleton)	Category-grouped collapsible feature table
FeatureStatusBadge	Status pill with fixed colour mapping; exports featureStatusLabel
FeatureComparisonSection	Baseline select + comparison table + retry on error
FeatureTrendCardsSection	10 HealthMetricCard s from useUploadFactorTrends ; distinguishes computing/failed/no-data
HealthMetricCard	Compact GraphWorkspace + ECharts trend + status pill + refresh
HealthThresholdsTable	7-column threshold/latest/delta/range table
HealthInfoBanner	Green when thresholds exist, neutral otherwise
HealthEmptyState	Dashed empty panel
SensorThresholdConfig	Caution/warning limit display
StatusHealthSection	Alternative plots-derived health view (implemented, not mounted)

3.18.5 components/equipment

Table 44 — 3.18.5 components/equipment

Component	Role
EquipmentForm	FormProvider + wizard state + submit orchestration
EquipmentPageShell	Thin positioning wrapper
DigitalTwinHeader	Hero header with backdrop
FormStepper	Progress bar + 6 step nodes with reachability rules
MachineVisualizationPanel	9 machine SVGs + pulse rings
SensorMountingDiagram	DE/NDE/shaft schematic with orientation arrows and markers
AssetHealthPanel	5 ProgressRing s (AI, sensor, data, diagnostic, PM readiness) — not mounted
AssetIntelligencePanel	Sticky live-analysis sidebar with score, completion, fleet count, alerts, actions — not mounted
CompletenessEngine	5 per-section completion bars — not mounted
industrial/ProgressRing	SVG ring with strokeDasharray/offset and a centred percentage
industrial/CriticalityIndicator	Criticality banner with reliability-impact text
industrial/IndustrialEmptyState	Amber-ruled empty notice

3.18.6 components/settings

Table 45 — 3.18.6 components/settings

Component	Role
SettingsSectionCard	Card shell with icon tile, title, description, configurable body padding
SettingsTabNav	Two module tabs; <code>platform</code> disabled
ToggleSwitch	Accessible peer-checked CSS switch
vibration/DeviceInfoCard	Device identity + mode
vibration/ChannelConfigurationSection	Editable channel table
vibration/ChannelMappingOverview	Readiness tiles
vibration/ThresholdConfigurationSection	Editable threshold table with validation
vibration/ThresholdCoverageMatrix	Channel × parameter status matrix
vibration/SettingsPageActions	Sticky Save/Reset/Cancel bar
vibration/VibrationSettingsModule	Composes all of the above around <code>useVibrationSettings</code>

3.18.7 components/brand

Table 46 — 3.18.7 components/brand

Component	Role
HeroIntelligenceBg	Page-header backdrop: waveform paths, a 6-node sensor mesh with 6 links, 16 FFT bars, orange depth washes; uses <code>useId()</code> to namespace SVG gradient IDs
LoginIntelligenceBg	610-line animated login canvas: 10 sensor nodes, 10 links with 20 travelling pulses, 5 + 2 drifting waves, 24 FFT bars, a gauge needle; <code>variant</code> selects the left or right composition
SensorPulseRings	Three concentric amber rings on an 18 s cycle
VibrationWave	Two-stroke decorative sine accent
VibrationIntelligenceBg	Deprecated shim: <code>variant="page"</code> renders <code>null</code> , otherwise delegates to <code>HeroIntelligenceBg</code>

3.19 Complete lib/ Reference

Table 47 — 3.19 Complete lib/ Reference

Module	Exports	Purpose
auth-storage.ts	authStorage	sessionStorage wrapper for sv_access_token / sv_refresh_token
auth-debug.ts	authLog	[Auth] console logging gated on import.meta.env.DEV
role-access.ts	ALL_ROLES, WRITE_ROLES, ADMIN_ROLES, ROLE_LABELS, primaryRole, roleLabel, hasAnyRole	Role constants and helpers
utils.ts	cn	twMerge(clsx(...))
card-hover.ts / card-sizing.ts	cardHover, cardSizing	Design-token maps for the two card contracts
chart-constants.ts	GRAPH_PRIMARY_HEIGHT 580, GRAPH_COMPACT_HEIGHT 220, GRAPH_FULLSCREEN_MIN_HEIGHT 400	Chart heights
chart-layout.ts	computeFullscreenChartHeight	$\max(\text{round}(\text{vh} \times 0.92) - 132, 360)$
chart-data.ts	downsampleSeries, downsampleWaveformSeries, resolveSpectrumPeak, findSpectrumPeak	Decimation (uniform vs min/max bucket) and peak detection
chart-bounds.ts	computeYAxisBounds, expandBoundsForThresholds, computeSymmetricYAxisBounds, computeOrbitAxisBounds	Axis range maths
chart-statistics.ts	computeChartStatistics, sliceValuesByZoomPercent, formatStatValue	Live statistics for the visible window
chart-hints.ts	chartHintFor, autoscaleTitleFor	Per-plot-type interaction hints
chart-thresholds.ts	getPlotThresholds, thresholdValues, plotThresholdsFromSet, echartsThresholdMarkLineConfig (deprecated), echartsThresholdValues	Legacy two-level API bridged onto threshold-overlay
chart-reference-lines.ts	buildNyquistReferenceLine, buildHarmonicReferenceLines, mergeSpectrumReferenceLines	Vertical spectrum markers
threshold-overlay.ts	14 exports incl. THRESHOLD_LEVEL_META, buildThresholdSeriesOverlay, computeShadeUpperBound, mergeMarkLineConfigs	The complete three-level overlay engine
feature-threshold-lines.ts	resolveFeatureThresholdLines, graphThresholdsToHealthMetric	Mirrors backend threshold rules into display values
industrial-viz-standards.ts	INDUSTRIAL_AXIS_GRID, INDUSTRIAL_TRACE_COLORS, INDUSTRIAL_REFERENCE_LINE, ISO_10816_VELOCITY_ZONES_REFERENCE, CREST_FACTOR_DISPLAY_GUIDE, KURTOSIS_DISPLAY_GUIDE, nyquistFrequencyHz, frequencyResolutionHz, rpmToHz, parseUnitFromAxisLabel, formatAmplitudeWithUnit, readMetadataNumber, buildVizContextFromPlot	Documented visualisation standards; ISO zones are explicitly <i>not</i> auto-applied
echarts-theme.ts	ECHARTS_BRAND, CHART_GRID, CHART_TOOLBOX_OFF, CHART_X_AXIS_DATA_ZOOM, fixedYAxisConfig, baseAxisStyle, industrialAxisConfig, baseTooltip, formatFrequencyHz, formatMagnitude	Shared ECharts theme; X-axis-only zoom by design
echarts-diagnostic-option.ts	buildDiagnosticChartOption	Switch over plot_type
waveform-option.ts, fft-spectrum-option.ts, envelope-spectrum-option.ts, orbit-option.ts, trend-option.ts, health-trend-option.ts	Per-type builders	See §3.9.5
waveform-time-axis.ts	DEFAULT_SAMPLE_RATE_HZ 25600, resolveSampleRateHz, generateTimeAxisMs, buildTimeWaveformChartPairs, withGeneratedTimeAxis, analyzeTimeAxis, formatTimeMs	Display-only time-axis regeneration + a diagnostics report explaining <i>why</i> the backend axis was rejected
graph-interactions.ts	adaptiveLineWidth, readDataZoomRange, zoomChart, resetChartZoom, autoscaleChart, applyAdaptiveLineWidth, setCrosshairEnabled	Imperative ECharts control
health-metrics.ts	rms, peak, crestFactor, skewness, kurtosisExcess, computeHealthMetrics, computeScalarFromSamples, buildThresholdRows, channelLabel, extractWaveformSamples	Client-side DSP for the plots-derived health view
health-feature-fallback.ts	mapHealthLevelToFeatureStatus, buildFallbackFeatureItems, summarizeFeatureItems, deriveHealthState, buildFallbackChannelOverview	Fallbacks when the feature API returns nothing
health-feature-compare-fallback.ts	buildCompareItemsFromPlots	Client-side baseline comparison from raw plots
vibration-features.ts	VIBRATION_FEATURE_CATALOG (10), FEATURE_CATEGORY_ORDER/LABELS, resolveVibrationFeatureKey, getFeatureDefinition, formatFeatureUnit	The canonical feature catalogue and alias resolver

Module	Exports	Purpose
feature-display.ts	enrichFeatureStatusItems , enrichFeatureCompareItems , groupFeatureStatusItems , groupFeatureCompareItems , formatFeatureValue , formatCompareValue	Guarantees all 10 features render, grouped by category
feature-api-normalize.ts	normalizeUploadFeaturesResponse , normalizeFeatureCompareResponse , formatDifferencePercent	Defensive API adaptation
vibration-settings-defaults.ts	DEFAULT_DEVICE , createDefaultChannels , createDefaultThresholds , createDefaultVibrationSettings	Seeded settings state
vibration-settings-utils.ts	channelLabel , formatThresholdParameterLabel , isChannelFullyConfigured , isChannelPartiallyConfigured , getChannelReadinessStatus , isThresholdValid , getThresholdCoverageStatus , settingsStatesEqual , findThresholdRow , createEmptyChannel , renumberChannels , getDeviceMaxChannelCount , canAddChannelRow , canRemoveChannelRow	Settings domain logic
form-intelligence.ts	FORM_STEPS , getStepCompletion , getFormCompletion , getDataCompleteness , getSensorCoverage , getDiagnosticReadiness , getPMReadiness , getAIReadinessScore , getMissingAlerts , getSensorStatus , getRecommendedActions , getAssetStatusLabel	Client-side readiness scoring
industrial-metadata.ts	CRITICALITY_IMPACT , FIELD_HINTS (17 entries), COMPLETENESS_SECTIONS , MACHINE_SPECS (10 machine types + default), EMPTY_STATE_MESSAGES	Domain copy and guidance
upload-format.ts	getUploadFilename , formatCaptureDate , formatCaptureTime , formatCaptureSelection , formatShortDayLabel , formatRangeLabel , toDateKey , groupUploadsByDay	Capture labelling with date-fns and native fallbacks

3.20 Complete types/ Reference

Table 48 — 3.20 Complete types/ Reference

Module	Contents
auth.ts	LoginRequest , TokenResponse , UserMeResponse
equipment.ts	sensorSchema , equipmentSchema (Zod),inferred SensorFormData / EquipmentFormData , EquipmentListItem , EquipmentOut , AIReadiness , PaginatedEquipment , CRITICALITY_COLORS , CRITICALITY_DOT , ASSET_STATUS_COLORS
measurements.ts	PLOT_TYPES (5), PlotType , PLOT_LABELS , PlotConfig , PlotConfigInput , SensorDataUpload , PlotSeries , AllPlotsResponse
baseline.ts	Baseline , BaselineListResponse , BaselineCreateFromUpload
features.ts	FeatureMonitorStatus , FeatureStatusItem , FeatureSummaryCounts , ChannelHealthOverviewData , FeatureCompareItem , UploadFeaturesResponse , FeatureCompareResponse
factor-trends.ts	FactorTrendSeries , UploadFactorTrendsResponse
health-status.ts	HEALTH_CHANNEL_COUNT 8, HealthMetricKey (10), HealthStatusCardKey (7), HEALTH_STATUS_CARD_KEYS , TREND_TAB_METRIC_KEYS (6), HealthStatusLevel , HealthMetricTrend , HealthThresholdRow , HealthStatusSnapshot
vibration-settings.ts	VIBRATION_CHANNEL_COUNT 8, ChannelAxis , ChannelDataType , EngineeringUnit , ThresholdParameter (= VibrationFeatureKey), THRESHOLD_PARAMETERS (derived from the feature catalogue), ChannelConfig , ThresholdConfig , VibrationDeviceInfo , VibrationSettingsState , ChannelReadinessStatus , ThresholdCoverageStatus , SettingsModuleId
analysis-tabs.ts	AnalysisTabId , ANALYSIS_TABS (4 entries with Lucide icons)

4.0 Backend Documentation

4.1 Folder Structure and Layering

backend/app/

- config.py
- database.py
- main.py
- dependencies/
- models/
- schemas/
- crud/
- routers/
- services/

- ← settings (env-bound)
- ← engine / session / Base / get_db
- ← application assembly
- ← FastAPI Depends providers (auth)
- ← SQLAlchemey ORM (persistence shape)
- ← Pydantic (wire shape)
- ← data access (no HTTP, no business rules)
- ← HTTP endpoints (no SQL, no DSP)
- ← business logic and algorithms

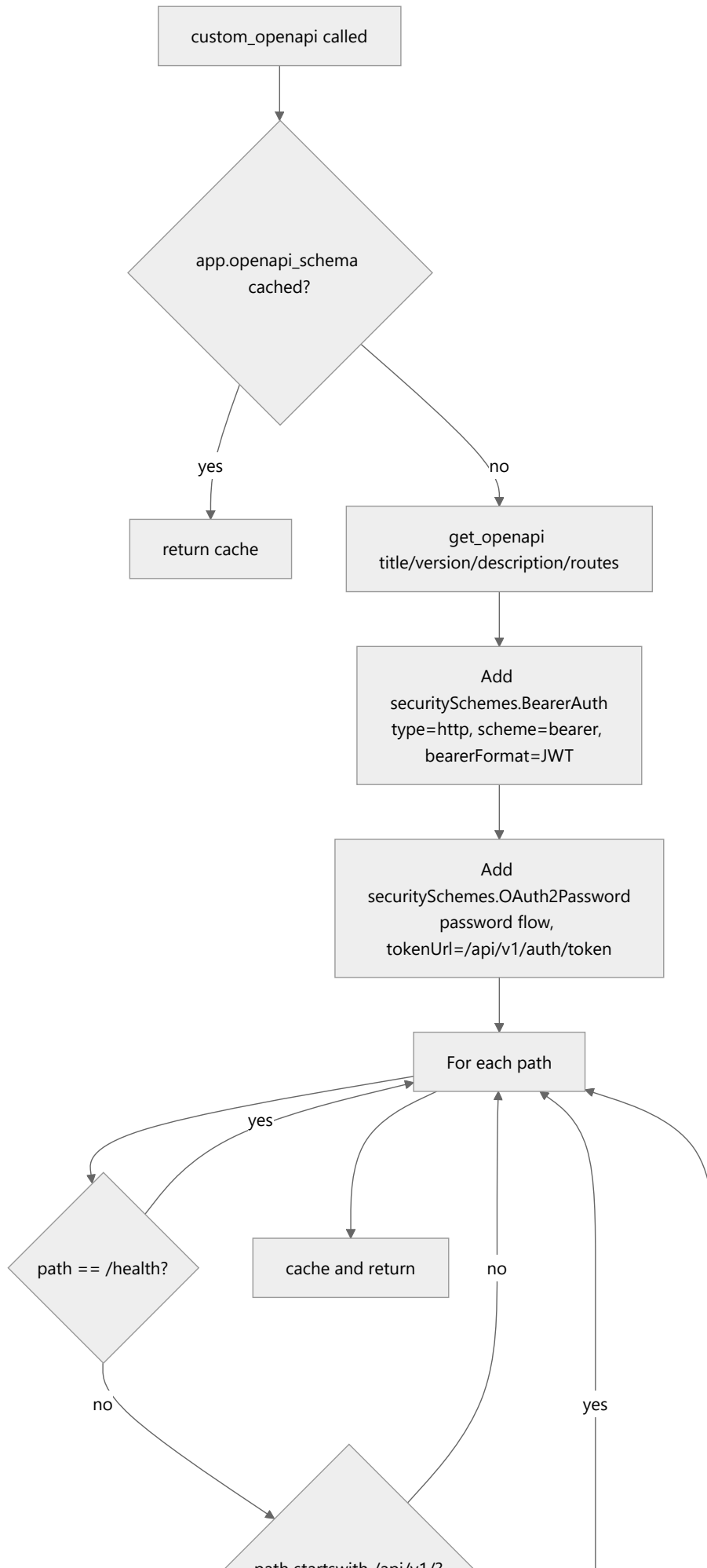
Dependency direction is strictly downward: `routers` → `services` → `crud` → `models`. `schemas` is imported by `routers` and `services`; `crud` never imports `routers`; `services` never raise `HTTPException` (they raise `ValueError`, which `routers` translate).

4.2 Application Startup — `app/main.py`

Table 49 — 4.2 Application Startup — `app/main.py`

Element	Detail
<code>lifespan</code>	Async context manager. Opens a <code>SessionLocal</code> , calls <code>seed_super_admin(db)</code> then <code>seed_role_users(db)</code> , closes the session in <code>finally</code> , then <code>yield s</code> . No shutdown logic.
<code>FastAPI(...)</code>	<code>title="AI Vibration Intelligence Platform"</code> , <code>version="1.1.0"</code> , and a Markdown <code>description</code> containing Swagger login instructions and a table of the three dev accounts
<code>CORSMiddleware</code>	6 allowed origins, all methods, all headers, credentials enabled
<code>include_router</code>	<code>auth</code> , <code>baselines</code> , <code>equipment</code> , <code>lookups</code> , <code>measurements</code>
Directory creation	<code>os.makedirs(settings.upload_dir)</code> and <code>os.makedirs(settings.measurement_upload_dir)</code> at import time
<code>custom_openapi()</code>	Post-processes the generated schema
<code>GET /health</code>	Returns <code>{"status": "ok", "service": "AI Vibration Intelligence Platform"}</code> ; unauthenticated

4.2.1 `custom_openapi()` in detail



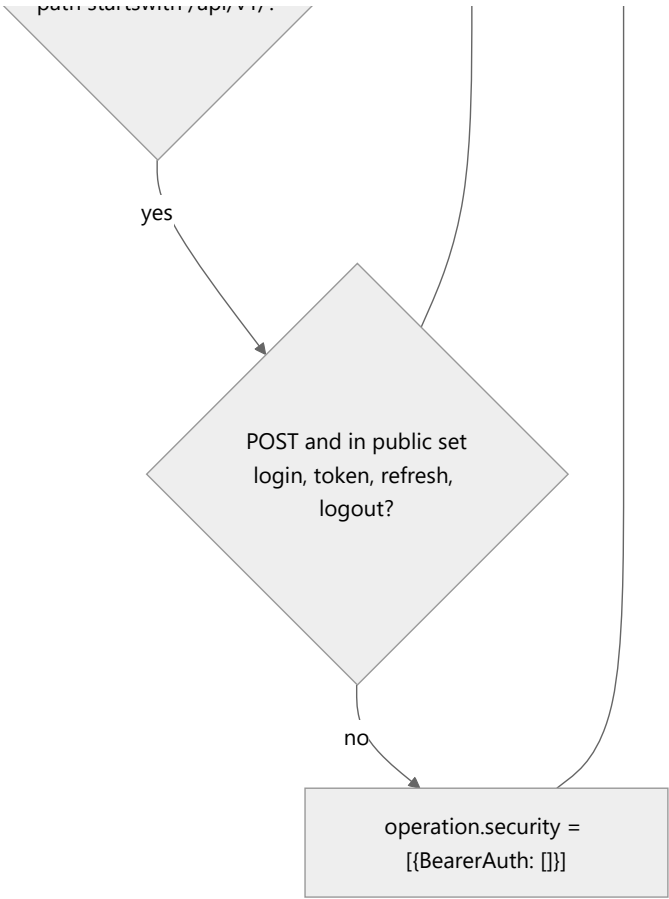


Figure 17 — 4.2.1 custom_openapi() in detail

The effect is that Swagger UI shows a padlock on every protected operation and lets a developer authorise once with a pasted access token.

4.3 Configuration — app/config.py

```
class Settings(BaseSettings):
    database_url: str # REQUIRED - no default
    secret_key: str = "change-in-production"
    upload_dir: str = "uploads"
    measurement_upload_dir: str = "uploads/measurements"
    max_image_size_mb: int = 10
    max_pdf_size_mb: int = 50
    jwt_secret: str = ""
    jwt_algorithm: str = "HS256"
    jwt_access_expire_minutes: int = 30
    jwt_refresh_expire_days: int = 7
    initial_admin_email/password/name
    seed_admin_email/password/name
    seed_user_email/password/name
    @property
    def effective_jwt_secret(self) -> str:
        return self.jwt_secret or self.secret_key
class Config:
    env_file = ".env"
    extra = "ignore"
```

Table 50 — 4.3 Configuration — app/config.py

Design point	Consequence
database_url has no default	The process refuses to start without it — a fail-fast guard against silently connecting to the wrong database
effective_jwt_secret	JWT_SECRET can be rotated independently of SECRET_KEY ; if unset, SECRET_KEY is used
extra = "ignore"	The shared root .env contains Postgres/pgAdmin/Vite keys that are not Settings fields; ignoring extras prevents startup failure
env_file = ".env"	Read relative to the process CWD — hence copy ../.env .env in START.md and setup_and_run.bat
Environment variables win over the file	Docker Compose passes DATABASE_URL , SECRET_KEY , UPLOAD_DIR=/app/uploads , JWT_SECRET , and the six seed variables explicitly

4.4 Database Access — app/database.py

```
engine = create_engine(settings.database_url, pool_pre_ping=True)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Table 51 — 4.4 Database Access — app/database.py

Choice	Reason
pool_pre_ping=True	Issues a lightweight <code>SELECT 1</code> before handing out a pooled connection, which prevents "server closed the connection unexpectedly" after idle periods or a Postgres restart
autocommit=False	Explicit transaction boundaries; every CRUD write calls <code>db.commit()</code>
autoflush=False	Prevents surprise flushes mid-query; the code flushes explicitly with <code>db.flush()</code> where an ID is needed before commit (e.g. <code>create_equipment</code> , <code>create_user</code>)
Generator dependency	FastAPI guarantees the <code>finally</code> block runs after the response is produced, so sessions are always returned to the pool

No global exception handler exists, so an uncaught exception rolls back implicitly on session close and returns a FastAPI 500.

4.5 Dependencies — app/dependencies/auth.py

Table 52 — 4.5 Dependencies — app/dependencies/auth.py

Symbol	Type	Behaviour
WRITE_ROLES	{"super_admin", "admin"}	Roles permitted to mutate data
bearer_scheme	HTTPBearer(auto_error=False)	<code>auto_error=False</code> lets the code emit its own 401 body and <code>WWW-Authenticate</code> header
_user_role(user)	str	Prefers <code>user.role</code> ; otherwise derives from the <code>roles</code> relationship, mapping <code>super_admin</code> → <code>super_admin</code> , any of <code>admin</code> / <code>plant_admin</code> / <code>engineer</code> → <code>admin</code> , else <code>user</code>
get_current_user(credentials, db)	User	401 when: no credentials, non-bearer scheme, decode failure, bad <code>sub</code> , user missing, <code>is_active == False</code>
require_write_access(current_user)	User	403 "Insufficient permissions" when <code>_user_role(...)</code> is not in <code>WRITE_ROLES</code>

`require_write_access` depends on `get_current_user`, so listing it in a route's `dependencies=[...]` implies authentication as well.

4.6 Models — app/models/

`models/__init__.py` re-exports every model class. `alembic/env.py` imports `app.models` precisely so that `Base.metadata` is fully populated for autogenerate and `target_metadata` comparison.

4.6.1 Equipment (equipment_masters)

Table 53 — 4.6.1 Equipment (equipment_masters)

Group	Columns
Identity	<code>id</code> UUID PK (default <code>uuid.uuid4</code>)
Location	<code>plant_name</code> , <code>area</code> , <code>line</code> — all <code>String(255)</code> NOT NULL
Asset	<code>machine_name</code> (255, NOT NULL), <code>machine_id</code> (100, nullable, unique , indexed), <code>machine_type</code> (50, NOT NULL), <code>machine_criticality</code> (20, NOT NULL), <code>manufacturer</code> , <code>model</code> , <code>serial_number</code>
Mechanical	<code>rated_power_kw</code> Numeric(10,2), <code>rated_rpm</code> Integer, <code>drive_type</code> , <code>load_type</code> , <code>foundation_type</code> , <code>coupling_details</code>
Rotating	<code>bearing_details</code> Text, <code>bearing_number_de/nde</code> , <code>gearbox_ratio</code> Numeric(8,3), <code>gear_teeth</code> , <code>motor_pole_count</code> , <code>fan_blades</code> , <code>pump_vanes</code> , <code>direction_of_rotation</code>
Operating	<code>operating_speed_min/max</code> , <code>load_range_min/max</code> Numeric(5,2), <code>normal_operating_load</code> Numeric(5,2), <code>process_details</code> Text, <code>operating_environment</code> ARRAY(String)
Lubrication	<code>lubrication_type</code> , <code>installation_date</code> Date, <code>last_maintenance_date</code> Date, <code>maintenance_notes</code> Text
Image	<code>equipment_image_path</code> String(500)
AI readiness	<code>asset_status</code> (default "Active"), <code>machine_train_configured</code> , <code>bearing_database_mapped</code> , <code>operating_mode_configured</code> — Booleans defaulting to <code>False</code>
Timestamps	<code>created_at</code> , <code>updated_at</code> (<code>onupdate=datetime.utcnow</code>)
Relationship	<code>sensors</code> → <code>SensorConfiguration</code> , <code>cascade="all, delete-orphan"</code>

4.6.2 SensorConfiguration (`sensor_configurations`)

`id`, `equipment_id` (FK CASCADE, NOT NULL), `sensor_type` (NOT NULL), `mounting_location` (NOT NULL), `orientation` (NOT NULL), `mounting_method`, `sensitivity` Numeric(10,4), `sensitivity_unit`, `sampling_rate` (string label), `sampling_rate_custom` Integer, `frequency_range`, `frequency_range_custom_min/max`, `is_active` (default `True`), `device_id` String(64) **unique + indexed**, `created_at`. Back-reference `equipment`.

`device_id` is the join key for the edge acquisition API and is deliberately a free-form MAC-style string rather than a UUID.

4.6.3 Measurement models (`models/measurement.py`)

Table 54 — 4.6.3 Measurement models (*models/measurement.py*)

Model	Table	Purpose	Notable columns
PlotConfiguration	plot_configurations	One processing profile per sensor	sensor_id unique , channel_count, active_channel, sampling_rate_hz Numeric(12,4) default 25600, fft_lines default 1600, frequency_max_hz, data_type default acceleration, enabled_plots JSONB
SensorDataUpload	sensor_data_uploads	Upload lifecycle record	pdf_path, parsed_data_path, sample_count, three status triplets (parse_*, plots_*, features_*), original_filename, source (default manual), created_at, parsed_at
PlotResult	plot_results	Cached plot series	plot_type, channel, title, x_label, y_label, x_data / y_data JSONB, metadata_ mapped to column metadata, point_count, sampling_rate_hz, fft_lines, frequency_max_hz, config_fingerprint, status
MeasurementUploadData	measurement_upload_data	Durable copy of the file	upload_id unique , file_content LargeBinary, parsed_data JSONB, file_format, channel_count, sample_count
SensorBaseline	sensor_baselines	Append-only reference captures	source_upload_id (FK SET NULL), name, description, labels JSONB, file_content, parsed_data, sampling_rate_hz, is_primary, captured_at
BaselinePlotResult	baseline_plot_results	Cached baseline plots	Same shape as PlotResult, keyed by baseline_id
FeatureDefinition	feature_definitions	Feature catalogue	code unique , name, unit, description, sort_order, is_active
FeatureThresholdRule	feature_threshold_rules	Evaluation rules	feature_code FK, rule_type, machine_type (nullable = global), normal_max, warning_max, normal_min, warning_min, metadata_, is_active
MeasurementChannelFeature	measurement_channel_features	Scalar feature values	upload_id, sensor_id, channel, feature_code, value Numeric(18,8), unit, status, metadata_, computed_at
MeasurementChannelFeatureTrend	measurement_channel_feature_trends	Per-segment series	Adds segment_index, time_s
BaselineChannelFeature	baseline_channel_features	Baseline feature values	Same as measurement features but keyed by baseline_id; status defaults to normal

The `metadata_` **naming pattern**. `metadata` is reserved on SQLAlchemy declarative classes, so the attribute is named `metadata_` and explicitly mapped: `Column("metadata", JSONB, ...)`. Pydantic response models expose it back as `metadata`.

4.6.4 User models (`models/user.py`)

Table 55 — 4.6.4 User models (`models/user.py`)

Model	Table	Columns
Role	roles	id, name (unique, indexed), description, created_at
User	users	id, email (unique, indexed), password_hash, full_name, role (String(50), default "user"), is_active, must_change_password, last_login_at, created_at, updated_at
UserRole	user_roles	Composite PK (user_id, role_id), assigned_at
RefreshToken	refresh_tokens	id, user_id (indexed), token_hash (String(64), unique, indexed), expires_at, revoked_at, created_at

`User.roles` is a many-to-many via `secondary="user_roles"`. The scalar `users.role` column (added in migration 008) is the **authoritative** value used by `_user_role`, `create_access_token`, and `user_to_me_dict`; the M2M table is the legacy path retained for backwards compatibility.

4.7 Schemas — `app/schemas/`

4.7.1 `schemas/equipment.py`

- `SensorConfigBase` — all sensor fields; `device_id` carries `max_length=64` and a description.
- `SensorConfigCreate` — inherits base unchanged.
- `SensorConfigUpdate` — every field `Optional` for PATCH semantics.
- `SensorConfigOut` — adds `id`, `equipment_id`, `created_at`; `from_attributes = True`.
- `EquipmentBase` — 40 fields. Location and asset identity default to `""` rather than being required, which lets the frontend save partial drafts. Contains one validator:

```
@field_validator("machine_id", mode="before")
def normalize_machine_id(cls, v): return None if v == "" else v
```

This converts an empty string to `NULL`, which is essential because `machine_id` is `UNIQUE` — many empty strings would collide, many `NULL`s do not.

- `EquipmentCreate` — adds `sensors: Optional[List[SensorConfigCreate]] = []`.
- `EquipmentUpdate` — 36 optional fields; used with `model_dump(exclude_unset=True)` so omitted keys are untouched.
- `EquipmentOut`, `EquipmentListItem`, `AIReadinessOut`, `PaginatedEquipment`.

4.7.2 schemas/measurement.py

```
PLOT_TYPES = ["time_waveform", "circular_time_waveform", "fft_spectrum", "envelope_spectrum", "trend_plot"]
PLOT_TYPE_ALIASES = {"psd": "circular_time_waveform", "rms_trend": "trend_plot"}
DATA_TYPES = ["acceleration", "velocity", "displacement"]
```

`PlotConfigBase` constraints: `channel_count` 1–32, `active_channel` ≥ 0 , `sampling_rate_hz` > 0 , `fft_lines` 64–65536, `frequency_max_hz` > 0 .

Four validators:

1. `validate_data_type` — membership in `DATA_TYPES`.
2. `validate_plots` — canonicalises aliases, drops unknowns, de-duplicates, and rejects an empty result.
3. `PlotConfigCreate.validate_channel_index` (model validator) — `active_channel < channel_count`.
4. `PlotConfigOut.normalize_plots_on_read` + `clamp_channel_on_read` — repairs legacy rows on read: unknown plot types are dropped, an empty list becomes all five types, and an out-of-range `active_channel` is clamped to `channel_count - 1` using `object.__setattr__`.

`SensorDataUploadOut` carries a non-persisted `has_stored_data: bool` that routers populate from a `measurement_upload_data` existence check.

4.7.3 schemas/feature.py

Eight models: `ChannelFeatureOut`, `FeaturesSummaryOut`, `ChannelHealthOverviewOut`, `UploadFeaturesOut`, `FactorTrendSeriesOut`, `UploadFactorTrendsOut`, `BaselineFeaturesOut`, `FeatureCompareItemOut`, `FeatureCompareOut`.

4.7.4 schemas/auth.py, schemas/baseline.py, schemas/acquisition.py

- `LoginRequest` uses `EmailStr` (so a malformed email returns 422 before any DB access) and `password: str = Field(min_length=1)`.
- `TokenResponse` — `access_token`, `refresh_token`, `token_type="bearer"`, `expires_in` (seconds).
- `UserMeResponse` — includes `roles: List[str]` and `plants: List[str]`.
- `BaselineOut` adds two computed fields, `plot_count` and `plots_status`, filled by the router helper.
- `EdgeAcquisitionConfigOut` mirrors the Sensovibe edge JSON contract exactly, including camelCase field names (`acquisitionFormula`, `totalChannelCount`, `windowType`, `overlapPercentage`) and the `platformSensorId` field documented as "Internal UUID — use for upload API until device_id upload is added".

4.8 CRUD Layer — app/crud/

4.8.1 crud/equipment.py

Table 56 — 4.8.1 crud/equipment.py

Function	Behaviour
<code>get_equipment_list(db, page, page_size, plant_name, machine_type, machine_criticality)</code>	<code>plant_name</code> uses <code>ILIKE %value%</code> ; type and criticality use equality; returns (total, items) with <code>ORDER BY created_at DESC</code> and offset/limit
<code>get_equipment_by_id</code> , <code>get_equipment_by_machine_id</code>	Single-row lookups
<code>create_equipment(db, data)</code>	<code>model_dump(exclude={"sensors"})</code> → <code>Equipment</code> ; <code>db.flush()</code> to obtain the id; inserts each nested sensor; single <code>commit</code>
<code>update_equipment(db, id, data)</code>	<code>model_dump(exclude_unset=True)</code> then <code>setattr</code> per field
<code>delete_equipment</code>	<code>db.delete</code> → ORM cascade removes sensors, and <code>DB ON DELETE CASCADE</code> removes all descendants
<code>update_image_path(db, id, path)</code>	Sets <code>equipment_image_path</code> (also used with <code>None</code> to clear)
<code>get_sensors_by_equipment</code> , <code>get_sensor_by_id</code> , <code>get_sensor_by_device_id</code> , <code>create_sensor</code> , <code>update_sensor</code> , <code>delete_sensor</code>	Sensor CRUD
<code>compute_ai_readiness(equipment)</code>	Five boolean checks → <code>score = int(sum(checks)/len(checks)*100)</code>

The five readiness checks are: `machine_train_configured`, `asset_status` not in `(None, "")`, `len(sensors) > 0`, `bearing_database_mapped`, and `operating_speed_min` is not `None` and `operating_speed_max` is not `None`. Each is worth 20 %.

4.8.2 crud/measurement.py

Plot-config functions (`get_plot_config_by_sensor`, `create_plot_config` — which raises `ValueError` if one already exists — `update_plot_config`, `upsert_plot_config`), upload-lifecycle functions (`create_upload_record`, `mark_upload_parsed`, `mark_upload_failed`, `mark_upload_plots_ready`, `mark_upload_plots_failed`, `get_upload_by_id`), listing (`list_uploads_by_sensor` with date/status filters and pagination, `get_stored_upload_ids`), and plot-result access (`get_plot_results`, `delete_plot_results`).

Date filtering uses `_date_start(d) = datetime.combine(d, time.min)` and `_date_end(d) = datetime.combine(d, time.max)` so a single-day range is inclusive of the whole day.

`config_to_dict` converts a `PlotConfiguration` row into the plain dict the services expect; `default_config_dict(channel_count)` supplies `{channel_count, active_channel: 0, sampling_rate_hz: 25600.0, fft_lines: 1600, frequency_max_hz: None, data_type: "acceleration", enabled_plots: PLOT_TYPES}` when no configuration row exists.

4.8.3 crud/baseline.py

`save_upload_data`, `get_upload_data_by_upload_id`, `create_baseline` (clears the previous primary flag first when `set_as_primary`), `get_baseline_by_id`, `list_baselines_by_sensor` (newest first), `get_primary_baseline`, `set_baseline_primary`, `delete_baseline_plot_results`, `count_baseline_plot_results` (only `status == "ready"`), `get_baseline_plot_results`.

4.8.4 crud/feature.py

`get_active_threshold_rules(db, machine_type)` implements a **specific-then-global** lookup: if a `machine_type` is supplied and machine-specific active rules exist, they are returned; otherwise the rules with `machine_type IS NULL` are returned. (The current caller passes no machine type, so the global set is always used.)

Also: `get_feature_definitions` (active, ordered by `sort_order`), `get_definition_map`, `delete_measurement_features`, `delete_measurement_feature_trends`, `get_measurement_features`, `get_measurement_feature_trends`, `delete_baseline_features`, `get_baseline_features`, `mark_upload_features_ready`, `mark_upload_features_failed`.

4.8.5 crud/user.py

`SUPPORTED_ROLES = ["super_admin", "admin", "user"]`. `primary_role(role_names)` collapses a role list to one canonical role. `get_user_by_email` **lowercases** the input before comparison, and `create_user` lowercases on write — so email is effectively case-insensitive. All user reads use `joinedload(User.roles)` to avoid N+1 queries. Token functions: `create_refresh_token_record`, `get_refresh_token_by_hash` (with a nested `joinedload` down to `User.roles`), `revoke_refresh_token`, `revoke_all_user_refresh_tokens`, `super_admin_exists`.

4.9 Services — app/services/

4.9.1 auth_service.py

Table 57 — 4.9.1 auth_service.py

Function	Detail
<code>pwd_context</code>	<code>CryptContext(schemes=["bcrypt"], deprecated="auto")</code>
<code>hash_password(p)</code> / <code>verify_password(p, h)</code>	bcrypt via passlib
<code>create_access_token(user_id, roles)</code>	Payload <code>{sub, exp, type:"access", roles}</code> ; HS256; returns <code>(token, expires_in_seconds)</code>
<code>decode_access_token(token)</code>	Verifies the signature and <code>exp</code> , then rejects any token whose <code>type != "access"</code>
<code>_hash_refresh_token(t)</code>	<code>hashlib.sha256(t.encode()).hexdigest()</code> — 64 hex chars, matching <code>String(64)</code>
<code>create_refresh_token(db, user_id)</code>	<code>secrets.token_urlsafe(48)</code> (≈64 chars, 384 bits of entropy); stores only the hash
<code>validate_refresh_token(db, plain)</code>	Raises <code>ValueError</code> for: unknown hash, revoked, expired (naive datetimes are coerced to UTC), inactive user
<code>authenticate_user(db, email, password)</code>	Returns <code>None</code> for unknown user, inactive user, or wrong password — the caller emits one generic 401 so the three cases are indistinguishable to an attacker
<code>user_to_me_dict(user)</code>	Prefers the scalar <code>user.role</code> ; falls back to the M2M names or <code>["user"]</code> ; <code>plants</code> is always <code>[]</code>

4.9.2 seed.py

`seed_super_admin(db)` :

- If a super admin already exists, it additionally clears a stale `must_change_password` flag on the configured `INITIAL_ADMIN_EMAIL` account (a legacy-data repair) and returns.
- Otherwise, if both `INITIAL_ADMIN_EMAIL` and `INITIAL_ADMIN_PASSWORD` are set, it creates the account with `role_names=["super_admin"]` and `must_change_password=False`; if not, it logs a warning and does nothing.

`seed_role_users(db)` calls `_seed_user_if_missing` twice, for `SEED_ADMIN_*` (role `admin`) and `SEED_USER_*` (role `user`). Both are no-ops when the env values are blank or the email already exists. The whole routine is therefore idempotent and safe on every restart.

4.9.3 pdf_parser.py

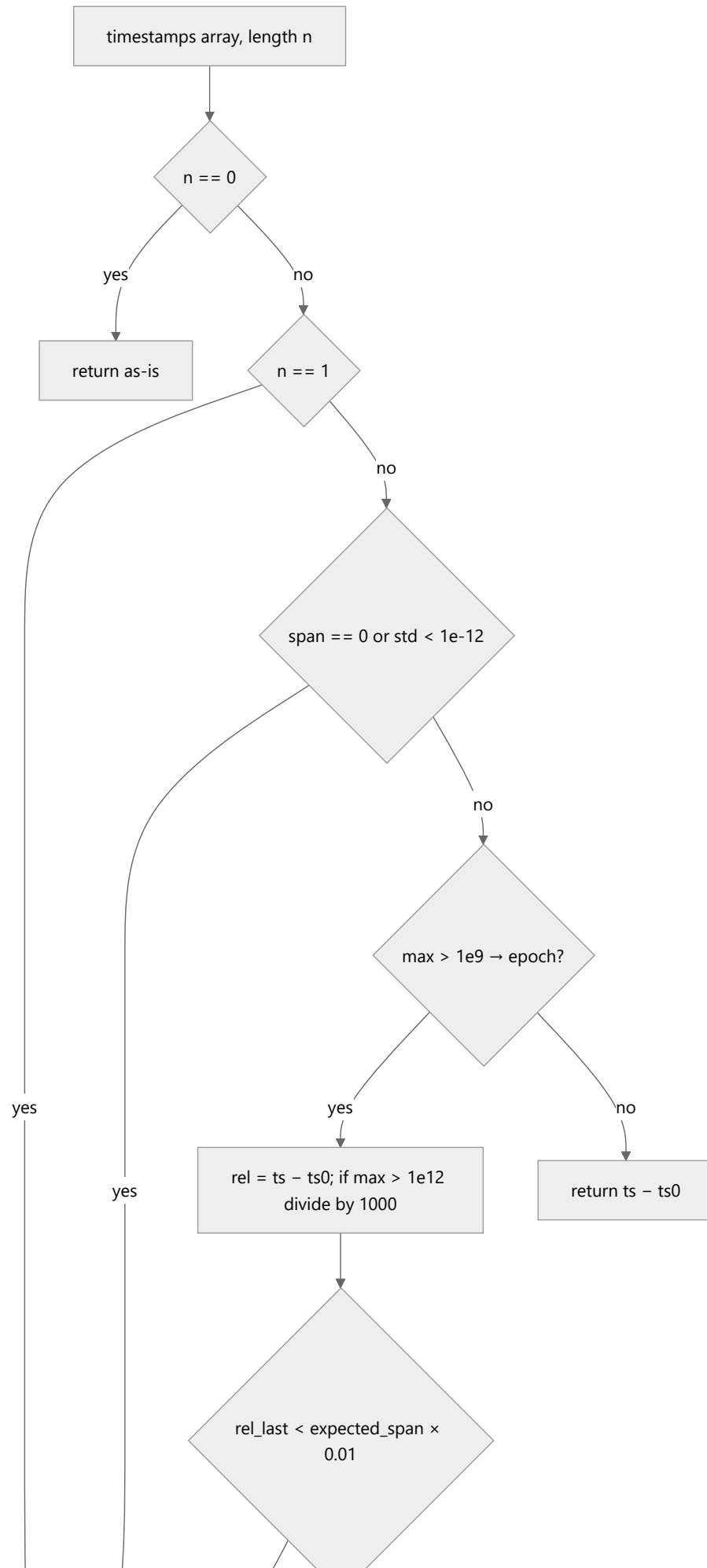
Supported inputs (from the module docstring): `timestamp_,ch0,ch1,...` (Excel export), `timestamp,ch0,ch1,...`, and tab- or space-separated rows with epoch or float timestamps.

Table 58 — 4.9.3 pdf_parser.py

Function	Behaviour
<code>_clean_cell(v)</code>	Strips whitespace, <code>"</code> , <code>'</code> , and the BOM
<code>_split_line(line)</code>	Delimiter precedence: tab → semicolon (only when no comma) → comma → any whitespace run
<code>_is_timestamp_header(cell)</code>	Normalised membership in <code>{timestamp,time,t,index,sample,datetime,date}</code> or any cell starting with <code>timestamp</code>
<code>_is_header_row(parts)</code>	True when the first cell is a timestamp header or any later cell matches <code>^ch(\d+)\$</code> (case-insensitive)
<code>_detect_channel_count_from_header/_text</code>	Counts <code>chN</code> columns
<code>_parse_numeric_row(parts, n)</code>	<code>float()</code> the timestamp and the first <code>n</code> values; pads short rows with <code>0.0</code> ; returns <code>None</code> on <code>ValueError</code> / <code>IndexError</code>
<code>parse_measurement_text(text, channel_count)</code>	Detects the effective channel count (header wins over the argument), iterates lines, skips blanks/headers/unparseable rows, and appends <code>0.0</code> for missing channels. Raises <code>ValueError</code> with a diagnostic message when no row parsed
<code>parse_sensor_pdf(path, n)</code>	<code>pdfplumber</code> : prefers <code>page.extract_tables()</code> (Excel→PDF preserves table structure) and joins cells with commas; falls back to <code>page.extract_text()</code> . Raises if nothing extractable
<code>parse_sensor_csv(path, n)</code>	Tries encodings <code>utf-8-sig</code> , <code>utf-8</code> , <code>latin-1</code> in order
<code>parse_sensor_file(path, n)</code>	Dispatches on extension; raises for anything other than <code>.csv</code> / <code>.pdf</code>
<code>parse_pdf_text</code>	Backwards-compatible alias of <code>parse_measurement_text</code>

4.9.4 signal_processing.py

`resolve_time_seconds(timestamps, fs)` — the timestamp heuristic, applied in order:



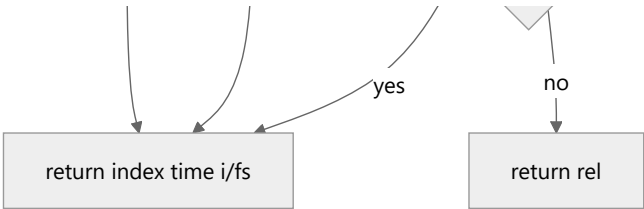


Figure 18 — 4.9.4 signal_processing.py

The `x * 0.01` test catches the common case where every row in a batch shares one epoch second: the apparent span is far smaller than `(n-1)/fs`, so index-derived time is used instead.

Table 59 — 4.9.4 signal_processing.py

Function	Output
<code>compute_time_waveform(ts, samples, fs)</code>	<code>{x: resolved time (s), y: samples, x_label:"Time (s)", y_label:"Amplitude", title:"Time Waveform", metadata:{plot_style:"line"}}</code>
<code>compute_circular_time_waveform(ts, samples, max_points=2048)</code>	Decimates by striding, then maps <code>θ = linspace(0, 2π, n, endpoint=False)</code> , <code>x = A*cos θ</code> , <code>y = A*sin θ</code> . Requires ≥ 4 samples
<code>compute_fft_spectrum(samples, fs, fft_lines, frequency_max_hz)</code>	Truncates to <code>min(fft_lines, n)</code> , applies a Hann window (<code>np.hanning</code>), computes <code>abs(fft(windowed))[:n//2] * 2/n</code> (single-sided amplitude scaling), builds <code>fftfreq(n, 1/fs)[:n//2]</code> , optionally masks to <code>frequency_max_hz</code> . Metadata records <code>fft_lines</code> and <code>sampling_rate_hz</code>
<code>compute_envelope_spectrum(...)</code>	Hilbert transform $\rightarrow$ <code>abs(analytic)</code> $\rightarrow$ subtract the mean (removes the DC pedestal) $\rightarrow$ FFT of the envelope. Retitled "Envelope Spectrum", y-label "Envelope Magnitude"
<code>compute_trend_plot(ts, samples, fs, num_segments=32)</code>	Splits into 32 equal segments, computes RMS per segment, and places each point at the segment mid-time

4.9.5 plot_generator.py

`PLOT_COMPUTERS` is a dict of five lambdas mapping a plot type to its signal-processing call with the right configuration arguments — the registry that makes adding a sixth plot type a one-line change.

Table 60 — 4.9.5 plot_generator.py

Function	Behaviour
<code>normalize_plot_types(enabled)</code>	Canonicalises aliases, filters to known computers, de-duplicates, falls back to all five
<code>load_parsed_data(path)</code> / <code>save_parsed_data(path, data)</code>	JSON I/O; <code>save</code> creates parent directories
<code>resolve_active_channel(parsed, requested)</code>	Returns the requested channel if it exists in the parsed data, else the lowest available channel, else 0
<code>generate_plot(parsed, plot_type, channel, config)</code>	Canonicalise $\rightarrow$ validate $\rightarrow$ resolve channel $\rightarrow$ look up samples $\rightarrow$ run the computer $\rightarrow$ wrap in <code>PlotSeriesOut</code>
<code>generate_all_plots(upload_id, sensor_id, path, config)</code>	Loads, resolves the active channel once, and generates every enabled plot for that channel

4.9.6 plot_storage.py

```
ALGORITHM_VERSION = "v1"

def compute_config_fingerprint(config) -> str:
    payload = {
        "algorithm_version": ALGORITHM_VERSION,
        "sampling_rate_hz": float(config["sampling_rate_hz"]),
        "fft_lines": config.get("fft_lines"),
        "frequency_max_hz": config.get("frequency_max_hz"),
        "data_type": config.get("data_type", "acceleration"),
        "enabled_plots": sorted(normalize_plot_types(config.get("enabled_plots"))),
    }
    return hashlib.sha256(json.dumps(payload, sort_keys=True, default=str).encode()).hexdigest()[:32]
```

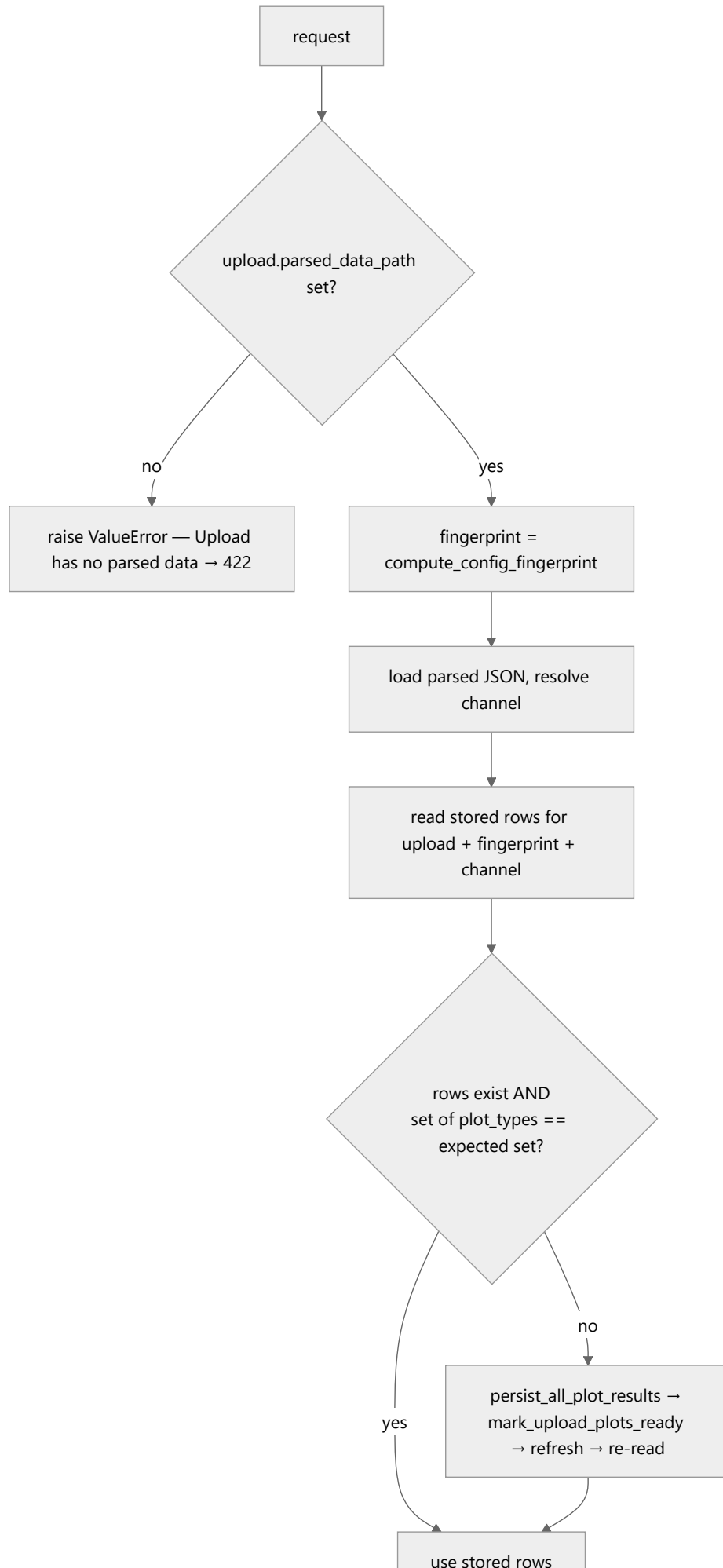
Note what is **excluded**: `active_channel` and `channel_count`. Changing the viewed channel must not invalidate the cache, because plots are computed for *all* channels and stored per channel.

```
persist_all_plot_results(db, upload, parsed_path, config):

    1. Load parsed JSON, compute the fingerprint, normalise the enabled list.
    2. delete_plot_results(upload.id, fingerprint) — idempotent re-computation.
```

3. For each available channel × each enabled plot type: `generate_plot` → build a `PlotResult` row.
4. `db.add_all(rows)` + `commit`. Returns the row count.

`get_or_load_all_plots(db, upload, config, channel)` — the cache-or-compute read path:



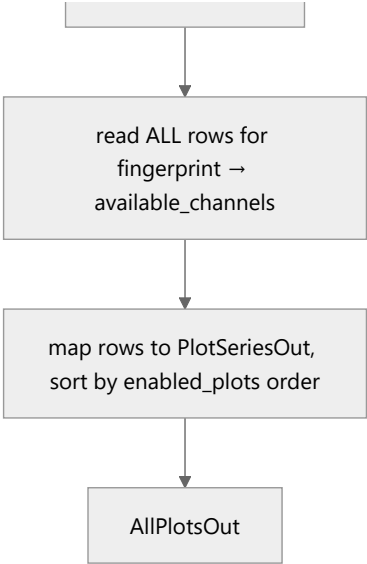


Figure 19 — 4.9.6 `plot_storage.py`

The set comparison (not merely "any rows exist") means enabling a sixth plot type later automatically triggers recomputation.

`get_or_load_single_plot` delegates to the above and filters, raising `ValueError` when the requested type is absent for the channel.

4.9.7 `baseline_storage.py`

`persist_baseline_plot_results(db, baseline, parsed_data, config)` mirrors `persist_all_plot_results` but takes the parsed dict directly from the baseline row (no filesystem dependency) and writes `BaselinePlotResult` rows. `baseline_plot_to_series(row)` maps a row back to `PlotSeriesOut`.

4.9.8 `feature_extraction.py`

Constants: `SHAFT_FREQ_MIN_HZ = 5.0`, `SHAFT_FREQ_MAX_HZ = 120.0`, `FFT_BAND_MAX_HZ = 500.0`, `SEGMENT_COUNT = 32`.

The ten features computed by `extract_channel_features(samples, fs)`:

Table 61 — 4.9.8 `feature_extraction.py`

#	feature_code	Formula	Unit
1	rms	$\sqrt{\text{mean}(x^2)}$	scaled_eng
2	peak	$\max()$	x
3	crest_factor	peak / rms (0 when $\text{rms} < 1\text{e-}30$)	dimensionless
4	kurtosis	Excess kurtosis $\mu_4 / \mu_2^2 - 3$ (0 when variance $< 1\text{e-}30$)	dimensionless
5	fft_band_energy_0_500	$\sum \text{spectrum}^2$ over $f \leq 500$ Hz	scaled_eng_sq
6	amplitude_1x	Magnitude at the estimated shaft frequency	scaled_eng
7	amplitude_2x	Magnitude at $2 \times f_{\text{shaft}}$	scaled_eng
8	amplitude_3x	Magnitude at $3 \times f_{\text{shaft}}$	scaled_eng
9	envelope_rms	$\sqrt{\text{mean}(\text{hilbert}(x)^2)}$	hilbert(x)
10	noise_floor	$20 \cdot \log_{10}(\max(\text{mean}(\text{spectrum}), 1\text{e-}30))$	dB

Shaft-frequency estimation (`_estimate_shaft_hz`): take the FFT bin with maximum magnitude inside 5–120 Hz (i.e. 300–7200 RPM, the practical range for industrial rotating machinery). If no bin falls in that band, use the global maximum. `_magnitude_at_freq` then picks the nearest bin via `argmin(|freqs - target|)`. Features 6–8 carry metadata `{estimated_shaft_hz, sampling_rate_hz, sample_count}` so the estimate is auditable.

`extract_segment_trends(samples, fs, num_segments=32)` splits the signal into equal segments, runs the full feature extraction on each, and returns `{trend_x, trend_y, value, unit, metadata}` per feature code, where `trend_x` is the mid-point time of each segment in seconds and `value` is the whole-signal scalar.

Performance characteristic: this function calls `extract_channel_features` once per segment **per feature code**, i.e. $10 \times 32 = 320$ full extractions per channel, each performing an FFT and a Hilbert transform on the segment. This is the dominant cost of the upload pipeline and the reason the frontend polls `features_status` every 3 seconds and sets a 120-second HTTP timeout.

`extract_all_channels` and `extract_all_channel_trends` iterate `range(effective_channel_count)` and skip channels with fewer than 4 samples.

4.9.9 threshold_evaluator.py

Four status constants (`normal` , `warning` , `critical` , `no_baseline`) and a `ThresholdRule` dataclass with `from_row()` for ORM conversion.

`evaluate_feature(code, value, rule, channel_rms, baseline_value)` supports five rule types:

Table 62 — 4.9.9 threshold_evaluator.py

rule_type	Logic
absolute_max	<code>value ≤ normal_max</code> → normal; <code>≤ warning_max</code> → warning; else critical. Returns normal if <code>warning_max</code> is <code>None</code>
absolute_db	Identical arithmetic (dB values are negative, so the ordering still holds)
range	Normal when <code>normal_min ≤ v ≤ normal_max</code> ; warning when <code>warning_min ≤ v ≤ warning_max</code> ; else critical. Missing bounds default to <code>0.0 / inf</code>
percent_rms	<code>pct = 100·v/channel_rms</code> ; <code>< normal_max</code> → normal; <code>< warning_max</code> → warning; else critical. Returns normal when RMS is missing or <code>< 1e-30</code>
percent_baseline	Returns <code>no_baseline</code> when there is no baseline value. Otherwise <code>pct = 100·v/baseline</code> ; <code>≤ normal_max</code> (default 120) → normal; <code>≤ warning_max</code> (default 150) → warning; <code>≤ metadata.critical_percent</code> → warning; else critical
anything else	<code>normal</code> (safe default)

`status_to_health_level(status)` maps to the display strings `Critical` , `Warning` , `Normal` , `No baseline` .

4.9.10 feature_storage.py

Table 63 — 4.9.10 feature_storage.py

Function	Behaviour
<code>_load_parsed_for_upload(db, upload)</code>	Prefers <code>measurement_upload_data.parsed_data</code> (DB); falls back to the JSON file; raises <code>ValueError</code> if neither is available
<code>ensure_upload_features_ready(db, upload, fs)</code>	Returns immediately when <code>features_status == "ready"</code> and rows actually exist (guards against a status/row mismatch). Requires <code>parse_status == "parsed"</code> . Resets a <code>failed</code> status to <code>pending</code> before retrying. Computes, persists, marks ready
<code>_baseline_ref_map(db, sensor_id)</code>	Builds <code>{(channel, feature_code): value}</code> from the sensor's primary baseline; empty dict when there is none
<code>persist_upload_features_and_trends(db, upload, parsed, fs)</code>	The core writer — see below
<code>copy_upload_features_to_baseline(db, upload_id, baseline)</code>	Copies feature rows into <code>baseline_channel_features</code> , forcing <code>status = "normal"</code> (a baseline is by definition the reference)
<code>features_summary_from_rows(rows)</code>	Counts by status plus <code>total</code> ; unknown statuses are counted as <code>normal</code>

`persist_upload_features_and_trends` flow:

1. Load active threshold rules → `{feature_code: rule}` .
2. Build the baseline reference map.
3. `extract_all_channels` and `extract_all_channel_trends` .
4. Delete any existing feature and trend rows for the upload (idempotent recompute).
5. For each channel: capture `channel_rms` , then for each of the 10 codes evaluate the status and build a `MeasurementChannelFeature` ; for each trend point build a `MeasurementChannelFeatureTrend` .
6. `db.bulk_save_objects(...)` for both lists, then a single `commit` . Returns `(feature_rows, trend_rows)` .

`bulk_save_objects` is used deliberately: a typical 8-channel upload writes `8 × 10 = 80` feature rows and `8 × 10 × 32 = 2560` trend rows, and the bulk path skips per-object ORM identity-map overhead.

4.9.11 acquisition_config.py

Defaults used when a sensor has no plot configuration: `DEFAULT_SAMPLE_RATE_HZ = 256_000.0` , `DEFAULT_LOR = 51_200` , `DEFAULT_FMAX_HZ = 15_000.0` , `DEFAULT_CHANNEL_COUNT = 8` , `DEFAULT_WINDOW = "HANNING"` , `DEFAULT_MINUTES = "1"` , `DEFAULT_AVERAGING = 1` , `DEFAULT_OVERLAP = 0` .

`compute_acquisition_formula(sample_rate, lor, overlap, average_count)` returns:

Table 64 — 4.9.11 acquisition_config.py

Field	Formula
<code>frequencyResolutionHz</code>	<code>sample_rate / lor</code>
<code>blockTimeSeconds</code>	<code>lor / sample_rate</code>
<code>requiredSamples</code>	<code>lor</code>
<code>totalAcquisitionTimeSeconds</code>	<code>blockTime × averageCount</code>
<code>stepSizeSamples</code>	<code>lor × (1 - overlap)</code> , floored at <code>lor</code> when the result is ≤ 0
<code>sampleRateHz</code> , <code>overlapDecimal</code> , <code>averageCount</code> , <code>fmaxHz</code> , <code>lor</code>	Pass-through

`_axis_from_orientation` maps the sensor's orientation to `HORIZONTAL` / `AXIAL` / `VERTICAL` (default). `build_channels` emits one entry per channel with `channelIndex` starting at `1` (edge convention) while the platform's own channels are 0-based (`ch0`).

`build_edge_acquisition_config(sensor, plot_config)` prefers the sensor's saved plot configuration and falls back to the defaults, then assembles the full payload, including `sensorId` (the `device_id`, or the sensor UUID as a fallback) and `platformSensorId` (always the UUID).

4.10 Routers — endpoint inventory

Table 65 — 4.10 Routers — endpoint inventory

Router	Prefix	Tag	Router-level dependency	Endpoints
<code>auth</code>	<code>/api/v1/auth</code>	Authentication	none	5
<code>equipment</code>	<code>/api/v1/equipment</code>	Equipment	<code>get_current_user</code>	13
<code>lookups</code>	<code>/api/v1/lookups</code>	Lookups	<code>get_current_user</code>	2
<code>measurements</code>	<code>/api/v1/measurements</code>	Measurements	<code>get_current_user</code>	14
<code>baselines</code>	<code>/api/v1/baselines</code>	Baselines	<code>get_current_user</code>	8
(app)	<code>/health</code>	—	none	1

Full request/response documentation is in **Section 5.0**.

4.11 Middleware, Filters, and Interceptors

The backend registers exactly **one** middleware: `CORSMiddleware` . There is no logging middleware, no rate limiter, no request-ID injector, and no global exception handler.

Cross-cutting behaviour is instead implemented through FastAPI's dependency system:

Table 66 — 4.11 Middleware, Filters, and Interceptors

Concern	Mechanism
Authentication	<code>dependencies=[Depends(get_current_user)]</code> at router level
Authorisation	<code>dependencies=[Depends(require_write_access)]</code> at route level
Session lifecycle	<code>Depends(get_db)</code> generator
Validation	Pydantic request models
Serialisation	<code>response_model=</code> on every route
OpenAPI security metadata	<code>app.openapi = custom_openapi</code>

4.12 Logging

Two call sites use `logging` :

Table 67 — 4.12 Logging

Location	Logger	Messages
<code>services/seed.py</code>	<code>logging.getLogger("uvicorn")</code>	"Super admin already exists — skipping seed", "Seeded super admin user: ...", "Seeded admin user: ...", "Cleared must_change_password for seeded admin (legacy flag): ...", and a warning when the initial-admin env variables are absent
<code>routers/equipment.py::create_equipment</code>	<code>logging.getLogger("uvicorn")</code> (imported inline)	<code>[CREATE_EQUIPMENT]</code> <code>plant_name=...</code> <code>area=...</code> <code>machine_name=...</code> <code>machine_type=...</code>

Everything else relies on Uvicorn's default access log. Alembic logging is configured in `alembic.ini`: root and `sqlalchemy.engine` at `WARN`, `alembic` at `INFO`, formatted as `%(levelname)-5.5s [% (name)s] %(message)s` to `stderr`.

4.13 Caching

There is no in-memory or external cache (no Redis, no `functools.lru_cache`). Caching is **database-backed and content-addressed**:

Table 68 — 4.13 Caching

Cache	Table	Key	Invalidation
Plot series	<code>plot_results</code>	<code>(upload_id, plot_type, channel, config_fingerprint)</code> — a unique constraint	New fingerprint ⇒ new rows; <code>delete_plot_results(upload_id, fingerprint)</code> before each recompute
Baseline plot series	<code>baseline_plot_results</code>	<code>(baseline_id, plot_type, channel, config_fingerprint)</code>	Same pattern
Feature values	<code>measurement_channel_features</code>	<code>(upload_id, channel, feature_code)</code>	Full delete-then-insert per upload
Feature trends	<code>measurement_channel_feature_trends</code>	<code>(upload_id, channel, feature_code, segment_index)</code>	Full delete-then-insert per upload
OpenAPI schema	Process memory	<code>app.openapi_schema</code>	Process restart

On the client, TanStack Query provides a second cache layer with a 30-second default `staleTime`.

4.14 Transactions

Table 69 — 4.14 Transactions

Pattern	Example
Single-statement commit	Most CRUD writes: <code>mutate</code> → <code>db.commit()</code> → <code>db.refresh()</code>
Flush-then-insert-children	<code>create_equipment</code> : <code>add(equipment)</code> → <code>flush()</code> (assigns the PK) → add sensors → single <code>commit()</code> — so equipment and its sensors are atomic
Bulk insert in one transaction	<code>persist_upload_features_and_trends</code> : two <code>bulk_save_objects</code> calls then one <code>commit</code>
Delete-then-insert	<code>persist_all_plot_results</code> commits the delete inside <code>delete_plot_results</code> , then commits the inserts — two transactions , so a crash between them leaves the upload with no cached plots (self-healing: the next read recomputes)
Multi-step without an outer transaction	<code>POST /measurements/upload</code> commits at each stage (record → parsed → upload data → plots → features). A failure part-way leaves an upload row whose status columns record exactly how far it got — the design intentionally favours partial visibility over all-or-nothing

4.15 Dependency Injection

FastAPI's `Depends` is the only DI mechanism. Three provider levels are used:

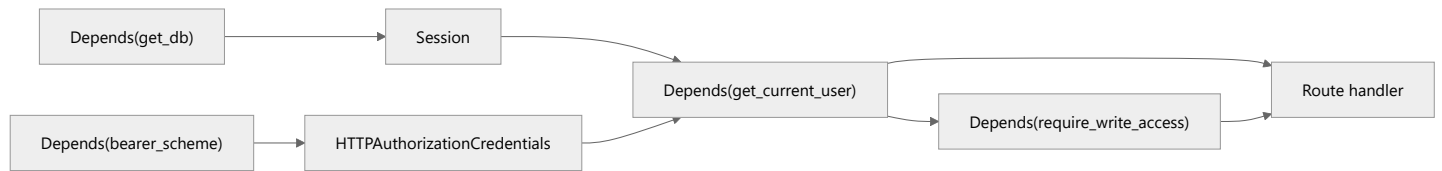


Figure 20 — 4.15 Dependency Injection

Router-level `dependencies=[...]` apply to every route in the router without appearing in the handler signature (used for authentication). Route-level `dependencies=[...]` add write-role enforcement. Handler parameters (`db: Session = Depends(get_db)`) inject values the handler actually uses.

4.16 Scheduler, Async Processing, Background Jobs

None exist. There is no Celery, APScheduler, `BackgroundTasks`, or cron integration. Every operation is synchronous within its request:

- Parsing, plot computation, and feature extraction all run inline inside `POST /measurements/upload`.
- On-demand recomputation happens inside the corresponding `GET (ensure_upload_features_ready, get_or_load_all_plots)`.

The two `async def` handlers (`upload_image`, `upload_sensor_data`, `upload_baseline`) are `async` only because `await file.read()` is required by Starlette's `UploadFile`; all subsequent work is CPU-bound and blocking.

Operational consequence. Under Unicorn's default single worker, a large upload's feature extraction blocks the event loop. Section 11 discusses this and the available mitigations.

4.17 Exception Handling (backend)

Table 70 — 4.17 Exception Handling (backend)

Layer	Behaviour
Services	Raise <code>ValueError</code> with a human-readable message; never raise <code>HTTPException</code>
Routers	Translate: <code>except ValueError as e: raise HTTPException(422, str(e))</code>
Auth dependency	Raises 401 with <code>headers={"WWW-Authenticate": "Bearer"}</code>
Non-fatal stage failure	<code>try/except Exception</code> around plots and features inside the upload endpoint; the error string is written to <code>plots_error</code> / <code>features_error</code> and the request still succeeds
Fatal stage failure	Parse failure calls <code>mark_upload_failed</code> then raises 422 <code>f"PDF parsing failed: {e}"</code>
Best-effort operations	<code>POST /auth/logout</code> swallows <code>ValueError</code> from an invalid token and still returns 204
Cleanup	<code>POST /baselines/upload</code> removes its temp file in a <code>finally</code> block
Unhandled	No custom handler — FastAPI returns a generic 500

4.18 Alembic Migrations

Table 71 — 4.18 Alembic Migrations

Rev	Down-rev	Title	Objects created / altered
001	—	initial schema	<code>equipment_masters</code> (40 cols), unique <code>uq_equipment_machine_id</code> , indexes on <code>plant_name</code> and <code>machine_type</code> ; <code>sensor_configurations</code> + index on <code>equipment_id</code>
002	001	<code>machine_id</code> nullable	<code>ALTER equipment_masters.machine_id → NULL</code>
003	002	measurement tables	<code>plot_configurations</code> + unique index on <code>sensor_id</code> ; <code>sensor_data_uploads</code> + index on <code>sensor_id</code>
004	003	auth tables	<code>roles</code> (+unique name index), <code>users</code> (+unique email index), <code>user_roles</code> , <code>refresh_tokens</code> (+2 indexes); bulk-inserts 5 roles: <code>super_admin</code> , <code>plant_admin</code> , <code>engineer</code> , <code>operator</code> , <code>viewer</code>
005	004	<code>sensor device_id</code>	<code>sensor_configurations.device_id</code> + unique index
006	005	<code>plot_results</code>	<code>plot_results</code> + 2 indexes + unique (<code>upload_id</code> , <code>plot_type</code> , <code>channel</code> , <code>config_fingerprint</code>); adds <code>plots_status</code> , <code>plots_error</code> , <code>plots_computed_at</code> to <code>sensor_data_uploads</code>
007	006	upload data and baselines	<code>measurement_upload_data</code> , <code>sensor_baselines</code> (+2 indexes), <code>baseline_plot_results</code> (+index +unique constraint)
008	007	user role column	Adds <code>users.role</code> default 'user'; backfills <code>super_admin</code> and <code>admin</code> from <code>user_roles</code> ; inserts roles <code>admin</code> and <code>user</code> with <code>ON CONFLICT DO NOTHING</code> ; adds <code>CHECK (role IN ('super_admin', 'admin', 'user'))</code>
009	008	upload history fields	Adds <code>original_filename</code> and <code>source</code> to <code>sensor_data_uploads</code> ; backfills <code>original_filename</code> from <code>measurement_upload_data</code> ; creates the composite index (<code>sensor_id</code> , <code>created_at</code>)
010	009	channel features and trends	Idempotent (uses <code>_table_exists</code> / <code>_column_exists</code> via <code>sa.inspect</code>). Adds <code>features_status/_error/_computed_at</code> ; creates <code>feature_definitions</code> , <code>feature_threshold_rules</code> (+index), <code>measurement_channel_features</code> (+unique +index), <code>baseline_channel_features</code> (+unique +index); seeds 10 feature definitions and 10 threshold rules only when the tables are empty
011	010	feature trends table	Creates <code>measurement_channel_feature_trends</code> (+unique +composite index); returns early if the table already exists

Migrations 010 and 011 are explicitly written to recover from a partially applied 010 — their docstrings say *"idempotent for partial installs"* and *"partial 010 recovery"*.

4.18.1 alembic/env.py

```
load_dotenv(os.path.join(os.path.dirname(__file__), '..', '..', '.env')) # repo-root .env
sys.path.insert(0, os.path.join(os.path.dirname(__file__), '..')) # make `app` importable
from app.database import Base
import app.models # register all models
config.set_main_option("sqlalchemy.url", os.environ["DATABASE_URL"])
target_metadata = Base.metadata
```

The placeholder URL in `alembic.ini` (`driver://user:pass@localhost/dbname`) is always overwritten. Online mode uses `poolclass=pool.NullPool` so migrations do not hold pooled connections.

4.19 Backend Utility Scripts

Table 72 — 4.19 Backend Utility Scripts

Script	Purpose	Notes
<code>scripts/create_sample_sensor_pdf.py</code>	Generates <code>backend/sample_sensor_data.pdf</code> with a <code>timestamp</code> , <code>ch0</code> , <code>ch1</code> header and 512 rows at <code>fs = 25600</code> . <code>ch0 = 0.5·sin(2π·120t) + 0.1·sin(2π·480t)</code> , <code>ch1 = 0.3·sin(2π·60t)</code>	Requires <code>reportlab</code> , which is not in <code>requirements.txt</code> ; the script prints an install hint and re-raises
<code>scripts/test_auth_phase1.py</code>	End-to-end auth smoke test with <code>fastapi.testclient.TestClient</code> : seed → login → <code>/me</code> (asserts <code>super_admin</code> in roles) → refresh → logout (204) → <code>/me</code> without auth (401)	Hits the real configured database; not a pytest test and not part of a suite

4.20 Edge Acquisition Script — `scripts/vibration.py`

A standalone reader for a Xilinx ZedBoard-class device, outside the FastAPI process:

- Connects with `iio.Context("ip:192.168.1.34")` and `ctx.set_timeout(0)`.
- Finds device `cf_axi_adc`; exits with an error if absent.
- Enables channels `voltage0 ... voltage7` and records each channel's `scale` attribute.
- Allocates `iio.Buffer(dev, 4096)` and loops on `buf.refill()`.
- Reads raw `int32`, reshapes to `(-1, 8)` interleaved, then **shifts left 8 and arithmetic-shifts right 8** to discard an 8-bit status header while sign-extending the 24-bit ADC value.
- Prints the latest raw count per channel on one refreshing console line.
- Handles `KeyboardInterrupt` gracefully and `OSError` with a hardware troubleshooting hint.

This script demonstrates the acquisition side of the contract that `/api/v1/measurements/acquisition` serves configuration for; it does not itself call the API.

5.0 REST API Documentation

5.1 Conventions

Table 73 — 5.1 Conventions

Aspect	Value
Base URL	http://<host>:8000
API prefix	/api/v1
Auth header	Authorization: Bearer <access_token>
Content type	application/json unless a table says multipart/form-data
Interactive docs	/docs (Swagger UI), /redoc , /openapi.json
Error body	{"detail": "<string>"} for HTTPException; {"detail": [{loc, msg, type}, ...]} for Pydantic 422

Authentication column legend. *Public* = no token. *Auth* = any valid token. *Write* = `super_admin` or `admin` only (role `user` receives 403).

5.2 Complete Endpoint Index (46 endpoints)

Table 74 — 5.2 Complete Endpoint Index (46 endpoints)

#	Method	Path	Auth	Purpose
1	GET	/health	Public	Liveness probe
2	POST	/api/v1/auth/login	Public	JSON login
3	POST	/api/v1/auth/token	Public	OAuth2 form login (Swagger)
4	POST	/api/v1/auth/refresh	Public	Rotate token pair
5	POST	/api/v1/auth/logout	Public	Revoke a refresh token
6	GET	/api/v1/auth/me	Auth	Current user profile
7	POST	/api/v1/equipment/	Write	Create equipment (+ nested sensors)
8	GET	/api/v1/equipment/	Auth	Paginated, filtered list
9	GET	/api/v1/equipment/{equipment_id}	Auth	Full equipment record
10	PUT	/api/v1/equipment/{equipment_id}	Write	Update (partial semantics)
11	PATCH	/api/v1/equipment/{equipment_id}	Write	Update (partial semantics)
12	DELETE	/api/v1/equipment/{equipment_id}	Write	Delete + cascade
13	POST	/api/v1/equipment/{equipment_id}/image	Write	Upload equipment image
14	GET	/api/v1/equipment/{equipment_id}/image	Auth	Download image file
15	DELETE	/api/v1/equipment/{equipment_id}/image	Write	Delete image
16	GET	/api/v1/equipment/{equipment_id}/sensors	Auth	List sensors
17	POST	/api/v1/equipment/{equipment_id}/sensors	Write	Add sensor
18	PUT	/api/v1/equipment/{equipment_id}/sensors/{sensor_id}	Write	Update sensor
19	DELETE	/api/v1/equipment/{equipment_id}/sensors/{sensor_id}	Write	Delete sensor
20	GET	/api/v1/equipment/{equipment_id}/ai-readiness	Auth	Readiness score
21	GET	/api/v1/lookups/	Auth	All 18 lookup lists
22	GET	/api/v1/lookups/{lookup_name}	Auth	One lookup list
23	POST	/api/v1/measurements/configure	Write	Upsert plot configuration
24	GET	/api/v1/measurements/configure/{sensor_id}	Auth	Read plot configuration
25	PUT	/api/v1/measurements/configure/{sensor_id}	Write	Update plot configuration
26	GET	/api/v1/measurements/acquisition	Auth	Edge config by <code>device_id</code> query
27	GET	/api/v1/measurements/acquisition/by-sensor/{sensor_id}	Auth	Edge config by sensor UUID
28	GET	/api/v1/measurements/acquisition/{device_id}	Auth	Edge config by <code>device_id</code> path
29	POST	/api/v1/measurements/upload	Write	Upload + parse + plots + features
30	GET	/api/v1/measurements/uploads	Auth	List uploads for a sensor
31	GET	/api/v1/measurements/uploads/{upload_id}	Auth	Single upload record
32	GET	/api/v1/measurements/uploads/{upload_id}/plots	Auth	All plots for a channel
33	GET	/api/v1/measurements/uploads/{upload_id}/plots/{plot_type}	Auth	One plot
34	GET	/api/v1/measurements/plot-types	Auth	Supported plot types
35	GET	/api/v1/measurements/uploads/{upload_id}/features	Auth	10 scalar features + summary
36	GET	/api/v1/measurements/uploads/{upload_id}/factor-trends	Auth	Per-feature segment trends
37	GET	/api/v1/measurements/uploads/{upload_id}/features/compare	Auth	Features vs baseline
38	GET	/api/v1/baselines	Auth	List baselines for a sensor
39	GET	/api/v1/baselines/primary	Auth	Primary baseline
40	GET	/api/v1/baselines/{baseline_id}	Auth	Single baseline
41	PATCH	/api/v1/baselines/{baseline_id}/primary	Write	Set/clear primary flag
42	POST	/api/v1/baselines/upload	Write	Upload a file directly as a baseline
43	POST	/api/v1/baselines/from-upload/{upload_id}	Write	Promote an upload to a baseline
44	GET	/api/v1/baselines/{baseline_id}/plots	Auth	Baseline plots
45	GET	/api/v1/baselines/{baseline_id}/plots/{plot_type}	Auth	One baseline plot

#	Method	Path	Auth	Purpose
46	GET	/api/v1/baselines/{baseline_id}/features	Auth	Baseline feature values

5.3 Health

5.3.1 GET /health

Liveness probe for orchestrators. No authentication (explicitly excluded from the OpenAPI security sweep).

Response 200

```
{ "status": "ok", "service": "AI Vibration Intelligence Platform" }
```

5.4 Authentication API

5.4.1 POST /api/v1/auth/login

Table 75 — 5.4.1 POST /api/v1/auth/login

Property	Value
Purpose	Exchange email + password for a token pair
Auth	Public
Headers	Content-Type: application/json
Controller	routers/auth.py::login
Service	auth_service.authenticate_user , create_access_token , create_refresh_token
Repository	crud/user.get_user_by_email , update_last_login , create_refresh_token_record
Tables	users , roles , user_roles (read), refresh_tokens (insert)

Request body — LoginRequest

Table 76 — 5.4.1 POST /api/v1/auth/login

Field	Type	Rules
email	EmailStr	Must be a syntactically valid address (422 otherwise)
password	string	min_length=1

Response 200 — TokenResponse

Table 77 — 5.4.1 POST /api/v1/auth/login

Field	Type	Notes
access_token	string	JWT HS256
refresh_token	string	Opaque, 48-byte URL-safe
token_type	string	Always "bearer"
expires_in	int	Access-token lifetime in seconds (1800 by default)

Errors — 401 "Incorrect email or password" (unknown email, wrong password, or inactive user); 422 on schema violation.

Business logic — the three failure causes are deliberately collapsed into one message to avoid account enumeration. last_login_at is updated **before** tokens are issued.

Request example

```
curl -X POST http://localhost:8000/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"admin@vibration.com","password":"Admin@2024"}'
```

Response example

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refresh_token": "9Kx3q7...W2",
  "token_type": "bearer",
  "expires_in": 1800
}
```

Sequence diagram — see §2.6.

5.4.2 POST /api/v1/auth/token

OAuth2-password-flow variant used by the Swagger **Authorize** dialog. Body is `application/x-www-form-urlencoded` with `username` (the email) and `password`. Behaviour and responses are otherwise identical to `/login`.

5.4.3 POST /api/v1/auth/refresh

Table 78 — 5.4.3 POST /api/v1/auth/refresh

Property	Value
Purpose	Rotate a refresh token into a fresh token pair
Auth	Public (the refresh token is the credential)
Body	<code>{"refresh_token": "<string, min_length 1>"}</code>
Response 200	<code>TokenResponse</code> (both tokens are new)
Errors	401 "Invalid refresh token" / "Refresh token revoked" / "Refresh token expired" / "User inactive"

Business logic. Validate → revoke the presented token → issue a new pair. Single-use rotation; a replayed token fails with "Refresh token revoked".

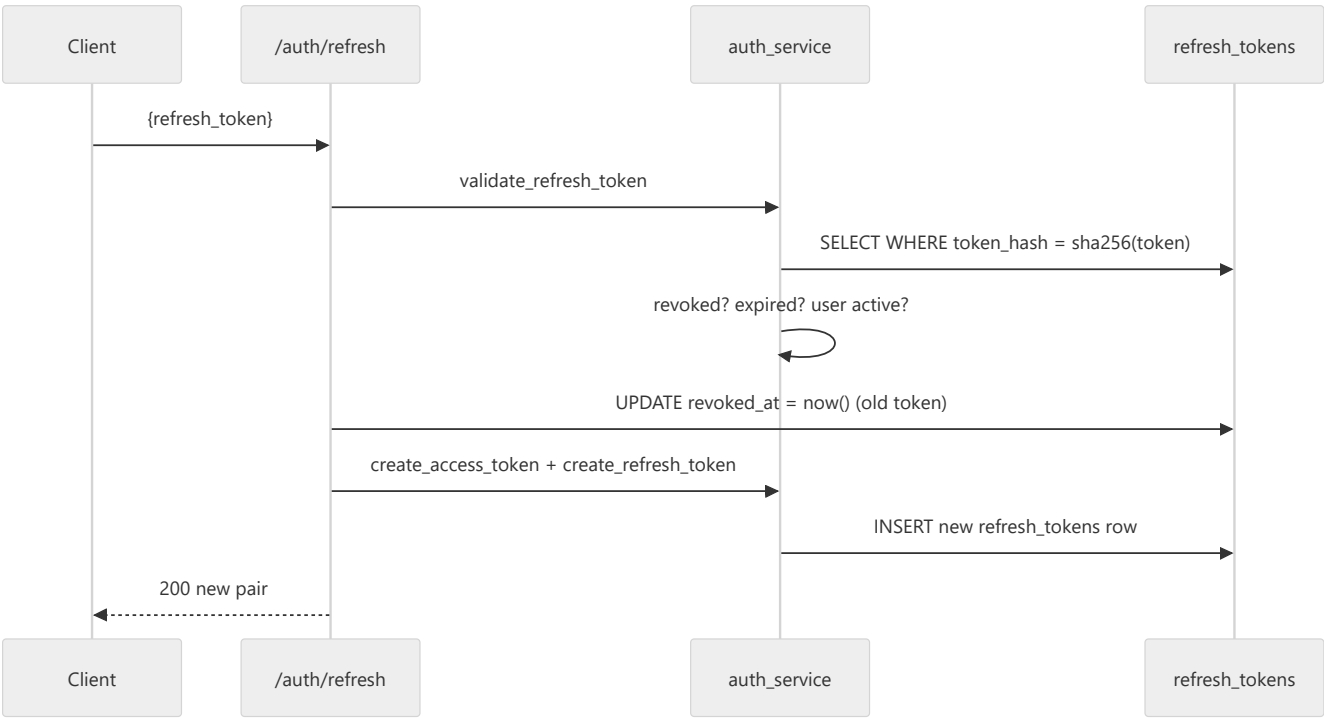


Figure 21 — 5.4.3 POST /api/v1/auth/refresh

5.4.4 POST /api/v1/auth/logout

Body `{"refresh_token": "..."}.` Returns **204 No Content**. Validation errors are swallowed (except `ValueError: pass`), so logout is idempotent and always succeeds — an already-revoked or unknown token still yields 204. Access tokens are **not** invalidated and remain usable until they expire.

5.4.5 GET /api/v1/auth/me

Table 79 — 5.4.5 GET /api/v1/auth/me

Property	Value
Auth	Auth (Bearer)
Response 200	UserMeResponse : id , email , full_name , is_active , must_change_password , roles[] , plants[] , last_login_at
Errors	401 for missing/invalid/expired token or inactive user

plants is always [] — user_to_me_dict hard-codes it because plant scoping is not yet modelled.

```
{
  "id": "3f2a...",
  "email": "admin@vibration.com",
  "full_name": "Platform Administrator",
  "is_active": true,
  "must_change_password": false,
  "roles": ["super_admin"],
  "plants": [],
  "last_login_at": "2026-07-25T09:14:22.113Z"
}
```

5.5 Equipment API

All routes inherit Depends(get_current_user) ; write routes add Depends(require_write_access) .

5.5.1 POST /api/v1/equipment/ — create

Table 80 — 5.5.1 POST /api/v1/equipment/ — create

Property	Value
Auth	Write · Status 201
Controller	create_equipment
Repository	crud.get_equipment_by_machine_id , crud.create_equipment
Tables	equipment_masters (insert), sensor_configurations (insert)

Request body — EquipmentCreate — all 40 EquipmentBase fields plus sensors: SensorConfigCreate[] .

Validation rules:

Table 81 — 5.5.1 POST /api/v1/equipment/ — create

Field	Rule
machine_id	"" is coerced to null by normalize_machine_id ; a non-null duplicate returns 409
rated_power_kw , gearbox_ratio , load_range_* , normal_operating_load	Decimal
operating_environment	Array of strings → Postgres text[]
installation_date , last_maintenance_date	ISO YYYY-MM-DD
Each sensor	sensor_type , mounting_location , orientation required; device_id ≤ 64 chars

Business logic. Logs the four identity fields; performs the duplicate check only when machine_id is truthy; delegates to crud.create_equipment , which flushes to obtain the parent ID before inserting sensors so both land in one transaction.

Responses — 201 EquipmentOut (includes id , sensors[] , created_at , updated_at); 409 "Machine ID 'X' already exists" ; 401; 403; 422.

Request example

```
{
  "plant_name": "Pune Plant", "area": "Utilities", "line": "Cooling Water Line",
  "machine_name": "Cooling Water Pump P-204", "machine_id": "PUMP-P204",
  "machine_type": "Pump", "machine_criticality": "Critical",
  "manufacturer": "KSB", "model": "Etanorm SYT 100-250",
  "rated_power_kw": 75, "rated_rpm": 1480,
  "drive_type": "Direct Drive", "load_type": "Constant Load",
  "bearing_number_de": "6205", "bearing_number_nde": "6204", "pump_vanes": 7,
  "operating_speed_min": 1400, "operating_speed_max": 1500,
  "operating_environment": ["Indoor", "Wet Area"],
  "asset_status": "Active",
  "sensors": [
    { "sensor_type": "IEPE Accelerometer", "mounting_location": "Bearing Housing DE",
      "orientation": "Horizontal", "mounting_method": "Stud Mounted",
      "sensitivity": 100.0, "sensitivity_unit": "mV/g",
      "sampling_rate": "25600 Hz", "frequency_range": "0-10000 Hz",
      "is_active": true, "device_id": "11:AA:BB:CC:DD:EE" }
  ]
}
```

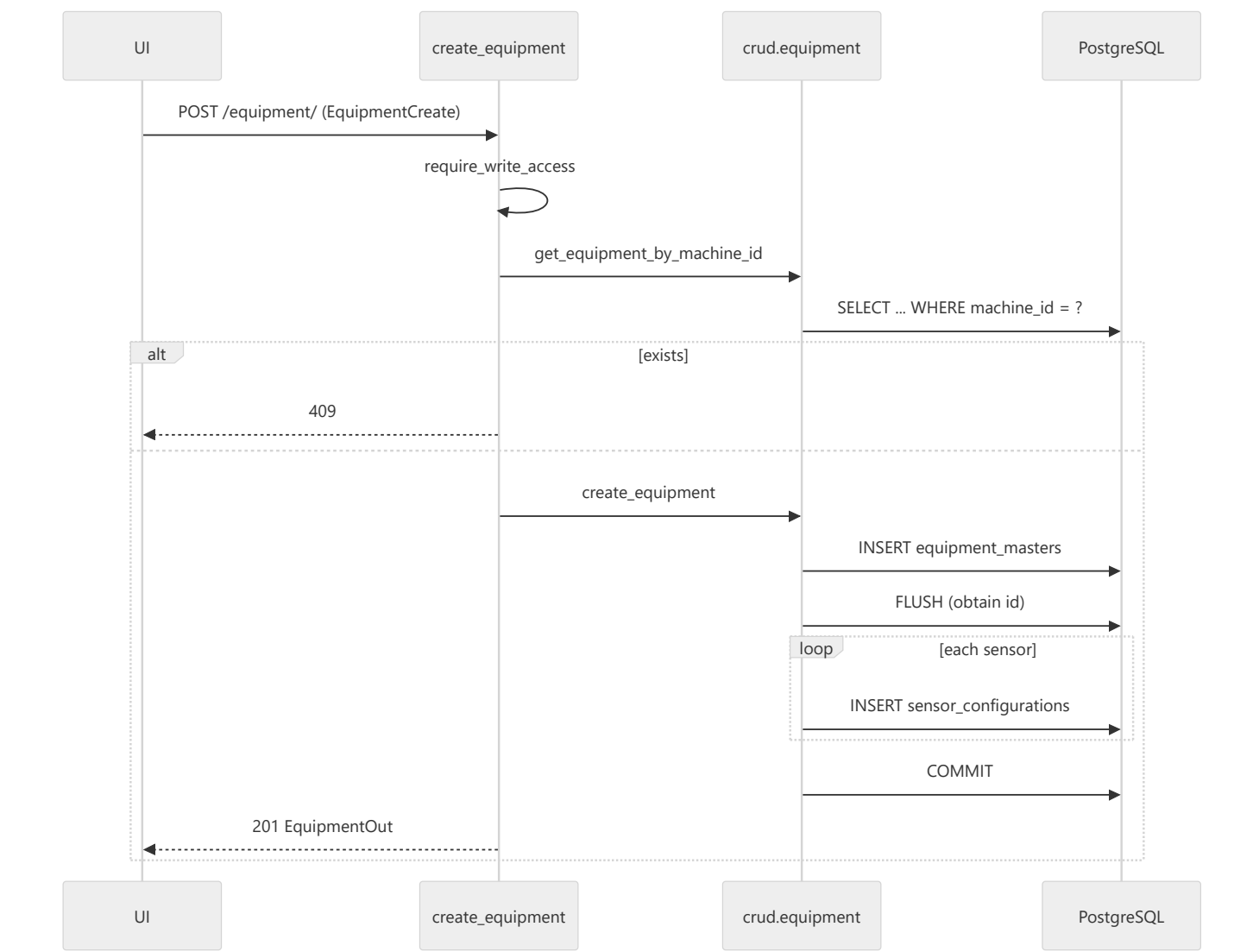


Figure 22 — 5.5.1 POST /api/v1/equipment/ — create

5.5.2 GET /api/v1/equipment/ — list

Query parameters

Table 82 — 5.5.2 GET /api/v1/equipment/ — list

Name	Type	Default	Rules
page	int	1	ge=1
page_size	int	20	ge=1, le=100
plant_name	string?	—	Case-insensitive partial (ILIKE %v%)
machine_type	string?	—	Exact
machine_criticality	string?	—	Exact

Response 200 — `PaginatedEquipment`: {total, page, page_size, items: `EquipmentListItem`[]} where each item carries `id`, `plant_name`, `area`, `line`, `machine_name`, `machine_id`, `machine_type`, `machine_criticality`, `manufacturer`, `asset_status`, `equipment_image_path`, `created_at`. Ordered `created_at DESC`.

GET /api/v1/equipment/?page=1&page_size=20&machine_type=Pump&machine_criticality=Critical

5.5.3 GET /api/v1/equipment/{equipment_id}

Path parameter `equipment_id`: UUID (non-UUID → 422). Returns the full `EquipmentOut` including the nested `sensors[]`. 404 "Equipment not found".

5.5.4 PUT and PATCH /api/v1/equipment/{equipment_id}

Both call the same handler logic (`crud.update_equipment`) with `EquipmentUpdate`, and both use `exclude_unset=True` — so `PUT` behaves as a partial update, not a replace. Response 200 `EquipmentOut`; 404 when the id is unknown; 403 for role `user`.

Note: unlike create, update does **not** check `machine_id` uniqueness in application code. A colliding value is rejected by the database's `uq_equipment_machine_id` constraint, surfacing as a 500 rather than a 409.

5.5.5 DELETE /api/v1/equipment/{equipment_id}

Status 204. Removes the equipment row; ORM `delete-orphan` plus database `ON DELETE CASCADE` remove sensors and, transitively, plot configurations, uploads, upload data, plot results, baselines, baseline plots, and all feature/trend rows for those sensors. 404 when absent.

5.5.6 POST /api/v1/equipment/{equipment_id}/image

Table 83 — 5.5.6 POST /api/v1/equipment/{equipment_id}/image

Property	Value
Auth	Write · <code>multipart/form-data</code> · field <code>file</code>
Allowed MIME	<code>image/jpeg</code> , <code>image/png</code> , <code>image/webp</code> , <code>image/gif</code>
Max size	<code>settings.max_image_size_mb</code> = 10 MB
Storage	<code>{upload_dir}/{equipment_id}.{ext}</code> — the extension is taken from the original filename, defaulting to <code>jpg</code>
Response 200	<code>EquipmentOut</code> with the updated <code>equipment_image_path</code>
Errors	404 equipment not found; 400 wrong type; 400 "Image exceeds 10MB limit"

Because the filename is deterministic, re-uploading replaces the previous image — unless the extension differs, in which case the old file is orphaned on disk while the path column points at the new one.

5.5.7 GET /api/v1/equipment/{equipment_id}/image

Returns a `FileResponse` streaming the stored file. 404 when the equipment is missing, when `equipment_image_path` is null, or when the path no longer exists on disk ("Image file not found on disk").

5.5.8 DELETE /api/v1/equipment/{equipment_id}/image

Status 204. Removes the file when present, then sets `equipment_image_path` = `NULL`. 404 when the equipment is unknown.

5.5.9 Sensor endpoints

Table 84 — 5.5.9 Sensor endpoints

Endpoint	Auth	Behaviour
GET /{equipment_id}/sensors	Auth	404 if the equipment does not exist, else <code>SensorConfigOut[]</code>
POST /{equipment_id}/sensors	Write	201 <code>SensorConfigOut</code> ; 404 if the parent is missing
PUT /{equipment_id}/sensors/{sensor_id}	Write	Partial update via <code>SensorConfigUpdate</code> + <code>exclude_unset</code> ; 404 "Sensor not found" . The handler does not verify the sensor belongs to <code>equipment_id</code> — the path segment is contextual only
DELETE /{equipment_id}/sensors/{sensor_id}	Write	204; cascades to that sensor's plot config, uploads, plots, baselines, and features

5.5.10 GET /api/v1/equipment/{equipment_id}/ai-readiness

Response 200 — `AIReadinessOut`

Table 85 — 5.5.10 GET /api/v1/equipment/{equipment_id}/ai-readiness

Field	Meaning
<code>equipment_id</code>	Echoed UUID
<code>score_percent</code>	$\text{int}(\text{sum}(\text{checks})/5 \times 100)$ → one of 0, 20, 40, 60, 80, 100
<code>machine_train_configured</code>	The stored boolean
<code>asset_status_set</code>	<code>asset_status</code> not in (None, "")
<code>sensor_coverage</code>	<code>len(sensors) > 0</code>
<code>bearing_database_mapped</code>	The stored boolean
<code>operating_mode_configured</code>	Both <code>operating_speed_min</code> and <code>operating_speed_max</code> are set

```
{ "equipment_id": "3f2a...", "score_percent": 80,
  "machine_train_configured": true, "asset_status_set": true,
  "sensor_coverage": true, "bearing_database_mapped": false,
  "operating_mode_configured": true }
```

The frontend's `getAIReadinessScore` in `lib/form-intelligence.ts` uses a **different**, four-dimension formula (data completeness, sensor coverage, diagnostic readiness, PM readiness). The two scores are independent and will not agree.

5.6 Lookups API

5.6.1 GET /api/v1/lookups/

Returns the entire `LOOKUPS` dictionary — a static, in-code catalogue with no database access.

Table 86 — 5.6.1 GET /api/v1/lookups/

Key	Count	Values
machine-types	13	Motor, Pump, Fan, Blower, Compressor, Gearbox, Turbine, Generator, DG Set, Conveyor, Crusher, Mixer, Agitator
machine-criticality	4	Low, Medium, High, Critical
drive-types	6	Direct, Belt, Gear, Chain, VFD, Hydraulic Drive
load-types	5	Constant, Variable, Intermittent, Cyclic, Shock Load
foundation-types	5	Concrete Foundation, Steel Structure, Skid Mounted, Base Frame, Suspended Structure
coupling-types	9	Flexible, Grid, Gear, Jaw, Disc, Tyre, Chain, Fluid, Direct
motor-pole-counts	6	2, 4, 6, 8, 10, 12 (integers)
direction-of-rotation	3	Clockwise, Counter-Clockwise, Bidirectional
operating-environments	13	Indoor ... Food Grade Area
lubrication-types	6	Grease, Oil Bath, Oil Mist, Forced Oil, Splash, Automatic
sensor-types	14	IEPE Accelerometer ... RPM Sensor
mounting-locations	11	Bearing Housing DE ... Custom
sensor-orientations	5	Horizontal, Vertical, Axial, Radial, Tangential
mounting-methods	9	Stud Mounted ... Custom
sensitivity-units	5	mV/g, mV/mm/s, mV/μm, mA, V
sampling-rates	9	512 Hz ... 65536 Hz, Custom
frequency-ranges	7	0-500 Hz ... 0-20000 Hz, Custom
asset-status	4	Active, Inactive, Under Maintenance, Decommissioned

5.6.2 GET /api/v1/lookups/{lookup_name}

Returns {"lookup": "<name>", "values": [...]}; 404 "Lookup '<name>' not found" for an unknown key.

The frontend currently hard-codes the same option lists inside its tab components rather than calling these endpoints, although `getLookup` / `getAllLookups` exist in `api/equipment.ts`. Keeping both in sync is a maintenance obligation.

5.7 Measurements API

5.7.1 POST /api/v1/measurements/configure

Table 87 — 5.7.1 POST /api/v1/measurements/configure

Property	Value
Purpose	Create or update (upsert) the processing profile for a sensor
Auth	Write
Body	PlotConfigCreate
Tables	plot_configurations

Body fields

Table 88 — 5.7.1 POST /api/v1/measurements/configure

Field	Type	Default	Rules
sensor_id	UUID	—	Must exist (404 otherwise)
channel_count	int	—	1–32
active_channel	int	0	≥ 0 and $< \text{channel_count}$ (model validator)
sampling_rate_hz	float	25600	> 0
fft_lines	int	1600	64–65536
frequency_max_hz	float?	null	> 0
data_type	string	acceleration	$\in \{\text{acceleration, velocity, displacement}\}$
enabled_plots	string[]	all 5	Aliases canonicalised, unknowns dropped, must be non-empty after normalisation

Response 200 PlotConfigOut (adds id, created_at, updated_at).

5.7.2 GET /api/v1/measurements/configure/{sensor_id}

200 PlotConfigOut; 404 "Plot configuration not found for this sensor". Read-time repair applies: legacy plot names are canonicalised and an out-of-range active_channel is clamped.

5.7.3 PUT /api/v1/measurements/configure/{sensor_id}

Write. Body PlotConfigUpdate (all optional). 404 when the sensor or the configuration is missing. Sets updated_at.

5.7.4 Edge acquisition endpoints (26–28)

Three shapes of the same operation:

Table 89 — 5.7.4 Edge acquisition endpoints (26–28)

Endpoint	Lookup key	Intended use
GET /acquisition?device_id=11:AA:BB:CC:DD:EE	sensor_configurations.device_id	Recommended — safe for MAC addresses containing :
GET /acquisition/{device_id}	same	Path form; awkward when the ID contains :
GET /acquisition/by-sensor/{sensor_id}	sensor_configurations.id	Testing before a device_id is assigned

Response 200 — EdgeAcquisitionConfigOut

```
{
  "acquisitionFormula": {
    "frequencyResolutionHz": 16.0, "blockTimeSeconds": 0.0625,
    "sampleRateHz": 25600.0, "requiredSamples": 1600.0,
    "overlapDecimal": 0.0, "totalAcquisitionTimeSeconds": 0.0625,
    "averageCount": 1.0, "fmaxHz": 15000.0, "lor": 1600.0,
    "stepSizeSamples": 1600.0
  },
  "minutes": "1", "averaging": 1, "sensitivityMvPerG": 100.0,
  "totalChannelCount": 8, "averageCount": 1,
  "lastAveraging": null, "lastOverlapping": null,
  "lor": "1600", "fmax": "15000", "windowType": "HANNING",
  "sensorId": "11:AA:BB:CC:DD:EE",
  "channels": [
    {"transducerType": "IEPE Accelerometer", "signalType": "VIBRATION", "channelIndex": 1, "machineAxis": "HORIZONTAL"}
  ],
  "success": true, "overlapping": 0, "ksp": "25",
  "id": 1, "overlapPercentage": 0,
  "platformSensorId": "9c1d..."
}
```

Errors — 404 for /acquisition and /acquisition/{device_id}: "No sensor found with device_id 'X'. Set device_id on the sensor via equipment API, then save plot config."; 404 "Sensor not found" for the by-sensor form.

Note. These routes inherit the router's get_current_user dependency, so an edge device must present a bearer token.

5.7.5 POST /api/v1/measurements/upload

Table 90 — 5.7.5 POST /api/v1/measurements/upload

Property	Value
Purpose	Upload a capture and run the full processing pipeline
Auth	Write · Status 201
Content type	<code>multipart/form-data</code>
Tables written	<code>sensor_data_uploads</code> , <code>measurement_upload_data</code> , <code>plot_results</code> , <code>measurement_channel_features</code> , <code>measurement_channel_feature_trends</code>
Files written	<code>{measurement_upload_dir}/{upload_id}.csv</code>

Form fields

Table 91 — 5.7.5 POST /api/v1/measurements/upload

Field	Type	Rules
<code>sensor_id</code>	UUID	Must exist → 404 with guidance on where to find a real id
<code>channel_count</code>	int	<code>ge=1</code> , <code>le=32</code>
<code>file</code>	file	<code>.csv</code> / <code>.pdf</code> by extension or MIME ∈ {application/pdf, text/csv, application/csv, text/plain}; ≤ 50 MB

Accepted file layouts

```
timestamp_,ch0,ch1,ch2,ch3,ch4,ch5,ch6
1777747212,0.01234,-0.00871,0.00042,...
```

Also accepted: header `timestamp`; tab-, semicolon-, comma-, or whitespace-separated values; PDFs whose pages contain extractable tables or text.

Response 201 — SensorDataUploadOut

Table 92 — 5.7.5 POST /api/v1/measurements/upload

Field	Meaning
<code>id</code> , <code>sensor_id</code> , <code>original_filename</code> , <code>source</code>	Identity (<code>source</code> is always <code>"manual"</code> for this endpoint)
<code>channel_count</code> , <code>sample_count</code>	Capture dimensions
<code>parse_status</code> / <code>parse_error</code>	<code>pending</code> → <code>parsed</code> <code>failed</code>
<code>plots_status</code> / <code>plots_error</code> / <code>plots_computed_at</code>	<code>pending</code> → <code>ready</code> <code>failed</code>
<code>features_status</code> / <code>features_error</code> / <code>features_computed_at</code>	<code>pending</code> → <code>ready</code> <code>failed</code>
<code>created_at</code> , <code>parsed_at</code>	Timestamps
<code>has_stored_data</code>	<code>true</code> — the bytes are in <code>measurement_upload_data</code>

Errors — 404 sensor; 400 file type; 400 `"File exceeds 50MB limit"`; 422 `"PDF parsing failed: <detail>"`.

Full sequence diagram — see §1.8.1.

```
curl -X POST http://localhost:8000/api/v1/measurements/upload \
-H "Authorization: Bearer $TOKEN" \
-F "sensor_id=9c1d..." -F "channel_count=8" -F "file=@capture.csv"
```

5.7.6 GET /api/v1/measurements/uploads

Query parameters

Table 93 — 5.7.6 GET /api/v1/measurements/uploads

Name	Type	Default	Rules
<code>sensor_id</code>	UUID	required	404 when unknown
<code>from_date</code>	date?	—	Inclusive from 00:00:00
<code>to_date</code>	date?	—	Inclusive to 23:59:59.999999
<code>parse_status</code>	string?	—	Exact match
<code>plots_status</code>	string?	—	Exact match
<code>page</code>	int	1	<code>ge=1</code>
<code>page_size</code>	int	50	<code>ge=1</code> , <code>le=200</code>

Response 200 — `PaginatedUploadListOut`: {items, total, page, page_size}, newest first. `has_stored_data` is resolved for the whole page with **one** `IN` query (`get_stored_upload_ids`) rather than per row.

Errors — 400 `"from_date must be on or before to_date"`; 404 sensor.

5.7.7 GET /api/v1/measurements/uploads/{upload_id}

Single `SensorDataUploadOut`; 404 `"Upload not found"`.

5.7.8 GET /api/v1/measurements/uploads/{upload_id}/plots

Table 94 — 5.7.8 GET /api/v1/measurements/uploads/{upload_id}/plots

Property	Value
Query	<code>channel: int?</code> with <code>ge=0</code> , <code>le=31</code> — overrides the saved <code>active_channel</code>
Response 200	<code>AllPlotsOut</code> : {upload_id, sensor_id, channel, available_channels[], plots[]}
Errors	404 upload; 422 <code>"Upload not parsed: <error or status>"</code> ; 422 from <code>ValueError</code> (e.g. <code>"Upload has no parsed data"</code>)

Each `plots[]` entry is a `PlotSeriesOut`: {plot_type, title, x_label, y_label, x[], y[], channel, metadata}. Plots are returned in the order given by `enabled_plots`.

Cache-or-compute behaviour — see §4.9.6. A first request after a configuration change recomputes and persists before responding.

```
{
  "upload_id": "6d2b...", "sensor_id": "9c1d...", "channel": 0,
  "available_channels": [0,1,2,3,4,5,6,7],
  "plots": [
    { "plot_type": "time_waveform", "title": "Time Waveform",
      "x_label": "Time (s)", "y_label": "Amplitude",
      "x": [0.0,0.0000390625,...], "y": [0.0123,-0.0087,...],
      "channel": 0, "metadata": {"plot_style": "line"} },
    { "plot_type": "fft_spectrum", "title": "FFT Spectrum",
      "x_label": "Frequency (Hz)", "y_label": "Magnitude",
      "x": [0.0,16.0,32.0,...], "y": [0.0001,0.0342,...],
      "channel": 0,
      "metadata": {"plot_style": "line", "fft_lines": 1600, "sampling_rate_hz": 25600.0} }
  ]
}
```

5.7.9 GET /api/v1/measurements/uploads/{upload_id}/plots/{plot_type}

`plot_type` must be one of the five canonical values, else 400 `"Invalid plot type. Allowed: [...]"`. Returns a single `PlotSeriesOut`.

5.7.10 GET /api/v1/measurements/plot-types

{`"plot_types": ["time_waveform", "circular_time_waveform", "fft_spectrum", "envelope_spectrum", "trend_plot"]`}

5.7.11 GET /api/v1/measurements/uploads/{upload_id}/features

Table 95 — 5.7.11 GET /api/v1/measurements/uploads/{upload_id}/features

Property	Value
Query	<code>channel: int?</code> (<code>ge=0</code> , <code>le=31</code>) — omit for all channels
Response 200	<code>UploadFeaturesOut</code>
Errors	404 upload; 422 not parsed; 422 <code>"Feature compute failed: <detail>"</code>
Tables	<code>sensor_data_uploads</code> , <code>feature_definitions</code> , <code>feature_threshold_rules</code> , <code>sensor_baselines</code> , <code>baseline_channel_features</code> , <code>measurement_channel_features</code>

Business logic. Resolves the plot configuration for the sampling rate, calls `ensure_upload_features_ready` (which computes on demand for legacy uploads whose `features_status` is still `pending`), then reads rows, builds `items`, `summary`, and `channel_overview`.

`channel_overview.health_state` is the worst status present: any `critical` → `Critical`; else any `warning` → `Warning`; else all `normal` → `Normal`; else `status_to_health_level(first)`.

```
{
  "upload_id": "6d2b...", "sensor_id": "9c1d...", "channel": 0,
  "features_status": "ready", "features_error": null,
  "features_computed_at": "2026-07-25T09:20:11.442Z",
  "items": [
    { "channel": 0, "feature_code": "rms", "feature_name": "RMS", "value": 0.00841,
      "unit": "scaled_eng", "status": "normal", "metadata": {},
      "computed_at": "2026-07-25T09:20:11.442Z",
    },
    { "channel": 0, "feature_code": "amplitude_1x", "feature_name": "1X Amplitude",
      "value": 0.00219, "unit": "scaled_eng", "status": "normal",
      "metadata": { "estimated_shaft_hz": 24.6, "sampling_rate_hz": 25600.0, "sample_count": 4096 },
      "computed_at": "2026-07-25T09:20:11.442Z"
    },
  ],
  "summary": { "normal": 9, "warning": 1, "critical": 0, "no_baseline": 0, "total": 10 },
  "channel_overview": { "health_state": "Warning", "feature_count": 10,
    "computed_at": "2026-07-25T09:20:11.442Z",
    "baseline_name": null, "baseline_id": null
  }
}
```

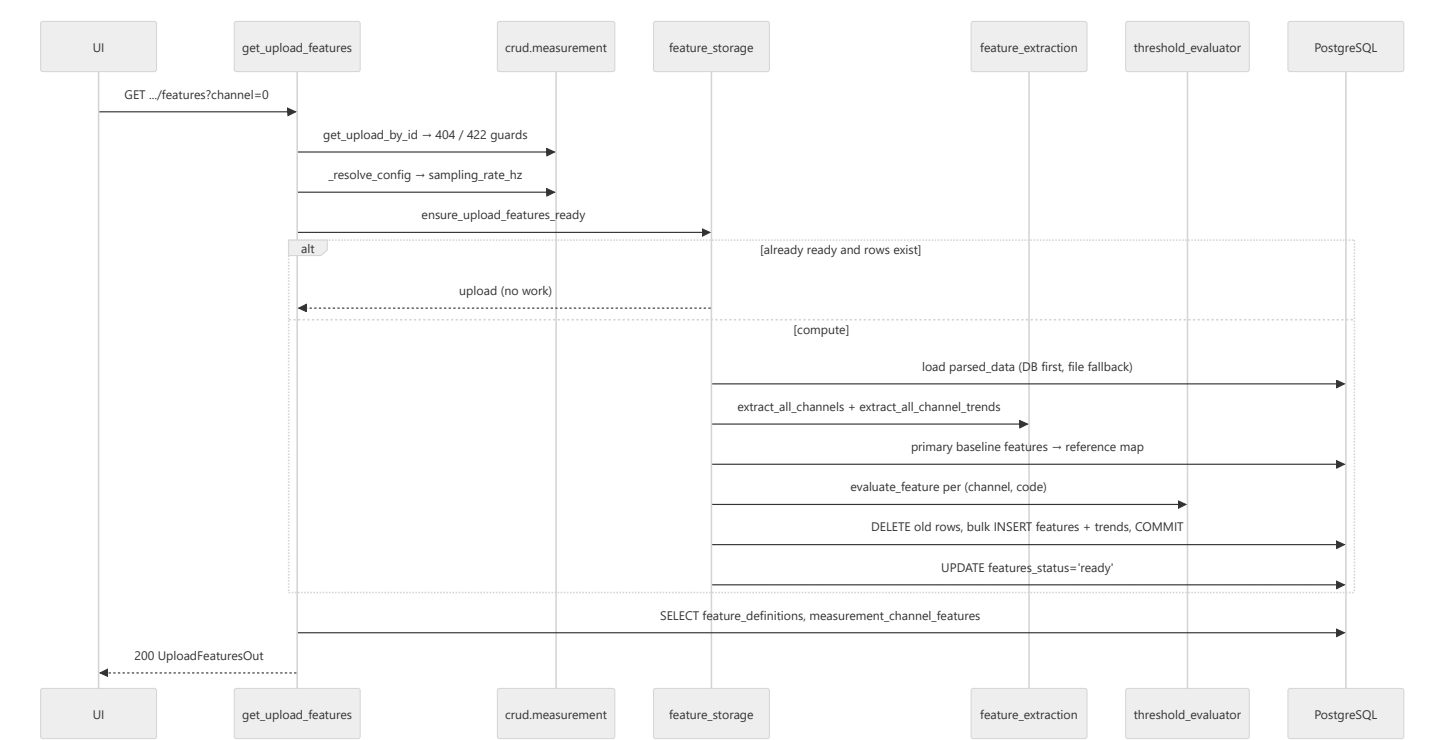


Figure 23 — 5.7.11 GET /api/v1/measurements/uploads/{upload_id}/features

5.7.12 GET /api/v1/measurements/uploads/{upload_id}/factor-trends

Table 96 — 5.7.12 GET /api/v1/measurements/uploads/{upload_id}/factor-trends

Property	Value
Query	channel: int default 0, ge=0, le=31 (required semantics — a single channel only)
Response 200	UploadFactorTrendsOut
Errors	404 upload; 422 not parsed; 422 feature-compute failure

Merges the scalar row and the ordered trend rows for each feature code, then sorts the result by `feature_definitions.sort_order` (unknown codes sort last with key 999).

```
{
  "upload_id": "6d2b...", "sensor_id": "9c1d...", "channel": 0,
  "features_status": "ready", "sampling_rate_hz": 25600.0,
  "factors": [
    { "feature_code": "rms", "feature_name": "RMS", "unit": "scaled_eng",
      "value": 0.00841, "status": "normal",
      "trend_x": [0.0025, 0.0075, "...32 values..."],
      "trend_y": [0.0081, 0.0086, "...32 values..."]}
  ]
}
```

5.7.13 GET /api/v1/measurements/uploads/{upload_id}/features/compare

Table 97 — 5.7.13 GET /api/v1/measurements/uploads/{upload_id}/features/compare

Property	Value
Query	baseline_id: UUID? (falls back to the sensor's primary baseline), channel: int?
Response 200	FeatureCompareOut
Errors	404 upload; 404 "No primary baseline for this sensor"; 404 "Baseline not found"; 422 feature-compute failure

percent_of_baseline = 100 × upload_value / baseline_value, computed only when the baseline value exists and exceeds 1e-30; otherwise null. Baseline values are keyed by (channel, feature_code), so a channel mismatch yields null rather than a wrong comparison.

```
{
  "upload_id": "6d2b...", "baseline_id": "1a7f...", "channel": 0,
  "items": [
    { "channel": 0, "feature_code": "rms", "feature_name": "RMS", "unit": "scaled_eng",
      "upload_value": 0.00841, "baseline_value": 0.00612,
      "percent_of_baseline": 137.4, "status": "warning" }
  ],
  "summary": { "normal": 9, "warning": 1, "critical": 0, "no_baseline": 0, "total": 10 }
}
```

5.8 Baselines API

5.8.1 GET /api/v1/baselines

Query sensor_id: UUID (required). Returns BaselineListOut: {sensor_id, total, primary_baseline_id, items[]}, newest first. 404 "Sensor not found".

Each item's plots_status is derived, not stored:

```
expected = channel_count × 5
plot_count ≥ expected and expected > 0 → "ready"
plot_count > 0 → "partial"
otherwise → "pending"
```

5.8.2 GET /api/v1/baselines/primary

Query sensor_id: UUID. 200 BaselineOut; 404 "No primary baseline set for this sensor". The frontend converts this 404 into null.

5.8.3 GET /api/v1/baselines/{baseline_id}

200 BaselineOut; 404 "Baseline not found".

5.8.4 PATCH /api/v1/baselines/{baseline_id}/primary

Write. Body {"is_primary": true} (default true). When setting to true, all other baselines for the same sensor are cleared first, guaranteeing at most one primary per sensor. Returns the updated BaselineOut. No baseline is deleted.

5.8.5 POST /api/v1/baselines/upload

Table 98 — 5.8.5 POST /api/v1/baselines/upload

Property	Value
Auth	Write · 201 · <code>multipart/form-data</code>
Fields	<code>sensor_id</code> UUID, <code>channel_count</code> int (1-32), <code>name</code> string, <code>description</code> string?, <code>set_as_primary</code> bool (default false), <code>file</code>
Behaviour	Writes a temp file → parses → always deletes the temp file in finally → creates the baseline with the raw bytes and parsed data → computes and persists baseline plots
Errors	404 sensor; 422 "Parse failed: ..."; 422 "Baseline plot compute failed: ..."

Unlike the upload endpoint, this route stores **no** row in `sensor_data_uploads` and computes **no** feature rows — a directly uploaded baseline has plots but no `baseline_channel_features`.

5.8.6 POST `/api/v1/baselines/from-upload/{upload_id}`

Table 99 — 5.8.6 POST `/api/v1/baselines/from-upload/{upload_id}`

Property	Value
Auth	Write · 201
Body	<code>BaselineCreateFromUpload</code> : <code>name</code> (1-200), <code>description?</code> , <code>labels</code> : <code>string[]</code> , <code>set_as_primary</code> (default false), <code>captured_at?</code>
Errors	404 "Parsed upload not found"; 422 "Upload file data not in DB. Re-upload the file after migration 007."; 422 "Baseline plot compute failed: ..."

Business logic. Reads `measurement_upload_data` (bytes + parsed JSON) — never the filesystem — so the baseline is durable. Copies both into a new `sensor_baselines` row, persists baseline plot results, then calls `copy_upload_features_to_baseline`, which copies the upload's feature values into `baseline_channel_features` with `status` forced to `normal`.

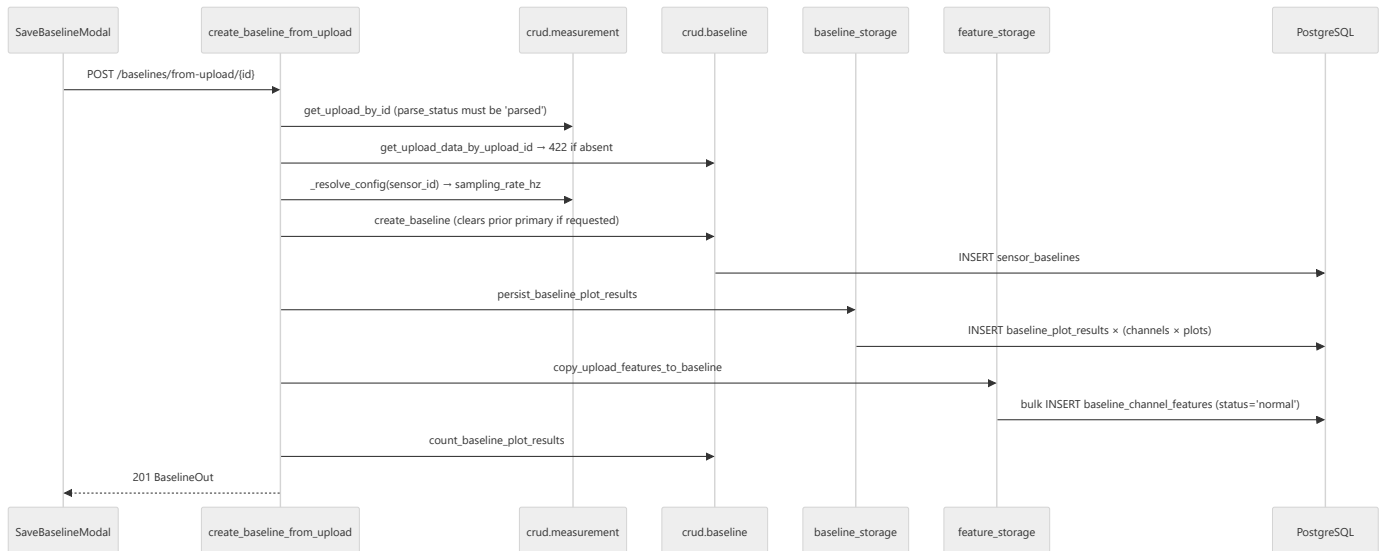


Figure 24 — 5.8.6 POST `/api/v1/baselines/from-upload/{upload_id}`

5.8.7 GET `/api/v1/baselines/{baseline_id}/plots`

Query `channel`: `int?`. Resolves the sensor's configuration, computes the fingerprint, and reads stored rows; if none exist it computes and persists them, then re-reads. Returns `AllPlotsOut` — note that `upload_id` in the payload carries the **baseline id** (schema reuse). 404 baseline; 422 on compute failure.

5.8.8 GET `/api/v1/baselines/{baseline_id}/plots/{plot_type}`

Validates the plot type (400), delegates to the previous endpoint, and filters. 404 "Plot {plot_type} not found".

5.8.9 GET `/api/v1/baselines/{baseline_id}/features`

Query `channel`: `int?`. Returns `BaselineFeaturesOut` with `items` and a `summary` counted directly from the rows. 404 baseline. Baselines created via `POST /baselines/upload` return an empty list because that path never populates features.

5.9 Cross-Cutting API Behaviour

5.9.1 Which endpoints require write access

Table 100 — 5.9.1 Which endpoints require write access

Router	Write endpoints
Equipment	create, PUT, PATCH, DELETE, image upload, image delete, sensor add/update/delete
Measurements	POST /configure , PUT /configure/{sensor_id} , POST /upload
Baselines	PATCH /{id}/primary , POST /upload , POST /from-upload/{id}
Auth / Lookups	none

Everything else is readable by all three roles.

5.9.2 Validation-rule summary

Table 101 — 5.9.2 Validation-rule summary

Constraint	Endpoints
page ≥ 1 ; page_size 1–100 (equipment) / 1–200 (uploads)	list endpoints
channel 0–31	all plot/feature/trend endpoints
channel_count 1–32	configure, upload, baseline upload
fft_lines 64–65536	configure
sampling_rate_hz > 0 , frequency_max_hz > 0	configure
active_channel < channel_count	configure (create)
data_type ∈ {acceleration, velocity, displacement}	configure
enabled_plots non-empty after canonicalisation	configure
name 1–200 chars	baseline from-upload
password min length 1; email valid address	login
refresh_token min length 1	refresh, logout
Image ≤ 10 MB, MIME in the allow-list	equipment image
Measurement file ≤ 50 MB, .csv / .pdf	upload, baseline upload
from_date ≤ to_date	uploads list

5.9.3 Endpoint → table matrix

Table 102 — 5.9.3 Endpoint → table matrix

Endpoint group	Tables read	Tables written
Auth	users , roles , user_roles , refresh_tokens	users.last_login_at , refresh_tokens
Equipment	equipment_masters , sensor_configurations	both
Lookups	—	—
Configure	sensor_configurations	plot_configurations
Acquisition	sensor_configurations , plot_configurations	—
Upload	sensor_configurations , plot_configurations , feature_definitions , feature_threshold_rules , sensor_baselines , baseline_channel_features	sensor_data_uploads , measurement_upload_data , plot_results , measurement_channel_features , measurement_channel_feature_trends
Plots	sensor_data_uploads , plot_configurations , plot_results	plot_results (on cache miss)
Features / trends / compare	as above + feature tables + baselines	feature tables (on demand)
Baselines	sensor_baselines , baseline_plot_results , baseline_channel_features , measurement_upload_data	all three baseline tables

6.0 Database Documentation

6.1 Database Identity

Table 103 — 6.1 Database Identity

Property	Value	Source
Engine	PostgreSQL 16	<code>docker-compose.yml</code> → <code>image: postgres:16</code>
Database name	<code>vibration_platform</code>	<code>.env</code> → <code>POSTGRES_DB</code>
Owner / role	<code>vibration_user</code>	<code>.env</code> → <code>POSTGRES_USER</code>
Host port	5433 (mapped to container 5432)	<code>docker-compose.yml</code>
Connection URL (container network)	<code>postgresql://vibration_user:***@postgres:5432/vibration_platform</code>	<code>docker-compose.yml</code> → <code>DATABASE_URL</code>
Driver	<code>psycopg2-binary 2.9.9</code>	<code>requirements.txt</code>
Storage engine	PostgreSQL uses a single integrated storage engine; tables are heap-organised with MVCC. There is no per-table engine choice as in MySQL.	—
Character set / encoding	UTF8 — the default for the <code>postgres:16</code> image (<code>initdb</code> default, <code>en_US.utf8</code> locale). No <code>ENCODING</code> override is issued anywhere in the code.	—
Collation	Image default (<code>en_US.utf8</code>). No per-column <code>COLLATE</code> clause exists in any migration.	
Schema	<code>public</code> (no explicit schema is set)	
UUID generation	<code>gen_random_uuid()</code> as a server default in migrations; <code>uuid.uuid4</code> as a Python-side default in the ORM. <code>gen_random_uuid()</code> is built into PostgreSQL 13+, so no <code>pgcrypto</code> extension is required.	
Migration tool	Alembic — 11 revisions, head <code>011</code>	
Table count	17	
Column count	215	

6.2 Entity Relationship Diagram

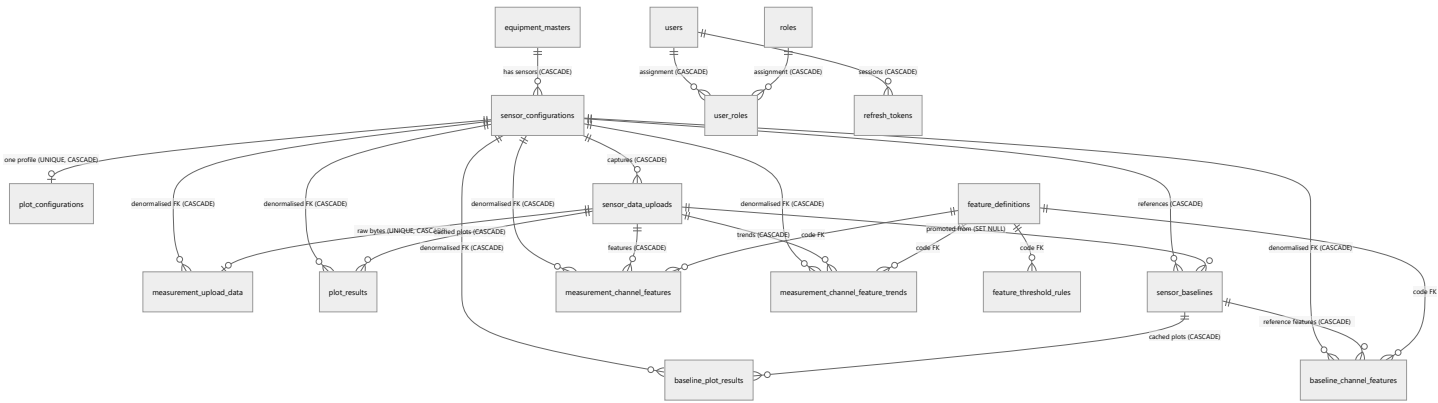


Figure 25 — 6.2 Entity Relationship Diagram

6.3 Relationship Catalogue

Table 104 — 6.3 Relationship Catalogue

#	Parent	Child	FK column	Cardinality	On delete	Declared in
1	equipment_masters	sensor_configurations	equipment_id	1 : N	CASCADE	001
2	sensor_configurations	plot_configurations	sensor_id (UNIQUE)	1 : 0..1	CASCADE	003
3	sensor_configurations	sensor_data_uploads	sensor_id	1 : N	CASCADE	003
4	sensor_data_uploads	plot_results	upload_id	1 : N	CASCADE	006
5	sensor_configurations	plot_results	sensor_id	1 : N	CASCADE	006
6	sensor_data_uploads	measurement_upload_data	upload_id (UNIQUE)	1 : 0..1	CASCADE	007
7	sensor_configurations	measurement_upload_data	sensor_id	1 : N	CASCADE	007
8	sensor_configurations	sensor_baselines	sensor_id	1 : N	CASCADE	007
9	sensor_data_uploads	sensor_baselines	source_upload_id	1 : N	SET NULL	007
10	sensor_baselines	baseline_plot_results	baseline_id	1 : N	CASCADE	007
11	sensor_configurations	baseline_plot_results	sensor_id	1 : N	CASCADE	007
12	feature_definitions	feature_threshold_rules	feature_code → code	1 : N	(default NO ACTION)	010
13	sensor_data_uploads	measurement_channel_features	upload_id	1 : N	CASCADE	010
14	sensor_configurations	measurement_channel_features	sensor_id	1 : N	CASCADE	010
15	feature_definitions	measurement_channel_features	feature_code	1 : N	NO ACTION	010
16	sensor_baselines	baseline_channel_features	baseline_id	1 : N	CASCADE	010
17	sensor_configurations	baseline_channel_features	sensor_id	1 : N	CASCADE	010
18	feature_definitions	baseline_channel_features	feature_code	1 : N	NO ACTION	010
19	sensor_data_uploads	measurement_channel_feature_trends	upload_id	1 : N	CASCADE	011
20	sensor_configurations	measurement_channel_feature_trends	sensor_id	1 : N	CASCADE	011
21	feature_definitions	measurement_channel_feature_trends	feature_code	1 : N	NO ACTION	011
22	users	user_roles	user_id (PK part)	1 : N	CASCADE	004
23	roles	user_roles	role_id (PK part)	1 : N	CASCADE	004
24	users	refresh_tokens	user_id	1 : N	CASCADE	004

Why relationship 9 differs. `sensor_baselines.source_upload_id` is `ON DELETE SET NULL` so that deleting a measurement upload never destroys the baseline promoted from it. The baseline keeps its own copy of the file bytes and parsed data, so it remains fully self-sufficient — only the provenance link is lost.

Why `sensor_id` is repeated on child tables. `plot_results`, `measurement_upload_data`, `baseline_plot_results`, and all three feature tables carry a direct `sensor_id` in addition to their `upload_id` / `baseline_id`. This is a deliberate denormalisation: it lets a query filter by sensor without joining through the upload or baseline, and it guarantees the row is removed when the sensor is deleted even if the intermediate path changes.

6.4 Table Reference

Legend for the **Nullable** column: NO = NOT NULL ; YES = nullable; PK = primary key.

6.4.1 equipment_masters

Purpose. One row per physical machine — the digital twin master record. Every downstream measurement is ultimately traceable to a row here. **Used by.** `crud/equipment.py` ; endpoints 7–20; the Equipment Master pages; the analysis page's equipment/sensor selectors. **CRUD.** Create (`POST /equipment/`), Read (list + detail), Update (`PUT / PATCH`), Delete (`DELETE` , cascading).

Table 105 — 6.4.1 equipment_masters

Column	Type	Nullable	Default	Purpose
id	UUID	PK	gen_random_uuid()	Surrogate key
plant_name	VARCHAR(255)	NO	—	Plant in the asset hierarchy; filterable with ILIKE
area	VARCHAR(255)	NO	—	Functional area within the plant
line	VARCHAR(255)	NO	—	Production line
machine_name	VARCHAR(255)	NO	—	Human-readable machine label
machine_id	VARCHAR(100)	YES	NULL	Plant asset code. UNIQUE ; empty strings are coerced to NULL so multiple blanks are allowed
machine_type	VARCHAR(50)	NO	—	One of the 13 machine-types lookup values; drives conditional form fields and diagnostic expectations
machine_criticality	VARCHAR(20)	NO	—	Low / Medium / High / Critical
manufacturer	VARCHAR(255)	YES	—	OEM
model	VARCHAR(255)	YES	—	Model designation
serial_number	VARCHAR(100)	YES	—	OEM serial
rated_power_kw	NUMERIC(10,2)	YES	—	Nameplate power; used to normalise amplitude against load
rated_rpm	INTEGER	YES	—	Nameplate speed; the basis for shaft-frequency and harmonic markers
drive_type	VARCHAR(50)	YES	—	Direct / Belt / Gear / Chain / VFD / Hydraulic
load_type	VARCHAR(50)	YES	—	Constant / Variable / Intermittent / Cyclic / Shock
foundation_type	VARCHAR(50)	YES	—	Affects expected vibration transmission
coupling_details	VARCHAR(50)	YES	—	Coupling family
bearing_details	TEXT	YES	—	Free-text bearing specification
bearing_number_de	VARCHAR(100)	YES	—	Drive-end bearing designation; required for BPFO/BPFI computation
bearing_number_nde	VARCHAR(100)	YES	—	Non-drive-end bearing designation
gearbox_ratio	NUMERIC(8,3)	YES	—	Reduction ratio
gear_teeth	INTEGER	YES	—	Tooth count for gear-mesh frequency
motor_pole_count	INTEGER	YES	—	2–12; for electrical fault frequencies
fan_blades	INTEGER	YES	—	Blade-pass frequency input
pump_vanes	INTEGER	YES	—	Vane-pass frequency input
direction_of_rotation	VARCHAR(30)	YES	—	Clockwise / Counter-Clockwise / Bidirectional
operating_speed_min	INTEGER	YES	—	Lower speed envelope; part of AI readiness
operating_speed_max	INTEGER	YES	—	Upper speed envelope; part of AI readiness
load_range_min	NUMERIC(5,2)	YES	—	Minimum load %
load_range_max	NUMERIC(5,2)	YES	—	Maximum load %
normal_operating_load	NUMERIC(5,2)	YES	—	Typical load %
process_details	TEXT	YES	—	Process narrative
operating_environment	VARCHAR[]	YES	—	PostgreSQL text array of the 13 environment values
lubrication_type	VARCHAR(50)	YES	—	Grease / Oil Bath / ...
installation_date	DATE	YES	—	Commissioning date
last_maintenance_date	DATE	YES	—	Last intervention
maintenance_notes	TEXT	YES	—	Free text
equipment_image_path	VARCHAR(500)	YES	—	Filesystem path to the uploaded photo
asset_status	VARCHAR(30)	YES	'Active'	Active / Inactive / Under Maintenance / Decommissioned
machine_train_configured	BOOLEAN	YES	false	AI readiness flag (manual)
bearing_database_mapped	BOOLEAN	YES	false	AI readiness flag (manual)
operating_mode_configured	BOOLEAN	YES	false	AI readiness flag (manual)
created_at	TIMESTAMP TZ	YES	now()	Insert time; the list sort key
updated_at	TIMESTAMP TZ	YES	now()	ORM sets onupdate=utcnow

Constraints. PK `id`; UNIQUE `uq_equipment_machine_id (machine_id)`. **Indexes.** `ix_equipment_plant_name (plant_name)`, `ix_equipment_machine_type (machine_type)`, plus the unique index backing `machine_id`. **Performance note.** `plant_name` filtering uses `ILIKE '%value%'`, which cannot use the plain B-tree index — a leading-wildcard match forces a sequential scan. See §14.6.

6.4.2 `sensor_configurations`

Purpose. A measurement point on a machine — the unit that owns captures, plots, features, and baselines. **Used by.** `crud/equipment.py` sensor functions; endpoints 16–19; every measurement and baseline endpoint (via `sensor_id`).

Table 106 — 6.4.2 `sensor_configurations`

Column	Type	Nullable	Default	Purpose
<code>id</code>	UUID	PK	<code>gen_random_uuid()</code>	Surrogate key; used as <code>sensor_id</code> everywhere downstream
<code>equipment_id</code>	UUID	NO	—	FK → <code>equipment_masters.id</code> CASCADE
<code>sensor_type</code>	VARCHAR(50)	NO	—	One of 14 sensor types; influences the edge <code>transducerType</code>
<code>mounting_location</code>	VARCHAR(100)	NO	—	Bearing Housing DE, Motor NDE, ...
<code>orientation</code>	VARCHAR(30)	NO	—	Horizontal / Vertical / Axial / Radial / Tangential; mapped to the edge <code>machineAxis</code>
<code>mounting_method</code>	VARCHAR(50)	YES	—	Stud / Magnetic / Adhesive / ...
<code>sensitivity</code>	NUMERIC(10,4)	YES	—	Transducer sensitivity; emitted as <code>sensitivityMvPerG</code>
<code>sensitivity_unit</code>	VARCHAR(20)	YES	—	mV/g, mV/mm/s, mV/μm, mA, V
<code>sampling_rate</code>	VARCHAR(20)	YES	—	Human label such as <code>"25600 Hz"</code> or <code>"Custom"</code>
<code>sampling_rate_custom</code>	INTEGER	YES	—	Numeric value when the label is <code>Custom</code>
<code>frequency_range</code>	VARCHAR(20)	YES	—	Label such as <code>"0-10000 Hz"</code>
<code>frequency_range_custom_min</code>	INTEGER	YES	—	Custom lower bound
<code>frequency_range_custom_max</code>	INTEGER	YES	—	Custom upper bound
<code>is_active</code>	BOOLEAN	YES	<code>true</code>	Soft enable/disable
<code>device_id</code>	VARCHAR(64)	YES	NULL	UNIQUE MAC-style edge identifier; the join key for <code>/measurements/acquisition</code>
<code>created_at</code>	TIMESTAMPTZ	YES	<code>now()</code>	Insert time

Constraints. PK `id`; FK `equipment_id`; UNIQUE index on `device_id`. **Indexes.** `ix_sensor_equipment_id (equipment_id)`, `ix_sensor_configurations_device_id (device_id)` UNIQUE.

Design observation: the sensor stores sampling rate and frequency range as *labels* for documentation, while the numeric values actually used for processing live in `plot_configurations`. The two are not synchronised by any code path.

6.4.3 `plot_configurations`

Purpose. The single processing profile per sensor. Its contents feed `compute_config_fingerprint`, so changing any of `sampling_rate_hz`, `fft_lines`, `frequency_max_hz`, `data_type`, or `enabled_plots` invalidates every cached plot for that sensor's uploads.

Table 107 — 6.4.3 `plot_configurations`

Column	Type	Nullable	Default	Purpose
id	UUID	PK	gen_random_uuid()	Surrogate key
sensor_id	UUID	NO	—	FK → sensor_configurations.id CASCADE; UNIQUE (one profile per sensor)
channel_count	INTEGER	NO	1	1–32
active_channel	INTEGER	NO	0	Default channel for plot reads; clamped on read
sampling_rate_hz	NUMERIC(12,4)	NO	25600	Governs the FFT frequency axis and all trend time axes
fft_lines	INTEGER	NO	1600	Sample count truncation before the FFT (64–65536)
frequency_max_hz	NUMERIC(12,4)	YES	NULL	Optional spectrum cut-off
data_type	VARCHAR(30)	NO	'acceleration'	acceleration / velocity / displacement
enabled_plots	JSONB	NO	'[]'	Array of plot-type strings; an empty array is treated as "all five" on read
created_at	TIMESTAMPTZ	YES	now()	Insert time
updated_at	TIMESTAMPTZ	YES	now()	Set explicitly by update_plot_config

Indexes. ix_plot_config_sensor_id (sensor_id) **UNIQUE**.

6.4.4 sensor_data_uploads

Purpose. The lifecycle record for one capture. Its three status triplets make the processing pipeline observable.

Table 108 — 6.4.4 sensor_data_uploads

Column	Type	Nullable	Default	Purpose
id	UUID	PK	gen_random_uuid()	Also used as the on-disk filename stem
sensor_id	UUID	NO	—	FK → sensor_configurations.id CASCADE
channel_count	INTEGER	NO	—	Channels requested at upload time
pdf_path	VARCHAR(500)	NO	—	Path to the original file (CSV or PDF, despite the column name)
parsed_data_path	VARCHAR(500)	YES	—	Path to the parsed JSON sidecar
sample_count	INTEGER	YES	—	Rows parsed
parse_status	VARCHAR(20)	NO	'pending'	pending → parsed failed
parse_error	TEXT	YES	—	Parser message on failure
plots_status	VARCHAR(20)	NO	'pending'	pending → ready failed
plots_error	TEXT	YES	—	Plot computation message
plots_computed_at	TIMESTAMPTZ	YES	—	Last successful plot computation
features_status	VARCHAR(20)	NO	'pending'	pending → ready failed
features_error	TEXT	YES	—	Feature computation message
features_computed_at	TIMESTAMP	YES	—	Last successful feature computation (note: added in 010 as naive DateTime)
original_filename	VARCHAR(255)	YES	—	User-supplied filename; backfilled by 009
source	VARCHAR(20)	NO	'manual'	Ingestion origin; only manual is produced today
created_at	TIMESTAMPTZ	YES	now()	Capture-record creation, used as the timeline position
parsed_at	TIMESTAMPTZ	YES	—	Parse completion

Indexes. ix_sensor_upload_sensor_id (sensor_id), ix_sensor_data_uploads_sensor_id_created_at (sensor_id, created_at). **Why the composite index exists.** Every timeline query is WHERE sensor_id = ? AND created_at BETWEEN ? AND ? ORDER BY created_at DESC — the composite index serves the filter and the sort together.

6.4.5 measurement_upload_data

Purpose. The durable copy of the capture: original bytes plus the parsed arrays, both in PostgreSQL. This is what makes POST /baselines/from-upload/{id} independent of the filesystem.

Table 109 — 6.4.5 measurement_upload_data

Column	Type	Nullable	Default	Purpose
id	UUID	PK	gen_random_uuid()	Surrogate key
upload_id	UUID	NO	—	FK → sensor_data_uploads.id CASCADE; UNIQUE (1:0.1)
sensor_id	UUID	NO	—	FK → sensor_configurations.id CASCADE
original_filename	VARCHAR(255)	NO	—	As uploaded
file_format	VARCHAR(10)	NO	—	csv or pdf
file_content	BYTEA	NO	—	The complete original file
parsed_data	JSONB	NO	—	{timestamps, channels, sample_count, channel_count, detected_channel_count}
channel_count	INTEGER	NO	—	Effective channel count
sample_count	INTEGER	NO	—	Row count
created_at	TIMESTAMPZ	YES	now()	Insert time

Indexes. ix_measurement_upload_data_sensor_id (sensor_id); unique index backing upload_id. **Storage note.** BYTEA and JSONB values beyond ~2 kB are TOAST-compressed and stored out of line, so wide-row reads that do not select these columns remain cheap. A 50 MB upload therefore does not slow down SELECT upload_id FROM measurement_upload_data.

6.4.6 plot_results

Purpose. The plot cache. One row per (upload, plot type, channel, fingerprint).

Table 110 — 6.4.6 plot_results

Column	Type	Nullable	Default	Purpose
id	UUID	PK	gen_random_uuid()	Surrogate key
upload_id	UUID	NO	—	FK → sensor_data_uploads.id CASCADE
sensor_id	UUID	NO	—	FK → sensor_configurations.id CASCADE
plot_type	VARCHAR(40)	NO	—	One of the five canonical types
channel	INTEGER	NO	—	0-based channel index
title	VARCHAR(120)	NO	—	Display title from the DSP routine
x_label	VARCHAR(80)	NO	—	Axis label
y_label	VARCHAR(80)	NO	—	Axis label
x_data	JSONB	NO	—	X array
y_data	JSONB	NO	—	Y array
metadata	JSONB	NO	'{'	Plot-specific extras (plot_style , fft_lines , sampling_rate_hz , num_segments , samples); mapped to metadata_ in Python
point_count	INTEGER	NO	—	len(x) , stored for cheap size inspection
sampling_rate_hz	NUMERIC(12,4)	NO	—	Rate used at computation time
fft_lines	INTEGER	YES	—	Lines used
frequency_max_hz	NUMERIC(12,4)	YES	—	Cut-off used
config_fingerprint	VARCHAR(64)	NO	—	32-hex-char SHA-256 prefix of the processing configuration
computed_at	TIMESTAMPZ	NO	now()	Computation timestamp
status	VARCHAR(20)	NO	'ready'	Only ready rows are read

Constraints. UNIQUE uq_plot_results_upload_plot_channel_fingerprint (upload_id, plot_type, channel, config_fingerprint). **Indexes.** ix_plot_results_upload_id , ix_plot_results_sensor_id .

6.4.7 sensor_baselines

Purpose. Append-only reference captures. The code comment is explicit: "Historical baseline records — all rows kept (append-only) for RAG / learning."

Table 111 — 6.4.7 sensor_baselines

Column	Type	Nullable	Default	Purpose
id	UUID	PK	gen_random_uuid()	Surrogate key
sensor_id	UUID	NO	—	FK → sensor_configurations.id CASCADE
source_upload_id	UUID	YES	NULL	FK → sensor_data_uploads.id SET NULL; NULL for direct uploads
name	VARCHAR(200)	NO	—	User-supplied label
description	TEXT	YES	—	Free text
labels	JSONB	NO	'[]'	Tag array for future classification
original_filename	VARCHAR(255)	NO	—	Provenance
file_format	VARCHAR(10)	NO	—	csv / pdf
file_content	BYTEA	NO	—	Complete original bytes — the reason a baseline survives upload deletion
parsed_data	JSONB	NO	—	Parsed channel arrays
channel_count	INTEGER	NO	—	Channels
sample_count	INTEGER	NO	—	Samples
sampling_rate_hz	NUMERIC(12,4)	NO	—	Rate captured from the plot configuration at creation time
is_primary	BOOLEAN	NO	false	Display/comparison default; at most one true per sensor, enforced in application code
captured_at	TIMESTAMPTZ	YES	—	Physical capture time; defaults to utcnow() when not supplied
created_at	TIMESTAMPTZ	YES	now()	Row creation; the list sort key

Indexes. ix_sensor_baselines_sensor_id, ix_sensor_baselines_created_at. **Integrity note.** The "one primary per sensor" rule is enforced by create_baseline and set_baseline_primary (both clear the previous primary first), **not** by a partial unique index. A concurrent double set-primary could therefore produce two primaries; get_primary_baseline mitigates this by ordering created_at DESC and taking the first.

6.4.8 baseline_plot_results

Identical column set to plot_results with baseline_id in place of upload_id:

Table 112 — 6.4.8 baseline_plot_results

Column	Type	Nullable	Default
id	UUID	PK	gen_random_uuid()
baseline_id	UUID	NO	FK → sensor_baselines.id CASCADE
sensor_id	UUID	NO	FK → sensor_configurations.id CASCADE
plot_type	VARCHAR(40)	NO	—
channel	INTEGER	NO	—
title / x_label / y_label	VARCHAR(120/80/80)	NO	—
x_data / y_data	JSONB	NO	—
metadata	JSONB	NO	'{'
point_count	INTEGER	NO	—
sampling_rate_hz	NUMERIC(12,4)	NO	—
fft_lines	INTEGER	YES	—
frequency_max_hz	NUMERIC(12,4)	YES	—
config_fingerprint	VARCHAR(64)	NO	—
computed_at	TIMESTAMPTZ	NO	now()
status	VARCHAR(20)	NO	'ready'

Constraints. UNIQUE uq_baseline_plot_results_baseline_plot_channel_fingerprint. **Indexes.** ix_baseline_plot_results_baseline_id.

6.4.9 feature_definitions

Purpose. The catalogue of the ten computed features — the reference table for names, units, and display order.

Table 113 — 6.4.9 feature_definitions

Column	Type	Nullable	Default	Purpose
id	UUID	PK	(Python-side <code>uuid4</code>)	Surrogate key
code	VARCHAR(40)	NO	—	UNIQUE ; the FK target for all feature tables
name	VARCHAR(120)	NO	—	Display name
unit	VARCHAR(30)	NO	—	<code>scaled_eng</code> , <code>scaled_eng_sq</code> , <code>dimensionless</code> , <code>dB</code>
description	TEXT	YES	—	Explanation of the metric
sort_order	INTEGER	NO	<code>0</code>	Display order (0–9)
is_active	BOOLEAN	NO	<code>true</code>	Soft disable

Seeded rows (migration 010).

Table 114 — 6.4.9 feature_definitions

code	name	unit	sort_order	description
rms	RMS	scaled_eng	0	Root mean square amplitude
peak	Peak	scaled_eng	1	Maximum absolute amplitude
crest_factor	Crest Factor	dimensionless	2	Peak / RMS
kurtosis	Kurtosis	dimensionless	3	Excess kurtosis
fft_band_energy_0_500	FFT Band Energy (0-500 Hz)	scaled_eng_sq	4	Sum of squared FFT magnitudes 0-500 Hz
amplitude_1x	1X Amplitude	scaled_eng	5	FFT magnitude at running speed
amplitude_2x	2X Amplitude	scaled_eng	6	FFT magnitude at 2× running speed
amplitude_3x	3X Amplitude	scaled_eng	7	FFT magnitude at 3× running speed
envelope_rms	Envelope RMS	scaled_eng	8	RMS of Hilbert envelope
noise_floor	Noise Floor	dB	9	Mean FFT magnitude in dB

6.4.10 feature_threshold_rules

Purpose. Configurable evaluation rules turning a feature value into a status.

Table 115 — 6.4.10 feature_threshold_rules

Column	Type	Nullable	Default	Purpose
id	UUID	PK	uuid4	Surrogate key
feature_code	VARCHAR(40)	NO	—	FK → feature_definitions.code
rule_type	VARCHAR(30)	NO	—	<code>absolute_max</code> , <code>absolute_db</code> , <code>range</code> , <code>percent_rms</code> , <code>percent_baseline</code>
machine_type	VARCHAR(80)	YES	NULL	NULL = global rule; a value scopes the rule to a machine type
normal_max	NUMERIC(18,8)	YES	—	Upper bound of the normal band
warning_max	NUMERIC(18,8)	YES	—	Upper bound of the warning band
normal_min	NUMERIC(18,8)	YES	—	Lower bound (range rules)
warning_min	NUMERIC(18,8)	YES	—	Lower warning bound (range rules)
metadata	JSONB	NO	'{}'	Extra parameters, e.g. <code>{"critical_percent": 150.0}</code>
is_active	BOOLEAN	NO	true	Soft disable

Indexes. ix_feature_threshold_rules_code_machine (feature_code, machine_type).

Seeded rows (migration 010). All ten are global (machine_type = NULL).

Table 116 — 6.4.10 feature_threshold_rules

feature_code	rule_type	normal_max	warning_max	normal_min	warning_min	metadata
rms	absolute_max	0.01	0.02	—	—	{}
peak	absolute_max	0.05	0.10	—	—	{}
crest_factor	range	3.0	5.0	1.4	3.0	{}
kurtosis	absolute_max	3.5	5.0	—	—	{}
fft_band_energy_0_500	percent_baseline	120.0	150.0	—	—	{}
amplitude_1x	percent_rms	20.0	40.0	—	—	{}
amplitude_2x	percent_rms	10.0	20.0	—	—	{}
amplitude_3x	percent_rms	5.0	15.0	—	—	{}
envelope_rms	percent_baseline	100.0	125.0	—	—	{"critical_percent": 150.0}
noise_floor	absolute_db	-60.0	-54.0	—	—	{}

The migration tuple order is (code, rule_type, machine_type, normal_max, warning_max, normal_min, warning_min, metadata) — note that machine_type is the third element and is None for every seeded rule, which is why the frontend mirror in lib/feature-threshold-lines.ts uses the same numbers with no machine scoping.

6.4.11 measurement_channel_features

Purpose. Evaluated scalar feature values per upload and channel — the source for the Status (Health) tables and summary cards.

Table 117 — 6.4.11 measurement_channel_features

Column	Type	Nullable	Default	Purpose
id	UUID	PK	uuid4	Surrogate key
upload_id	UUID	NO	—	FK → sensor_data_uploads.id CASCADE
sensor_id	UUID	NO	—	FK → sensor_configurations.id CASCADE
channel	INTEGER	NO	—	0-based channel
feature_code	VARCHAR(40)	NO	—	FK → feature_definitions.code
value	NUMERIC(18,8)	NO	—	Computed value
unit	VARCHAR(30)	NO	—	Copied from the extractor
status	VARCHAR(20)	NO	—	normal / warning / critical / no_baseline
metadata	JSONB	NO	'{}'	e.g. {estimated_shaft_hz, sampling_rate_hz, sample_count} or {band_hz:[0,500]}
computed_at	TIMESTAMP	NO	—	Single timestamp shared by the whole batch

Constraints. UNIQUE uq_measurement_channel_feature (upload_id, channel, feature_code). **Indexes.** ix_measurement_channel_features_upload (upload_id). **Row volume.** channels × 10. An 8-channel upload writes 80 rows.

6.4.12 measurement_channel_feature_trends

Purpose. Per-segment feature series inside a single capture — the data behind the ten trend cards.

Table 118 — 6.4.12 measurement_channel_feature_trends

Column	Type	Nullable	Default	Purpose
<code>id</code>	UUID	PK	<code>uuid4</code>	Surrogate key
<code>upload_id</code>	UUID	NO	—	FK → <code>sensor_data_uploads.id</code> CASCADE
<code>sensor_id</code>	UUID	NO	—	FK → <code>sensor_configurations.id</code> CASCADE
<code>channel</code>	INTEGER	NO	—	0-based channel
<code>feature_code</code>	VARCHAR(40)	NO	—	FK → <code>feature_definitions.code</code>
<code>segment_index</code>	INTEGER	NO	—	0-based segment number
<code>time_s</code>	NUMERIC(18,8)	NO	—	Segment mid-point time in seconds
<code>value</code>	NUMERIC(18,8)	NO	—	Feature value for that segment
<code>computed_at</code>	TIMESTAMP	NO	—	Batch timestamp

Constraints. UNIQUE `uq_measurement_channel_feature_trend` (`upload_id`, `channel`, `feature_code`, `segment_index`). **Indexes.**

`ix_measurement_channel_feature_trends_upload` (`upload_id`, `channel`) — composite, because every read is scoped to one channel. **Row volume.** `channels` × 10 × ~32. An 8-channel upload writes about 2560 rows. This is by far the largest table by row count.

6.4.13 `baseline_channel_features`

Purpose. The reference feature values a capture is compared against.

Table 119 — 6.4.13 `baseline_channel_features`

Column	Type	Nullable	Default	Purpose
<code>id</code>	UUID	PK	<code>uuid4</code>	Surrogate key
<code>baseline_id</code>	UUID	NO	—	FK → <code>sensor_baselines.id</code> CASCADE
<code>sensor_id</code>	UUID	NO	—	FK → <code>sensor_configurations.id</code> CASCADE
<code>channel</code>	INTEGER	NO	—	0-based channel
<code>feature_code</code>	VARCHAR(40)	NO	—	FK → <code>feature_definitions.code</code>
<code>value</code>	NUMERIC(18,8)	NO	—	Reference value
<code>unit</code>	VARCHAR(30)	NO	—	Unit
<code>status</code>	VARCHAR(20)	NO	<code>'normal'</code>	Always written as <code>normal</code> by <code>copy_upload_features_to_baseline</code>
<code>metadata</code>	JSONB	NO	<code>'{}'</code>	Copied from the source feature row
<code>computed_at</code>	TIMESTAMP	NO	—	Copy timestamp

Constraints. UNIQUE `uq_baseline_channel_feature` (`baseline_id`, `channel`, `feature_code`). **Indexes.** `ix_baseline_channel_features_baseline` (`baseline_id`).

6.4.14 `roles`

Table 120 — 6.4.14 `roles`

Column	Type	Nullable	Default	Purpose
<code>id</code>	UUID	PK	<code>gen_random_uuid()</code>	Surrogate key
<code>name</code>	VARCHAR(50)	NO	—	UNIQUE role name
<code>description</code>	VARCHAR(255)	YES	—	Human description
<code>created_at</code>	TIMESTAMPTZ	YES	<code>now()</code>	Insert time

Seeded rows. Migration 004 inserts five: `super_admin` ("Full platform access"), `plant_admin` ("Manage users and data within assigned plants"), `engineer` ("Equipment and vibration analysis CRUD"), `operator` ("View dashboards and plots"), `viewer` ("Read-only access"). Migration 008 adds two more: `admin` ("Application administrator with full write access.") and `user` ("Read-only platform user.") with `ON CONFLICT (name) DO NOTHING`.

Total: 7 rows. Only three (`super_admin`, `admin`, `user`) are used by the current authorisation logic; the other four are legacy and are still mapped defensively by `_user_role` / `primary_role` (`plant_admin` and `engineer` collapse to `admin`).

Indexes. `ix_roles_name` (`name`) UNIQUE.

6.4.15 users

Table 121 — 6.4.15 users

Column	Type	Nullable	Default	Purpose
id	UUID	PK	gen_random_uuid()	Surrogate key; the JWT sub
email	VARCHAR(255)	NO	—	UNIQUE, stored lower-case, the login identifier
password_hash	VARCHAR(255)	NO	—	bcrypt hash (never the password)
full_name	VARCHAR(150)	NO	—	Display name
role	VARCHAR(50)	NO	'user'	Authoritative role; CHECK constrained
is_active	BOOLEAN	NO	true	Disabling a user invalidates authentication immediately
must_change_password	BOOLEAN	NO	false	Forces the /change-password redirect
last_login_at	TIMESTAMPTZ	YES	—	Updated on every successful login
created_at	TIMESTAMPTZ	YES	now()	Insert time
updated_at	TIMESTAMPTZ	YES	now()	ORM onupdate

Constraints. PK id; UNIQUE email; CHECK ck_users_role_allowed: role IN ('super_admin', 'admin', 'user'). **Indexes.** ix_users_email (email) UNIQUE.

6.4.16 user_roles

Table 122 — 6.4.16 user_roles

Column	Type	Nullable	Default	Purpose
user_id	UUID	PK part	—	FK → users.id CASCADE
role_id	UUID	PK part	—	FK → roles.id CASCADE
assigned_at	TIMESTAMPTZ	YES	now()	Assignment time

Composite primary key (user_id, role_id) prevents duplicate assignments. This table is the legacy many-to-many path; users.role is authoritative.

6.4.17 refresh_tokens

Table 123 — 6.4.17 refresh_tokens

Column	Type	Nullable	Default	Purpose
id	UUID	PK	gen_random_uuid()	Surrogate key
user_id	UUID	NO	—	FK → users.id CASCADE
token_hash	VARCHAR(64)	NO	—	UNIQUE SHA-256 hex digest of the plaintext token
expires_at	TIMESTAMPTZ	NO	—	Issue time + jwt_refresh_expire_days (7)
revoked_at	TIMESTAMPTZ	YES	NULL	Set on logout and on rotation
created_at	TIMESTAMPTZ	YES	now()	Issue time

Indexes. ix_refresh_tokens_token_hash (token_hash) UNIQUE, ix_refresh_tokens_user_id (user_id). **Retention.** Rows are never deleted — only marked revoked. The table grows by one row per login and per refresh. See §14.7 for the recommended cleanup job.

6.5 Constraint Summary

6.5.1 Primary keys

Every table uses a surrogate UUID primary key except user_roles, which uses the composite natural key (user_id, role_id).

6.5.2 Unique constraints and unique indexes

Table 124 — 6.5.2 Unique constraints and unique indexes

Object	Table	Columns
uq_equipment_machine_id	equipment_masters	machine_id
ix_sensor_configurations_device_id	sensor_configurations	device_id
ix_plot_config_sensor_id	plot_configurations	sensor_id
column-level UNIQUE	measurement_upload_data	upload_id
uq_plot_results_upload_plot_channel_fingerprint	plot_results	upload_id, plot_type, channel, config_fingerprint
uq_baseline_plot_results_baseline_plot_channel_fingerprint	baseline_plot_results	baseline_id, plot_type, channel, config_fingerprint
column-level UNIQUE	feature_definitions	code
uq_measurement_channel_feature	measurement_channel_features	upload_id, channel, feature_code
uq_measurement_channel_feature_trend	measurement_channel_feature_trends	upload_id, channel, feature_code, segment_index
uq_baseline_channel_feature	baseline_channel_features	baseline_id, channel, feature_code
ix_roles_name	roles	name
ix_users_email	users	email
ix_refresh_tokens_token_hash	refresh_tokens	token_hash

6.5.3 Check constraints

Table 125 — 6.5.3 Check constraints

Constraint	Table	Definition
ck_users_role_allowed	users	role IN ('super_admin','admin','user')

This is the only `CHECK` constraint in the schema. All other value domains (plot types, statuses, machine types, rule types) are enforin application code — Pydantic validators, service-layer constants, or the frontend lookup lists — not in the database.

6.5.4 Default values

Table 126 — 6.5.4 Default values

Kind	Examples
Server-side (<code>server_default</code>)	<code>gen_random_uuid()</code> for PKs, <code>now()</code> for timestamps, <code>'pending'</code> for statuses, <code>'ready'</code> for plot status, <code>'Active'</code> for <code>asset_status</code> , <code>'manual'</code> for <code>source</code> , <code>'user'</code> for <code>users.role</code> , <code>true / false</code> for booleans, <code>'[]'::jsonb / '{}'::jsonb</code> for JSONB
Python-side (ORM <code>default=</code>)	<code>uuid.uuid4</code> for feature-table PKs, <code>datetime.utcnow</code> for <code>computed_at</code> , <code>list / dict</code> for JSONB attributes

Note the mixture: tables created in migrations 001–009 use `server_default` for IDs, while the feature tables created in 010–011 rely on the Python-side `uuid4` default. Inserting into the feature tables with raw SQL therefore requires an explicit `id`.

6.5.5 Complete index inventory (24 indexes)

Table 127 — 6.5.5 Complete index inventory (24 indexes)

Index	Table	Columns	Unique	Purpose
PK indexes (17)	all tables	primary key	✓	Identity lookup
<code>ix_equipment_plant_name</code>	<code>equipment_masters</code>	<code>plant_name</code>	—	Plant filter (limited use — see §14.6)
<code>ix_equipment_machine_type</code>	<code>equipment_masters</code>	<code>machine_type</code>	—	Type filter
<code>uq_equipment_machine_id</code>	<code>equipment_masters</code>	<code>machine_id</code>	✓	Asset-code uniqueness + duplicate check
<code>ix_sensor_equipment_id</code>	<code>sensor_configurations</code>	<code>equipment_id</code>	—	Sensor list per machine
<code>ix_sensor_configurations_device_id</code>	<code>sensor_configurations</code>	<code>device_id</code>	✓	Edge acquisition lookup
<code>ix_plot_config_sensor_id</code>	<code>plot_configurations</code>	<code>sensor_id</code>	✓	Config lookup + one-per-sensor rule
<code>ix_sensor_upload_sensor_id</code>	<code>sensor_data_uploads</code>	<code>sensor_id</code>	—	Uploads per sensor
<code>ix_sensor_data_uploads_sensor_id_created_at</code>	<code>sensor_data_uploads</code>	<code>sensor_id</code> , <code>created_at</code>	—	Timeline range query + sort
<code>ix_plot_results_upload_id</code>	<code>plot_results</code>	<code>upload_id</code>	—	Cache read
<code>ix_plot_results_sensor_id</code>	<code>plot_results</code>	<code>sensor_id</code>	—	Sensor-scoped reads
<code>uq_plot_results_*</code>	<code>plot_results</code>	4 columns	✓	Cache identity
<code>ix_measurement_upload_data_sensor_id</code>	<code>measurement_upload_data</code>	<code>sensor_id</code>	—	Sensor-scoped reads
<code>ix_sensor_baselines_sensor_id</code>	<code>sensor_baselines</code>	<code>sensor_id</code>	—	Baseline list
<code>ix_sensor_baselines_created_at</code>	<code>sensor_baselines</code>	<code>created_at</code>	—	Chronological ordering
<code>ix_baseline_plot_results_baseline_id</code>	<code>baseline_plot_results</code>	<code>baseline_id</code>	—	Cache read
<code>uq_baseline_plot_results_*</code>	<code>baseline_plot_results</code>	4 columns	✓	Cache identity
<code>ix_feature_threshold_rules_code_machine</code>	<code>feature_threshold_rules</code>	<code>feature_code</code> , <code>machine_type</code>	—	Rule resolution
<code>ix_measurement_channel_features_upload</code>	<code>measurement_channel_features</code>	<code>upload_id</code>	—	Feature read
<code>uq_measurement_channel_feature</code>	<code>measurement_channel_features</code>	3 columns	✓	One value per feature/channel
<code>ix_measurement_channel_feature_trends_upload</code>	<code>measurement_channel_feature_trends</code>	<code>upload_id</code> , <code>channel</code>	—	Trend read (channel-scoped)
<code>uq_measurement_channel_feature_trend</code>	<code>measurement_channel_feature_trends</code>	4 columns	✓	One value per segment
<code>ix_baseline_channel_features_baseline</code>	<code>baseline_channel_features</code>	<code>baseline_id</code>	—	Reference read
<code>uq_baseline_channel_feature</code>	<code>baseline_channel_features</code>	3 columns	✓	One value per feature/channel
<code>ix_roles_name</code>	<code>roles</code>	<code>name</code>	✓	Role lookup by name
<code>ix_users_email</code>	<code>users</code>	<code>email</code>	✓	Login lookup
<code>ix_refresh_tokens_token_hash</code>	<code>refresh_tokens</code>	<code>token_hash</code>	✓	Refresh validation
<code>ix_refresh_tokens_user_id</code>	<code>refresh_tokens</code>	<code>user_id</code>	—	Bulk revocation

6.6 Triggers, Views, Stored Procedures, Functions

None of these objects exist in the schema. A full search of `backend/alembic/versions/` finds no `CREATE TRIGGER`, `CREATE VIEW`, `CREATE FUNCTION`, or `CREATE PROCEDURE` statement. The only database-side executable code invoked anywhere is the built-in `gen_random_uuid()` and `now()` functions used as column defaults.

Everything that would conventionally be a trigger is done in Python:

Table 128 — 6.6 Triggers, Views, Stored Procedures, Functions

Conventional trigger	Where it is done instead
<code>updated_at</code> maintenance	SQLAlchemy <code>onupdate=datetime.utcnow</code> on <code>Equipment.updated_at</code> , <code>User.updated_at</code> ; explicit assignment in <code>update_plot_config</code>
Cascading soft-state updates	Explicit <code>mark_upload_*</code> functions in <code>crud/measurement.py</code> and <code>crud/feature.py</code>
Single-primary-baseline enforcement	<code>crud/baseline.py::create_baseline</code> / <code>set_baseline_primary</code>
Audit trail	Not implemented (<code>created_at</code> / <code>updated_at</code> only)

6.7 Normalisation Analysis

The schema is essentially in **Third Normal Form**, with four deliberate, documented denormalisations.

Normal-form assessment

Table 129 — 6.7 Normalisation Analysis

Table group	Form	Reasoning
equipment_masters	3NF	All 40 attributes depend on the machine's identity alone. <code>operating_environment</code> uses a PostgreSQL array rather than a junction table — strictly a 1NF deviation, but the array is a closed, non-queried lookup set used only for display and filtering in the UI
sensor_configurations , plot_configurations	3NF	Single-key dependency; the 1:1 with <code>plot_configurations</code> is enforced by a unique index
sensor_data_uploads , measurement_upload_data	3NF (vertical partition)	The heavy <code>BYTEA</code> / <code>JSONB</code> payload is split into its own table so lifecycle queries never touch it
feature_definitions , feature_threshold_rules	3NF	Classic reference/rule pair keyed by <code>code</code>
Feature/trend/plot tables	3NF w.r.t. their composite business keys	Each non-key column depends on the whole key (<code>upload_id</code> , <code>channel</code> , <code>feature_code</code> [, <code>segment_index</code>])
users , roles , user_roles , refresh_tokens	3NF	Standard RBAC shape

Intentional denormalisations

Table 130 — 6.7 Normalisation Analysis

#	Denormalisation	Rationale	Risk accepted
1	<code>sensor_id</code> duplicated on 6 child tables	Enables sensor-scoped queries and cascade deletion without multi-level joins	The value could theoretically diverge from the parent's <code>sensor_id</code> ; nothing enforces agreement
2	<code>plot_results</code> / <code>baseline_plot_results</code> store computed arrays	FFT and Hilbert transforms are far too expensive to run per page view	Stale rows if the algorithm changes without bumping <code>ALGORITHM_VERSION</code>
3	<code>users.role</code> alongside <code>user_roles</code>	A single scalar avoids a join on every authenticated request and is <code>CHECK</code> -constrained	Two sources of truth; <code>_user_role</code> prefers the scalar and falls back to the relationship
4	<code>sampling_rate_hz</code> , <code>fft_lines</code> , <code>frequency_max_hz</code> copied onto every plot row	Makes each cached row self-describing and auditable without reading <code>plot_configurations</code>	Redundant storage (negligible relative to the arrays)

Missing normalisation, noted honestly. `plant_name` , `area` , and `line` are free-text strings on every equipment row rather than a normalised plant hierarchy. This is why the `PLANTS` list in the frontend is hard-coded and why plant selection filters nothing — there is no plant entity to filter on.

6.8 Data-Volume Model

For one 8-channel capture of 4096 samples per channel:

Table 131 — 6.8 Data-Volume Model

Table	Rows	Approximate size
sensor_data_uploads	1	< 1 kB
measurement_upload_data	1	file bytes + parsed JSONB (≈ 2 × the CSV size)
plot_results	8 channels × 5 plots = 40	Each row holds two arrays; the time waveform alone is 2 × 4096 numbers
measurement_channel_features	8 × 10 = 80	< 1 kB each
measurement_channel_feature_trends	8 × 10 × 32 = 2560	< 200 B each
Total per capture	≈ 2682 rows	Dominated by <code>plot_results</code> JSONB

Promoting that capture to a baseline adds 1 + 40 + 80 = **121 rows** plus another copy of the file bytes and parsed data.

6.9 SQL Examples

The application never issues raw SQL except inside migrations; these queries are the SQL equivalents of what SQLAlchemy generates, and are useful for operational inspection.

Equipment list with filters (endpoint 8)

```
SELECT count(*) FROM equipment_masters
WHERE plant_name ILIKE '%Pune%' AND machine_type = 'Pump';

SELECT id, plant_name, area, line, machine_name, machine_id,
       machine_type, machine_criticality, manufacturer,
       asset_status, equipment_image_path, created_at
FROM equipment_masters
WHERE plant_name ILIKE '%Pune%' AND machine_type = 'Pump'
ORDER BY created_at DESC
OFFSET 0 LIMIT 20;
```

Timeline query (endpoint 30) — uses `ix_sensor_data_uploads_sensor_id_created_at`

```
SELECT * FROM sensor_data_uploads
WHERE sensor_id = $1
      AND created_at >= $2::date
      AND created_at <= ($3::date + time '23:59:59.999999')
ORDER BY created_at DESC
OFFSET 0 LIMIT 50;
```

Stored-data probe for a whole page in one round trip

```
SELECT upload_id FROM measurement_upload_data
WHERE upload_id IN ($1, $2, $3, ...);
```

Plot cache read (endpoint 32)

```
SELECT * FROM plot_results
WHERE upload_id = $1
      AND config_fingerprint = $2
      AND status = 'ready'
      AND channel = $3
ORDER BY channel, plot_type;
```

Feature read with definition names (endpoint 35)

```
SELECT f.channel, f.feature_code, d.name AS feature_name,
       f.value, f.unit, f.status, f.metadata, f.computed_at
FROM measurement_channel_features f
JOIN feature_definitions d ON d.code = f.feature_code
WHERE f.upload_id = $1 AND f.channel = $2
ORDER BY f.channel, f.feature_code;
```

Baseline comparison (endpoint 37)

```
SELECT u.feature_code, u.channel,
       u.value AS upload_value,
       b.value AS baseline_value,
       CASE WHEN b.value > 1e-30
            THEN 100.0 * u.value / b.value END AS percent_of_baseline,
       u.status
FROM measurement_channel_features u
LEFT JOIN baseline_channel_features b
      ON b.baseline_id = $2
      AND b.channel = u.channel
      AND b.feature_code = u.feature_code
WHERE u.upload_id = $1 AND u.channel = $3;
```

Trend series for one channel (endpoint 36)

```
SELECT feature_code, segment_index, time_s, value
FROM measurement_channel_feature_trends
WHERE upload_id = $1 AND channel = $2
ORDER BY feature_code, segment_index;
```

Threshold rule resolution

```
-- machine-specific first
SELECT * FROM feature_threshold_rules
WHERE is_active = true AND machine_type = 'Pump';
-- fall back to global
SELECT * FROM feature_threshold_rules
WHERE is_active = true AND machine_type IS NULL;
```

Refresh-token validation

```
SELECT rt.*, u.*
FROM refresh_tokens rt
JOIN users u ON u.id = rt.user_id
WHERE rt.token_hash = $1;
-- then: revoked_at IS NULL, expires_at > now(), u.is_active
```

Operational queries

```
-- Uploads whose pipeline did not complete
SELECT id, original_filename, parse_status, plots_status, features_status,
       parse_error, plots_error, features_error, created_at
FROM sensor_data_uploads
WHERE parse_status <> 'parsed'
      OR plots_status <> 'ready'
      OR features_status <> 'ready'
ORDER BY created_at DESC;

-- Sensors with more than one primary baseline (integrity check)
SELECT sensor_id, count(*) FROM sensor_baselines
WHERE is_primary GROUP BY sensor_id HAVING count(*) > 1;

-- Distinct cached fingerprints per upload (cache-churn indicator)
SELECT upload_id, count(DISTINCT config_fingerprint) AS fingerprints
FROM plot_results GROUP BY upload_id HAVING count(DISTINCT config_fingerprint) > 1;

-- Largest tables
SELECT relname, pg_size_pretty(pg_total_relation_size(relid)) AS size
FROM pg_catalog.pg_statio_user_tables ORDER BY pg_total_relation_size(relid) DESC;
```

6.10 Performance Considerations (database)

Table 132 — 6.10 Performance Considerations (database)

Consideration	Detail
Cache-first reads	Plot endpoints read <code>plot_results</code> instead of recomputing; only a fingerprint change or an incomplete plot-type set triggers computation
Bulk writes	<code>bulk_save_objects</code> for the 80 + 2560 feature/trend rows avoids per-object ORM overhead
Batched existence probe	<code>get_stored_upload_ids</code> uses one <code>IN</code> query per page instead of one query per row
Eager loading	Every user read uses <code>joinedload(User.roles)</code> ; refresh validation uses a nested <code>joinedload</code> down to <code>User.roles</code>
Composite indexes matched to access paths	<code>(sensor_id, created_at)</code> for the timeline; <code>(upload_id, channel)</code> for trends; <code>(feature_code, machine_type)</code> for rules
TOAST	Large <code>BYTEA</code> / <code>JSONB</code> values are stored out of line and compressed automatically
<code>pool_pre_ping</code>	Prevents stale-connection errors at the cost of one extra round trip per checkout
Known weaknesses	<code>ILIKE '%...%'</code> cannot use a B-tree index; <code>plot_results</code> rows are read whole (both arrays) even when only one axis is needed; <code>refresh_tokens</code> grows unboundedly; per-page KPI counts in the equipment list are computed from the current page, not the whole table

6.11 Backup and Recovery

The repository contains no backup automation. The recoverable state is:

Table 133 — 6.11 Backup and Recovery

Asset	Location	Recovery
Database	Docker volume <code>postgres_data</code>	<code>pg_dump</code> / <code>pg_restore</code> or a volume snapshot
Uploaded images	Docker volume <code>uploads_data</code> → <code>/app/uploads</code>	Volume snapshot. Not reconstructible from the database
Measurement files and parsed JSON	<code>uploads_data</code> → <code>/app/uploads/measurements</code>	Reconstructible: <code>measurement_upload_data.file_content</code> and <code>.parsed_data</code> hold the same information. <code>sensor_baselines</code> likewise
Schema	<code>backend/alembic/versions</code>	<code>alembic upgrade head</code> recreates the structure

Because the parsed JSON sidecar is only a cache of `measurement_upload_data.parsed_data`, losing the measurements directory does not lose data — but `get_or_load_all_plots` reads the `file`, not the database, so plot reads for pre-existing uploads would fail with "Upload has no parsed data" until the sidecar is regenerated. Feature computation is unaffected because `_load_parsed_for_upload` prefers the database.

6.12 Migration Runbook

```
# apply everything
cd backend && alembic upgrade head

# inspect
alembic current
alembic history --verbose

# step back one revision
alembic downgrade -1

# regenerate a revision after changing models
alembic revision --autogenerate -m "description"
```

In Docker the backend image runs `alembic upgrade head && uvicorn app.main:app`, so migrations are applied automatically on every container start.

Downgrade coverage. Every revision implements `downgrade()`. Revisions 010 and 011 guard their downgrades with existence checks, so they are safe to run against a partially migrated database. Note that `downgrade` of 001 drops both initial tables and therefore all data.

6.13 Data Dictionary Quick Reference

Table 134 — 6.13 Data Dictionary Quick Reference

Table	Rows per capture	Primary access path	Cascade parent
<code>equipment_masters</code>	—	<code>id</code> , <code>machine_id</code> , filters	—
<code>sensor_configurations</code>	—	<code>equipment_id</code> , <code>device_id</code>	<code>equipment_masters</code>
<code>plot_configurations</code>	—	<code>sensor_id</code>	<code>sensor_configurations</code>
<code>sensor_data_uploads</code>	1	(<code>sensor_id</code> , <code>created_at</code>)	<code>sensor_configurations</code>
<code>measurement_upload_data</code>	1	<code>upload_id</code>	<code>sensor_data_uploads</code>
<code>plot_results</code>	40	(<code>upload_id</code> , <code>fingerprint</code> , <code>channel</code>)	<code>sensor_data_uploads</code>
<code>measurement_channel_features</code>	80	<code>upload_id</code>	<code>sensor_data_uploads</code>
<code>measurement_channel_feature_trends</code>	2560	(<code>upload_id</code> , <code>channel</code>)	<code>sensor_data_uploads</code>
<code>sensor_baselines</code>	0 or 1	<code>sensor_id</code> , <code>is_primary</code>	<code>sensor_configurations</code>
<code>baseline_plot_results</code>	0 or 40	<code>baseline_id</code>	<code>sensor_baselines</code>
<code>baseline_channel_features</code>	0 or 80	<code>baseline_id</code>	<code>sensor_baselines</code>
<code>feature_definitions</code>	10 (static)	<code>code</code>	—
<code>feature_threshold_rules</code>	10 (static)	(<code>feature_code</code> , <code>machine_type</code>)	—
<code>users</code> / <code>roles</code> / <code>user_roles</code> / <code>refresh_tokens</code>	—	<code>email</code> / <code>name</code> / <code>composite</code> / <code>token_hash</code>	—

6.14 End-to-End Database Flow

The canonical request path from browser to disk and back:

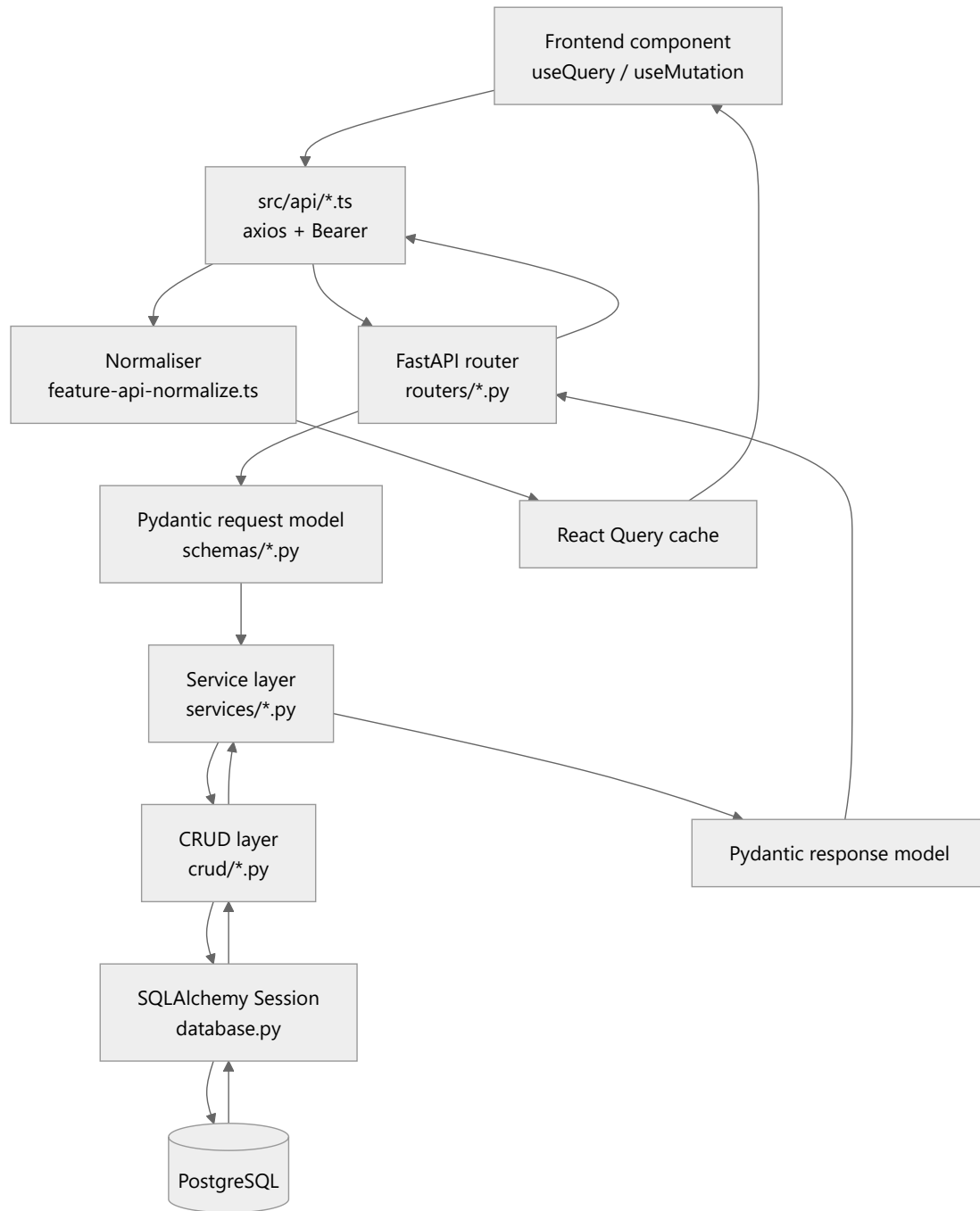


Figure 26 — 6.14 End-to-End Database Flow

Worked example — reading the FFT spectrum of channel 2 of an upload:

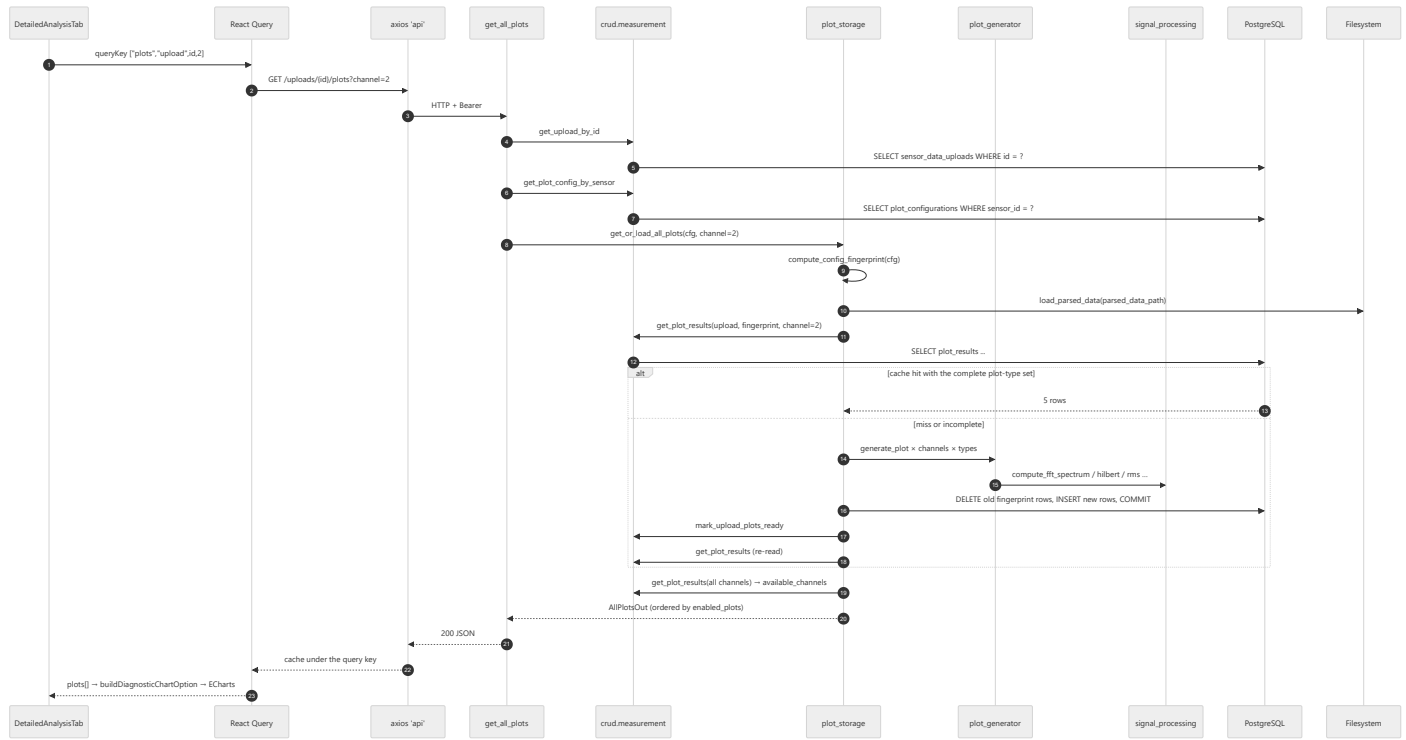


Figure 27 — 6.14 End-to-End Database Flow

7.0 Authentication & Security

7.1 Authentication Model

Table 135 — 7.1 Authentication Model

Property	Value
Scheme	Bearer token (<code>Authorization: Bearer <jwt></code>)
Access token	JWT, HS256, 30-minute default lifetime
Refresh token	Opaque random string, 7-day default lifetime, stored hashed
Session storage (client)	<code>sessionStorage</code> — cleared when the browser tab closes
Server session state	Only <code>refresh_tokens</code> ; access tokens are stateless
Password hashing	bcrypt via <code>passlib.CryptContext(schemes=["bcrypt"], deprecated="auto")</code>
Identity claim	<code>sub</code> = <code>users.id</code> (UUID string)

7.1.1 Access-token payload

```
{
  "sub": "3f2a1c8e-...-9d",
  "exp": 1780000000,
  "type": "access",
  "roles": ["super_admin"]
}
```

`decode_access_token` verifies the signature and expiry via `jose.jwt.decode` , then applies an additional guard: `if payload.get("type") != "access": raise JWTError("Invalid token type")` . This makes it impossible to present a token minted for another purpose as an access token.

The **roles claim is informational only**. `get_current_user` ignores it and re-reads the user from the database on every request, so a role change or a deactivation takes effect immediately rather than at the next token refresh.

7.1.2 Refresh-token design

Table 136 — 7.1.2 Refresh-token design

Aspect	Implementation	Security property
Generation	<code>secrets.token_urlsafe(48)</code>	384 bits from a CSPRNG
Storage	<code>sha256(token)</code> hex in <code>refresh_tokens.token_hash</code>	A database dump does not yield usable tokens
Lookup	By hash, on a unique index	Constant-time index lookup, no plaintext comparison
Revocation	<code>revoked_at</code> timestamp	Server-side invalidation, unlike a JWT
Rotation	Every refresh revokes the presented token and issues a new one	Replay of a used token fails
Bulk revocation	<code>revoke_all_user_refresh_tokens(user_id)</code> exists in <code>crud/user.py</code>	Available for "sign out everywhere"; no endpoint calls it today

7.2 Authorisation Model

7.2.1 Role hierarchy

Table 137 — 7.2.1 Role hierarchy

Role	Read	Write	Settings page	Source of truth
<code>super_admin</code>	✓	✓	✓	<code>users.role</code>
<code>admin</code>	✓	✓	✓	<code>users.role</code>
<code>user</code>	✓	✗ (403)	✗ (redirect to <code>/unauthorized</code>)	<code>users.role</code>

Legacy role names are collapsed defensively: `plant_admin` and `engineer` → `admin` ; anything unrecognised → `user` .

7.2.2 Enforcement points

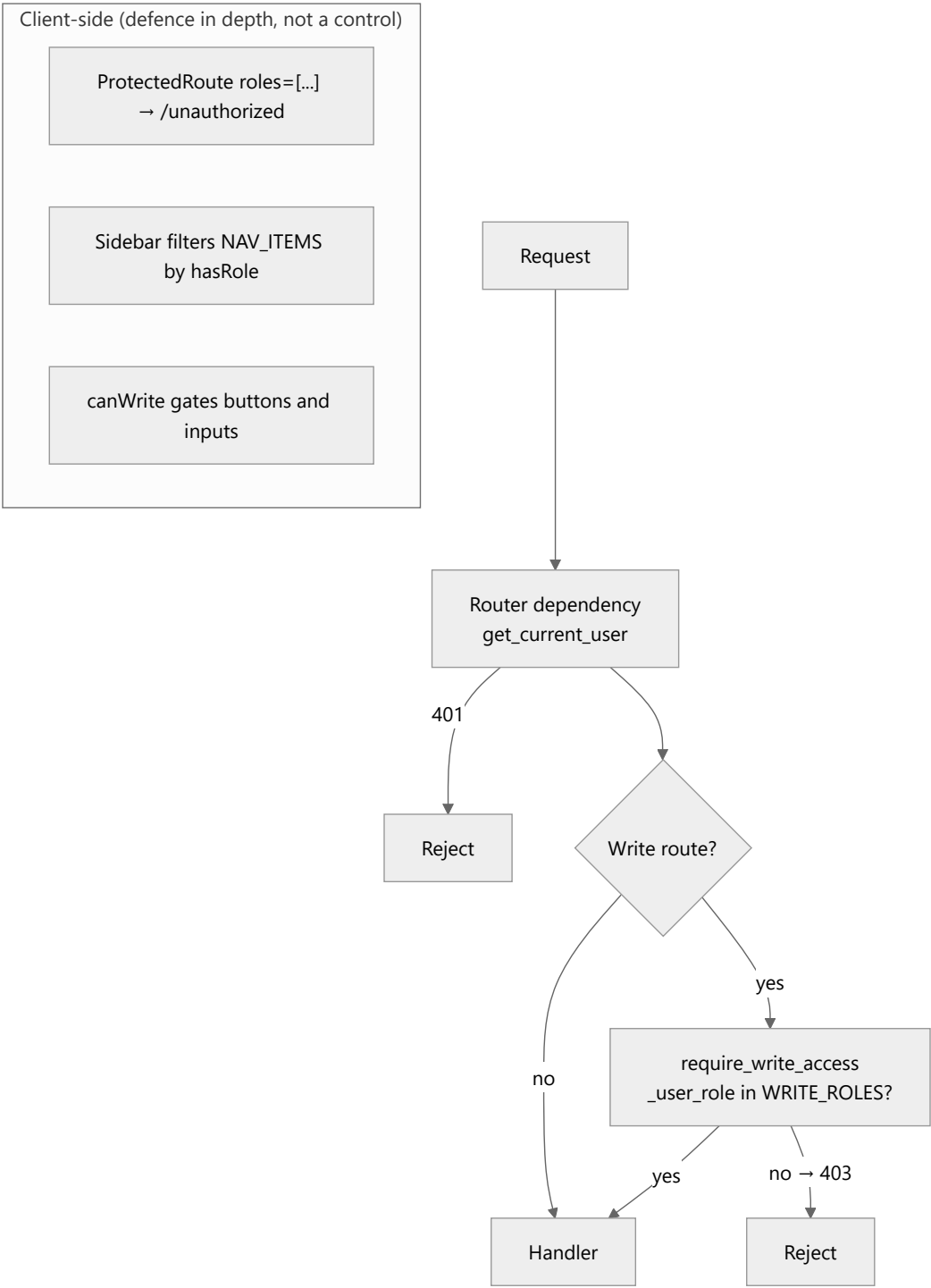


Figure 28 — 7.2.2 Enforcement points

The client-side checks are **usability affordances only**. Every mutating endpoint independently enforces `require_write_access`, so bypassing the UI does not bypass authorisation.

7.2.3 Client-side gating inventory

Table 138 — 7.2.3 Client-side gating inventory

Location	Gate
App.tsx	<code>ALL_ROLES</code> on Dashboard/Equipment/Analysis; <code>WRITE_ROLES</code> on equipment create/edit; <code>ADMIN_ROLES</code> on Settings
Sidebar.tsx	<code>NAV_ITEMS.filter(i => hasRole(i.roles))</code> — Settings is hidden for role <code>user</code>
EquipmentMasterList.tsx	<code>canWrite</code> hides Add / Edit / Delete
VibrationAnalysis.tsx	<code>canWrite</code> disables the file input and upload button, and shows "Read-only users cannot upload sensor data."
DetailedAnalysisTab.tsx	<code>canWrite</code> disables Save Plot Configuration
AnalysisSummaryPanel.tsx	"Save as baseline" rendered only when <code>canWrite</code>
BaselineManagementPanel.tsx	"Set as Primary" rendered only when <code>canWrite</code>

7.3 Sessions and Cookies

The platform uses **no cookies and no server-side sessions**. Tokens live in `sessionStorage` and are attached by an axios request interceptor.

Table 139 — 7.3 Sessions and Cookies

Option	Chosen	Consequence
<code>sessionStorage</code>	✓	Cleared on tab close; not shared across tabs; not sent automatically, so CSRF is structurally impossible
<code>localStorage</code>	✗	Would survive restarts but widen the XSS exfiltration window
<code>httpOnly</code> cookie	✗	Would be XSS-proof for token theft but require CSRF defences

`allow_credentials=True` is set on the CORS middleware even though no cookies are used — harmless but unnecessary.

7.4 Password Storage

Table 140 — 7.4 Password Storage

Property	Value
Algorithm	<code>bcrypt (passlib + bcrypt 4.0.1)</code>
Cost factor	<code>passlib</code> default (12 rounds for <code>bcrypt</code>)
Salt	Generated per hash by <code>bcrypt</code> , embedded in the digest
Column	<code>users.password_hash VARCHAR(255)</code>
Verification	<code>pwd_context.verify(plain, hash)</code> — constant-time within <code>bcrypt</code>
Upgrade path	<code>deprecated="auto"</code> lets <code>passlib</code> mark older schemes for rehash if a scheme is ever added
Plaintext	Never stored, never logged. <code>Login.tsx</code> clears the password state immediately after a successful sign-in
Policy	No complexity, length, rotation, or reuse policy is enforced — only <code>min_length=1</code> at the API boundary

7.5 Encryption

Table 141 — 7.5 Encryption

Layer	Status
At rest	Not implemented. <code>file_content</code> is stored as plain <code>BYTEA</code> ; no column-level encryption; no <code>pgcrypto</code> usage
In transit	Not configured in this repository. Everything is HTTP: <code>nginx</code> listens on 4173 without TLS, <code>Uvicorn</code> on 8000 without certificates, and CORS origins are all <code>http://</code> . TLS is expected to be terminated by an upstream proxy in production
Application-level hashing	<code>bcrypt</code> for passwords, <code>SHA-256</code> for refresh tokens, <code>SHA-256</code> for configuration fingerprints (non-security use)
Secrets	Supplied through environment variables; committed <code>.env</code> contains development values (see §7.11)

7.6 Role and Permission Management

There is **no user-management API**. Accounts are created only by the startup seeding routine:

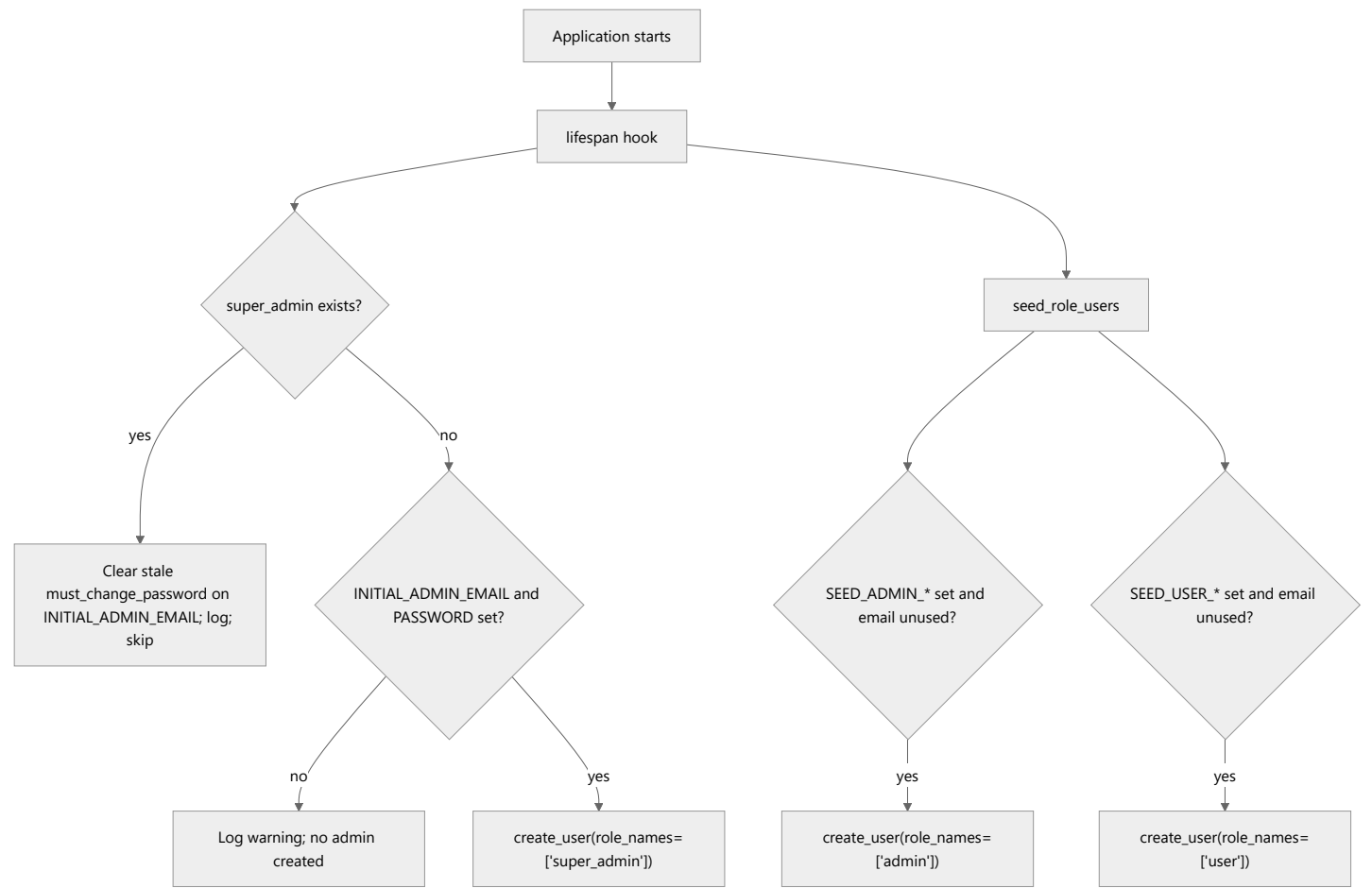


Figure 29 — 7.6 Role and Permission Management

`create_user` writes both the scalar `users.role` (via `primary_role`) and the `user_roles` rows, keeping the two representations consistent for seeded accounts.

Development accounts documented in the API description (`main.py`):

Table 142 — 7.6 Role and Permission Management

Role	Email	Password	Access
super_admin	admin@vibration.com	Admin@2024	Full read + write
admin	plantadmin@vibration.com	PlantAdmin@2024	Read + write
user	viewer@vibration.com	Viewer@2024	Read only (GET)

Only the first is seeded by the committed `.env` (`INITIAL_ADMIN_*`); the other two require `SEED_ADMIN_*` / `SEED_USER_*` to be set, which the committed `.env` does not do.

7.7 Security Filters and Middleware

Table 143 — 7.7 Security Filters and Middleware

Control	Status	Detail
CORS	✓	Explicit six-origin allow-list; no wildcard
Bearer extraction	✓	<code>HTTPBearer(auto_error=False)</code> + custom 401 with <code>WWW-Authenticate: Bearer</code>
Write-role guard	✓	<code>require_write_access</code>
Request validation	✓	Pydantic on every body, query, and path parameter
Security headers (HSTS, CSP, X-Frame-Options, X-Content-Type-Options)	✗	No middleware sets them
Rate limiting	✗	See §7.10
Request-size limit	Partial	Enforced per endpoint after the body is read, not at the server edge
Audit logging	✗	Only <code>created_at</code> / <code>updated_at</code> columns and one equipment-create log line
Trusted-host / host-header validation	✗	Not configured

7.8 CORS

```
app.add_middleware(
  CORSMiddleware,
  allow_origins=[
    "http://localhost:5173", "http://localhost:4173", "http://localhost:3000",
    "http://127.0.0.1:5173", "http://127.0.0.1:4173", "http://127.0.0.1:3000",
  ],
  allow_credentials=True, allow_methods=["*"], allow_headers=["*"],
)
```

Enumerating origins rather than using `["*"]` is correct and is the reason `allow_credentials=True` is even legal (browsers reject the wildcard-plus-credentials combination). **Production deployment requires adding the real frontend origin to this list** — otherwise every browser request fails preflight.

7.9 CSRF

No CSRF protection exists, and none is required by the current design: the credential is a header value read from `sessionStorage`, and a cross-site request cannot read another origin's `sessionStorage` or forge the `Authorization` header. Cookie authentication would immediately create a CSRF requirement.

7.10 Rate Limiting and Brute-Force Resistance

Table 144 — 7.10 Rate Limiting and Brute-Force Resistance

Aspect	Status
Server-side rate limiting	None. No <code>slowapi</code> , no reverse-proxy limit, no per-IP counter
Account lockout	None
Client-side deterrent	<code>Login.tsx</code> counts failures and displays "Multiple failed login attempts detected." after five — cosmetic only, resets on reload
Natural cost barrier	bcrypt at 12 rounds makes each verification ~100 ms of CPU, which throttles online guessing but also makes the login endpoint a cheap denial-of-service target
Timing side channel	<code>authenticate_user</code> returns early for an unknown email (no bcrypt run) but performs a full bcrypt verification for a known email, so response time distinguishes the two cases

7.11 Input Validation and Injection Resistance

7.11.1 SQL injection

Not reachable through the application code. Every query is built with SQLAlchemy ORM expressions, which parameterise values. Even pattern matching is parameterised:

```
query.filter(Equipment.plant_name.ilike(f"%{plant_name}%"))
```

The f-string builds the *pattern*, which is then bound as a parameter — the value never becomes SQL text. Type coercion adds a second barrier: `equipment_id: UUID` rejects anything that is not a UUID with a 422 before any database access.

The only raw SQL in the repository is inside migrations (`op.execute`, `sa.text`) with either literal statements or bound parameters — not user input.

7.11.2 Cross-site scripting

Table 145 — 7.11.2 Cross-site scripting

Vector	Mitigation
Rendered data	React escapes all interpolated text by default
<code>dangerouslySetInnerHTML</code>	Not used anywhere in <code>src/</code>
Chart tooltips	ECharts tooltip formatters build HTML strings containing <code>plot.title</code> , <code>plot.x_label</code> , <code>plot.y_label</code> , and numeric values. These originate from server-generated constants (<code>"FFT Spectrum"</code> , <code>"Frequency (Hz)"</code>), not user input, so no injection path exists today. If plot titles ever became user-editable, these formatters would need escaping
Uploaded filenames	Rendered as text only
SVG assets	Static files in the repository, not user-supplied
Content-Security-Policy	Not set

7.11.3 File-upload validation

Table 146 — 7.11.3 File-upload validation

Check	Images	Measurements
Extension	Derived for storage	<code>.csv</code> / <code>.pdf</code> accepted
MIME	Allow-list of four image types	Allow-list of four text/PDF types (OR extension match)
Size	10 MB	50 MB
Content inspection	None — no magic-byte check	Parser rejects unparseable content with 422
Filename handling	Only the extension is taken from the user; the stored name is <code>{equipment_id}.{ext}</code>	Stored name is <code>{upload_id}.csv</code>
Path traversal	Not possible — filenames are server-generated UUIDs	
Execution risk	Files are written to <code>uploads/</code> , served only through <code>FileResponse</code> for images; measurement files are never served back	

The size check happens **after** `await file.read()`, so a 1 GB upload is fully buffered in memory before rejection. See §14.9.

7.11.4 Other validation surfaces

Table 147 — 7.11.4 Other validation surfaces

Surface	Validation
Numeric ranges	<code>channel</code> 0–31, <code>channel_count</code> 1–32, <code>fft_lines</code> 64–65536, <code>page_size</code> $\leq$ 100/200
Enumerations	<code>data_type</code> , plot types, <code>users.role</code> (DB CHECK)
Emails	Pydantic <code>EmailStr</code> on login; the frontend also regex-checks
Dates	<code>from_date</code> $\leq$ <code>to_date</code> explicit check
UUIDs	Path/query type coercion
Business invariants	<code>active_channel < channel_count</code> ; unique <code>machine_id</code> ; <code>danger > warning</code> (client-side, Settings)

7.12 Security Posture Summary

Implemented well

- 1. bcrypt password hashing with per-hash salts.
- 2. Hashed, revocable, single-use-rotating refresh tokens.
- 3. Short-lived access tokens with an explicit `type` guard.
- 4. Database re-validation of the user on every request, so deactivation is immediate.
- 5. Server-side authorisation independent of the UI.
- 6. Parameterised queries throughout — no SQL injection surface.
- 7. React's default escaping and no `dangerouslySetInnerHTML`.
- 8. Explicit CORS origin allow-list.
- 9. Credential errors that do not reveal whether an account exists.
- 10. `sessionStorage` rather than `localStorage`, and no cookies (hence no CSRF surface).

Gaps, stated plainly

Table 148 — 7.12 Security Posture Summary

#	Gap	Impact	Location
1	Development secrets committed to <code>.env</code>	<code>SECRET_KEY=vibration-platform-secret-key-change-in-production</code> , <code>POSTGRES_PASSWORD</code> , <code>INITIAL_ADMIN_PASSWORD=Admin@2024</code> , and the pgAdmin password are all in the repository. Anyone with repository access can forge JWTs against any deployment reusing these values	<code>.env</code> (note: <code>.gitignore</code> lists <code>.env</code> , so it is ignored going forward, but the file is present in the working tree)
2	No TLS in the shipped configuration	Tokens and passwords traverse plain HTTP	<code>nginx.conf</code> , compose ports, CORS origins
3	No rate limiting or lockout	Online password guessing and login-endpoint DoS are unmitigated	Application-wide
4	No password policy	A one-character password is accepted	<code>schemas/auth.py</code>
5	No password-change endpoint	<code>must_change_password</code> can force a redirect that the user cannot satisfy — a lock-out condition	<code>routers/auth.py</code> , <code>pages/ChangePassword.tsx</code>
6	No security headers	No CSP, HSTS, <code>X-Frame-Options</code> , or <code>X-Content-Type-Options</code>	<code>main.py</code>
7	No user-management API	Roles can only be changed by direct SQL	—
8	Refresh tokens accumulate forever	Unbounded table growth; revoked rows are never purged	<code>refresh_tokens</code>
9	Access tokens survive logout	Logout revokes only the refresh token; the access token stays valid for up to 30 minutes	<code>routers/auth.py::logout</code>
10	Size limits applied after full buffering	Memory-exhaustion vector on large uploads	<code>upload_image</code> , <code>upload_sensor_data</code> , <code>upload_baseline</code>
11	Login timing distinguishes known from unknown emails	Account enumeration by response time	<code>auth_service.authenticate_user</code>
12	Sensor update does not verify parent ownership	<code>PUT /equipment/{A}/sensors/{S}</code> succeeds even when sensor <code>S</code> belongs to equipment <code>B</code>	<code>routers/equipment.py::update_sensor</code>
13	Edge acquisition endpoints require a user token	An edge device must hold user credentials; there is no device-credential concept	<code>routers/measurements.py</code>
14	No audit trail	Who created, changed, or deleted an asset is not recorded	Schema-wide

Priority remediation order

1. Rotate every secret and remove the committed `.env` from history; inject secrets at deploy time.
2. Terminate TLS and add HSTS plus the standard security headers.
3. Add rate limiting on `/auth/login`, `/auth/token`, and `/auth/refresh`.
4. Implement `POST /auth/change-password` (and enforce a password policy) so `must_change_password` is satisfiable.
5. Add the real production origin to the CORS allow-list.
6. Stream uploads with an edge-level body-size limit.
7. Add a scheduled purge of expired/revoked refresh tokens.
8. Add an ownership check to the sensor update and delete routes.

8.0 Business Logic Documentation

This section documents each functional module in the terms requested: purpose, workflow, business rules, dependencies, data flow, user interaction, backend processing, database operations, and response generation.

8.1 Module BL-1 — Identity and Session Management

Purpose. Establish and maintain an authenticated, role-scoped session.

Workflow.

1. The user submits email and password.
2. The server authenticates, updates `last_login_at`, and issues an access/refresh pair.
3. The client stores both in `sessionStorage` and immediately fetches `/auth/me`.
4. On any 401 the client silently refreshes and retries the original request once.
5. Logout revokes the refresh token server-side and clears client state.

Business rules.

Table 149 — 8.1 Module BL-1 — Identity and Session Management

#	Rule	Enforced in
BR-1.1	Email comparison is case-insensitive (stored and queried lower-case)	<code>crud/user.py</code>
BR-1.2	An inactive user cannot authenticate and cannot use an existing access token	<code>authenticate_user</code> , <code>get_current_user</code>
BR-1.3	Unknown email, wrong password, and inactive user all return the same 401 message	<code>routers/auth.py::login</code>
BR-1.4	A refresh token is single-use: presenting it revokes it and mints a new pair	<code>routers/auth.py::refresh_token</code>
BR-1.5	Only tokens carrying <code>type == "access"</code> are accepted as access tokens	<code>decode_access_token</code>
BR-1.6	Logout is idempotent — an invalid or already-revoked token still returns 204	<code>routers/auth.py::logout</code>
BR-1.7	<code>must_change_password</code> forces navigation to <code>/change-password</code> and blocks every other route	<code>ProtectedRoute</code>
BR-1.8	The user record is re-read from the database on every request; the JWT <code>roles</code> claim is never trusted	<code>get_current_user</code>

Dependencies. `auth_service`, `crud/user`, `dependencies/auth`; client-side `AuthContext`, `api/client.ts`, `auth-storage.ts`.

Data flow. `Login.tsx` → `AuthContext.login` → `authClient` → `/auth/login` → `users` + `refresh_tokens` → `TokenResponse` → `sessionStorage` → `/auth/me` → context state → route guards and nav filtering.

Database operations. `SELECT users JOIN roles`; `UPDATE users SET last_login_at`; `INSERT refresh_tokens`; `UPDATE refresh_tokens SET revoked_at`.

Response generation. `TokenResponse` on login/refresh; `UserMeResponse` on `/me`; 204 on logout.

8.2 Module BL-2 — Equipment Master Data

Purpose. Maintain the machine digital twin that gives every vibration measurement its engineering context.

Workflow. Six-step wizard → validate → create or update → optionally upload an image → return to the register.

Business rules.

Table 150 — 8.2 Module BL-2 — Equipment Master Data

#	Rule	Enforced in
BR-2.1	<code>machine_id</code> must be unique when supplied; an empty string is stored as <code>NULL</code> so multiple blanks are allowed	<code>normalize_machine_id</code> validator + <code>uq_equipment_machine_id</code> + the 409 pre-check
BR-2.2	<code>plant_name</code> , <code>area</code> , <code>line</code> , <code>machine_name</code> , <code>machine_type</code> , <code>machine_criticality</code> are NOT <code>NULL</code> in the database but default to "" in the schema, so partial drafts are accepted	<code>EquipmentBase</code> , migration 001
BR-2.3	Sensors supplied in the create payload are inserted in the same transaction as the equipment	<code>crud.create_equipment</code> (<code>flush</code> then insert)
BR-2.4	Update semantics are partial for both <code>PUT</code> and <code>PATCH</code> (<code>exclude_unset=True</code>)	<code>crud.update_equipment</code>
BR-2.5	Deleting equipment cascades to sensors and, transitively, to every measurement artefact of those sensors	ORM <code>delete-orphan</code> + DB <code>ON DELETE CASCADE</code>
BR-2.6	Rotating-component fields are shown only when relevant to the machine type or drive type	<code>RotatingComponentsTab</code>
BR-2.7	Equipment images are limited to four MIME types and 10 MB, and are stored as <code>{equipment_id}.{ext}</code>	<code>upload_image</code>
BR-2.8	AI readiness is five equally weighted checks, each worth 20 %	<code>compute_ai_readiness</code>

Backend processing. Duplicate check → `Equipment(**data)` → flush → insert sensors → commit → refresh → serialise with nested sensors.

Database operations. `equipment_masters` and `sensor_configurations` INSERT/SELECT/UPDATE/DELETE.

8.3 Module BL-3 — Sensor and Acquisition Configuration

Purpose. Define how a sensor's raw signal is to be processed, and expose that definition to edge acquisition devices.

Business rules.

Table 151 — 8.3 Module BL-3 — Sensor and Acquisition Configuration

#	Rule	Enforced in
BR-3.1	Exactly one <code>plot_configurations</code> row per sensor	Unique index + <code>upsert_plot_config</code>
BR-3.2	<code>active_channel</code> must be less than <code>channel_count</code> at creation; on read an out-of-range value is clamped rather than rejected	<code>PlotConfigCreate.validate_channel_index</code> , <code>PlotConfigOut.clamp_channel_on_read</code>
BR-3.3	Legacy plot names (<code>psd</code> , <code>rms_trend</code>) are canonicalised on both write and read	<code>PLOT_TYPE_ALIASES</code>
BR-3.4	An empty or fully invalid <code>enabled_plots</code> list is rejected on write and treated as "all five" on read	<code>validate_plots</code> , <code>normalize_plots_on_read</code>
BR-3.5	When a sensor has no configuration row, processing uses the documented defaults (25600 Hz, 1600 lines, acceleration, all five plots)	<code>default_config_dict</code>
BR-3.6	Edge configuration is resolved by <code>device_id</code> ; if the sensor has no <code>device_id</code> , the payload falls back to the sensor UUID as <code>sensorId</code>	<code>build_edge_acquisition_config</code>
BR-3.7	Edge channel indices are 1-based; platform channels are 0-based	<code>build_channels</code> vs <code>ch0...chN</code>
BR-3.8	Changing any fingerprinted configuration field invalidates every cached plot for that sensor's uploads	<code>compute_config_fingerprint</code>

Acquisition mathematics.

```
frequencyResolutionHz      = sampleRateHz / LOR
blockTimeSeconds           = LOR / sampleRateHz
totalAcquisitionTimeSeconds= blockTimeSeconds × averageCount
stepSizeSamples            = LOR × (1 - overlapDecimal)  [floored at LOR]
```

8.4 Module BL-4 — Measurement Ingestion

Purpose. Turn a heterogeneous sensor export into validated, queryable, analysis-ready data.

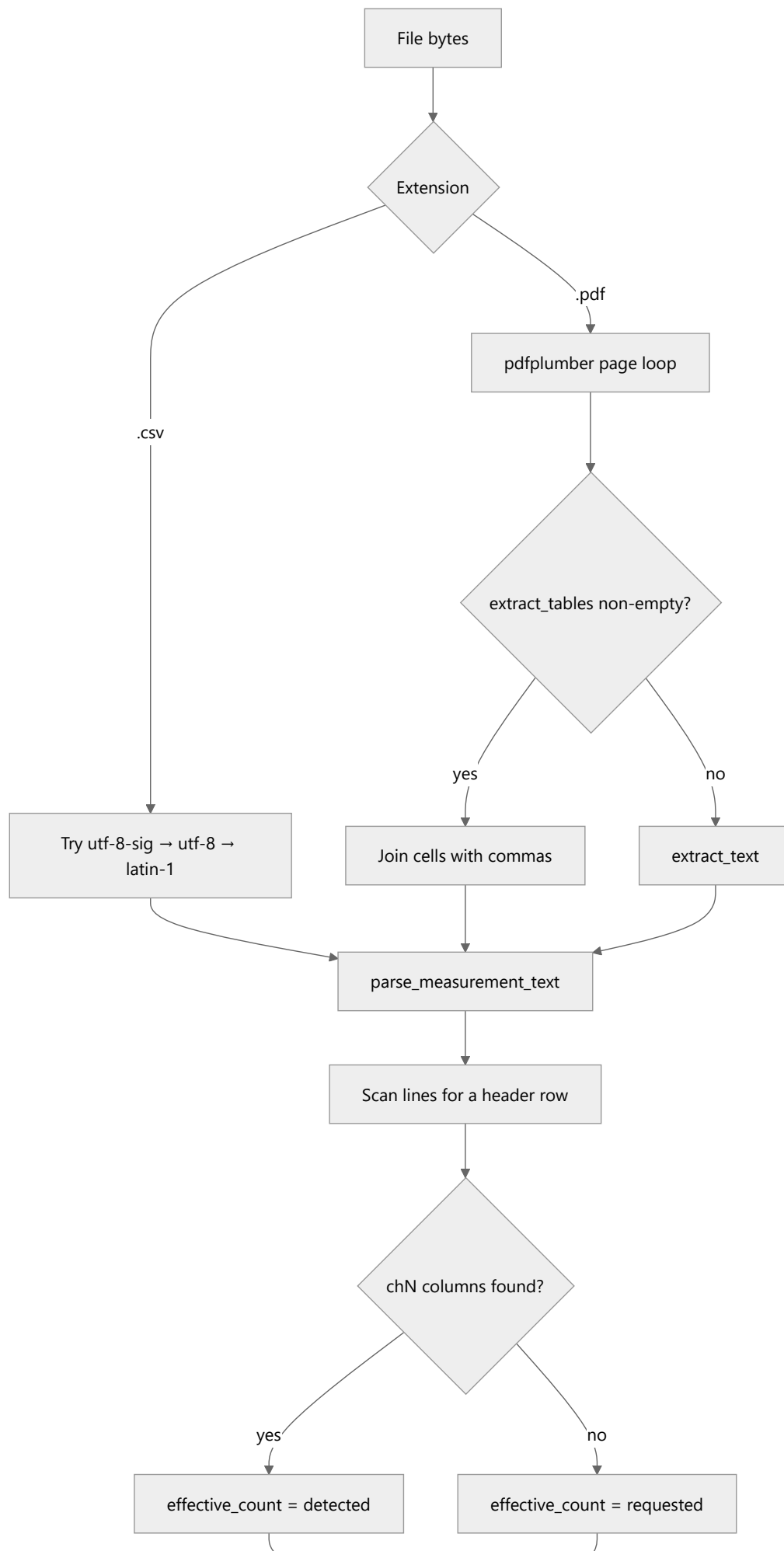
Workflow. Validate → persist raw → parse → persist parsed (disk + DB) → compute plots → compute features and trends → return the lifecycle record.

Business rules.

Table 152 — 8.4 Module BL-4 — Measurement Ingestion

#	Rule	Enforced in
BR-4.1	Only <code>.csv</code> / <code>.pdf</code> by extension, or one of four MIME types	<code>_allowed_upload</code>
BR-4.2	Maximum file size 50 MB	<code>settings.max_pdf_size_mb</code>
BR-4.3	The channel count detected from the file header overrides the user-supplied value	<code>parse_measurement_text</code>
BR-4.4	Rows with fewer values than the channel count are zero-padded; unparseable rows are skipped silently	<code>_parse_numeric_row</code> , the main loop
BR-4.5	A file that yields zero valid rows is a hard failure (422) and the upload is marked <code>parse_status='failed'</code>	<code>parse_measurement_text</code> + <code>mark_upload_failed</code>
BR-4.6	Plot-computation failure and feature-computation failure are non-fatal ; the upload still returns 201 with the error recorded	<code>upload_sensor_data</code> try/except blocks
BR-4.7	The original bytes and the parsed arrays are always persisted to PostgreSQL, not only to disk	<code>baseline_crud.save_upload_data</code>
BR-4.8	Timestamps that look like a shared epoch batch ID are replaced by index-derived time	<code>resolve_time_seconds</code>

Parsing decision flow.



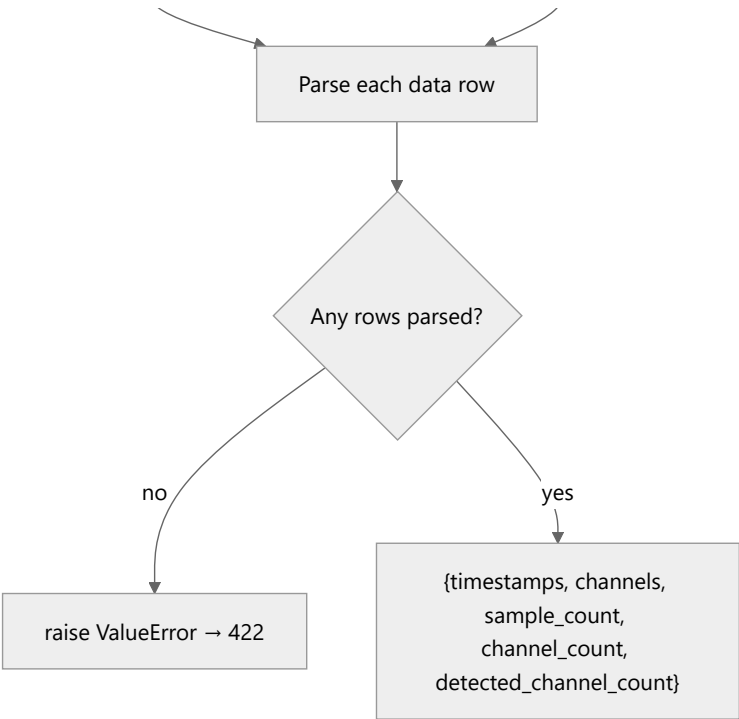


Figure 30 — 8.4 Module BL-4 — Measurement Ingestion

8.5 Module BL-5 — Signal Processing and Plot Generation

Purpose. Convert time-series samples into the five diagnostic views used by vibration analysts.

Business rules.

Table 153 — 8.5 Module BL-5 — Signal Processing and Plot Generation

#	Rule	Enforced in
BR-5.1	Plots are computed for every available channel, then read per channel	<code>persist_all_plot_results</code>
BR-5.2	A cache entry is valid only when the stored plot-type set exactly equals the enabled set for the current fingerprint	<code>get_or_load_all_plots</code>
BR-5.3	The fingerprint excludes <code>active_channel</code> and <code>channel_count</code> so channel switching never invalidates the cache	<code>compute_config_fingerprint</code>
BR-5.4	FFT and envelope spectra apply a Hann window and single-sided $2/n$ scaling	<code>compute_fft_spectrum</code>
BR-5.5	The envelope spectrum removes the envelope's mean before the FFT to suppress the DC pedestal	<code>compute_envelope_spectrum</code>
BR-5.6	Circular waveform and trend plots need at least 4 samples; FFT needs at least 4	<code>compute_*_guards</code>
BR-5.7	The trend plot uses 32 equal segments with RMS per segment	<code>compute_trend_plot</code>
BR-5.8	A requested channel that does not exist falls back to the lowest available channel	<code>resolve_active_channel</code>

Display-side rules (frontend).

Table 154 — 8.5 Module BL-5 — Signal Processing and Plot Generation

#	Rule	Enforced in
BR-5.9	Time waveforms are displayed on a frontend-generated millisecond axis; amplitudes are never altered	<code>withGeneratedTimeAxis</code>
BR-5.10	Waveform and orbit series use min/max bucket decimation so peaks survive; spectra use uniform decimation	<code>downsampleWaveformSeries</code> , <code>downsampleSeries</code>
BR-5.11	Waveform Y axes are symmetric about zero; spectrum Y axes are floored at zero	<code>computeSymmetricYAxisBounds</code> , <code>fixedYAxisConfig</code>
BR-5.12	Zoom and pan act on the X axis only; the Y axis stays fixed so amplitude comparisons remain valid	<code>CHART_X_AXIS_DATA_ZOOM</code>
BR-5.13	Statistics recompute against the visible zoom window	<code>sliceValuesByZoomPercent</code>
BR-5.14	Spectra display a Nyquist marker and, when RPM metadata exists, $1\times/2\times/3\times$ harmonic markers	<code>chart-reference-lines.ts</code>

8.6 Module BL-6 — Feature Extraction and Health Evaluation

Purpose. Reduce each channel to ten interpretable scalars, classify each against rules and a baseline, and expose intra-capture trends.

Workflow.

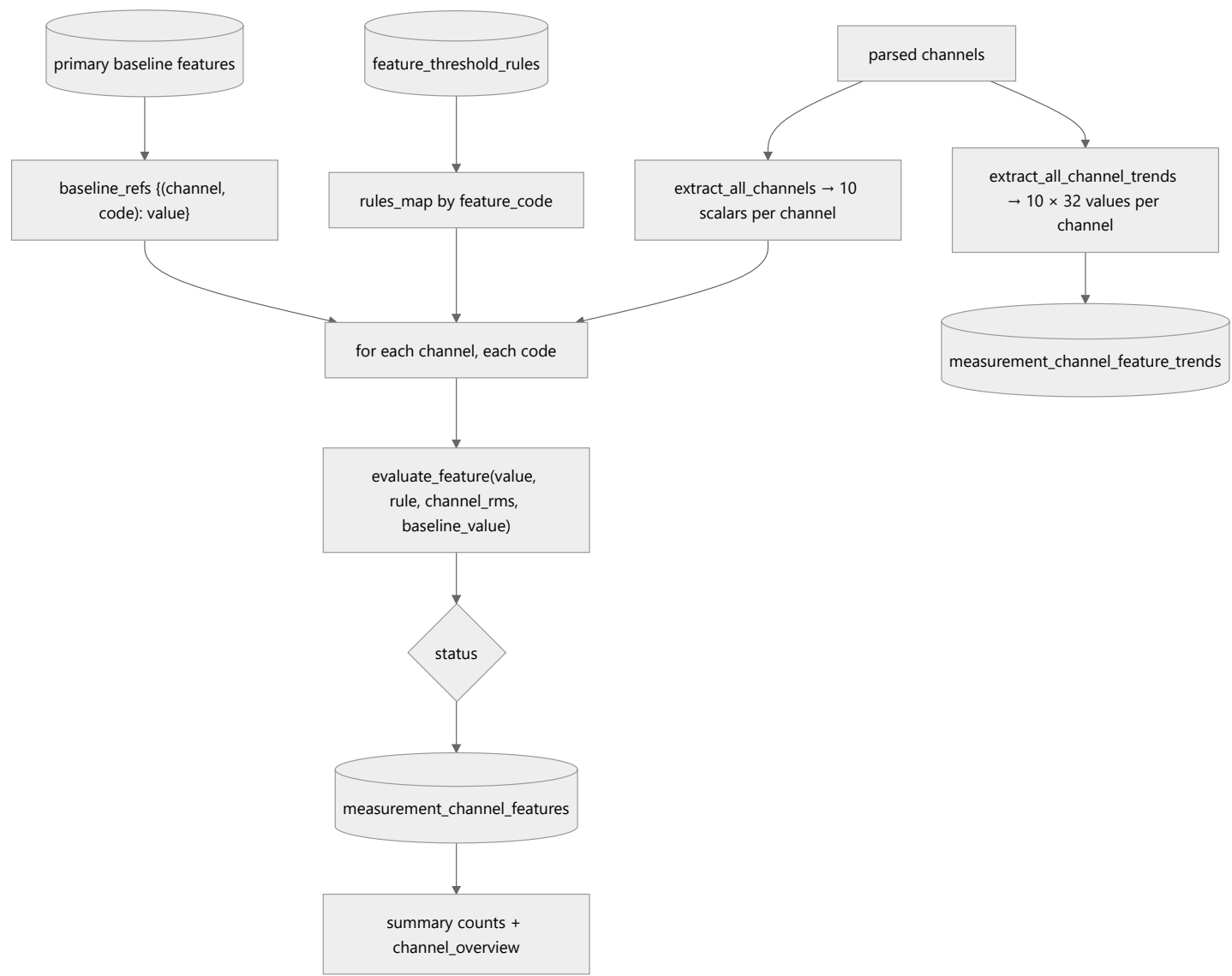


Figure 31 — 8.6 Module BL-6 — Feature Extraction and Health Evaluation

Business rules.

Table 155 — 8.6 Module BL-6 — Feature Extraction and Health Evaluation

#	Rule	Enforced in
BR-6.1	Exactly ten features per channel, in the fixed <code>FEATURE_CODES</code> order	<code>feature_extraction.py</code>
BR-6.2	Shaft speed is estimated as the strongest FFT bin between 5 Hz and 120 Hz (300–7200 RPM)	<code>_estimate_shaft_hz</code>
BR-6.3	1×/2×/3× amplitudes are read at the nearest bin to the estimated shaft frequency and its multiples	<code>_magnitude_at_freq</code>
BR-6.4	Crest factor and kurtosis return 0 when RMS or variance is below <code>1e-30</code> (division guard)	<code>extract_channel_features</code>
BR-6.5	Channels with fewer than 4 samples are skipped entirely	<code>extract_all_channels</code>
BR-6.6	<code>percent_baseline</code> rules return <code>no_baseline</code> — not <code>normal</code> — when no baseline exists, so the UI can distinguish "healthy" from "unknown"	<code>evaluate_feature</code>
BR-6.7	Recomputation always deletes existing rows first, so features are never duplicated or partially stale	<code>persist_upload_features_and_trends</code>
BR-6.8	Features are computed on demand for legacy uploads whose status is still <code>pending</code> , and a <code>failed</code> status is reset to <code>pending</code> before a retry	<code>ensure_upload_features_ready</code>
BR-6.9	A <code>ready</code> status with zero rows is treated as not ready	<code>ensure_upload_features_ready</code>
BR-6.10	Channel health is the worst status present: critical > warning > normal	<code>_channel_health_overview</code>
BR-6.11	Baseline feature rows are always written with <code>status = normal</code>	<code>copy_upload_features_to_baseline</code>
BR-6.12	The UI always renders all ten features, inserting <code>no_baseline</code> placeholders for any the API omitted	<code>enrichFeatureStatusItems</code>

Threshold evaluation matrix.

Table 156 — 8.6 Module BL-6 — Feature Extraction and Health Evaluation

Feature	Rule type	Normal	Warning	Critical
RMS	<code>absolute_max</code>	≤ 0.01	≤ 0.02	> 0.02
Peak	<code>absolute_max</code>	≤ 0.05	≤ 0.10	> 0.10
Crest Factor	<code>range</code>	1.4–3.0	3.0–5.0 (via warning bounds)	outside
Kurtosis	<code>absolute_max</code>	≤ 3.5	≤ 5.0	> 5.0
FFT Band Energy	<code>percent_baseline</code>	≤ 120 %	≤ 150 %	> 150 % (or <code>no_baseline</code>)
1X Amplitude	<code>percent_rms</code>	< 20 % of RMS	< 40 %	≥ 40 %
2X Amplitude	<code>percent_rms</code>	< 10 %	< 20 %	≥ 20 %
3X Amplitude	<code>percent_rms</code>	< 5 %	< 15 %	≥ 15 %
Envelope RMS	<code>percent_baseline</code>	≤ 100 %	≤ 125 %, and ≤ 150 % still warning	> 150 %
Noise Floor	<code>absolute_db</code>	≤ −60 dB	≤ −54 dB	> −54 dB

8.7 Module BL-7 — Baseline Management

Purpose. Preserve known-good reference captures and make them the comparison target for health evaluation.

Business rules.

Table 157 — 8.7 Module BL-7 — Baseline Management

#	Rule	Enforced in
BR-7.1	Baselines are append-only; there is no delete or update endpoint	<code>crud/baseline.py</code> , <code>routers/baselines.py</code>
BR-7.2	At most one primary baseline per sensor; setting a new primary clears the previous one	<code>create_baseline</code> , <code>set_baseline_primary</code>
BR-7.3	Promotion reads bytes and parsed data from <code>measurement_upload_data</code> , never from disk	<code>create_baseline_from_upload</code>
BR-7.4	An upload without stored data cannot be promoted (422 with a re-upload instruction)	<code>create_baseline_from_upload</code>
BR-7.5	Promotion copies the upload's feature values into <code>baseline_channel_features</code> with <code>status = normal</code>	<code>copy_upload_features_to_baseline</code>
BR-7.6	Direct baseline upload computes plots but not features	<code>upload_baseline</code>
BR-7.7	<code>plots_status</code> is derived at read time: <code>ready</code> when <code>plot_count ≥ channel_count × 5</code> , <code>partial</code> when <code>> 0</code> , else <code>pending</code>	<code>_baseline_out</code>
BR-7.8	Deleting the source upload nulls <code>source_upload_id</code> but preserves the baseline	FK ON DELETE SET NULL
BR-7.9	The comparison baseline defaults to the primary, then the first in the list, and can be overridden per view	<code>useFeatureHealthDashboard</code>

8.8 Module BL-8 — Vibration Settings (client-side)

Purpose. Let an administrator map device channels to engineering meaning and define per-parameter alarm limits.

Business rules.

Table 158 — 8.8 Module BL-8 — Vibration Settings (client-side)

#	Rule	Enforced in
BR-8.1	Channel count cannot exceed <code>device.maxChannelCount</code> (default 8)	<code>canAddChannelRow</code> , <code>migrateSettings</code>
BR-8.2	Removing a channel rennumbers the remainder sequentially	<code>renumberChannels</code>
BR-8.3	A channel is "configured" only when axis, data type, unit, and measurement point name are all present	<code>isChannelFullyConfigured</code>
BR-8.4	An enabled threshold row requires both limits and <code>danger > warning</code> ; save is blocked otherwise	<code>isThresholdValid</code> , <code>handleSave</code>
BR-8.5	The threshold parameter list is derived from the shared feature catalogue, so Settings and the Status tab can never diverge	THRESHOLD_PARAMETERS from VIBRATION_FEATURE_CATALOG
BR-8.6	Persisted settings are migrated against the current parameter catalogue on load; values for surviving parameters are preserved	<code>migrateThresholds</code>
BR-8.7	Rows are read-only until explicitly put into edit mode	<code>editingChannels</code> , <code>editingThresholds</code>

Important limitation. These settings are stored only in `localStorage` and are **not** transmitted to the backend. The thresholds actually applied during evaluation are the seeded rows in `feature_threshold_rules` . The Settings module is therefore a configuration surface awaiting a persistence API.

8.9 Module BL-9 — Data Visualisation and Interaction

Purpose. Present diagnostic data to industrial standards with the interactions an analyst expects.

Business rules.

Table 159 — 8.9 Module BL-9 — Data Visualisation and Interaction

#	Rule	Enforced in
BR-9.1	Alarm zones use a fixed visual language: green solid (normal), amber dashed (warning), red dotted (critical)	THRESHOLD_LEVEL_META
BR-9.2	Threshold crossings are marked, capped at 24 per level	findThresholdCrossings
BR-9.3	ISO 10816 velocity zones are documented but never auto-applied — they depend on machine class and must come from configuration	ISO_10816_VELOCITY_ZONES_REFERENCE
BR-9.4	Trace colours are fixed per plot type (navy waveform/orbit, amber FFT/trend, burnt orange envelope)	INDUSTRIAL_TRACE_COLORS
BR-9.5	Line width thins as the user zooms in	adaptiveLineWidth
BR-9.6	Fullscreen falls back to a portal overlay when the native Fullscreen API is unavailable	GraphWorkspace
BR-9.7	Export is PNG at 2× pixel ratio on the warm-white plot background	handleExport

9.0 Complete User Flows

9.1 Flow map

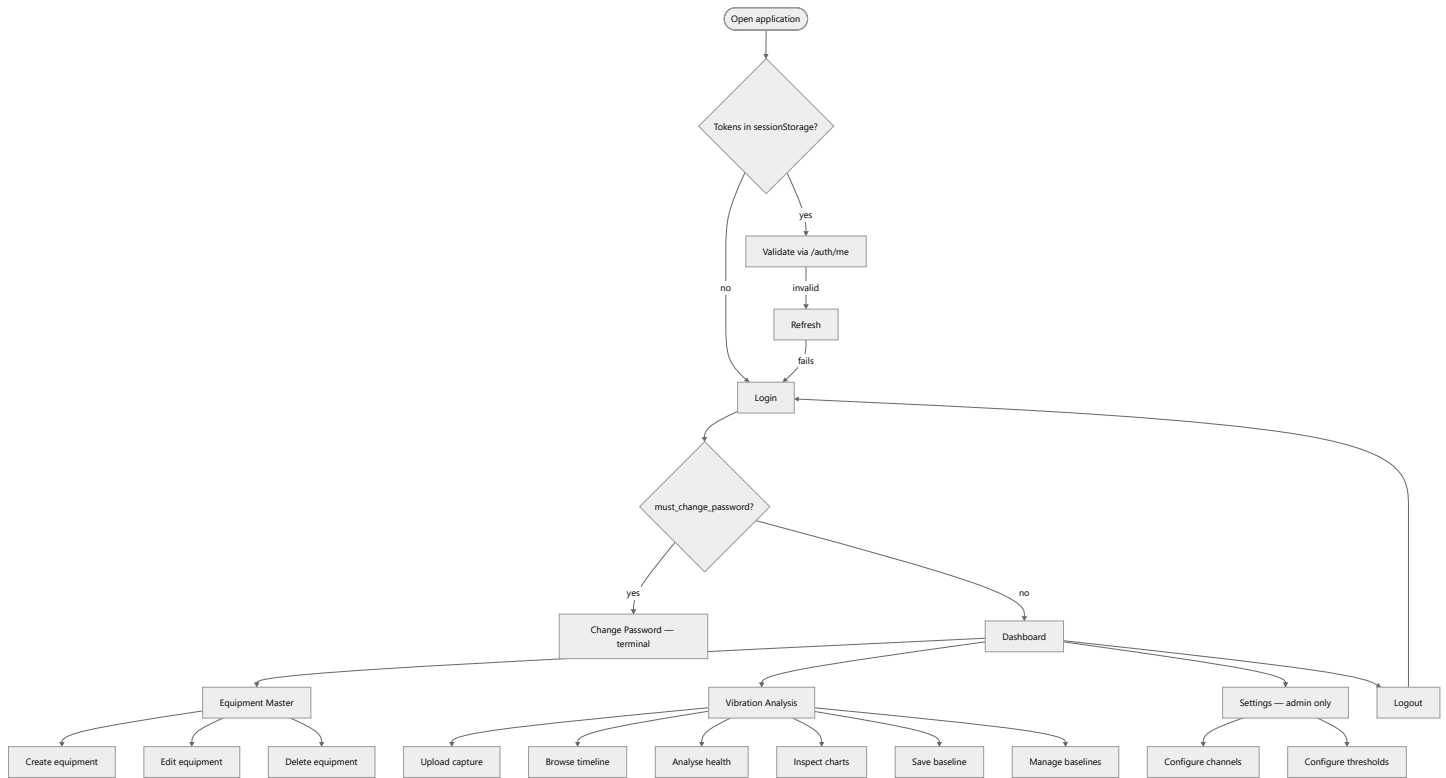


Figure 32 — 9.1 Flow map

9.2 UF-1 — Login

Table 160 — 9.2 UF-1 — Login

Step	User action	System response
1	Navigates to any protected URL	<code>ProtectedRoute</code> redirects to <code>/login</code> with <code>state.from</code> set
2	Enters email and password	Field errors clear as they type; Caps Lock warning appears if active
3	Submits	Client validation → <code>POST /auth/login</code>
4	—	Tokens stored; <code>GET /auth/me</code> ; context populated
5	—	Success toast; redirect to the original destination
Alt 3a	Invalid credentials	401 → inline error; failure counter increments; after 5, a warning banner appears
Alt 4a	<code>must_change_password</code>	Redirect to <code>/change-password</code>

9.3 UF-2 — Create equipment

Table 161 — 9.3 UF-2 — Create equipment

Step	User action	System response
1	Clicks Add Equipment (visible only to write roles)	Navigates to <code>/equipment/new</code>
2	Step 1: location, identity, manufacturer, image	Live completeness updates; criticality dot colours the select
3	Steps 2-4: mechanical, rotating, operating	Step 3 shows only the fields relevant to the machine/drive type
4	Step 5: sensors	Six mounting rows drive the SVG diagram; additional sensors can be appended
5	Step 6: review	Read-only summary and asset-configuration checkboxes
6	Save & Finish	Zod validation → <code>POST /equipment/</code> → optional image upload → success toast → redirect after 1200 ms
Alt 6a	Duplicate <code>machine_id</code>	409 → error toast carrying the server <code>detail</code>
Alt 6b	Validation failure	react-hook-form blocks submission and surfaces field errors

9.4 UF-3 — Upload and analyse a capture

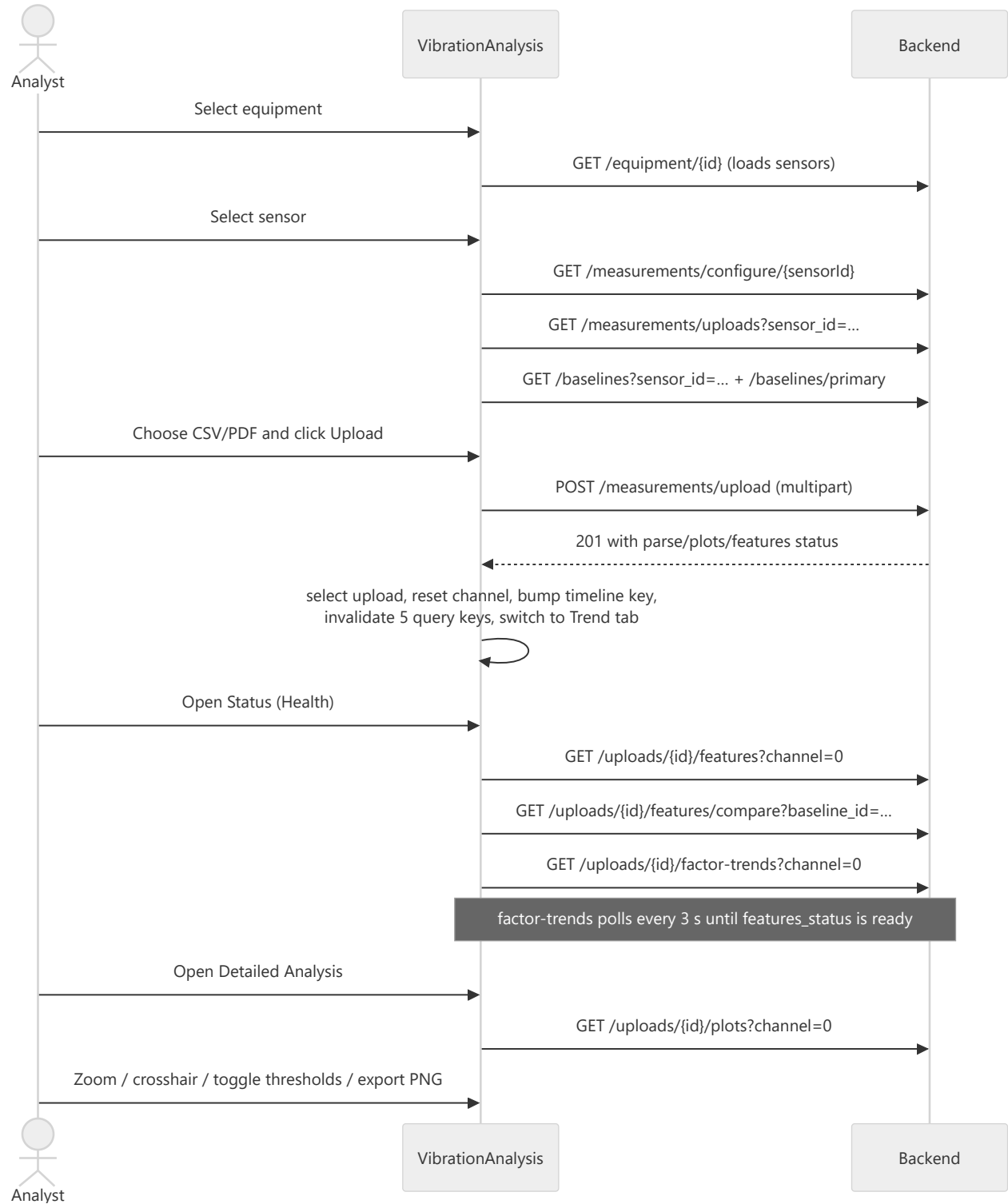


Figure 33 — 9.4 UF-3 — Upload and analyse a capture

9.5 UF-4 — Browse capture history

1. Select equipment and sensor.
2. The Capture Timeline defaults to the last 30 days.
3. Adjust From/To — the query refetches; day chips rebuild.
4. Click a day chip to filter to that day, or **All** to clear.
5. Click a dot, or use Previous/Next, to select a capture (`n of N` shows the position).
6. The Selected Capture panel and all four analysis tabs update to the chosen capture.

9.6 UF-5 — Create and use a baseline

Table 162 — 9.6 UF-5 — Create and use a baseline

Step	Action	Result
1	Select a parsed capture representing healthy operation	Capture becomes active
2	Open Detailed Analysis → Save as baseline	Modal opens with a pre-filled name <code>Baseline <timestamp></code>
3	Confirm name, description, and "set as primary"	<code>POST /baselines/from-upload/{uploadId}</code>
4	—	Baseline row + baseline plots + copied features are created; baseline queries invalidate
5	Open Status (Health)	Comparison automatically targets the new primary baseline
6	Optionally switch the comparison baseline in the dropdown	<code>features/compare</code> refetches for the chosen baseline
7	Optionally Load for Analysis in Baseline Management	<code>plotSource</code> switches to <code>baseline</code> ; charts render <code>GET /baselines/{id}/plots</code>

9.7 UF-6 — Configure vibration settings

1. Open **Settings** (admin roles only; the nav item is hidden for role `user`).
2. Review the device card.
3. Press **Edit** on a channel row, set axis / data type / unit / point name / active, press **Done**.
4. Watch the Channel Mapping Overview change from grey → amber → green.
5. Press **Edit** on a threshold row, enter warning and danger limits, enable it.
6. Invalid rows show "Danger must exceed warning" and block saving.
7. **Save Changes** persists to `localStorage` and clears all edit states.
8. **Reset** or **Cancel** restores the last saved state.

9.8 UF-7 — Search, filter, and paginate the register

Table 163 — 9.8 UF-7 — Search, filter, and paginate the register

Control	Scope	Mechanism
Search box	Current page only	Client-side match on name, ID, plant
Type filter	Whole dataset	Query key change → server refetch
Criticality filter	Whole dataset	Query key change → server refetch
Previous / Next	Whole dataset	<code>page</code> state → server refetch; shown only when <code>total > 20</code>

9.9 UF-8 — Export a chart

Open any chart → adjust zoom and thresholds as desired → click the download icon → the browser saves `sensovibe-{plot_type}-ch{n}.png` (or `sensovibe-health-{metric}-{channel}.png`) rendered at 2× on the `#FFDF8` background.

9.10 UF-9 — Logout

Click the user menu → **Sign Out** → `POST /auth/logout` (best effort) → `sessionStorage` cleared → context reset → redirect to `/login`. The access token remains technically valid until it expires; the refresh token is revoked immediately.

9.11 UF-10 — Session expiry during work

1. The access token expires while the analyst is working.
2. The next request returns 401.

3. The interceptor refreshes silently and retries; concurrent requests queue and are released with the new token.
4. If the refresh token is also expired or revoked, `auth:session-expired` fires: state is cleared, a "Session expired" toast appears, and the user is redirected to `/login` — where `state.from` preserves the page they were on.

9.12 UF-11 — Read-only user journey

Table 164 — 9.12 UF-11 — Read-only user journey

Capability	Available
View dashboard, equipment register, equipment detail	✓
View analysis: timeline, charts, health, statistics, baselines	✓
Export chart PNGs	✓
Add / edit / delete equipment	✗ (buttons hidden; API returns 403)
Upload captures	✗ (input disabled; explanatory text shown)
Save plot configuration	✗ (button disabled)
Save or set primary baselines	✗ (buttons hidden)
Open Settings	✗ (nav item hidden; direct URL → <code>/unauthorized</code>)

10.0 Module Documentation

10.1 Module inventory

Table 165 — 10.1 Module inventory

ID	Module	Frontend files	Backend files	Tables	Endpoints
M-1	Authentication & Session	pages/Login , pages/Unauthorized , pages/ChangePassword , contexts/AuthContext , components/auth/ProtectedRoute , api/auth , api/client , lib/auth-storage , lib/auth-debug , lib/role-access , types/auth	routers/auth , services/auth_service , services/seed , crud/user , dependencies/auth , models/user , schemas/auth	users , roles , user_roles , refresh_tokens	2–6
M-2	Equipment Master	pages/EquipmentMasterList , pages/EquipmentMaster , components/equipment/** (18 files) , api/equipment , types/equipment , lib/form-intelligence , lib/industrial-metadata	routers/equipment , routers/lookups , crud/equipment , models/equipment , models/sensor , schemas/equipment	equipment_masters , sensor_configurations	7–22
M-3	Measurement & Plot Configuration	components/analysis/workspace/DetailedAnalysisTab , api/measurements	routers/measurements (configure + acquisition) , crud/measurement , services/acquisition_config , schemas/measurement , schemas/acquisition	plot_configurations , sensor_configurations	23–28
M-4	Measurement Ingestion	pages/VibrationAnalysis (upload card) , components/analysis/CaptureTimeline*	routers/measurements (upload/list) , services/pdf_parser , services/plot_generator , crud/measurement , crud/baseline	sensor_data_uploads , measurement_upload_data	29–31
M-5	Signal Processing & Charts	components/charts/** , components/analysis/charts/** , lib/*-option.ts , lib/chart-*.ts , lib/threshold-overlay , lib/graph-interactions , lib/waveform-time-axis , lib/echarts-theme , lib/industrial-viz-standards , hooks/useEchartsResize	services/signal_processing , services/plot_generator , services/plot_storage	plot_results	32–34
M-6	Feature Analytics & Health	components/analysis/health/** (13 files) , hooks/useFeatureHealthDashboard , hooks/useUploadFactorTrends , hooks/useHealthStatusData , lib/feature-* , lib/health-* , lib/vibration-features , types/features , types/factor-trends , types/health-status	services/feature_extraction , services/feature_storage , services/threshold_evaluator , crud/feature , schemas/feature	feature_definitions , feature_threshold_rules , measurement_channel_features , measurement_channel_feature_trends , baseline_channel_features	35–37
M-7	Baseline Management	components/analysis/baseline/** , components/analysis/SaveBaselineModal , api/baselines , types/baseline	routers/baselines , crud/baseline , services/baseline_storage	sensor_baselines , baseline_plot_results , baseline_channel_features	38–46
M-8	Vibration Settings	pages/Settings , components/settings/** (10 files) , hooks/useVibrationSettings , lib/vibration-settings-* , types/vibration-settings	—	— (localStorage)	—
M-9	Application Shell & Design System	App , main , components/layout/** , components/ui/** , components/brand/** , contexts/LayoutContext , contexts/ThemeContext , lib/card-* , lib/utils , index.css , tailwind.config.js	—	—	—

10.2 M-1 Authentication & Session

Purpose. Identity, session lifecycle, and role resolution. **Workflow.** §2.6, §8.1. **Business rules.** BR-1.1 ... BR-1.8. **Dependencies.** passlib/bcrypt, python-jose, axios interceptors, React context. **Notable design.** Two axios instances prevent refresh recursion; a window `CustomEvent` bridges the non-React interceptor to React state.

10.3 M-2 Equipment Master

Purpose. Asset register and digital twin. **Workflow.** §9.3. **Business rules.** BR-2.1 ... BR-2.8. **Notable design.** All six wizard tabs stay mounted so uncontrolled inputs never lose state; `SectionCard` / `GlassCard` enforce the documented card contracts; readiness scoring exists in two independent variants (server: 5 checks; client: 4 dimensions).

10.4 M-3 Measurement & Plot Configuration

Purpose. Define processing parameters and expose them to edge devices. **Business rules.** BR-3.1 ... BR-3.8. **Notable design.** Read-time repair of legacy rows (`normalize_plots_on_read` , `clamp_channel_on_read`) means old configurations never break the UI; three URL shapes exist for the same acquisition lookup to accommodate MAC addresses containing colons.

10.5 M-4 Measurement Ingestion

Purpose. Accept and normalise sensor exports. **Business rules.** BR-4.1 ... BR-4.8. **Notable design.** Dual persistence (disk + database) with the database as the authority for reproduction; per-stage status columns give the UI a truthful progress model; non-fatal downstream failures keep the capture usable.

10.6 M-5 Signal Processing & Charts

Purpose. DSP and industrial-standard visualisation. **Business rules.** BR-5.1 ... BR-5.14. **Notable design.** Content-addressed plot cache; a chart shell that is independent of the charting library; X-only zoom to preserve amplitude comparability; documented references to Randall/Antoni and Smith for every visual convention.

10.7 M-6 Feature Analytics & Health

Purpose. Ten features, five rule types, four statuses, per-segment trends. **Business rules.** BR-6.1 ... BR-6.12. **Notable design.** Five distinct rule types cover absolute, ratio-to-RMS, and ratio-to-baseline semantics; `no_baseline` is a first-class status so "unknown" is never displayed as "healthy"; the frontend guarantees a stable ten-row table regardless of API completeness.

10.8 M-7 Baseline Management

Purpose. Reference captures for comparison and future learning. **Business rules.** BR-7.1 ... BR-7.9. **Notable design.** Append-only with a display-only `is_primary` flag; self-sufficient rows (bytes + parsed data) so provenance loss is not data loss.

10.9 M-8 Vibration Settings

Purpose. Channel mapping and alarm limits. **Business rules.** BR-8.1 ... BR-8.7. **Notable design.** Draft/saved separation with row-level edit mode and forward-compatible migration of persisted state; parameter list derived from the shared feature catalogue.

10.10 M-9 Application Shell & Design System

Purpose. Consistent chrome, tokens, and interaction language. **Notable design.** Two written design contracts (`CARD_SIZING.md` , `CARD_HOVER.md`) that components implement through token maps; an enlarged type scale for control-room readability; `prefers-reduced-motion` honoured in two places.

10.11 Code Walkthrough — Application Entry to Shutdown

10.11.1 Backend

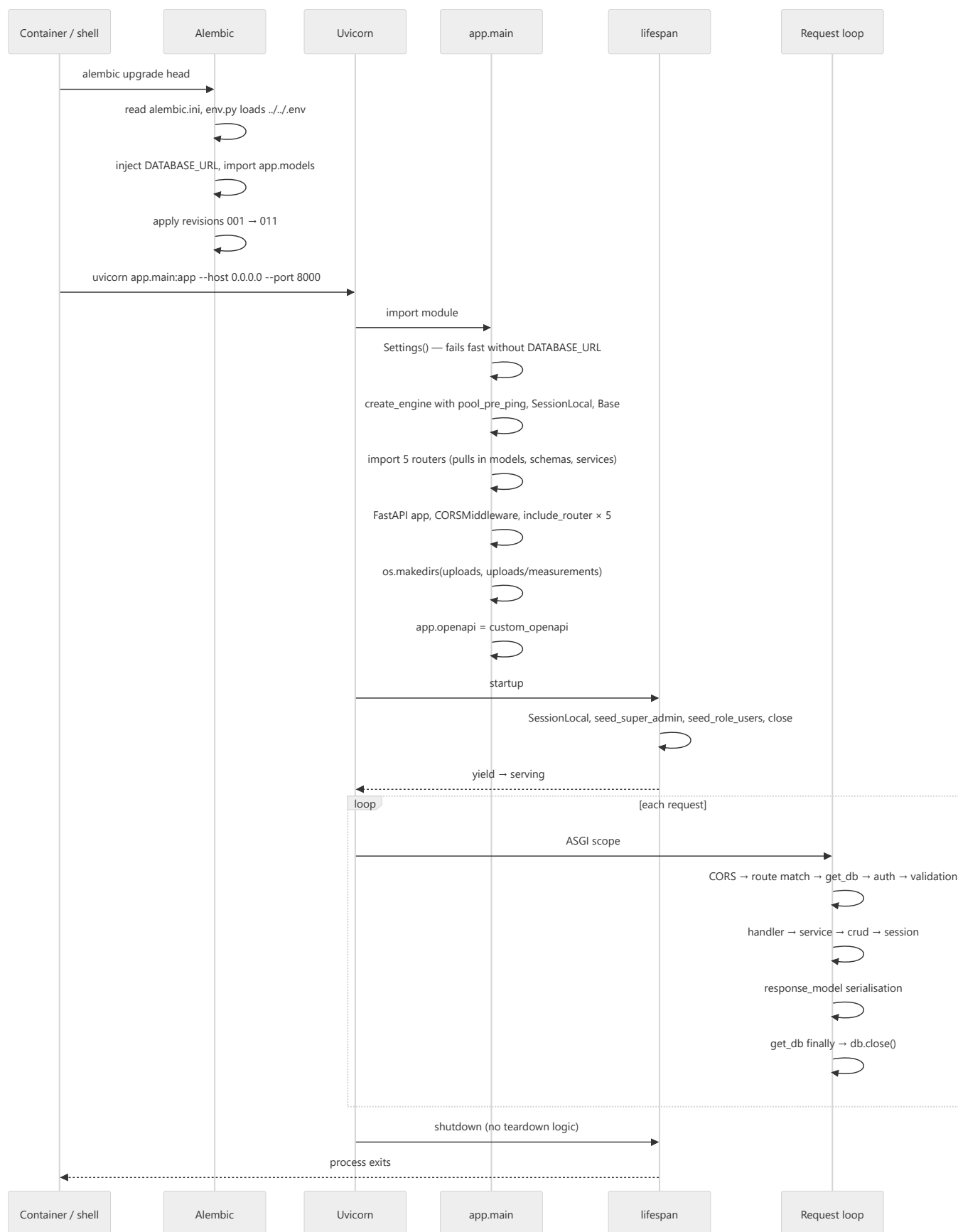


Figure 34 — 10.11.1 Backend

Shutdown. The `lifespan` context manager has no code after `yield`, so shutdown is a plain process exit. Open sessions are closed by their `get_db` generators; the SQLAlchemy pool is released by the interpreter.

10.11.2 Frontend

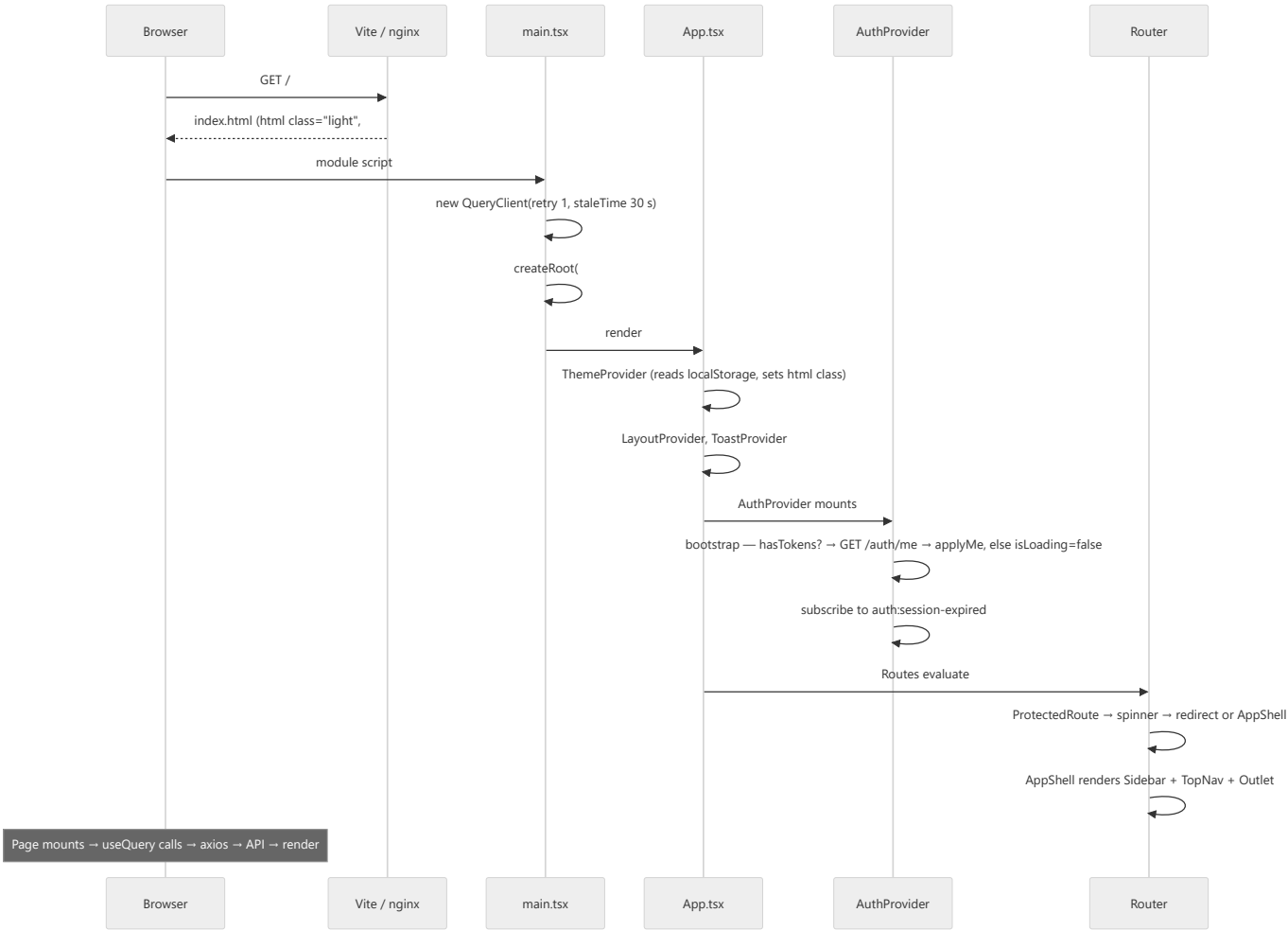


Figure 35 — 10.11.2 Frontend

Teardown. `AuthProvider` removes its window listener on unmount; `useEchartsResize` disconnects its `ResizeObserver` and clears timers; `GraphWorkspace` removes the fullscreen and keydown listeners and restores `document.body.style.overflow`; `MultiSelect` removes its outside-click listener; `ToastProvider` timers are fire-and-forget.

10.12 UI Element Catalogue

Cross-reference of UI element types per screen (detailed screen documentation is in §3.17).

Table 166 — 10.12 UI Element Catalogue

Screen	Cards	Buttons	Forms	Tables	Charts	Dialogs	Filters	Search	Pagination	Responsive behaviour
Login	Form panel, features panel	Submit, password toggle	Email + password	—	Animated SVG backdrop	—	—	—	—	flex-col → lg:flex-row; left panel lg:w-[54%]; features grid 1→2 columns
Dashboard	ComingSoon card, 4 KPI cards, CTA card	Go to Equipment Master, Open Equipment Master	—	—	—	—	—	—	—	KPI grid 1→2→4 columns
Equipment List	4 KPI cards, filter card, table card, AI tip strip	Add, Edit, Delete, More, Prev/Next	Search + 2 selects	7-column register	—	window.confirm on delete	Type, Criticality	Name/ID/Plant	Prev/Next with range label	KPI grid 2→4; table scrolls horizontally
Equipment Wizard	DigitalTwinHeader, FormStepper, 6 tab card groups, MachineVisualizationPanel	Back, Continue, Save & Finish, step nodes, Add Sensor, Remove Sensor, image clear	40+ fields, MultiSelect, date pickers, checkboxes, dynamic sensor array	Mounting table, sensors review	Machine SVG, mounting diagram, progress rings	—	—	—	—	3-column tab layouts collapse to 1; aside becomes full-width above xl
Vibration Analysis	Equipment/Sensor card, Baseline panel, Upload card, Timeline card, Selected Capture card, Workspace card	Upload, Remove File, Save Config, Save as baseline, Set as Primary, Load for Analysis, Retry, 10 toolbar actions, channel buttons, tab buttons, day chips, Prev/Next	2 selects, file input, 3 numeric inputs, date range, baseline select	Feature status, comparison, statistics	5 diagnostic types + 10 health trend cards	SaveBaselineModal	Date range, day chips, baseline status filter	Baseline name/date	Timeline n of N	Tab nav 1→2→4 columns; charts fill width; tables scroll
Settings	Device card, 4 section cards, sticky action bar	Add Row, Edit/Done, Reset Row, Delete, Save, Reset, Cancel, module tabs	Channel selects + text, threshold numeric inputs, toggles	Channel table, threshold table, coverage matrix	—	—	—	—	—	Tables scroll with sticky headers; action bar sticks to the bottom
Unauthorized	GlassCard	Return to Dashboard, Sign Out	—	—	Hero backdrop	—	—	—	—	Centred, max-width card
Change Password	GlassCard	Sign Out	—	—	—	—	—	—	—	Centred, max-width card
App Shell	—	Collapse, plant dropdown, bell, user menu, Sign Out	Search input (inert)	—	—	Two dropdown menus	Plant (inert)	Global (inert)	—	Sidebar 320→80 px; search hidden below md

11.0 Configuration Documentation

11.1 Configuration file inventory

Table 167 — 11.1 Configuration file inventory

File	Scope	Consumed by
<code>.env</code>	Whole stack	Docker Compose, backend <code>Settings</code> , Vite
<code>.gitignore</code>	Repository	git
<code>docker-compose.yml</code>	Orchestration	Docker Compose
<code>package-lock.json</code> (root)	—	Empty stub; no root <code>package.json</code> exists
<code>backend/requirements.txt</code>	Backend deps	pip
<code>backend/Dockerfile</code>	Backend image	Docker
<code>backend/.dockerignore</code>	Backend build context	Docker
<code>backend/alembic.ini</code>	Migration runner	Alembic
<code>backend/setup_and_run.bat</code>	Windows dev bootstrap	Developer
<code>frontend/package.json</code>	Frontend deps + scripts	npm
<code>frontend/vite.config.ts</code>	Dev server + build	Vite
<code>frontend/tsconfig.json</code> , <code>tsconfig.node.json</code>	Type checking	TypeScript
<code>frontend/tailwind.config.js</code>	Design tokens	Tailwind
<code>frontend/postcss.config.js</code>	CSS pipeline	PostCSS
<code>frontend/index.html</code>	SPA shell	Vite
<code>frontend/nginx.conf</code>	Production serving	nginx
<code>frontend/Dockerfile</code> , <code>.dockerignore</code>	Frontend image	Docker
<code>frontend/setup_and_run.bat</code>	Windows dev bootstrap	Developer
<code>START.md</code>	Runbook	Developer

Not present: ESLint, Prettier, EditorConfig, Husky, `pytest.ini` / `pyproject.toml` , `.github/workflows` , `Makefile` , `.nvmrc` , `.python-version` .

11.2 `.env` — complete reference

```
# PostgreSQL
POSTGRES_USER=vibration_user
POSTGRES_PASSWORD=vibration_pass_2024
POSTGRES_DB=vibration_platform
POSTGRES_HOST=localhost
POSTGRES_PORT=5432

# pgAdmin
PGADMIN_EMAIL=admin@vibration.com
PGADMIN_PASSWORD=admin2024

# Backend
DATABASE_URL=postgresql://vibration_user:vibration_pass_2024@postgres:5432/vibration_platform
SECRET_KEY=vibration-platform-secret-key-change-in-production
UPLOAD_DIR=uploads
INITIAL_ADMIN_EMAIL=admin@vibration.com
INITIAL_ADMIN_PASSWORD=Admin@2024
INITIAL_ADMIN_NAME=Platform Administrator

# Frontend
VITE_API_BASE_URL=http://localhost:8000
```

Table 168 — 11.2 `.env` — complete reference

Variable	Consumer	Required	Default	Notes
POSTGRES_USER	Compose (postgres, healthcheck, DATABASE_URL)	✓	—	
POSTGRES_PASSWORD	Compose	✓	—	
POSTGRES_DB	Compose	✓	—	
POSTGRES_HOST	Documentation only	✗	—	Not referenced by any code
POSTGRES_PORT	Documentation only	✗	—	Not referenced by any code
PGADMIN_EMAIL / PGADMIN_PASSWORD	Compose (pgadmin)	✓ for pgAdmin	—	
DATABASE_URL	Settings.database_url , alembic/env.py	✓ — startup fails without it	none	Host must be <code>postgres</code> in Docker, <code>localhost</code> when running the backend outside Docker
SECRET_KEY	Settings.secret_key ; JWT fallback	✗	change-in-production	
JWT_SECRET	Settings.jwt_secret	✗	"" → falls back to SECRET_KEY	Referenced by Compose but absent from the committed .env
UPLOAD_DIR	Settings.upload_dir	✗	uploads	Compose overrides to <code>/app/uploads</code>
MEASUREMENT_UPLOAD_DIR	Settings.measurement_upload_dir	✗	uploads/measurements	Never set anywhere
MAX_IMAGE_SIZE_MB	Settings.max_image_size_mb	✗	10	Never set
MAX_PDF_SIZE_MB	Settings.max_pdf_size_mb	✗	50	Never set
JWT_ALGORITHM	Settings.jwt_algorithm	✗	HS256	Never set
JWT_ACCESS_EXPIRE_MINUTES	Settings	✗	30	Never set
JWT_REFRESH_EXPIRE_DAYS	Settings	✗	7	Never set
INITIAL_ADMIN_EMAIL / _PASSWORD / _NAME	seed_super_admin	✗	"" / "" / Platform Administrator	Without the first two, no admin is created and a warning is logged
SEED_ADMIN_EMAIL / _PASSWORD / _NAME	seed_role_users	✗	"" / "" / Plant Administrator	Referenced by Compose; absent from the committed .env
SEED_USER_EMAIL / _PASSWORD / _NAME	seed_role_users	✗	"" / "" / Read Only User	Same
VITE_API_BASE_URL	api/client.ts	✗	http://localhost:8000	Inlined at build time, not runtime

Because `Settings.Config.extra = "ignore"` , the Postgres and pgAdmin keys are safely ignored by the backend even though they share the file.

11.3 docker-compose.yml

```
services:
  postgres: # postgres:16, port 5433:5432, volume postgres_data, healthcheck pg_isready 10s/5s/5
  pgadmin: # dpage/pgadmin4:latest, port 5050:80, volume pgadmin_data, depends_on postgres
  backend: # build ./backend, port 8000, env_file .env + 11 explicit env vars,
           # volume uploads_data:/app/uploads, depends_on postgres condition service_healthy
  frontend: # build ./frontend target preview, build arg VITE_API_BASE_URL=http://localhost:8000,
            # port 4173, depends_on backend
volumes: postgres_data, pgadmin_data, uploads_data
```

Explicit backend environment overrides: `DATABASE_URL` (rebuilt from the Postgres variables with host `postgres`), `SECRET_KEY` , `UPLOAD_DIR=/app/uploads` , `JWT_SECRET` , `INITIAL_ADMIN_*` (3), `SEED_ADMIN_*` (3), `SEED_USER_*` (3).

Port summary

Table 169 — 11.3 docker-compose.yml

Service	Host	Container
PostgreSQL	5433	5432
pgAdmin	5050	80
Backend	8000	8000
Frontend	4173	4173

11.4 backend/Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
RUN apt-get update && apt-get install -y gcc libjpeg-dev zlib1g-dev && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["sh", "-c", "alembic upgrade head && uvicorn app.main:app --host 0.0.0.0 --port 8000"]
```

gcc supports building any wheel-less dependency; libjpeg-dev and zlib1g-dev are Pillow's image-codec headers. Requirements are copied before the source so the dependency layer caches across code changes. Migrations run before the server starts, so a fresh volume is schema-complete on first boot.

11.5 frontend/Dockerfile

Three targets:

Table 170 — 11.5 frontend/Dockerfile

Target	Base	Purpose	Command
base	node:20-alpine	npm ci + copy source	—
dev	base	Hot-reload dev server	npm run dev -- --host 0.0.0.0 --port 5173
build	base	ARG VITE_API_BASE_URL → npm run build	—
preview	nginx:alpine	Serve /app/dist with the SPA fallback	nginx -g "daemon off;"

The header comments document both usage modes verbatim, including the Windows volume-mount form (-v "%cd%:/app" -v /app/node_modules).

11.6 frontend/nginx.conf

```
server {
  listen 4173;
  root /usr/share/nginx/html;
  index index.html;
  location / { try_files $uri $uri/ /index.html; }
}
```

try_files ... /index.html is required for client-side routing — without it a refresh on /analysis would return 404. No gzip, cache-control, or security headers are configured.

11.7 frontend/vite.config.ts

Table 171 — 11.7 frontend/vite.config.ts

Setting	Value	Note
plugins	react()	JSX transform + fast refresh
optimizeDeps.include	["plotly.js-dist-min", "react-plotly.js"]	Stale — neither package is installed or imported; a leftover from the pre-ECharts implementation. Harmless but should be removed
resolve.alias	@ → ./src	Matches the tsconfig path mapping
server.port	5173	
server.proxy["/api"]	http://localhost:8000 , changeOrigin: true	Available but unused, because all calls use the absolute baseUrl

11.8 frontend/tsconfig.json

Table 172 — 11.8 frontend/tsconfig.json

Option	Value	Effect
target / lib	ES2020 / ES2020 + DOM + DOM.Iterable	Modern output
module / moduleResolution	ESNext / bundler	Vite-native resolution
jsx	react-jsx	No <code>import React</code> requirement (though the code imports it anyway)
strict	true	Full strictness including <code>strictNullChecks</code>
noUnusedLocals / noUnusedParameters	false	Unused symbols do not fail the build
noFallthroughCasesInSwitch	true	Guards the plot-type switch
noEmit	true	Vite emits; <code>tsc</code> only type-checks
skipLibCheck	true	Skips <code>.d.ts</code> checking for speed
isolatedModules	true	Required for esbuild transpilation
paths	@/* → ./src/*	Absolute imports

`tsconfig.node.json` is a `composite` project covering only `vite.config.ts`.

11.9 frontend/tailwind.config.js

Table 173 — 11.9 frontend/tailwind.config.js

Section	Contents
darkMode	<code>["class"]</code> — toggled by <code>ThemeContext</code> on <code><html></code>
content	<code>./index.html</code> , <code>./src/**/*.{ts,tsx,js,jsx}</code>
colors	Semantic tokens bound to CSS variables + literal palettes: <code>signal</code> (light/dark/deep), <code>cta</code> (default/hover/foreground), <code>brand</code> (7 shades), <code>machine</code> (healthy/warning/critical/offline), <code>page.accent</code>
backgroundImage	3 signal gradients (horizontal, vertical, hover)
borderRadius	<code>lg / md / sm</code> from <code>--radius</code> ; <code>xl</code> 0.75rem; <code>2xl</code> 1rem
fontFamily	Inter → system-ui → sans-serif
fontSize	10 enlarged steps plus a custom <code>overline</code>
boxShadow	<code>card</code> , <code>card-hover</code> , <code>nav</code> , <code>logo</code> , <code>signal</code> , <code>cta</code> , <code>cta-hover</code>
keyframes / animation	<code>fade-up</code> (0.35 s ease-out)
plugins	<code>tailwindcss-animate</code>

11.10 backend/alembic.ini

`script_location = alembic`, `prepend_sys_path = .`, `version_path_separator = os`, and a placeholder `sqlalchemy.url` that `env.py` always overwrites. Logging: `root/sqlalchemy` at `WARN`, `alembic` at `INFO`, `stderr` handler, format `%(levelname)-5.5s [% (name)s] %(message)s`.

11.11 .gitignore

Ignores `.env`, `__pycache__`, `*.pyc`, `*.pyo`, `.venv`, `venv`, `node_modules`, `dist`, `build`, `.DS_Store`, `uploads`, `*.egg-info`, `.pytest_cache`, `alembic/versions`, `__pycache__`.

`.env` is ignored, yet the file exists in the working tree with real development credentials — see security gap #1 in §7.12.

11.12 Configuration precedence

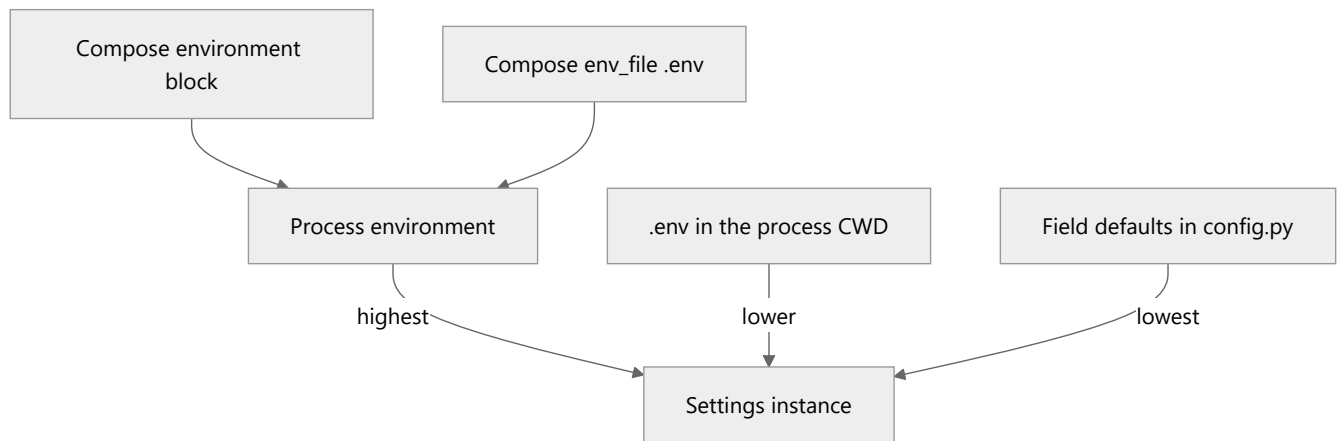


Figure 36 — 11.12 Configuration precedence

For the frontend, `VITE_API_BASE_URL` is resolved at **build** time: the Compose build arg becomes an `ENV` in the build stage, is inlined by Vite into the bundle, and cannot be changed without rebuilding.

12.0 Deployment Guide

12.1 Deployment topology

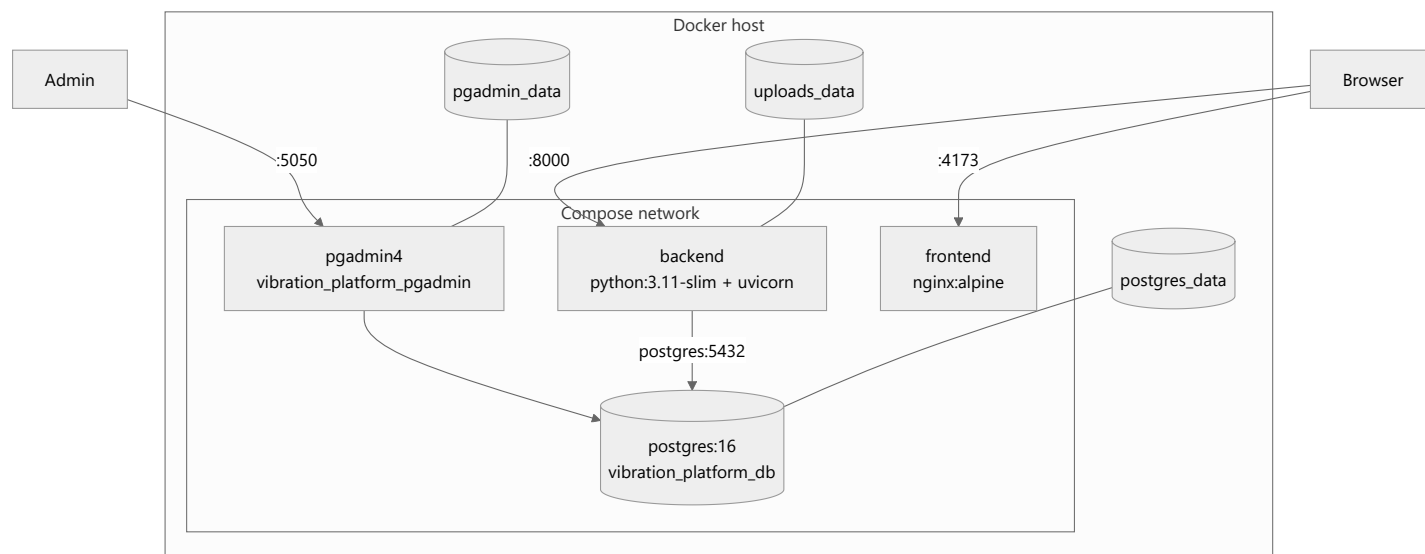


Figure 37 — 12.1 Deployment topology

12.2 Local development (from `START.md`)

Step 1 — database

```
docker-compose up -d          # postgres on 5433, pgAdmin on http://localhost:5050
```

Step 2 — backend

```
cd backend
python -m venv .venv
.venv\Scripts\activate          # Windows; source .venv/bin/activate on POSIX
pip install -r requirements.txt
copy ..\.env .env               # cp ../.env .env on POSIX
alembic upgrade head
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

API at <http://localhost:8000>, docs at <http://localhost:8000/docs>.

When running the backend outside Docker, `DATABASE_URL` must point at `localhost:5433`, not `postgres:5432`, because the container hostname does not resolve on the host and the published port is 5433.

Step 3 — frontend

```
cd frontend
npm install
npm run dev                    # http://localhost:5173
```

Step 4 — open <http://localhost:5173> and sign in with `admin@vibration.com / Admin@2024`.

Windows convenience scripts `backend/setup_and_run.bat` and `frontend/setup_and_run.bat` perform steps 2 and 3 respectively.

12.3 Full container deployment

```
docker-compose up -d --build
```

Startup order: `postgres` → healthcheck passes → `backend` (runs `alembic upgrade head`, then seeds users on lifespan) → `frontend`.

Table 174 — 12.3 Full container deployment

Endpoint	URL
Frontend	http://localhost:4173
Backend	http://localhost:8000
Swagger	http://localhost:8000/docs
ReDoc	http://localhost:8000/redoc
Health	http://localhost:8000/health
pgAdmin	http://localhost:5050
PostgreSQL	localhost:5433

Common operations:

```
docker-compose logs -f backend
docker-compose restart backend
docker-compose down           # stop, keep volumes
docker-compose down -v        # stop and DESTROY all data
docker-compose exec backend alembic current
docker-compose exec postgres psql -U vibration_user -d vibration_platform
```

12.4 Frontend build process

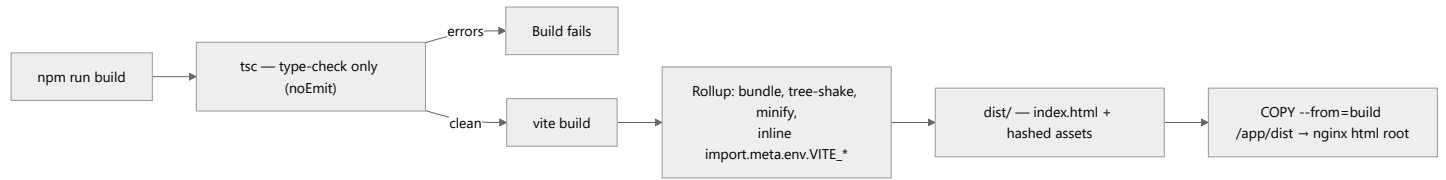


Figure 38 — 12.4 Frontend build process

Because type-checking gates the bundle, a type error blocks deployment — this is the project's strongest automated quality gate.

12.5 Production readiness checklist

The repository ships a development configuration. Before production use:

Table 175 — 12.5 Production readiness checklist

#	Action	Reason
1	Replace every secret (SECRET_KEY , JWT_SECRET , POSTGRES_PASSWORD , PGADMIN_PASSWORD , INITIAL_ADMIN_PASSWORD) and remove .env from the working tree	\$7.12 gap 1
2	Terminate TLS at a reverse proxy; add HSTS and the standard security headers	\$7.12 gap 2
3	Add the production origin to the CORS allow-list in main.py	Otherwise every request fails preflight
4	Rebuild the frontend with the production VITE_API_BASE_URL	The value is baked in at build time
5	Run Uvicorn with multiple workers or behind Gunicorn (-k uvicorn.workers.UvicornWorker)	Feature extraction is CPU-bound and blocks a single worker (\$14.3)
6	Remove the pgAdmin service or restrict it to an internal network	It exposes full database access on 5050
7	Do not publish PostgreSQL to the host	Only the backend needs it
8	Add rate limiting on the auth endpoints	\$7.10
9	Configure database backups of postgres_data and file backups of uploads_data	\$6.11
10	Add log aggregation and an uptime check against /health	No observability exists today
11	Set container resource limits	A large upload can consume significant memory (\$14.9)
12	Schedule a purge of expired/revoked refresh tokens	\$14.7

12.6 Reverse-proxy example

```
server {
    listen 443 ssl http2;
    server_name sensovibe.example.com;
    ssl_certificate /etc/ssl/certs/sensovibe.crt;
    ssl_certificate_key /etc/ssl/private/sensovibe.key;
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
    add_header X-Content-Type-Options nosniff always;
    add_header X-Frame-Options DENY always;

    location / {
        # SPA
        proxy_pass http://frontend:4173;
    }
    location /api/ {
        # API
        proxy_pass http://backend:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        client_max_body_size 60M; # above the 50 MB application limit
        proxy_read_timeout 180s; # feature computation can exceed 60s
    }
}
```

`client_max_body_size` and `proxy_read_timeout` are the two settings most likely to cause confusing failures if left at nginx defaults (1 MB and 60 s).

12.7 Cloud deployment notes

The stack is portable to any container platform. Points that need attention:

Table 176 — 12.7 Cloud deployment notes

Concern	Guidance
Managed PostgreSQL (RDS, Cloud SQL, Azure Database)	Set <code>DATABASE_URL</code> to the managed endpoint; drop the <code>postgres</code> service; keep <code>pool_pre_ping=True</code> (it protects against proxy-idle disconnects)
Object storage instead of a volume	<code>uploads/</code> is written with <code>open()</code> / <code>os.remove()</code> and read with <code>FileResponse</code> ; switching to S3/Blob requires code changes in <code>upload_image</code> , <code>get_image</code> , <code>delete_image</code> , <code>upload_sensor_data</code> , and <code>plot_generator.load_parsed_data</code>
Statelessness	Two of the three filesystem uses are already redundant with the database (<code>measurement_upload_data</code> , <code>sensor_baselines</code>). Only equipment images are filesystem-only
Horizontal scaling	Safe for reads. Concurrent writes to the same upload could duplicate plot computation, but the unique constraint on <code>plot_results</code> prevents duplicate rows
Migrations in multi-replica deployments	<code>alembic upgrade head</code> runs in the image <code>CMD</code> ; with several replicas starting together, run migrations as a separate init job instead
Health probe	<code>GET /health</code> — liveness. A readiness probe should additionally verify database connectivity

12.8 CI/CD

No CI/CD configuration exists in the repository — there is no `.github/workflows`, `.gitlab-ci.yml`, `Jenkinsfile`, or `azure-pipelines.yml`.

A pipeline matching the current toolchain would be:

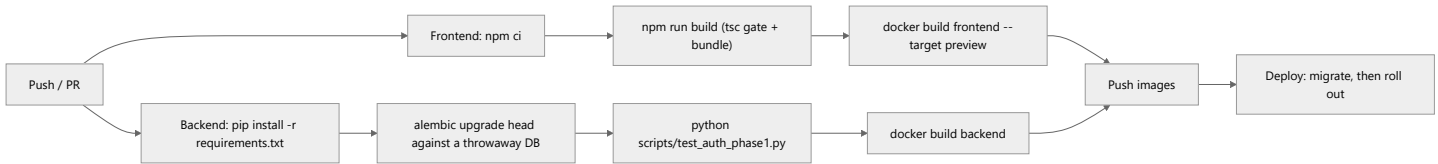


Figure 39 — 12.8 CI/CD

13.0 Testing Documentation

13.1 Current state — stated plainly

The repository contains **one** executable test artefact: `backend/scripts/test_auth_phase1.py`. There is no test framework configuration, no test directory, no frontend test tooling, and no coverage measurement.

Table 177 — 13.1 Current state — stated plainly

Test type	Present	Evidence
Backend unit tests	X	No <code>pytest</code> / <code>unittest</code> in <code>requirements.txt</code> ; no <code>tests/</code> directory
Backend integration tests	Partial	<code>scripts/test_auth_phase1.py</code> uses <code>fastapi.testclient.TestClient</code>
Frontend unit tests	X	No Vitest/Jest/Testing Library in <code>package.json</code>
Frontend E2E tests	X	No Playwright/Cypress
API contract tests	X	—
Load tests	X	—
Coverage	X	<code>.gitignore</code> mentions <code>.pytest_cache/</code> , indicating an intent that was never realised

13.2 The existing smoke test

`backend/scripts/test_auth_phase1.py`:

```
db = SessionLocal(); seed_super_admin(db); db.close()
client = TestClient(app)

r = client.post("/api/v1/auth/login", json={"email": "admin@vibration.com", "password": "Admin@2024"})
assert r.status_code == 200
r2 = client.get("/api/v1/auth/me", headers={"Authorization": f"Bearer {tokens['access_token']}"})
assert r2.status_code == 200 and "super_admin" in r2.json()["roles"]
r3 = client.post("/api/v1/auth/refresh", json={"refresh_token": tokens["refresh_token"]}); assert r3.status_code == 200
r4 = client.post("/api/v1/auth/logout", json={"refresh_token": r3.json()["refresh_token"]}); assert r4.status_code == 204
r5 = client.get("/api/v1/auth/me"); assert r5.status_code == 401
print("ALL TESTS PASSED")
```

Run with `cd backend && python scripts/test_auth_phase1.py`. It covers login, `/me`, refresh rotation, logout, and unauthenticated rejection. **It runs against the configured database**, so it must not be pointed at production.

13.3 Implicit quality gates

Table 178 — 13.3 Implicit quality gates

Gate	Mechanism	Catches
TypeScript strict compilation	<code>npm run build</code> runs <code>tsc</code> before bundling	Type errors, null-safety violations, wrong props, bad API shapes
Zod schemas	Runtime form validation	Malformed user input
Pydantic models	Runtime API validation	Malformed requests and non-conforming responses
Database constraints	PK, FK, UNIQUE, CHECK	Referential and domain violations
Alembic linearity	Single revision chain 001→011	Divergent schema history

13.4 Manual test plan

13.4.1 Authentication

Table 179 — 13.4.1 Authentication

ID	Case	Steps	Expected
TA-01	Valid login	admin@vibration.com / Admin@2024	Redirect to <code>/</code> , toast, user menu shows SUPER ADMIN
TA-02	Wrong password	Any user, bad password	401, inline error, no redirect
TA-03	Unknown email	nobody@x.com	Same message as TA-02
TA-04	Empty fields	Submit blank	Client-side field errors, no request sent
TA-05	Malformed email	<code>abc</code>	"Enter a valid email address"
TA-06	Caps Lock	Enable, focus password	Warning appears
TA-07	Five failures	Repeat TA-02 ×5	Warning banner appears
TA-08	Deep-link preservation	Open <code>/analysis</code> unauthenticated, log in	Lands on <code>/analysis</code>
TA-09	Token expiry	Wait 30 min, act	Silent refresh; work continues
TA-10	Refresh expiry	Revoke/expire refresh, act	"Session expired" toast, redirect to <code>/login</code>
TA-11	Logout	User menu → Sign Out	Redirect to <code>/login</code> ; back button does not restore the session
TA-12	Tab close	Close and reopen the tab	Login required (<code>sessionStorage</code>)

13.4.2 Authorisation

Table 180 — 13.4.2 Authorisation

ID	Case	Expected
TZ-01	Role <code>user</code> opens <code>/settings</code> directly	Redirect to <code>/unauthorized</code>
TZ-02	Role <code>user</code> views the equipment register	Add/Edit/Delete absent
TZ-03	Role <code>user</code> POST <code>/equipment/</code> via curl	403 "Insufficient permissions"
TZ-04	Role <code>user</code> on the analysis page	Upload disabled with an explanatory note
TZ-05	Role <code>admin</code> everywhere	Full write access; Settings visible
TZ-06	No token on a protected endpoint	401 with <code>WWW-Authenticate: Bearer</code>
TZ-07	Tampered JWT	401 "Invalid or expired token"
TZ-08	Deactivated user with a valid token	401 "User not found or inactive"

13.4.3 Equipment

Table 181 — 13.4.3 Equipment

ID	Case	Expected
TE-01	Create with all fields	201; appears first in the register
TE-02	Create with minimum fields	201 (schema defaults permit it)
TE-03	Duplicate <code>machine_id</code>	409; error toast with the server message
TE-04	Blank <code>machine_id</code> on two machines	Both succeed (<code>NULL</code> $\neq$ <code>NULL</code>)
TE-05	Machine type = Gearbox	Ratio and gear-teeth fields appear
TE-06	Machine type = Fan	Fan-blades field appears
TE-07	Machine type = Motor	Pole-count field appears
TE-08	Drive type = Gear Drive on a Pump	Gearbox fields appear
TE-09	Add 3 sensors	All persist and appear on the review tab
TE-10	Upload a 12 MB image	400 "Image exceeds 10MB limit"
TE-11	Upload a <code>.txt</code> renamed to <code>.jpg</code>	400 (MIME check)
TE-12	Edit and save	Values persist; <code>updated_at</code> changes
TE-13	Delete	Row disappears; sensors and all measurements cascade
TE-14	Search by partial name	Filters the current page
TE-15	Filter by type	Server refetch; <code>total</code> changes
TE-16	Paginate with 25+ machines	Prev/Next appear; range label correct
TE-17	Tab away and back mid-wizard	Entered values are retained

13.4.4 Measurement and analysis

Table 182 — 13.4.4 Measurement and analysis

ID	Case	Expected
TM-01	Upload a valid 8-channel CSV	201; all three statuses reach ready
TM-02	Upload a valid PDF	Parsed via table extraction
TM-03	Upload a <code>.docx</code>	400 "File must be a CSV or PDF"
TM-04	Upload a 60 MB file	400 "File exceeds 50MB limit"
TM-05	Upload a CSV with no numeric rows	422; <code>parse_status = failed</code> ; error visible
TM-06	Header declares ch0–ch6 but 8 requested	7 channels used (detected wins)
TM-07	Rows shorter than the channel count	Padded with 0.0; no error
TM-08	All timestamps identical	Time axis derived from the sample index
TM-09	Epoch timestamps	Converted to relative seconds, or index time if the span is implausible
TM-10	Switch channels	Plots reload for the new channel; cache reused
TM-11	Change sampling rate and save config	Next plot read recomputes (new fingerprint)
TM-12	Select a capture on the timeline	All four tabs update
TM-13	Narrow the date range	Timeline refetches; day chips rebuild
TM-14	Invalid range (from > to)	400 from the API
TM-15	Zoom a chart	Statistics recompute for the visible window
TM-16	Double-click a chart	Zoom resets
TM-17	Toggle thresholds	Lines, shading, and crossing markers appear/disappear
TM-18	Fullscreen a chart	Chart fills the viewport and resizes correctly; Escape exits
TM-19	Export PNG	File downloads with the expected name
TM-20	Open Statistics	12 rows consistent with the waveform

13.4.5 Features and health

Table 183 — 13.4.5 Features and health

ID	Case	Expected
TF-01	Open Status (Health) after upload	5 summary cards + overview + 10-row table
TF-02	No baseline exists	<code>percent_baseline</code> features show No Baseline
TF-03	Create a baseline, revisit	Those features are evaluated; comparison table populates
TF-04	Change the comparison baseline	Comparison refetches; % differences change
TF-05	First-ever trend load	"Computing factor trends..." then 10 cards; 3-second polling stops at ready
TF-06	Feature computation fails	Explicit failure message with a re-upload instruction
TF-07	Switch channel	All feature panels reload for the new channel
TF-08	API returns fewer than 10 features	Table still shows 10 rows, missing ones marked No Baseline

13.4.6 Baselines

Table 184 — 13.4.6 Baselines

ID	Case	Expected
TB-01	Save an upload as baseline with "set as primary"	Created; badge shows Primary; previous primary loses the badge
TB-02	Save a second baseline without primary	Both listed; primary unchanged
TB-03	Set a different baseline as primary	Exactly one Primary badge remains
TB-04	Load a baseline for analysis	Charts render baseline data; "Loaded for Analysis" badge shows
TB-05	Search baselines by name/date	List filters
TB-06	Filter by status	Only matching baselines remain
TB-07	Delete the source upload	Baseline survives; <code>source_upload_id</code> becomes null
TB-08	Promote an upload lacking stored data	422 with the re-upload instruction

13.4.7 Settings

Table 185 — 13.4.7 Settings

ID	Case	Expected
TS-01	Edit a channel row and save	Persisted; overview tile turns green
TS-02	Partially configure a channel	Tile shows amber "Partial"
TS-03	Add rows up to 8	Add button disables at the maximum
TS-04	Delete a channel	Remaining rows renumber sequentially
TS-05	Threshold with danger ≤ warning, enabled	Inline error; save blocked with an error toast
TS-06	Valid thresholds	Matrix cell turns green "Saved"
TS-07	Cancel after edits	Draft reverts to the saved state
TS-08	Reload the browser	Settings restore from <code>localStorage</code>
TS-09	Corrupt the <code>localStorage</code> value	Defaults load with an error banner

13.4.8 Responsive and accessibility

Table 186 — 13.4.8 Responsive and accessibility

ID	Case	Expected
TR-01	375 px width	No horizontal page scroll; tables scroll internally
TR-02	768 px	Grids collapse appropriately; search hidden below <code>md</code>
TR-03	1920 px	Full multi-column layouts
TR-04	Collapse the sidebar	Width animates to 80 px; icons remain
TR-05	Keyboard tab order	Focus rings visible on all interactive elements
TR-06	<code>prefers-reduced-motion</code>	Card lift and login animations suppressed

13.5 Recommended automated test suite

Backend (`pytest` + `httpx`)

Table 187 — 13.5 Recommended automated test suite

Layer	Targets
Unit — DSP	<code>compute_fft_spectrum</code> against a synthetic sine (peak at the right bin, amplitude within tolerance); <code>resolve_time_seconds</code> for all five branches; <code>compute_trend_plot</code> segment count
Unit — features	Each of the 10 extractors against known signals; <code>_estimate_shaft_hz</code> band behaviour; division guards at <code>1e-30</code>
Unit — thresholds	All five <code>rule_type</code> branches including <code>no_baseline</code>
Unit — parser	Comma/tab/semicolon/space, BOM, quoted cells, short rows, header detection, encoding fallbacks, zero-row failure
Unit — fingerprint	Stability across key order; change on each fingerprinted field; invariance to <code>active_channel</code> / <code>channel_count</code>
Integration	Full upload → plots → features pipeline on a fixture CSV; cache hit on a second read; recompute after a config change; baseline promotion copying features
Security	401/403 matrix for every endpoint × role; refresh rotation and replay; inactive-user rejection

Frontend (Vitest + Testing Library)

Table 188 — 13.5 Recommended automated test suite

Layer	Targets
Unit	<code>computeChartStatistics</code> , <code>downsampleWaveformSeries</code> (peak preservation), <code>computeSymmetricYAxisBounds</code> , <code>resolveVibrationFeatureKey</code> aliases, <code>enrichFeatureStatusItems</code> (always 10), <code>normalizeUploadFeaturesResponse</code> key aliasing, <code>isThresholdValid</code> , <code>migrateThresholds</code> , <code>groupUploadsByDay</code>
Component	<code>ProtectedRoute</code> decision matrix, <code>FeatureStatusTable</code> grouping, <code>GraphToolBar</code> action visibility, <code>SaveBaselineModal</code> validation
Integration	Login flow with a mocked API, upload flow, 401→refresh→retry through the interceptor

E2E (Playwright) — the eleven user flows in §9.

14.0 Performance & Optimisation

14.1 Implemented optimisations

Table 189 — 14.1 Implemented optimisations

#	Optimisation	Location	Effect
1	Content-addressed plot cache	<code>plot_results</code> , <code>baseline_plot_results</code>	Eliminates repeat FFT/Hilbert computation across page views and users
2	Bulk inserts	<code>bulk_save_objects</code> in <code>feature_storage</code>	~2640 rows per upload written without per-object ORM overhead
3	Batched existence probe	<code>get_stored_upload_ids</code>	One <code>IN</code> query per page instead of N queries
4	Eager loading	<code>joinedload(User.roles)</code> everywhere	Removes the N+1 on every authenticated request
5	Composite indexes aligned to access paths	<code>(sensor_id, created_at)</code> , <code>(upload_id, channel)</code> , <code>(feature_code, machine_type)</code>	Index-only filtering and ordering
6	<code>pool_pre_ping</code>	<code>database.py</code>	Avoids stale-connection failures
7	Server-side pagination	equipment and uploads	Bounded result sets
8	Two-tier client cache	React Query <code>staleTime</code> 30 s (60 s for trend plots)	Suppresses duplicate requests
9	Min/max bucket decimation	<code>downsampleWaveformSeries</code> (8192 pts)	Renders 100k+ sample waveforms without losing peaks
10	Uniform decimation for spectra	<code>downsampleSeries</code> (2000 pts)	Smooth spectrum rendering
11	Adaptive line width	<code>applyAdaptiveLineWidth</code>	Keeps dense traces legible while zoomed
12	<code>memo</code> on chart components	<code>DiagnosticChart</code> , <code>EchartsDiagnosticChart</code>	Avoids re-rendering charts on unrelated state changes
13	<code>useMemo</code> on option builders	All chart wrappers	Rebuilds ECharts options only when inputs change
14	<code>notMerge</code> + <code>lazyUpdate</code>	<code>EchartsGraphViewport</code>	Predictable, batched chart updates
15	Tabs hidden rather than unmounted	<code>EquipmentForm</code> , <code>VibrationAnalysis</code>	Preserves chart instances, zoom state, and uncontrolled input values
16	Status polling only while pending	<code>useUploadFactorTrends.refetchInterval</code>	Stops polling as soon as <code>features_status</code> is terminal
17	Canvas renderer	<code>opts={{renderer: "canvas"}}</code>	Faster than SVG for tens of thousands of points
18	Docker layer caching	Requirements/package files copied before source	Fast rebuilds
19	Vite production bundling	Rollup tree-shaking + minification	Small, cache-busted assets
20	TOAST for large payloads	PostgreSQL automatic	Wide rows stay cheap when the blob columns are not selected

14.2 Not implemented

Table 190 — 14.2 Not implemented

Item	Status	Consequence
Route-level code splitting (<code>React.lazy</code>)	X	Every page, including ECharts, is in the initial bundle
<code>manualChunks</code> in the Vite config	X	No vendor/app split
Virtualised lists/tables	X	A 200-row upload list renders every row
Image lazy loading	X	Only one image type is served
HTTP cache headers	X	nginx serves without <code>Cache-Control</code> ; plot responses are not cacheable by the browser
gzip/brotli	X	Not enabled in <code>nginx.conf</code> ; JSON plot payloads are highly compressible
Service worker / offline	X	—
Redis or in-process cache	X	The database is the only cache
Background job queue	X	See §14.3
Streaming uploads	X	See §14.9
Connection-pool tuning	X	SQLAlchemy defaults (pool size 5, overflow 10)

14.3 The dominant performance characteristic

Feature extraction is the most expensive operation in the system, and it runs **synchronously inside the HTTP request**.

Cost model per channel:

```
extract_segment_trends → for each of 10 feature codes:
  for each of ~32 segments:
    extract_channel_features(segment) → FFT + Hilbert + statistics
= 10 × 32 = 320 full extractions per channel
```

Each extraction performs `scipy.fft.fft` and `scipy.signal.hilbert` on the segment. For an 8-channel capture that is **2560 FFT+Hilbert pairs** on top of the whole-signal extraction. This is why:

- the API sets no server timeout but the client allows 120 s;
- the UI polls `features_status` every 3 seconds and shows "first load may take a few seconds";
- `ensure_upload_features_ready` short-circuits when rows already exist.

Impact. With Uvicorn's default single worker, a large upload blocks the event loop for the duration — other requests queue behind it.

Optimisation opportunities, in order of value:

1. **Compute once per segment, not once per feature.** `extract_channel_features` already returns all ten features. Restructuring `extract_segment_trends` to call it once per segment and fan the results out reduces the work by **10x** with no change in output.
2. Move the pipeline to a background worker (Celery/RQ/ BackgroundTasks) so `POST /upload` returns immediately with `features_status = "pending"` — the UI already handles that state.
3. Run multiple Uvicorn workers so CPU-bound work does not block other requests.
4. Cache the FFT per segment across the features that need it (`fft_band_energy` , `amplitude_1x/2x/3x` , `noise_floor` all recompute the same spectrum).

14.4 Payload sizes

Table 191 — 14.4 Payload sizes

Response	Approximate size	Note
GET /uploads/{id}/plots (4096-sample waveform + 4 more plots)	Hundreds of kB of JSON	Two float arrays per plot; not gzipped by default
GET /uploads/{id}/factor-trends	~10 × 32 × 2 floats	Small
GET /uploads/{id}/features	10 rows	Small
GET /equipment/ (20 items)	Small	Uses the slim <code>EquipmentListItem</code> projection

Enabling gzip at the proxy is the single highest-value network optimisation, because float-array JSON compresses extremely well.

14.5 Frontend rendering

Table 192 — 14.5 Frontend rendering

Aspect	Behaviour
Initial bundle	React + Router + Query + axios + ECharts + framer-motion + lucide + date-fns + zod + react-hook-form, all eagerly loaded
Chart mount	ECharts initialises a canvas per chart; the Status tab can mount 10 compact charts simultaneously
Resize handling	Triple-fire (<code>immediate</code> , <code>rAF</code> , <code>+150 ms</code>) plus <code>ResizeObserver</code> per chart
Fullscreen	Four scheduled resizes plus a <code>ResizeObserver</code>
Animation	<code>framer-motion</code> entrance animations on cards, staggered table rows (<code>delay: i × 0.03</code>)

The heaviest screen is Status (Health) with ten `HealthMetricCard` instances, each owning an ECharts instance and a `ResizeObserver` .

14.6 Known query weaknesses

Table 193 — 14.6 Known query weaknesses

Query	Weakness	Remedy
<code>plant_name ILIKE '%value%'</code>	Leading wildcard prevents B-tree index use → sequential scan	<code>pg_trgm</code> GIN index, or a prefix-only match
Equipment KPI counts	Computed from the current page in the frontend, so "Critical Assets" reflects 20 rows, not the fleet	Add server-side aggregate counts
<code>plot_results</code> reads	Always selects both full arrays	Add a projection when only metadata is needed
<code>refresh_tokens</code>	Grows by one row per login and per refresh, never pruned	Scheduled <code>DELETE WHERE expires_at < now() OR revoked_at IS NOT NULL</code>
Trend rows	~2560 rows per upload accumulate indefinitely	Retention policy or aggregation for old captures

14.7 Recommended maintenance jobs

```
-- Purge dead refresh tokens (safe: revoked or expired only)
DELETE FROM refresh_tokens
WHERE revoked_at IS NOT NULL OR expires_at < now() - interval '7 days';

-- Drop superseded plot caches (keep only the current fingerprint per upload)
DELETE FROM plot_results pr
WHERE pr.config_fingerprint <> (
  SELECT p2.config_fingerprint FROM plot_results p2
  WHERE p2.upload_id = pr.upload_id ORDER BY p2.computed_at DESC LIMIT 1
);

-- Routine statistics maintenance
VACUUM ANALYZE measurement_channel_feature_trends;
VACUUM ANALYZE plot_results;
```

14.8 Scalability profile

Table 194 — 14.8 Scalability profile

Dimension	Current ceiling	Limiting factor
Concurrent users (read)	High	Cached plots; stateless auth
Concurrent uploads	Low	Synchronous CPU-bound pipeline on a single worker
Captures per sensor	High	Indexed, paginated, date-filtered
Channels per capture	32	Schema and validation limit
Samples per capture	Bounded by the 50 MB file limit and memory	Whole file and all arrays held in memory during processing
Equipment records	High	Indexed and paginated
Horizontal scaling	Read-safe	Filesystem-backed equipment images are the only true local state

14.9 Memory profile of an upload

<code>await file.read()</code>	→ entire file in memory (up to 50 MB)
<code>parse_measurement_text</code>	→ Python lists of floats (~3–8× the CSV size)
<code>save_parsed_data</code>	→ JSON serialisation of the same structure
<code>save_upload_data</code>	→ bytes + parsed dict passed to the ORM
<code>persist_all_plot_results</code>	→ per-channel numpy arrays + result lists
<code>persist_upload_features_and_trends</code>	→ scalars + trends dicts + ~2640 ORM objects

Peak memory can be several multiples of the uploaded file size. The size check occurs **after** the full read, so a rejected 200 MB upload is still buffered first. Recommended mitigations: enforce the limit at the reverse proxy (`client_max_body_size`), stream to disk in chunks, and set container memory limits.

15.0 Troubleshooting & Error Handling

15.1 Error-handling architecture

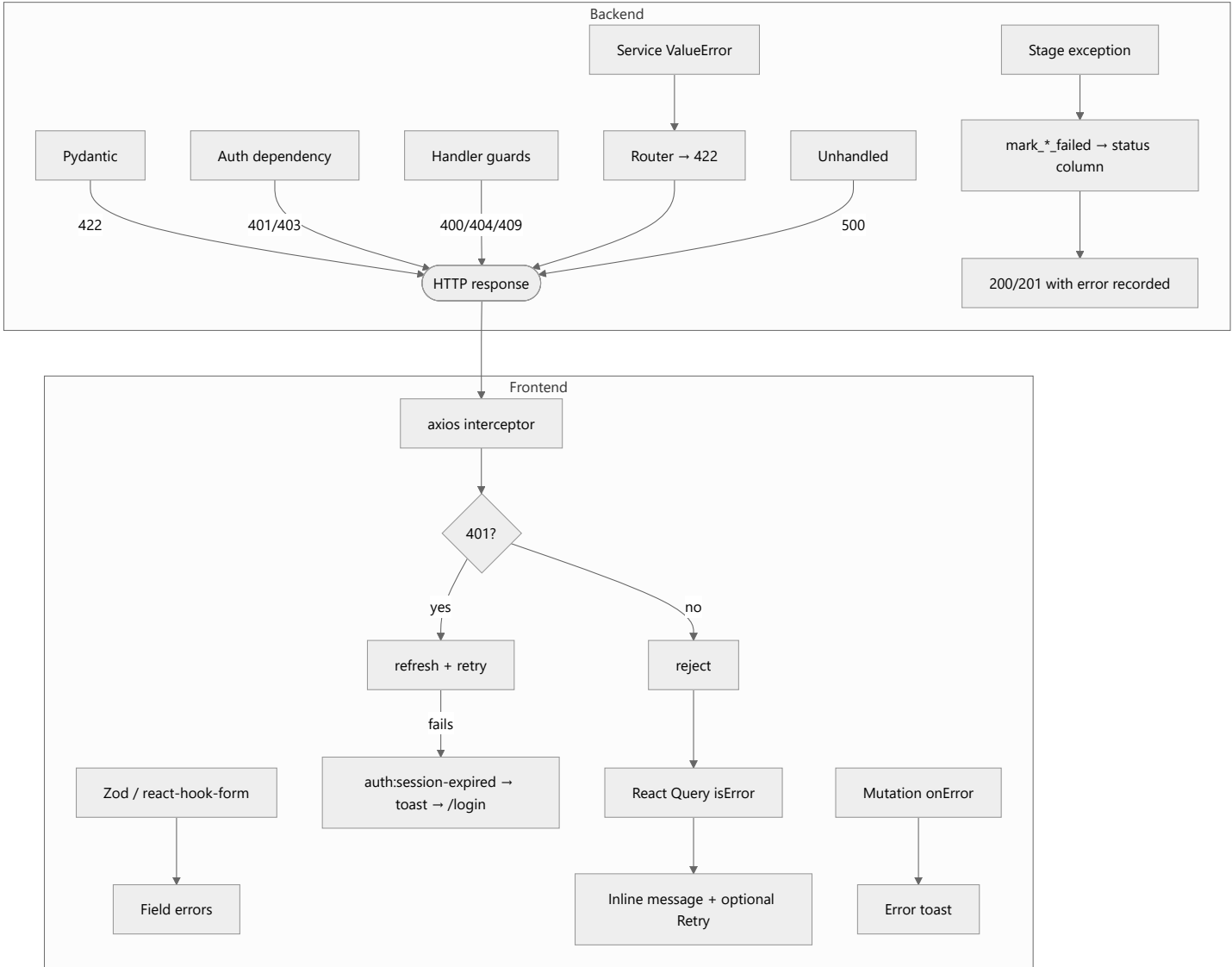


Figure 40 — 15.1 Error-handling architecture

15.2 Backend error catalogue

Table 195 — 15.2 Backend error catalogue

Status	Message	Endpoint(s)	Cause
400	File must be an image (JPEG, PNG, WebP, GIF)	equipment image	Disallowed MIME
400	Image exceeds 10MB limit	equipment image	Size
400	File must be a CSV or PDF	upload	Disallowed type
400	File exceeds 50MB limit	upload	Size
400	Invalid plot type. Allowed: [...]	single plot endpoints	Unknown plot type
400	from_date must be on or before to_date	uploads list	Range inversion
401	Not authenticated	any protected	Missing or non-bearer credentials
401	Invalid or expired token	any protected	Signature, expiry, type, or sub failure
401	User not found or inactive	any protected	Deleted or deactivated user
401	Incorrect email or password	login, token	Bad credentials
401	Invalid refresh token / Refresh token revoked / Refresh token expired / User inactive	refresh	Refresh validation
403	Insufficient permissions	all write endpoints	Role user
404	Equipment not found	equipment routes	Unknown id
404	Sensor not found	sensor, configure, uploads, baselines	Unknown id
404	Image not found / Image file not found on disk	image download	No path, or path missing
404	Lookup 'X' not found	lookups	Unknown key
404	Plot configuration not found for this sensor	configure GET/PUT	No profile
404	No sensor found with device_id 'X'. ...	acquisition	Unmapped device
404	Upload not found	upload routes	Unknown id
404	Baseline not found / No primary baseline set for this sensor / No primary baseline for this sensor	baseline routes, compare	Missing baseline
404	Parsed upload not found	from-upload	Upload absent or unparsed
404	Plot {type} not found	baseline single plot	Type not produced
409	Machine ID 'X' already exists	equipment create	Duplicate
422	Pydantic detail array	any	Schema violation
422	PDF parsing failed: ...	upload	Parser failure
422	Upload not parsed: ...	plots, features, trends	Wrong lifecycle state
422	Feature compute failed: ...	features, trends, compare	Extraction failure
422	Parse failed: ... / Baseline plot compute failed: ...	baseline upload / from-upload	Baseline pipeline
422	Upload file data not in DB. Re-upload the file after migration 007.	from-upload	Pre-migration upload
422	Upload has no parsed data	plots	Missing parsed_data_path
500	Generic	any	Unhandled exception (no global handler)

15.3 Frontend error surfaces

Table 196 — 15.3 Frontend error surfaces

Surface	Message pattern	Location
Login	Server <code>detail</code> , else "Sign in failed. Please check your credentials."	<code>Login.tsx</code>
Session	"Session expired" toast + redirect	<code>AuthContext</code>
Equipment save	Server <code>detail</code> , else "Failed to save equipment. Please try again."	<code>EquipmentForm</code>
Equipment delete	"Failed to delete equipment."	<code>EquipmentMasterList</code>
Equipment list	"Failed to load equipment." + "Make sure the backend is running on port 8000."	<code>EquipmentMasterList</code>
Equipment detail	"Failed to load equipment."	<code>EquipmentMaster</code>
Plots	"Failed to load plots: {detail}" or "Try another channel or select a different capture."	<code>DetailedAnalysisTab</code>
Features	"Unable to load feature data for CH-n."	<code>StatusHealthTab</code>
Comparison	"Unable to load feature comparison data." + Retry	<code>FeatureComparisonSection</code>
Trends	Server <code>detail</code> , else "Failed to load factor trends. Re-upload the file if features were not computed."	<code>FeatureTrendCardsSection</code>
Baselines	"Unable to load baselines for this sensor." + Retry	<code>BaselineManagementPanel</code>
Settings	"Unable to load saved vibration settings. Showing defaults."	<code>VibrationSettingsModule</code>
Settings validation	"Fix threshold rows where danger must be greater than warning before saving."	<code>VibrationSettingsModule</code>

15.4 Diagnostic runbook

Table 197 — 15.4 Diagnostic runbook

Symptom	Likely cause	Check	Fix
Backend exits immediately on start	<code>DATABASE_URL</code> unset	Container logs; <code>env</code>	Provide <code>DATABASE_URL</code> ; <code>copy ..\.env .env</code> for local runs
<code>could not translate host name "postgres"</code>	Backend running on the host with the container URL	<code>DATABASE_URL</code>	Use <code>localhost:5433</code> outside Docker
Login returns 401 for the seeded admin	No admin was seeded	Startup logs for "Seeded super admin user" or the warning	Set <code>INITIAL_ADMIN_EMAIL</code> / <code>_PASSWORD</code> and restart
All requests fail with a CORS error	Origin not in the allow-list	Browser console	Add the origin to <code>main.py</code>
Frontend calls the wrong API host	<code>VITE_API_BASE_URL</code> baked at build time	Network tab	Rebuild with the correct value
401 loop on every request	Refresh failing repeatedly	Network tab for <code>/auth/refresh</code>	Clear <code>sessionStorage</code> , re-login; verify <code>JWT_SECRET</code> did not change
Upload returns 422 "PDF parsing failed"	Unrecognised layout	The <code>detail</code> reports the configured and detected channel counts	Ensure a <code>timestamp,ch0,ch1,...</code> header and numeric rows
Upload succeeds, no plots	Plot stage failed non-fatally	<code>plots_status</code> / <code>plots_error</code> on the upload record	Read <code>plots_error</code> ; verify sample count ≥ 4
Upload succeeds, no features	Feature stage failed	<code>features_status</code> / <code>features_error</code>	Read <code>features_error</code> ; re-request <code>/features</code> to trigger recompute
Trend cards spin forever	<code>features_status</code> never becomes terminal	Poll the <code>/factor-trends</code> response	Check backend logs for an exception in extraction
Plots empty for a channel	Channel not present in the file	<code>available_channels</code> in the response	Select an available channel
Charts do not resize after fullscreen	Resize race	—	The four scheduled resizes normally cover it; toggling fullscreen again forces a resize
Baseline promotion returns 422	Upload predates migration 007	<code>measurement_upload_data</code> row missing	Re-upload the file
Settings lost after reload	<code>localStorage</code> cleared or corrupt	DevTools → Application → Local Storage	Reconfigure; the module falls back to defaults with a banner
Equipment image 404	Volume not mounted or file removed	<code>uploads_data</code> contents	Re-upload the image
Migration fails mid-way	Partially applied 010	<code>alembic current</code>	010/011 are idempotent — re-run <code>alembic upgrade head</code>
Slow first analysis load	Cold plot/feature cache	Timing of the first vs second request	Expected; subsequent loads are cached

15.5 Log locations

Table 198 — 15.5 Log locations

Source	Where
Uvicorn access and error logs	<code>docker-compose logs backend</code> (stdout)
Seeding messages	Same, logger <code>uvicorn</code>
Equipment-create trace	Same, <code>[CREATE_EQUIPMENT]</code> ...
Alembic	Same, INFO level
PostgreSQL	<code>docker-compose logs postgres</code>
nginx	<code>docker-compose logs frontend</code>
Frontend auth trace	Browser console, <code>[Auth]</code> prefix, dev builds only

16.0 Appendix

16.1 Glossary

Table 199 — 16.1 Glossary

Term	Definition
Baseline	A stored reference capture representing known-good machine condition, used as the comparison target for health evaluation
Capture / Upload	One measurement file ingested for a sensor, with its parsed arrays and derived artefacts
Channel	One signal stream within a capture (<code>ch0</code> , <code>ch1</code> , ...); typically one sensor axis
Config fingerprint	A 32-hex-character SHA-256 prefix of the processing parameters, used as the cache key for computed plots
Crest factor	Peak ÷ RMS; approximately 3 for Gaussian signals, elevated for impulsive faults
DE / NDE	Drive End / Non-Drive End — the two bearing positions on a rotating machine
Digital twin	The complete asset master record that gives measurements engineering context
Envelope spectrum	FFT of the Hilbert envelope; reveals amplitude-modulation sidebands characteristic of bearing defects
Excess kurtosis	Fourth standardised moment minus 3; ≈ 0 for Gaussian noise, higher for spiky signals
FFT	Fast Fourier Transform — converts a time signal to its frequency spectrum
Feature	A scalar descriptor of a channel (one of the ten in the catalogue)
Frequency resolution (Δf)	f_s / N — the spacing between FFT bins
Hann window	A tapering function applied before the FFT to reduce spectral leakage
Hilbert transform	Produces the analytic signal whose magnitude is the envelope
LOR	Lines of Resolution — the FFT line count used by edge acquisition
Nyquist frequency	$f_s / 2$ — the highest frequency representable at a given sample rate
Orbit / Circular waveform	Amplitude mapped onto a circle: $x = A \cdot \cos \theta$, $y = A \cdot \sin \theta$
Primary baseline	The baseline flagged as the default comparison target for a sensor
RMS	Root mean square — the energy-equivalent amplitude
Shaft frequency (1×)	Rotational frequency in Hz = RPM ÷ 60
Trend (segment)	A feature evaluated on successive time segments <i>within one capture</i>
1× / 2× / 3×	Amplitudes at the shaft frequency and its second and third harmonics

16.2 Abbreviations

Table 200 — 16.2 Abbreviations

Abbrev.	Expansion
ADC	Analogue-to-Digital Converter
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
BPFO / BPFi	Ball Pass Frequency Outer / Inner race
CM	Condition Monitoring
CMMS	Computerised Maintenance Management System
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSPRNG	Cryptographically Secure Pseudo-Random Number Generator
CSRF	Cross-Site Request Forgery
DSP	Digital Signal Processing
FK / PK	Foreign Key / Primary Key
HSTS	HTTP Strict Transport Security
IEPE	Integrated Electronics Piezo-Electric
IIO	Industrial I/O (Linux subsystem)
JSONB	PostgreSQL binary JSON type
JWT	JSON Web Token
MEMS	Micro-Electro-Mechanical System
MVCC	Multi-Version Concurrency Control
ORM	Object-Relational Mapper
PM	Preventive Maintenance
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
RTD	Resistance Temperature Detector
SPA	Single-Page Application
TOAST	The Oversized-Attribute Storage Technique (PostgreSQL)
UDP	User Datagram Protocol
UUID	Universally Unique Identifier
VFD	Variable Frequency Drive
XSS	Cross-Site Scripting

16.3 API summary table

Table 201 — 16.3 API summary table

#	Method	Path	Auth	Success	Primary tables
1	GET	/health	Public	200	—
2	POST	/api/v1/auth/login	Public	200	users, refresh_tokens
3	POST	/api/v1/auth/token	Public	200	users, refresh_tokens
4	POST	/api/v1/auth/refresh	Public	200	refresh_tokens
5	POST	/api/v1/auth/logout	Public	204	refresh_tokens
6	GET	/api/v1/auth/me	Auth	200	users, roles
7	POST	/api/v1/equipment/	Write	201	equipment_masters, sensor_configurations
8	GET	/api/v1/equipment/	Auth	200	equipment_masters
9	GET	/api/v1/equipment/{id}	Auth	200	equipment_masters, sensor_configurations
10	PUT	/api/v1/equipment/{id}	Write	200	equipment_masters
11	PATCH	/api/v1/equipment/{id}	Write	200	equipment_masters
12	DELETE	/api/v1/equipment/{id}	Write	204	equipment_masters (+cascade)
13	POST	/api/v1/equipment/{id}/image	Write	200	equipment_masters + FS
14	GET	/api/v1/equipment/{id}/image	Auth	200	equipment_masters + FS
15	DELETE	/api/v1/equipment/{id}/image	Write	204	equipment_masters + FS
16	GET	/api/v1/equipment/{id}/sensors	Auth	200	sensor_configurations
17	POST	/api/v1/equipment/{id}/sensors	Write	201	sensor_configurations
18	PUT	/api/v1/equipment/{id}/sensors/{sid}	Write	200	sensor_configurations
19	DELETE	/api/v1/equipment/{id}/sensors/{sid}	Write	204	sensor_configurations (+cascade)
20	GET	/api/v1/equipment/{id}/ai-readiness	Auth	200	equipment_masters, sensor_configurations
21	GET	/api/v1/lookups/	Auth	200	—
22	GET	/api/v1/lookups/{name}	Auth	200	—
23	POST	/api/v1/measurements/configure	Write	200	plot_configurations
24	GET	/api/v1/measurements/configure/{sensor_id}	Auth	200	plot_configurations
25	PUT	/api/v1/measurements/configure/{sensor_id}	Write	200	plot_configurations
26	GET	/api/v1/measurements/acquisition	Auth	200	sensor_configurations, plot_configurations
27	GET	/api/v1/measurements/acquisition/by-sensor/{sensor_id}	Auth	200	same
28	GET	/api/v1/measurements/acquisition/{device_id}	Auth	200	same
29	POST	/api/v1/measurements/upload	Write	201	5 tables + FS
30	GET	/api/v1/measurements/uploads	Auth	200	sensor_data_uploads, measurement_upload_data
31	GET	/api/v1/measurements/uploads/{id}	Auth	200	same
32	GET	/api/v1/measurements/uploads/{id}/plots	Auth	200	plot_results
33	GET	/api/v1/measurements/uploads/{id}/plots/{type}	Auth	200	plot_results
34	GET	/api/v1/measurements/plot-types	Auth	200	—
35	GET	/api/v1/measurements/uploads/{id}/features	Auth	200	measurement_channel_features + rules
36	GET	/api/v1/measurements/uploads/{id}/factor-trends	Auth	200	measurement_channel_feature_trends
37	GET	/api/v1/measurements/uploads/{id}/features/compare	Auth	200	measurement + baseline features
38	GET	/api/v1/baselines	Auth	200	sensor_baselines, baseline_plot_results
39	GET	/api/v1/baselines/primary	Auth	200	sensor_baselines
40	GET	/api/v1/baselines/{id}	Auth	200	sensor_baselines
41	PATCH	/api/v1/baselines/{id}/primary	Write	200	sensor_baselines
42	POST	/api/v1/baselines/upload	Write	201	sensor_baselines, baseline_plot_results
43	POST	/api/v1/baselines/from-upload/{id}	Write	201	3 baseline tables
44	GET	/api/v1/baselines/{id}/plots	Auth	200	baseline_plot_results
45	GET	/api/v1/baselines/{id}/plots/{type}	Auth	200	baseline_plot_results

#	Method	Path	Auth	Success	Primary tables
46	GET	/api/v1/baselines/{id}/features	Auth	200	baseline_channel_features

16.4 Database summary table

Table 202 — 16.4 Database summary table

#	Table	Columns	PK	FKs	Indexes	Purpose
1	equipment_masters	40	id	—	3	Machine master record
2	sensor_configurations	16	id	1	2	Measurement point
3	plot_configurations	11	id	1	1 (unique)	Processing profile
4	sensor_data_uploads	18	id	1	2	Capture lifecycle
5	measurement_upload_data	10	id	2	2	Durable file + parsed data
6	plot_results	18	id	2	3	Plot cache
7	sensor_baselines	16	id	2	2	Reference captures
8	baseline_plot_results	18	id	2	2	Baseline plot cache
9	feature_definitions	7	id	—	1 (unique code)	Feature catalogue
10	feature_threshold_rules	10	id	1	1	Evaluation rules
11	measurement_channel_features	10	id	3	2	Scalar feature values
12	measurement_channel_feature_trends	9	id	3	2	Segment trend values
13	baseline_channel_features	10	id	3	2	Reference feature values
14	roles	4	id	—	1 (unique)	Role catalogue
15	users	10	id	—	1 (unique) + CHECK	Accounts
16	user_roles	3	(user_id, role_id)	2	—	Role assignment
17	refresh_tokens	6	id	1	2	Session tokens

16.5 Backend class / module summary

Table 203 — 16.5 Backend class / module summary

Type	Name	File	Responsibility
Config	Settings	config.py	Environment-bound configuration
Infra	engine, SessionLocal, Base, get_db	database.py	Persistence plumbing
Entity	Equipment	models/equipment.py	equipment_masters
Entity	SensorConfiguration	models/sensor.py	sensor_configurations
Entity	PlotConfiguration, SensorDataUpload, PlotResult, MeasurementUploadData, SensorBaseline, BaselinePlotResult, FeatureDefinition, FeatureThresholdRule, MeasurementChannelFeature, MeasurementChannelFeatureTrend, BaselineChannelFeature	models/measurement.py	Measurement domain
Entity	Role, User, UserRole, RefreshToken	models/user.py	Identity domain
Controller	routers/auth.py	5 endpoints	Authentication
Controller	routers/equipment.py	14 endpoints	Asset master
Controller	routers/lookups.py	2 endpoints	Reference lists
Controller	routers/measurements.py	15 endpoints	Ingestion, plots, features
Controller	routers/baselines.py	9 endpoints	Baselines
Repository	crud/equipment.py	13 functions	Equipment + sensor access
Repository	crud/measurement.py	15 functions	Config + upload + plot access
Repository	crud/baseline.py	10 functions	Baseline access
Repository	crud/feature.py	11 functions	Feature + rule access
Repository	crud/user.py	12 functions	Identity access
Service	auth_service.py	9 functions	Hashing, JWT, refresh lifecycle
Service	seed.py	3 functions	Idempotent account seeding
Service	pdf_parser.py	11 functions	CSV/PDF → channel arrays
Service	signal_processing.py	6 functions	The five DSP routines
Service	plot_generator.py	6 functions + registry	Plot orchestration
Service	plot_storage.py	7 functions	Fingerprinting and caching
Service	baseline_storage.py	3 functions	Baseline plot persistence
Service	feature_extraction.py	9 functions	Ten features + trends
Service	feature_storage.py	6 functions	Evaluation and persistence
Service	threshold_evaluator.py	ThresholdRule + 2 functions	Rule evaluation
Service	acquisition_config.py	4 functions	Edge JSON generation
Dependency	dependencies/auth.py	4 symbols	AuthN / AuthZ

16.6 Frontend component summary

Table 204 — 16.6 Frontend component summary

Group	Count	Components
Pages	9	Login, Dashboard, EquipmentMasterList, EquipmentMaster (New/Edit), VibrationAnalysis, Settings, Unauthorized, ChangePassword, ModulePages
Layout	6	AppShell, Sidebar, TopNav, PageHero, ComingSoon, nav-config
UI primitives	6 (+2 docs)	Button, FormField (+4 inputs), GlassCard, MultiSelect, SectionCard, Toast
Auth	1	ProtectedRoute
Charts (shared)	6	GraphWorkspace, GraphToolbar, GraphStatisticsPanel, GraphChannelSelector, ThresholdZoneLegend, EchartsGraphViewport
Analysis (root)	12	AnalysisSectionHeader, AnalysisSummaryPanel, CaptureTimeline, CaptureTimelinePanel, CaptureTimelineSection, ChartHeader, CompactDateRangeBar, DiagnosticChart, PlotChart, PlotSelector, SaveBaselineModal, analysis-layout
Analysis / charts	4	EchartsDiagnosticChart + 3 deprecated re-exports
Analysis / baseline	3	BaselineManagementPanel, BaselineDetailCard, baseline-utils
Analysis / health	13	StatusHealthTab, StatusHealthSection, HealthChannelSelector, HealthSummaryCards, ChannelHealthOverviewCard, FeatureStatusTable, FeatureStatusBadge, FeatureComparisonSection, FeatureTrendCardsSection, HealthMetricCard, HealthThresholdsTable, HealthInfoBanner, HealthEmptyState, SensorThresholdConfig
Analysis / workspace	7	AnalysisWorkspace, AnalysisTabNav, DetailedAnalysisTab, StatisticsTab, TrendAnalysisTab, SelectedCapturePanel, BaselineSelectionPanel
Equipment	9	EquipmentForm, EquipmentPageShell, DigitalTwinHeader, FormStepper, MachineVisualizationPanel, SensorMountingDiagram, AssetHealthPanel, AssetIntelligencePanel, CompletenessEngine
Equipment / industrial	3	ProgressRing, CriticalityIndicator, IndustrialEmptyState
Equipment / tabs	6	Basic, Mechanical, Rotating, Operating, Sensors, Review
Settings	10	SettingsSectionCard, SettingsTabNav, ToggleSwitch, DeviceInfoCard, ChannelConfigurationSection, ChannelMappingOverview, ThresholdConfigurationSection, ThresholdCoverageMatrix, SettingsPageActions, VibrationSettingsModule
Brand	5	HeroIntelligenceBg, LoginIntelligenceBg, SensorPulseRings, VibrationWave, VibrationIntelligenceBg
Contexts	3	AuthContext, LayoutContext, ThemeContext
Hooks	6	useEchartsResize, useHealthStatusData, useFeatureHealthDashboard, useUploadFactorTrends, useHistoricalTrendData, useVibrationSettings
Lib modules	30	See §3.19
Type modules	9	See §3.20
API modules	5	client, auth, equipment, measurements, baselines

16.7 Folder-by-folder file index

Table 205 — 16.7 Folder-by-folder file index

Path	Why it exists	Connects to
<code>.env</code>	Single source of environment values for all three tiers	Compose, <code>Settings</code> , <code>alembic/env.py</code> , Vite
<code>docker-compose.yml</code>	Orchestrates the four services with correct ordering and volumes	All Dockerfiles, <code>.env</code>
<code>START.md</code>	Four-step local runbook	Developer workflow
<code>scripts/vibration.py</code>	Reference edge acquisition reader for a ZedBoard/IIO ADC	Conceptually pairs with <code>/measurements/acquisition</code>
<code>backend/Dockerfile</code>	Reproducible backend image; migrates then serves	<code>requirements.txt</code> , <code>app/</code>
<code>backend/alembic.ini</code>	Migration runner configuration	<code>alembic/env.py</code>
<code>backend/alembic/env.py</code>	Loads env, injects <code>DATABASE_URL</code> , imports models for autogenerate	<code>app.database</code> , <code>app.models</code>
<code>backend/alembic/versions/*</code>	The authoritative schema history (11 revisions)	Every table
<code>backend/app/config.py</code>	Fail-fast configuration binding	Everything
<code>backend/app/database.py</code>	Engine, session factory, declarative base, request-scoped session	All CRUD
<code>backend/app/main.py</code>	Application assembly, CORS, seeding, OpenAPI, health	All routers
<code>backend/app/dependencies/auth.py</code>	AuthN/AuthZ dependencies	All routers
<code>backend/app/models/*</code>	Persistence shape	CRUD, Alembic
<code>backend/app/schemas/*</code>	Wire shape and validation	Routers, services
<code>backend/app/crud/*</code>	Data access without HTTP or business rules	Routers, services
<code>backend/app/routers/*</code>	HTTP surface	Services, CRUD, schemas
<code>backend/app/services/*</code>	Business logic and algorithms	CRUD, models
<code>backend/scripts/*</code>	Standalone developer utilities	Not imported by the app
<code>frontend/index.html</code>	SPA shell	<code>main.tsx</code>
<code>frontend/vite.config.ts</code>	Dev server, alias, proxy, build	<code>tsconfig</code> , <code>src/</code>
<code>frontend/tailwind.config.js</code>	Design tokens consumed by every component	<code>index.css</code> , all components
<code>frontend/nginx.conf</code>	Production SPA serving with history fallback	<code>Dockerfile</code> preview stage
<code>frontend/src/main.tsx</code>	React root, Query client, Router	<code>App.tsx</code> , <code>index.css</code>
<code>frontend/src/App.tsx</code>	Provider stack and route table	Contexts, pages, guards
<code>frontend/src/index.css</code>	Tokens, base styles, 100+ component classes, keyframes	Every component
<code>frontend/src/api/*</code>	The only network boundary	Pages, hooks
<code>frontend/src/contexts/*</code>	Cross-cutting client state	App-wide
<code>frontend/src/hooks/*</code>	Query composition and view models	Analysis and settings components
<code>frontend/src/lib/*</code>	Pure maths, chart options, formatters, tokens	Components and hooks
<code>frontend/src/types/*</code>	Interfaces and driving constants	App-wide
<code>frontend/src/components/*</code>	Feature and shared UI	Pages
<code>frontend/src/images/*</code>	Brand and illustration assets with a typed barrel	Sidebar, empty states

16.8 Dependency reference

16.8.1 Backend

Table 206 — 16.8.1 Backend

Dependency	Version	Purpose	Where used
fastapi	0.115.0	HTTP framework, DI, OpenAPI	<code>main.py</code> , all routers
uvicorn[standard]	0.30.6	ASGI server	Dockerfile CMD, dev command
sqlalchemy	2.0.35	ORM and query building	<code>database.py</code> , all models and CRUD
alembic	1.13.3	Schema migrations	<code>alembic/</code>
psycopg2-binary	2.9.9	PostgreSQL driver	Engine URL
python-multipart	0.0.12	multipart parsing	<code>upload_image</code> , <code>upload_sensor_data</code> , <code>upload_baseline</code>
python-dotenv	1.0.1	<code>.env</code> loading	<code>alembic/env.py</code>
pillow	10.4.0	Image handling	Declared; not imported in <code>app/</code>
pydantic	2.9.2	Schemas and validators	All <code>schemas/</code>
pydantic-settings	2.5.2	Env-bound settings	<code>config.py</code>
aiofiles	24.1.0	Async file I/O	Declared; not imported
python-jose[cryptography]	3.3.0	JWT encode/decode	<code>auth_service.py</code> , <code>dependencies/auth.py</code>
passlib[bcrypt]	1.7.4	Password hashing context	<code>auth_service.py</code>
bcrypt	4.0.1	bcrypt backend	via passlib
email-validator	2.2.0	<code>EmailStr</code> support	<code>schemas/auth.py</code>
numpy	1.26.4	Array maths	<code>signal_processing.py</code> , <code>feature_extraction.py</code>
scipy	1.13.1	<code>fft</code> , <code>fftfreq</code> , <code>hilbert</code>	Same two modules
pdfplumber	0.11.4	PDF table/text extraction	<code>pdf_parser.py</code>
reportlab	—	Sample-PDF generation	<code>scripts/create_sample_sensor_pdf.py</code> ; not in requirements.txt

16.8.2 Frontend — runtime

Table 207 — 16.8.2 Frontend — runtime

Dependency	Version	Purpose	Where used
react / react-dom	^18.3.1	UI runtime	Everywhere
react-router-dom	^6.27.0	Routing and guards	<code>App.tsx</code> , <code>ProtectedRoute</code> , <code>Sidebar</code> , page navigation
@tanstack/react-query	^5.59.20	Server-state cache	<code>main.tsx</code> , all hooks and data pages
axios	^1.7.7	HTTP client + interceptors	<code>api/*</code> , <code>Login.tsx</code> (<code>isAxiosError</code>)
echarts	^6.1.0	Charting engine	<code>lib/*-option.ts</code> , chart components
echarts-for-react	^3.0.6	React binding	<code>EchartsGraphViewport</code> , <code>HealthMetricCard</code>
react-hook-form	^7.53.2	Form state, <code>useFieldArray</code>	Equipment wizard
@hookform/resolvers	^3.9.0	Zod ↔ RHF bridge	<code>EquipmentForm</code>
zod	^3.23.8	Schema validation	<code>types/equipment.ts</code>
framer-motion	^12.40.0	Animation	Cards, sidebar, toasts, table rows, login
lucide-react	^0.454.0	Icons	Every screen
tailwindcss	^3.4.14	Styling	All components
tailwindcss-animate	^1.0.7	Tailwind animation plugin	<code>tailwind.config.js</code>
clsx	^2.1.1	Conditional classes	<code>lib/utls.ts</code>
tailwind-merge	^2.5.4	Class conflict resolution	<code>lib/utls.ts</code>
date-fns	^4.1.0	Date formatting/arithmetic	<code>upload-format.ts</code> , timeline, trend hook
class-variance-authority	^0.7.0	Variant helper	Declared; no import found
@radix-ui/react-dialog, -dropdown-menu, -label, -popover, -select, -separator, -slot, -toast	^1.x– ^2.x	Headless primitives	Declared; no import found — modals and menus are hand-rolled

16.8.3 Frontend — development

Table 208 — 16.8.3 Frontend — development

Dependency	Version	Purpose
vite	^5.4.10	Dev server and bundler
@vitejs/plugin-react	^4.3.3	React transform and fast refresh
typescript	^5.6.3	Type checking (the build gate)
postcss	^8.4.47	CSS transform pipeline
autoprefixer	^10.4.20	Vendor prefixes
@types/node	^22.9.0	Node types for vite.config.ts
@types/react, @types/react-dom	^18.3.x	React type definitions

16.8.4 Infrastructure images

Table 209 — 16.8.4 Infrastructure images

Image	Tag	Role
postgres	16	Database
dpage/pgadmin4	latest	Database administration UI
python	3.11-slim	Backend base
node	20-alpine	Frontend build base
nginx	alpine	Frontend production server

16.9 Constants quick reference

Table 210 — 16.9 Constants quick reference

Constant	Value	Location
Access-token lifetime	30 minutes	<code>config.py</code>
Refresh-token lifetime	7 days	<code>config.py</code>
Refresh-token entropy	48 bytes URL-safe	<code>auth_service.py</code>
Default sampling rate	25600 Hz	<code>crud/measurement.py</code> , <code>schemas/measurement.py</code> , <code>lib/waveform-time-axis.ts</code>
Default FFT lines	1600	Same
Max image size	10 MB	<code>config.py</code>
Max measurement file size	50 MB	<code>config.py</code>
Channel range	1–32 (<code>channel</code> 0–31)	Schemas and query params
FFT lines range	64–65536	<code>schemas/measurement.py</code>
Trend segments	32	<code>signal_processing.py</code> , <code>feature_extraction.py</code> , <code>lib/health-metrics.ts</code>
Shaft-frequency search band	5–120 Hz	<code>feature_extraction.py</code>
FFT band-energy band	0–500 Hz	<code>feature_extraction.py</code>
Circular waveform max points (backend)	2048	<code>signal_processing.py</code>
Waveform/orbit max points (frontend)	8192	<code>chart-data.ts</code>
Spectrum max points (frontend)	2000	<code>chart-data.ts</code>
Primary chart height	580 px	<code>chart-constants.ts</code>
Compact chart height	220 px	<code>chart-constants.ts</code>
Fullscreen ratio / chrome	0.92 / 132 px, floor 360 px	<code>chart-layout.ts</code>
Threshold crossings per level	24	<code>threshold-overlay.ts</code>
React Query <code>staleTime</code>	30 000 ms (60 000 ms for trend plots)	<code>main.tsx</code> , <code>hooks</code>
Factor-trend poll interval	3000 ms	<code>useUploadFactorTrends.ts</code>
Long-request timeout	120 000 ms	<code>api/measurements.ts</code>
Toast lifetime	4000 ms	<code>Toast.tsx</code>
Sidebar widths	320 / 80 px	<code>Sidebar.tsx</code>
Equipment page size	20	<code>EquipmentMasterList.tsx</code>
Upload page size (client default)	200	<code>api/measurements.ts</code>
Health/vibration channel count	8	<code>types/health-status.ts</code> , <code>types/vibration-settings.ts</code>
Algorithm version (fingerprint)	<code>v1</code>	<code>plot_storage.py</code>

16.10 Diagram index

Every diagram is numbered sequentially and listed with its figure number in the **List of Figures** in the front matter. This index groups the same diagrams by subject and gives the section in which each appears.

Table 211 — 16.10 Diagram index

Subject	Section
Software architecture (4 tiers)	§1.7
High-level workflow	§1.8
Upload pipeline sequence	§1.8.1
Deployment topology	§2.1, §12.1
Dev API paths	§2.2.3
Frontend provider and route tree	§2.3
Backend request routing	§2.4
Application startup sequence	§2.4.1
Database cluster overview	§2.5
Login sequence	§2.6
Silent refresh with queueing	§2.6.1
API communication flow	§2.7
Data flow (ingestion → visualisation)	§2.8
Response lifecycle	§2.10
<code>ProtectedRoute</code> decision tree	§3.3.1
Chart architecture	§3.9.1
<code>custom_openapi</code> flow	§4.2.1
Backend startup sequence	§4.2.1, §10.11.1
Timestamp resolution heuristic	§4.9.4
Plot cache-or-compute flow	§4.9.6
Dependency injection graph	§4.15
Refresh rotation sequence	§5.4.3
Equipment create sequence	§5.5.1
Feature computation sequence	§5.7.11
Baseline promotion sequence	§5.8.6
Entity relationship diagram	§6.2
End-to-end database flow	§6.14
Plot read sequence	§6.14
Authorisation enforcement	§7.2.2
User seeding flow	§7.6
Parsing decision flow	§8.4
Feature evaluation flow	§8.6
User flow map	§9.1
Upload-and-analyse sequence	§9.4
Backend entry-to-shutdown	§10.11.1
Frontend entry-to-teardown	§10.11.2
Configuration precedence	§11.12
Frontend build pipeline	§12.4
CI/CD reference pipeline	§12.8
Error-handling architecture	§15.1

17.0 References

17.1 Domain and standards references cited in the code

Table 212 — 17.1 Domain and standards references cited in the code

Reference	Cited in	Used for
<i>Condition Monitoring with Vibration Signals</i> (Randall / Antoni)	<code>lib/industrial-viz-standards.ts</code> , <code>lib/chart-bounds.ts</code> , <code>lib/chart-reference-lines.ts</code> , <code>ThresholdZoneLegend.tsx</code>	Alarm-zone colour coding, harmonic/order markers, symmetric waveform display, crest-factor and kurtosis interpretation
<i>The Scientist and Engineer's Guide to Digital Signal Processing</i> (Steven W. Smith)	<code>lib/industrial-viz-standards.ts</code> , <code>lib/chart-reference-lines.ts</code>	FFT frequency axis, Nyquist limit $f_s/2$, frequency resolution $\Delta f = f_s/N$
ISO 10816-3 — Mechanical vibration: evaluation of machine vibration by measurements on non-rotating parts	<code>ISO_10816_VELOCITY_ZONES_REFERENCE</code> in <code>lib/industrial-viz-standards.ts</code>	Velocity-severity zones, documented but deliberately not auto-applied because limits depend on machine class and mounting

The code's own note on ISO 10816 is reproduced here because it is a design decision, not an omission:

"Zone limits depend on machine class and mounting. Configure per asset in Vibration Settings."

17.2 Technology documentation

Table 213 — 17.2 Technology documentation

Technology	Reference
FastAPI	https://fastapi.tiangolo.com
SQLAlchemy 2.0	https://docs.sqlalchemy.org/en/20/
Alembic	https://alembic.sqlalchemy.org
Pydantic v2	https://docs.pydantic.dev
PostgreSQL 16	https://www.postgresql.org/docs/16/
NumPy	https://numpy.org/doc/
SciPy signal / fft	https://docs.scipy.org/doc/scipy/reference/
pdfplumber	https://github.com/jsvine/pdfplumber
passlib	https://passlib.readthedocs.io
python-jose	https://python-jose.readthedocs.io
React 18	https://react.dev
React Router 6	https://reactrouter.com
TanStack Query 5	https://tanstack.com/query/latest
Apache ECharts	https://echarts.apache.org
Tailwind CSS	https://tailwindcss.com
Vite	https://vite.dev
Zod	https://zod.dev
React Hook Form	https://react-hook-form.com
Framer Motion	https://www.framer.com/motion/
Docker Compose	https://docs.docker.com/compose/

17.3 Internal source references

Table 214 — 17.3 Internal source references

Artefact	Path
Local runbook	START.md
Card sizing contract	frontend/src/components/ui/CARD_SIZING.md
Card hover contract	frontend/src/components/ui/CARD_HOVER.md
Swagger login instructions and dev accounts	backend/app/main.py (FastAPI description)
Interactive API documentation	http://localhost:8000/docs , /redoc , /openapi.json
Migration history	backend/alembic/versions/001...011
Auth smoke test	backend/scripts/test_auth_phase1.py
Sample data generator	backend/scripts/create_sample_sensor_pdf.py
Edge acquisition reference	scripts/vibration.py

End of document.